

Applications of Functional Modeling with Abstract Algebra

Higher-Order Effects and Automatic
Differentiation

Birthe van den Berg

Supervisors:

Prof. dr. ir. T. Schrijvers

Prof. dr. P. Dedecker

Dissertation presented in partial
fulfillment of the requirements for the
degree of Doctor of Engineering
Science (PhD): Computer Science

September 2023

Applications of Functional Modeling with Abstract Algebra

Higher-Order Effects and Automatic Differentiation

Birthe VAN DEN BERG

Examination committee:

Prof. dr. A. Bultheel, chair

Prof. dr. ir. T. Schrijvers, supervisor

Prof. dr. P. Dedecker, supervisor

Prof. dr. ir. F. Piessens (KU Leuven)

Prof. dr. ir. G. Janssens (KU Leuven)

Prof. dr. C. Bach Poulsen

(Delft University of Technology)

Dr. N. Wu

(Imperial College London)

Dissertation presented in partial fulfillment of the requirements for the degree of Doctor of Engineering Science (PhD): Computer Science

September 2023

© 2023 KU Leuven – Faculty of Engineering Science
Uitgegeven in eigen beheer, Birthe van den Berg, Celestijnenlaan 200A box 2402, B-3001 Leuven (Belgium)

Alle rechten voorbehouden. Niets uit deze uitgave mag worden vermenigvuldigd en/of openbaar gemaakt worden door middel van druk, fotokopie, microfilm, elektronisch of op welke andere wijze ook zonder voorafgaande schriftelijke toestemming van de uitgever.

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher.

Acknowledgements

I would like to take a moment here to express my gratitude to several people that have helped me in some way during the 4-year span of this work.

Thanks to my supervisor, Tom Schrijvers, and my co-supervisor Peter Dedecker, for their daily guidance, support and helpful advice.

Thanks to the members of my examination committee, Frank Piessens, Gerda Janssens, Casper Bach Poulsen and Nicolas Wu for reading the text.

Thanks to my computer science colleagues for the interesting discussions about the topic of my PhD, or off-topic at the coffee machine.

Special thanks to my office mates, Roger Bosman, Jesse Hoobergs, Pieter Delobelle, Gert-Jan Bottu, Koen Pauwels and César Santos, who have made 04.01 a nice place to work, even at +30°C.

Thanks to my colleagues and friends at the chemistry department, for introducing me to the world of fluorescence microscopy, for collaborating on the autofocus project, for practicing my presentation skills, for their simple—but not so simple—questions during the lab meetings and for their interest in my topic, which is not at all familiar to them.

Thanks to my co-authors for the nice collaborations.

A special thanks to James McKinna and Alexander Vandenbroucke for their critical mindset but above all their perseverance to improve our work.

Thanks to the many anonymous reviewers for their valuable time, expertise and diligent effort in critically reading and questioning my work.

Thanks to KU Leuven, the university that gave me tremendous opportunities during these 4 years.

Thanks to my family and friends for their unconditional support!

Abstract (EN)

Abstract algebra is the field that studies the underlying algebraic structures of a programming language, such as rings, monoids, and modules. It leverages insights into their fundamental properties and behavior. The goal of this work is to use abstract algebra to better understand two different concepts: computational effects and automatic differentiation.

The first part of this thesis studies higher-order effects & handlers, an extension of algebraic effects & handlers where effects can contain an internal computation. Examples are scoped effects, that have a computation in scope; latent effects, that defer a computation; and parallel effects, that parallelize different computations. We abstract over these and other classes of effects, using a generic framework that is based on a free-forgetful adjunction. This generalization retains the separation of syntax and semantics, one of the coveted properties of algebraic effects & handlers, but also adds a challenge in terms of forwarding unknown effects. We show that different classes of effects each form an instance of our framework. Furthermore, we define operational semantics for scoped effects by extending Eff, a calculus that supports algebraic effects as built-in primitives.

The second part of this thesis applies abstract algebra to differentiation algorithms. We show that the essence of (both symbolic and automatic) differentiation can be captured in a *oneliner*: an abstract differentiation function that is based on three algebraic structures. First, we introduce the notion of semimodules (modules over a semiring). Second, we generalize dual numbers into Nagata numbers, where the two components can have distinct types. Third, we need Kronecker's delta function, and Kronecker isomorphisms in order to devise correct-by-construction variants of automatic differentiation. We show that subsequent versions of the differentiation algorithm have improved computational complexity, starting from the naive symbolic differentiation via forward-mode automatic differentiation to the efficient reverse-mode with mutable arrays.

Thus, having generalized and formalized two popular programming concepts (computational effects and automatic differentiation), we aim to provide a better understanding of their underlying formalisms, and to facilitate programming with them.

Abstract (NL)

Abstracte algebra is het vakgebied dat de onderliggende algebraïsche structuren van een programmeertaal bestudeert, zoals ringen, monoïden en modules. Het geeft ons inzicht in de fundamentele eigenschappen en gedrag van die structuren. Het doel van dit werk is om abstracte algebra te gebruiken om twee verschillende concepten beter te begrijpen: computationele effecten en automatische differentiatie.

Het eerste deel van dit werk bestudeert hogere-orde effecten & handlers, een uitbreiding van algebraïsche effecten & handlers waarbij effecten een interne berekening kunnen bevatten. Voorbeelden zijn scoped effecten, die een berekening in scope hebben; latente effecten, die een berekening uitstellen; en parallele effecten, die verschillende berekeningen paralleliseren. We abstraheren over deze en andere klassen van effecten, met behulp van een generiek framework dat gebaseerd is op een free-forgetful adjunctie. Deze generalisatie behoudt de scheiding van syntaxis en semantiek, een van de begeerde eigenschappen van algebraïsche effecten & handlers, maar voegt ook een extra uitdaging toe, namelijk het “forwarden” van onbekende effecten. We laten zien dat verschillende klassen van effecten elk een instantie van ons framework vormen. Verder definiëren we een operationele semantiek voor scoped effecten als een uitbreiding van Eff, een calculus die algebraïsche effecten ondersteunt als ingebouwde primitieven.

Het tweede deel van dit werk past abstracte algebra toe op differentiatiealgoritmen. We laten zien dat symbolische differentiatie en automatische differentiatie kunnen worden gedefinieerd op basis van *een enkele* regel: een abstracte differentiatiefunctie die gebaseerd is op drie algebraïsche structuren. Ten eerste introduceren we het begrip semimodules (modules over een semiring). Ten tweede veralgemenen we duale getallen in Nagata-getallen, waarbij de twee componenten verschillende types kunnen hebben. Ten derde hebben we de Kroneckerdeltafunctie en Kroneckerisomorfismen nodig om varianten van automatische differentiatie te ontwikkelen die correct zijn vanaf constructie.

We laten zien dat opeenvolgende versies van het differentiatiealgoritme een verbeterde computationele complexiteit hebben, startend bij de naïeve symbolische differentiatie over voorwaartse automatische differentiatie naar de meer efficiënte achterwaartse differentiatie met muteerbare arrays.

Samengevat hebben we twee populaire programmeerconcepten veralgemeend en geformaliseerd: computationele effecten en automatische differentiatie. Met deze generalisatie streven we ernaar om hun onderliggende formalismen beter te begrijpen, en het programmeren ermee te faciliteren.

Contents

Abstract (EN)	iii
Abstract (NL)	v
Contents	vii
1 Introduction	1
1.1 Abstract Algebra	1
1.2 Functional Programming	2
1.3 Goal and Overview	3
I Effects & Handlers	7
2 Background	11
2.1 Algebraic Operations	11
2.2 Computations as Free Monads	12
2.3 Interpretation using Folds	13
2.4 Modularly Combining Effects	13
2.5 Non-Algebraic Effects and the Algebraicity Property	15
2.5.1 Algebraicity and Generic Operations	15
2.5.2 Higher-Order Effects	16
3 A Generic Free Monad for Modeling Effects	19
3.1 Effects and Their Interpretation	19
3.2 Instances of the Generic Monad	22
3.3 Algebraic Effects & Handlers	22
3.4 Modular Composition	24
4 Higher-Order Effects & Handlers	27
4.1 Scoped Effects & Handlers	27

4.2	Writer Effects & Handlers	30
4.3	Parallel Effects & Handlers	33
4.4	Latent Effects & Handlers	36
4.5	Bracketing Effects & Handlers	41
5	Categorical Foundations	45
5.1	Categories and Functors	45
5.2	Adjunctions	46
5.3	A Generalized Adjunction	49
5.3.1	The $Id \bowtie - \dashv App$ Adjunction	49
5.3.2	The $\overline{Free} \dashv \overline{U}$ Adjunction	50
5.3.3	Composing the Two Adjunctions	54
5.4	An Adjunction for Scoped Effects	54
5.4.1	Scoped Effects as Handlers are Not Modular	55
5.4.2	Eilenberg-Moore Algebras and Indexed Algebras	56
5.4.3	Functorial Algebras	58
6	A Calculus for Scoped Effects & Handlers	61
6.1	A Calculus for Algebraic Effects	62
6.2	Challenges & Design Choices	63
6.2.1	Eff with Koka-based Row Typing	63
6.2.2	Scoped Effects as Built-in Syntactic Primitives	64
6.2.3	Polymorphic Handlers	65
6.2.4	Forwarding Unknown Operations	66
6.3	The λ_{sc} -Calculus	67
6.3.1	Syntax	67
6.3.2	Operational Semantics	67
6.3.3	Type-and-Effect System	69
6.3.4	Metatheory and Type Inference	71
7	Summary	73
II	Automatic Differentiation	77
8	Background	81
8.1	A Symbolic Expression Language	81
8.2	Semirings	82
8.2.1	Commutative Semirings	83
8.2.2	Semiring Homomorphisms	85
8.3	Differentiation	85
8.3.1	Symbolic Differentiation	86
8.3.2	Dual Numbers	87

8.3.3	Forward-Mode Automatic Differentiation	88
8.3.4	Symbolic AD as Forward AD	89
9	Abstract Automatic Differentiation	91
9.1	Semimodules	92
9.2	Nagata Numbers	92
9.3	Kronecker's Delta	93
9.4	Automatic Differentiation as a One-Liner	94
9.5	Kronecker Isomorphisms	94
10	Forward-Mode and Reverse-Mode Automatic Differentiation	99
10.1	Forward-Mode Automatic Differentiation	99
10.1.1	Dense Function Space	99
10.1.2	Sparse Maps	101
10.2	Reverse-Mode Automatic Differentiation	103
10.2.1	Semimodule Homomorphisms	103
10.2.2	Cayley Representation	106
10.2.3	Mutable Arrays	109
10.3	Computational Complexity	111
11	Extensions	113
11.1	Overloading Initial Expressions	113
11.2	Additional Primitives	114
11.3	Non-Commutative Semirings	116
11.4	Higher Derivatives	118
11.5	Sharing Subexpressions	121
12	Applications	125
12.1	Automatic Differentiation in Prolog	125
12.1.1	Symbolic Expressions and Their Evaluation	126
12.1.2	Symbolic Differentiation	126
12.1.3	Forward-Mode Automatic Differentiation	127
12.1.4	Reverse-Mode Automatic Differentiation	128
12.2	Sum-Product Loop Programming	130
12.2.1	Parameter Estimation	130
12.2.2	Benchmarking our AD Versions	131
12.3	Matrices	132
12.4	Possible worlds	134
12.4.1	Facts, Worlds and Queries	135
12.4.2	Possible Worlds as Boolean Algebras	138
12.4.3	Probabilities as Commutative Semirings	139
12.4.4	Automatic Differentiation to Learn Probabilities	141

13 Summary	143
14 Conclusion	145
14.1 Global Summary	145
14.2 Future Work	146
14.3 Abstract Algebra is Elegant and General	147
Bibliography	149
List of Publications	166
A Isomorphisms	169
A.1 Algebraic Effects & Handlers	169
A.2 Scoped Effects & Handlers	170
A.3 Parallel Effects & Handlers	173
A.4 Writer Effect & Handler	176
A.5 Latent Effects & Handlers	177
A.6 Bracketing Effect & Handler	178
B Examples of Higher-Order Effects	181
B.1 Catch as a Scoped Effect	181
B.2 Reader as a Scoped Effect	183
B.3 Nondeterminism with Cut	185
B.4 Depth-Bounded Search	189
B.5 Concurrency	191
B.6 Amb	192
B.7 Two Ways of Modifying the Log with <code>censor</code>	194
B.8 Lambda by Call-by-Push-Value	195
C An Effect Library of Latent Effects	197
C.1 Reading	197
C.2 Mutating the State	198
C.3 Failing	199
C.4 Throwing and Catching Exceptions	200
C.5 Adding Arithmetics	201
C.6 Printing	201
C.7 Delaying	202
C.8 Abstracting	203
D λ_{sc}: Full Version	205
D.1 Syntax	206
D.2 Operational Semantics	207
D.3 Type-and-Effect System	208
D.4 Syntax-Directed Type-and-Effect System	212

D.5	Metatheory and Proofs	217
D.6	Type Inference	222
E	Derivation of the Second-Order Derivative	253
F	Proof of Lemma 1	255
G	Functions of Different Haskell Libraries	257
H	Derivation of <code>fwdad/4</code> from Symbolic Differentiation	263

Chapter 1

Introduction

Nowadays, computer systems have become indispensable to tackle societal and scientific challenges. The process of transforming problems into algorithms has emerged as a crucial skill. At the heart of this transformation lies the understanding of *decomposition*: most problems, however complex, can be broken down into smaller, manageable subproblems. These smaller building blocks can then be (re-)composed in order to build large and complex software systems.

Consequently, the design of programming languages that facilitate the composition of smaller building blocks has become essential: entire research fields focus on the underlying mathematical structures that make this possible.

In this work, we leverage abstract algebra to study the algebraic structures behind computational effects and automatic differentiation, modeling these concepts from a functional point of view.

1.1 Abstract Algebra: Theoretical Underpinnings of Programming Language Design

Abstract algebra is the study of algebraic structures, such as rings, fields and monoids, and the relations between them. In general, an algebraic structure consists of a set (e.g., natural numbers), and operations representing the relations between the elements in that set (e.g., addition). Although abstract algebra is rooted in the field of mathematics, it also forms the theoretical underpinnings

of programming language design: by leveraging abstract algebra, one can foster a deeper understanding of the underlying principles of computation.

In particular, we study the *denotational semantics* [137] of programming language features, i.e., a description or formalization of their meaning in terms of algebraic structures. In contrast, an operational semantics [136] is focused more on the execution of these features. Denotational semantics follow the principle of compositionality: the semantics should compose. For that reason, we not only study the algebraic structures themselves, but also the maps between them (called *homomorphisms*).

When talking about the fundamental structures on which (in particular functional) programming languages rely, we often refer to category theory instead of abstract algebra. Awodey [6] calls category theory a branch of abstract algebra that mainly looks at “structure-preserving transformations” between algebraic structures. Indeed, algebraic structures and their homomorphisms form categories, and their maps compose. Just like abstract algebra, category theory allows us to describe common properties or behavior of different algebraic structures.

Here, we use abstract algebra (and category theory) to define novel and existing algebraic structures, their properties, and their connection to other algebraic structures, and show how they underlie common programming language features.

1.2 Pure Functional Programming: A Paradigm without Computational Effects

To model programming language features, we can choose between different programming paradigms, which each employ their own thinking patterns, ways of looking at these features, and approach to solving computational problems. In the landscape of software development, various programming paradigms have emerged. For example, *declarative programming* stands out as a powerful approach that focuses on *what* needs to be accomplished. This contrasts with imperative programming, which deals with *how* to achieve a result or solution.

Functional programming is a declarative programming paradigm with functions as its primary building blocks. Functions compose so it is natural to model abstract algebra from a functional point of view.

Unlike imperative programming, which focuses on procedures for explicit state manipulation and control flow, functional programming emphasizes the evaluation of pure functions. The latter are functions that produce consistent

outputs solely based on their inputs, without any computational effects. These *computational effects* (also called *(side-)effects*) refer to the observable changes or interactions that occur during program execution. Examples of computational effects are input/output operations, nondeterminism, state manipulation, exceptions and concurrency.

Functional programming, which is based on the lambda-calculus [8], originates in academia, where it is used for research and development of programming language features. However, nowadays, functional programming goes beyond academia and is also seeing use in education and industry. For example, languages such as Elm and Clojure are increasingly used in web development. Apache Spark, a framework for distributed data processing, provides API's in Scala. Languages like Erlang and Elixir excel at building fault-tolerant concurrent systems. Haskell has become a popular language for embedding domain-specific languages, i.e., programming languages that are tailored to a specific problem domain. In the literature, applications of domain-specific languages can be found in various domains of discourse: music composition [69], financial contracts [115], combinatorial optimization [133], and fluorescence microscopy [154, 150].

Throughout this work, we use Haskell as a vehicle to illustrate our findings.

1.3 Goal and Overview

The goal of this thesis is the following:

We combine the elegance and generality of abstract algebra with the descriptive power of functional modeling, to unlock novel perspectives on well-known and lesser-known concepts in the design and application of programming languages. In particular, we apply abstract algebra and functional modeling to (1) computational effects and (2) automatic differentiation.

Although the common thread throughout this work is abstract algebra, discussing the mathematical structures, their relations and their properties, and thus focussing on the denotational semantics, we also provide applications and practical programming examples. In particular, we supply prototype implementations of all concepts we discuss in this work, and provide various examples in order to make these theoretical concepts accessible to the working programmer as well.

A Functional Model for Computational Effects In a purely functional setting, many attempts have been made to model computational effects. Since the

nineties, monads [105, 106, 167] have been the standard approach. Algebraic effects & handlers [119, 121], however, have been gaining significant interest in recent years, offering a modular approach to combining different kinds of effects, based on an algebraic model. In particular, it is their clean separation between *algebraic operations* (syntax) and *handlers* (semantics) that makes them so attractive. Composing different algebraic effects in a particular order using the coproduct of their operations [146] implies a different order of handlers and consequently leads to a different interpretation [178]. Handlers only know their part of the syntax, forwarding unknown effects to other handlers.

Although this modular technique of algebraic effects & handlers is desirable for every effectful program, not all effects fit the approach. The algebraicity property, which states that effectful operations commute with sequencing, is not satisfied for all kinds of effects. We refer to these non-algebraic effects as higher-order effects, as they contain an internal computation. For example, *scoped effects & handlers* [176, 117, 173] (e.g., catching exceptions, local variables) distinguish between an operation in scope and a continuation out of scope; *latent effects & handlers* [156] (e.g., lazy evaluation, staging) defer a computation until a later point of execution; *parallel effects & handlers* [174] (e.g., for-loops) parallelize a computation to improve performance or efficiency.

The proposal of Plotkin and Power [121] to model higher-order effects as handlers, was shown to lose modularity [173, 176], the coveted property of algebraic effects & handlers. In the literature, some higher-order effects have been modeled by ad-hoc algebraic structures, which fit their purpose of modeling a specific class of effects, but are rather unsuitable to modularly compose with other effects, and are often not even proven theoretically sound.

In Part I, we leverage abstract algebra to design a *generic* framework for higher-order effects that is theoretically sound. We achieve the desired modularity by extending the free monad definition of algebraic effects & handlers to support higher-order functors, and by providing a generic fold-style interpreter. Different classes of effects each form an instance of our framework. In addition to this denotational point of view, we also take an operational look at higher-order effects & handlers, in particular at a calculus that supports algebraic- and scoped effects & handlers as built-in primitives.

We show that this framework and its implementation facilitate practical programming with higher-order effects & handlers. We aim to align with the growing adoption of algebraic effects in various programming languages [18, 60, 91, 95] and libraries [82, 84, 129] to also include higher-order effects.

A Novel Perspective on Automatic Differentiation with the Algebra of Programming Many algorithms in artificial intelligence crucially depend not only on the efficient large-scale evaluation of arithmetic expressions, but also on the evaluation of their partial derivatives at specific points [1, 11]. Symbolic and numerical differentiation can be used for these computations but suffer from performance and accuracy issues, respectively. As a consequence, there is much interest in a third approach, called automatic differentiation (AD) [172], which efficiently and accurately evaluates the derivatives of any computation composed of basic arithmetic and exponential, logarithmic, and trigonometric functions. In particular, backpropagation, a key element of machine learning, can be defined as a special instance of AD.

The orthodox accounts of automatic differentiation, since its first introduction by Wengert [172], emphasize it as a whole-program transformation. It is coupled with a shift from evaluation of the primitive operations of the underlying language to those on the so-called *dual numbers* (due originally to Clifford [26], considerably predating their subsequent application to AD). Various AD modes arise as further optimizations of this basic picture in terms of the efficient organization and traversal of the underlying data structures, typically exploiting mutability.

The vast literature on AD covers extensions of Wengert’s original approach in all possible directions: language designs and extended expressivity [37, 114], novel applications or perspectives [78, 38, 36], correctness of AD variants [86, 71, 72, 142], libraries [85], and much more. The purpose here is not to extend this body of work even further, but rather to revisit the original algorithms from an algebraic perspective. Indeed, we argue that automatic differentiation is a surprisingly simple concept from the algebraic point of view. We capture the essence in a single line, in terms of three algebraic structures: (1) semimodules, which are modules over a semiring; (2) Nagata numbers, as a generalization of dual numbers; and (3) Kronecker’s delta function.

Following Bird and de Moor’s “Algebra of Programming” [13] we show how to (re-)derive existing AD algorithms from a basic executable specification, expressed in Haskell. New AD variants are correct by construction when we use Kronecker isomorphisms to develop them. The algebra involved consists of largely standard algebraic constructions, which—because they yield isomorphic mathematical objects—have no extensional consequences. Yet, they do establish the functional equivalence between the different algorithmic variants. At the algorithmic level, the various (isomorphic) changes of representation are precisely what allows us to exploit intensional aspects, specifically to change the computational complexity.

We provide common extensions of this approach to support a more expressive

language including transcendental functions and let-sharing; square matrices (which are not commutative); and higher derivatives. Finally, we apply the approach to two case studies in the field of probabilistic programming, where we learn probabilities using gradient descent.

This novel, algebraic perspective on AD intends to enhance understanding of the popular algorithm, and in particular the reverse-mode variant, which is used more often than forward-mode, but also perceived as more complex.

Scientific Output For the scientific output of this work, we refer to the preamble of the respective parts, and to the List of Publications for a full overview of the publications produced during the time span of the PhD.

Part I

Effects & Handlers

Preamble

This part is mainly based on an updated version of the following paper, which is currently under review in the Science of Computer Programming (SCP) journal.

- BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2023). A Framework for Higher-Order Effects & Handlers. <https://ssrn.com/abstract=4383536>

Section 4.4 has elements from the following paper:

- BIRTHE VAN DEN BERG, TOM SCHRIJVERS, CASPER BACH POULSEN, AND NICOLAS WU. (2021). Latent Effects for Reusable Language Components. In *Programming Languages and Systems: 19th Asian Symposium, APLAS 2021, Chicago, IL, USA, October 17–18, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 182–201. https://doi.org/10.1007/978-3-030-89051-3_11

Section 5.4 summarizes the following paper:

- ZHIXUAN YANG, MARCO PAVIOTTI, NICOLAS WU, BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2022). Structured Handling of Scoped Effects. In: Sergey, I. (eds) *Programming Languages and Systems. ESOP 2022. Lecture Notes in Computer Science*, vol 13240. Springer, Cham. https://doi.org/10.1007/978-3-030-99336-8_17

Chapter 6 contains an overview of the following paper, which is currently under review in the Logical Methods in Computer Science (LMCS) journal.

- ROGER BOSMAN*, BIRTHE VAN DEN BERG*, WENHAO TANG*, AND TOM SCHRIJVERS. (2023). A Calculus for Scoped Effects & Handlers. <https://arxiv.org/abs/2304.09697>

Abstract Algebraic effects & handlers are a modular approach for modeling side-effects in functional programming. Their syntax is defined in terms of a signature of *effectful operations*, encoded as a functor, which are plugged into the free monad; their semantics is defined by fold-style *handlers* that only interpret their part of the syntax and forward the rest. However, not all effects are algebraic: some need to access an *internal computation*. For example,

scoped effects distinguish between a computation in scope and out of scope; parallel effects parallelize over a computation, latent effects defer a computation. Separate representations have been proposed for these *higher-order effects*, often leading to expedient and complex monad definitions. We propose a generic framework for higher-order effects, generalizing algebraic effects & handlers. In particular, we design a generic free monad with higher-order effect signatures and a corresponding interpreter. Specializing this higher-order syntax leads to various definitions of previously defined (scoped, parallel, latent) and novel (writer, bracketing) effects. Furthermore, we develop the theoretical foundations of our framework: the generic free monad is based on a free-forgetful adjunction. We compare to an adjunction that is specific for scoped effects, and its alternatives. Next to this denotational view, we also define operational semantics in the form of a calculus with algebraic and scoped effects as first-class citizens. With this generic framework in place, we aim to facilitate programming with higher-order effects, following the trend of algebraic effects and their adoption in several programming languages and popular libraries.

Overview Chapter 2 provides the necessary background on algebraic effects & handlers and their key advantages. We show how we can modularly combine different algebraic effects. We argue that not all effects can be modelled algebraically and that there is a need for modular ways to model non-algebraic effects.

Chapter 3 presents a generic framework for modeling higher-order effects by means of a free monad with higher-order effect signatures, and a fold-style interpreter. Algebraic effects & handlers are a special case of this free monad and fold. We show how to obtain the same modularity as for algebraic effects, and under what boundary conditions this is possible. We discuss forwarding of higher-order effects.

Chapter 4 instantiates our generic framework for different higher-order effects from the literature (scoped effects, parallel effects, latent effects), but also for two novel classes of effects (writer effects, bracketing effects). For each of these, we provide practical programming examples.

Chapter 5 backs our free monad definition by a categorical model that is based on a free-forgetful adjunction. We compare this approach to the categorical model for scoped effects & handlers with functorial algebras and to alternative proposals to model scoped effects: as handlers, or using indexed algebras.

Chapter 6 presents the design of a calculus for scoped effects & handlers. It differs from the denotational approach of the previous chapters in the sense that

it models operational semantics, and a type-and-effect system, with algebraic and scoped effects & handlers as built-in primitives of the language. We prove type safety, and provide a type inference algorithm.

The code accompanying this part can be found at <https://github.com/birthevdb/higher-order-effects>.

Personal Contributions The common thread throughout this part of the thesis is the theoretical framework for higher-order effects, with its different instances, which is entirely my own contribution. In the project on “Structured Handling of Scoped Effects”, I was mainly responsible for writing code and examples in Haskell and OCaml¹. In the project on the calculus for scoped effects, Roger, Wenhao and I contributed equally to the design of the calculus. In the project on latent effects, I contributed to the design and implementation.

Acknowledgements I am very grateful for the productive conversations and good collaborations with Zhixuan Yang, Marco Paviotti, Nicolas Wu, Casper Bach Poulsen, Cas van der Rest, Roger Bosman and Wenhao Tang.

¹<https://github.com/birthevdb/Structured-Handling-of-Scoped-Effects>

Chapter 2

Background

This Chapter gives an overview of the approach of algebraic effects & handlers and motivates why not all computational effects fit this approach. Algebraic effects & handlers cleanly separate the syntax of effects from their semantics: at the syntax-level we distinguish *algebraic operations*, determined by their signatures, from *computations*, a recursive structure over these operations via the free monad. Furthermore, a recursion scheme (*handler*) interprets these computations.

2.1 Algebraic Operations

Following the approach of Plotkin and Pretnar [120], we model effectful operations by their *signature* σ . Effect signatures are modeled as functors and effectful operations as instances thereof.

```
class Functor  $\sigma$  where
  fmap :: ( $a \rightarrow b$ )  $\rightarrow \sigma a \rightarrow \sigma b$ 
```

Algebraic structures, such as functors, come with laws they must obey. For functors, these are (1) preservation of identities, i.e., $\text{fmap id} = \text{id}$, and (2) preservation of composition, i.e., $\text{fmap } (f \circ g) = \text{fmap } f \circ \text{fmap } g$.

An effect signature groups several effectful operations. For example, a signature for the state effect contains two operations: *Get* for reading and returning the state, and *Put* for modifying the state, overwriting it with the given value.

data *State* *s a* = *Get* (*s* → *a*) | *Put* *s a*

In fact, getting the state *Get* (*s* → *a*) has an extra parameter (*s*), which we ignore here for the sake of brevity. Similarly, *Put* *s* (*s* → *a*) is simplified since (*s* → *a*) is isomorphic to *a*.

2.2 Computations as Free Monads

Using a functor as signature, we can construct a recursive datatype to represent *computations*, which may be either pure or effectful. The computation may then call the operations included by the signature. This recursive datatype is known as the *free monad*:

```
data Free  $\sigma$  a where
  Var :: a          → Free  $\sigma$  a
  Op  ::  $\sigma$  (Free  $\sigma$  a) → Free  $\sigma$  a
instance Functor  $\sigma \Rightarrow$  Monad (Free  $\sigma$ )
  return      = Var
  Var x  $\gg$  k = k x
  Op t  $\gg$  k = Op (fmap ( $\gg$  k) t)
```

Here, the *Var* constructor represents pure computations, whereas *Op* recursively composes computations with a branching structure that is determined by signature σ . Isomorphically (using the co-Yoneda lemma), we can present this *Op* constructor as taking two arguments: an operation and a continuation [81].

$$Op :: \forall b. \underbrace{\sigma b}_{\text{operation}} \rightarrow \underbrace{(b \rightarrow Free\ \sigma\ a)}_{\text{continuation}} \rightarrow Free\ \sigma\ a$$

For example, the monad *Free* (*State* *String*) *Int* represents computations with a state of type *String* and a result of type *Int*. Often, we define constructors to ease programming with effectful computations. For example, for *State* *s*, we define *get* and *put* as computations that read and set the state, respectively.

```
get :: Free (State s) s
get = Op (Get return)

put  :: s → Free (State s) ()
put s = Op (Put s (return ()))
```

We use the monadic bind for sequencing effectful computations. For example, *incr* increments the state value and uses a sequence of *get* and *put*.

```
incr = get >>= \s → put (s + 1)
```

2.3 Interpretation using Folds

A recursion scheme is used to interpret free monads: we write a fold-style interpreter that gives semantics to the computations in the free monad by means of a generator and an algebra. In particular, interpreting a free monad $Free\ \sigma\ a$ into semantic domain b requires a generator $gen :: a \rightarrow b$ to transform pure computations into semantic domain b , and an algebra $alg :: \sigma\ b \rightarrow b$ to interpret the signature's effectful operations.

```
foldAlg :: Functor σ => (a → b) → (σ b → b) → Free σ a → b
foldAlg gen alg (Var x) = gen x
foldAlg gen alg (Op op) = alg (fmap (foldAlg gen alg) op)
```

For example, the following handler interprets $Free\ (State\ s)\ a$ into semantic domain $s \rightarrow (s, a)$ using this structural recursion scheme.

```
hState :: Free (State s) a → (s → (a, s))
hState = foldAlg (,) alg where
  alg (Get k)    = \s → k s s
  alg (Put s' k) = \_ → k s'
```

One can apply the above state handler to a computation that increments the state and returns the original state, with an initial value of 5.

```
>>> hState (incr >> return "done") 5
("done", 6)
```

The result is `("done", 6)`, with `"done"` the resulting value and 6 the (incremented) state.

2.4 Modularly Combining Effects

Often, a combination of multiple effects is desired. Such a composition is commonly modeled by the “Datatypes à la Carte” [146] approach, using the coproduct of functors, or in this case effect signatures (denoted by $+$)¹.

¹For the sake of readability [146], we have omitted *Inl* and *Inr* in our examples.

data $(\sigma_1 + \sigma_2) a = \text{Inl } (\sigma_1 a) \mid \text{Inr } (\sigma_2 a)$

For example, we can combine stateful computations with nondeterminism. Nondeterministic operations can either fail or branch.

data $\text{Choice } a = \text{Fail} \mid \text{Or } a a \quad \text{Free } (\text{State } s + \text{Choice}) a$

One could also combine effectful operations with *unknown* effects, which are then *forwarded* to other handlers. A separator ∇ distinguishes between the algebras to interpret and forward effects, respectively. We redefine the above state handler to include “other” (unknown) effects², forwarding them to be interpreted by other handlers, and passing the state to these handlers.

$h_{\text{State}}^+ :: \text{Functor } \sigma \Rightarrow \text{Free } (\text{State } s + \sigma) a \rightarrow (s \rightarrow \text{Free } \sigma (a, s))$
 $h_{\text{State}}^+ = \text{fold}_{\text{Alg}} \text{ gen } (\text{alg } \nabla \text{ fwd}) \text{ where}$
 $\text{gen } x = \lambda s \rightarrow \text{return } (x, s)$
 $\text{alg } (\text{Get } k) = \lambda s \rightarrow k \ s \ s$
 $\text{alg } (\text{Put } s' k) = \lambda s \rightarrow k \ s'$
 $\text{fwd op} = \lambda s \rightarrow \text{Op } (\text{fmap } (\$s) \text{ op})$

Similarly, a modular handler for nondeterminism could be defined as follows:

$h_{\text{ND}}^+ :: \text{Functor } \sigma \Rightarrow \text{Free } (\text{Choice} + \sigma) a \rightarrow \text{Free } \sigma [a]$
 $h_{\text{ND}}^+ = \text{fold}_{\text{Alg}} \text{ gen } (\text{alg } \nabla \text{ Op}) \text{ where}$
 $\text{gen } x = \text{return } [x]$
 $\text{alg } \text{Fail} = \text{return } []$
 $\text{alg } (\text{Or } p q) = (++) \ (\$) \ p \ (\$) \ q$

The modular combination of different effects at the level of interpretation is modeled by function composition: applying the two handlers in the correct order. A different order gives rise to different effect interactions [50, 178]. For example, by swapping the handlers for state and nondeterminism, we achieve local state (handling state first) versus global state (handling nondeterminism first) semantics [112]. We choose an initial state value of 5.

```
>>> hND+ (hState+ (or (incr >> return "a") (return "b"))) 5
[("a", 6), ("b", 5)] -- local state
>>> hState+ (hND+ (or (incr >> return "a") (return "b"))) 5
[("a", "b"), 6] -- global state
```

²In what follows, we denote handlers that are modular in terms of other algebraic effects with a superscript ⁺.

2.5 Non-Algebraic Effects and the Algebraicity Property

Not all effects are algebraic; some denote more advanced effectful computations [156, 173, 176].

2.5.1 Algebraicity and Generic Operations

Effectful operations op are algebraic effects iff (1) they have a type signature of the form $op :: \forall a. \sigma (M a) \rightarrow M a$ for some functor σ and monad M ; and (2) they satisfy the *algebraicity property*, which expresses that the operation commutes with sequencing:

$$(\gg=k) \circ op \equiv op \circ fmap (\gg=k)$$

Clearly, these requirements are satisfied by the Op constructor. It has signature $Op :: \forall a. \sigma (Free \sigma a) \rightarrow Free \sigma a$ where $Free \sigma$ is the monad, and the algebraicity property is built into the definition of $(\gg=)$ for $Free \sigma$.

Yet, effectful operations are often not given in the above form. Instead, many are given as so-called *generic effects* [121]: they have type signatures of the form $ge :: B \rightarrow M C$ for some types B and C , and do not satisfy any further properties. For example, `put` is usually given as a generic effect in terms of some type of state S .

$$put_{ge} :: S \rightarrow M ()$$

These generic effects can be systematically transformed into equivalent algebraic effects by means of the Cayley representation [65, 16]. The Cayley representation is best-known in its form for monoids [25, 70], but for monads it corresponds to a transformation with the codensity monad [128]. The idea is to represent a computation of type $M C$ by a polymorphic function $\forall a. (C \rightarrow M a) \rightarrow M a$ that satisfies *homogeneity*, which for monads is exactly the algebraicity property. For example, we get the algebraic counterpart of put_{ge} as follows:

$$\begin{aligned} put_{op} &:: \forall a. S \rightarrow ((\rightarrow M a) \rightarrow M a) \\ put_{op} s k &= put_{ge} s \gg= k \end{aligned}$$

If we abstract over part of this type signature (taking into account that $() \rightarrow r$ is isomorphic to r), we see that put_{op} has the signature of an algebraic operation with σ instantiated to $\sigma_{put} S$, where

data $\sigma_{\text{put}} s r = \text{Put } s r$

This is one of the two cases of the *State s* functor we have shown earlier.

2.5.2 Higher-Order Effects

Thus, with the help of the Cayley representation we can turn non-algebraic operations into algebraic ones. Yet, complications arise in the case of what are called higher-order effects [7]. Consider the following operation for catching an exception:

catch :: $M a \rightarrow (\text{Maybe } a \rightarrow M b) \rightarrow M b$

The idea is that **catch** $p q$ executes p , which may raise an exception. If it does, $q \text{ Nothing}$ is executed next. Otherwise, if p yields a result r , $q (\text{Just } r)$ is executed instead. Plotkin and Power [121] observed that the algebraicity property should not hold for this operation:

catch $p q \gg= k \not\equiv \text{catch } (p \gg= k) (\lambda x \rightarrow q x \gg= k)$

Indeed, if p does not throw an exception, but k does, the right-hand side suggests that k 's exception is caught. This is not the intended behavior. Yet, we can remedy the lack of algebraicity with the Cayley representation.

$$\begin{aligned} & \text{catch} : M a \rightarrow (\text{Maybe } a \rightarrow M b) \rightarrow M b \\ \cong & \quad \{- \text{Cayley representation} -\} \\ & \text{catch} : M a \rightarrow (\text{Maybe } a \rightarrow M b) \rightarrow (b \rightarrow M c) \rightarrow M c \\ \cong & \quad \{- \text{Functor abstraction} -\} \\ & \text{catch} : \sigma_{\text{catch}} (M c) \rightarrow M c \text{ with} \\ & \quad \sigma_{\text{catch}} r = (M c, \text{Maybe } a \rightarrow M b, b \rightarrow r) \end{aligned}$$

Here a new issue emerges: the signature functor σ_{catch} has a dependency on the monad M ! This dependency characterizes *higher-order effects* and distinguishes them from plain algebraic effects. We call the references to M in σ_{catch} the *internal computations*, in contrast to the continuation which is abstracted over by the functor.

Notably, in the case of the free monad, i.e., $M = \text{Free } \sigma_{\text{catch}}$, the dependency would be mutual. This mutual dependency complicates the fold and its algebras,

and forces us to move to a higher-order setting: we cannot just specify the type of the continuation's result in the effect signature, but also require the type of the internal computations. For that reason we move from endofunctors for plain algebraic effects to higher-order endofunctors for higher-order effects (Section 3.1).

The literature contains work on different classes of higher-order effects.

For example, *scoped effects & handlers* [117, 173, 176] distinguish between a scoped computation, representing the part of the program that is in scope, and a continuation, referring to the rest of the program, out of scope. Scoped effects require a reformulation of the algebraicity property as well as a more complex free monad to represent them. Examples of scoped effects & handlers are local variables, catching exceptions, local pruning of nondeterminism, and more. Furthermore, *parallel effects & handlers* [174] parallelize computations with algebraic effects, using a `for`-loop, for example. These effects contain a computation to parallelize, as well as a continuation. Moreover, *latent effects & handlers* [156] delay parts of an effectful program to be evaluated later in the execution. Their syntax also consists of a computation to be delayed, and a continuation. Lazy evaluation strategies in the presence of effects, and staging, among others, can be modeled by this approach.

Each of these classes of effects has been denotationally modeled by a *specialized version* of the free monad, making their representations ad hoc and inherently distinct. Moreover, the monads used to model parallel and latent effects have not been shown to be a free structure and, as far as we know, are not yet put on formal footing.

In what follows, we argue that we can generalize the framework for algebraic effects and handlers to model a broader range of higher-order effects. The generic framework retains the modularity of having separate syntax (by means of a free monad) and semantics (by means of a fold). We back the framework by categorical foundations, showing that (1) it is indeed a free monad, and (2) that it works on a range of examples. We relate the different classes of effects to their representations in the literature, if they already exist.

Chapter 3

A Generic Free Monad for Modeling Effects

This Chapter introduces our generic framework for higher-order effects.

3.1 Effects and Their Interpretation

Our framework for modeling higher-order effects uses a generic free monad and interpreter. This representation deliberately does not deviate far from the algebraic effects & handlers approach (Chapter 2). In particular, we use a free monad, with higher-order functors to include the internal computation in the framework, also featuring a coproduct. This free monad with higher-order functors is equivalent to the Hefty tree representation of Bach Poulsen and van der Rest [7].

Effect Signatures Higher-order effects need access to an internal computation. In order to reflect this in the signatures, a functor representation is not sufficiently powerful. We generalize effect signatures to include a higher-order functor $k: (* \rightarrow *) \rightarrow (* \rightarrow *)$ [173, 7], which is a mapping between two functors. The functor argument represents the internal computation so that $k f$ has the familiar form of an algebraic effect signature (a plain functor). We model it accordingly [173]:

class $(\forall f. \text{Functor } f \Rightarrow \text{Functor } (k f)) \Rightarrow H\text{Functor } k$ **where**
 $\text{hmap} :: (\text{Functor } f, \text{Functor } f') \Rightarrow f \rightsquigarrow f' \rightarrow k f \rightsquigarrow k f'$

Here, $(\rightsquigarrow)$ represents a natural transformation between two functors¹. For example, a higher-order signature for exceptions is the following:

data $\text{Exc } f \text{ } r = \text{Throw} \mid \forall a. \text{Catch } (f \text{ } a) \text{ } (\text{Maybe } a \rightarrow r)$

The continuation of catching an exception $(\text{Maybe } a \rightarrow r)$ depends on whether or not an exception was thrown in the internal computation $f \text{ } a$.

Computations Similar to the free monad for algebraic effects & handlers, we can now construct a free monad that uses the above generalization of signatures.

data $\text{Free}_H k \text{ } a$ **where**
 $\text{Var}_H :: a \rightarrow \text{Free}_H k \text{ } a$
 $\text{Op}_H :: k (\text{Free}_H k) (\text{Free}_H k \text{ } a) \rightarrow \text{Free}_H k \text{ } a$

This free monad is very similar to $\text{Free } f \text{ } a$, but the functor f is now replaced by a higher-order functor k , which reflects the effects with internal computation. We show that $\text{Free}_H k$ is indeed a monad for any higher-order functor k .

instance $H\text{Functor } k \Rightarrow \text{Monad } (\text{Free}_H k)$ **where**
 $\text{return} = \text{pure}$
 $\text{Var}_H x \gg= f = f \text{ } x$
 $\text{Op}_H \text{ } op \gg= f = \text{Op}_H (\text{fmap } (\gg= f) \text{ } op)$

Notice that $(\gg=)$ now contains a generalized algebraicity property for higher-order effects:

$$(\gg= f) \circ \text{Op}_H \equiv \text{Op}_H \circ \text{fmap } (\gg= f)$$

In Chapter 5 we show that $\text{Free}_H k$ is a *free* monad. Using the co-Yoneda lemma, we rewrite the Op_H constructor: it contains an operation, an internal computation and a continuation.

$$\text{Op}_H :: \forall f \text{ } b. \underbrace{(k f b)}_{\text{operation}} \rightarrow \underbrace{(f \rightsquigarrow \text{Free}_H k)}_{\text{internal comp.}} \rightarrow \underbrace{(b \rightarrow \text{Free}_H k \text{ } a)}_{\text{continuation}} \rightarrow \text{Free}_H k \text{ } a$$

¹**type** $f \rightsquigarrow g = \forall a. f \text{ } a \rightarrow g \text{ } a$

For example, $prog_{Exc}$ throws an exception if its argument is smaller than 0.

```

progExc :: Int → FreeH Exc String
progExc x = OpH (Catch (if x ≥ 0 then return x else OpH Throw) k)
  where k Nothing = return "Too small"
        k (Just x) = return (show x)

```

Interpretation We equip this free monad with a fold-style interpreter:

```

fold :: (HFunctor k, Pointed g)
      ⇒ (a → g b) → (∀ x. k g (g x) → g x) → (FreeH k a → g b)
fold gen alg (VarH x) = gen x
fold gen alg (OpH hop) = alg (hmap fold↘ (fmap (fold gen alg) hop))
  where fold↘ :: FreeH k ↘ g
        fold↘ (VarH x) = η x
        fold↘ (OpH t) = alg (hmap fold↘ (fmap fold↘ t))

```

This fold consists of two parts: one interpreting $Free_H k a$ into semantic domain $g b$, and another interpreting the internal computation: $Free_H k \rightsquigarrow g$. Consequently, from these two folds, one would expect two generators and two algebras. However, $fold_{\rightsquigarrow}$ relies on the fact that g is a pointed functor, which has an *implicit* generator $\forall a. a \rightarrow g a$, so that a single *explicit* generator suffices.

```

class Functor g ⇒ Pointed g where
  η :: a → g a

```

Furthermore, to keep things concise, we opt to reuse the same algebra for the two folds (the domain is universally quantified) to interpret the internal computation and the continuation consistently. This significantly reduces the handler's complexity but also implies that some effects are not supported (see Chapter 7).

For example, our exception handler interprets the result in terms of *Maybe*.

$h_{Exc} :: Free_H Exc a \rightarrow Maybe a$	$>>> h_{Exc} (prog_{Exc} 5)$
$h_{Exc} = fold \ Just \ alg_{Exc}$	$Just \ "5"$
$alg_{Exc} \ Throw = Nothing$	$>>> h_{Exc} (prog_{Exc} (-5))$
$alg_{Exc} (Catch \ c \ k) = k \ c$	$Just \ "Too \ small"$

3.2 Instances of the Generic Monad

Different effects (algebraic and higher-order) can be viewed as instances of our generic framework, defining an appropriate higher-order functor that maps their signature to the generic setting. In particular, *instantiating the framework* consists of four steps:

- (1) Map the effect signatures to a higher-order representation by defining a higher-order functor K that maps a functor F and type A onto a type of kind $*$.
- (2) Show that this K is indeed a higher-order functor.
- (3) Plug it in the generic free monad $Free_H$ and show that it is isomorphic to the specialized effect definition.
- (4) Use the generic fold function to write a handler for the effects and show that it is isomorphic to the specialized effect handler (if it exists).

In what follows, we instantiate our framework using these four steps for different classes of effects.

3.3 Algebraic Effects & Handlers

We follow the steps of Section 3.2 for algebraic effects, showing their specialization of the framework isomorphic to their definition in Chapter 2.

- (1) Our mapping ignores functor argument F , since algebraic effects do not have an internal computation. σ is a functor for algebraic operations. We define K_σ^{Alg} (left) and encode it as a datatype $K^{Alg} \sigma$ in Haskell (right).

$$K_\sigma^{Alg} F A = \sigma A \qquad \text{data } K^{Alg} \sigma f a \text{ where}$$

$$\text{Op} :: \sigma a \rightarrow K^{Alg} \sigma f a$$

- (2) This definition of K_σ^{Alg} is a higher-order functor:

$$\text{instance Functor } \sigma \Rightarrow HFunctor (K^{Alg} \sigma) \text{ where}$$

$$\text{hmap } k (\text{Op } x) = \text{Op } x$$

- (3) We can show that the following isomorphism holds (Appendix A):

$$\text{Free } \sigma \ a \cong \text{Free}_H (K^{\text{Alg}} \sigma) \ a$$

(4) The generic handler for algebraic effects is defined by h_{Alg} . In Appendix A we show that $h_{\text{Alg}} \text{ gen } \text{alg}$ is isomorphic to $\text{fold}_{\text{Alg}} \text{ gen } (\text{alg} \circ \text{Op})$.

$$\begin{aligned} h_{\text{Alg}} &:: (\text{Functor } \sigma, \text{Pointed } g) \\ &\Rightarrow (a \rightarrow g \ b) \rightarrow (\forall x. K^{\text{Alg}} \sigma \ g \ (g \ x) \rightarrow g \ x) \rightarrow \text{Free}_H (K^{\text{Alg}} \sigma) \ a \rightarrow g \ b \\ h_{\text{Alg}} &= \text{fold} \end{aligned}$$

Example: State We have already introduced the state effect as a functor (Chapter 2). The handler for state is re-defined in terms of Free_H .

$$\begin{aligned} h_{\text{State}} &:: \text{Free}_H (K^{\text{Alg}} (\text{State } s)) \ a \rightarrow (s \rightarrow (a, s)) \\ h_{\text{State}} &= \text{fold } (,) \ \text{alg}_{\text{State}} \\ \text{alg}_{\text{State}} &:: K^{\text{Alg}} (\text{State } s) ((\rightarrow) \ s) (s \rightarrow a) \rightarrow s \rightarrow a \\ \text{alg}_{\text{State}} (\text{Op } (\text{Get } k)) &= \lambda s \rightarrow k \ s \ s \\ \text{alg}_{\text{State}} (\text{Op } (\text{Put } s' \ k)) &= \lambda _ \rightarrow k \ s' \end{aligned}$$

Notice the similarity between this handler and the handler in Section 2.3.

We define a smart constructor `incr` to increment the state and revisit the example of Chapter 2. Here, and in the remainder of this work, we omit the details of implementing these smart constructors.

```
>>> hState (incr >> return "done") 5
("done", 6)
```

The result is `("done", 6)`, with `"done"` the resulting value and 6 the (incremented) state.

Example: Nondeterminism A handler for nondeterminism interprets computations in terms of a list with all possible results. We re-define the handler of Section 2.3 in terms of Free_H .

$$\begin{aligned} h_{\text{ND}} &:: \text{Free}_H (K^{\text{Alg}} \text{Choice}) \ a \rightarrow [a] & \text{alg}_{\text{ND}} &:: K^{\text{Alg}} \text{Choice } [] \ [a] \rightarrow [a] \\ h_{\text{ND}} &= \text{fold return } \text{alg}_{\text{ND}} & \text{alg}_{\text{ND}} (\text{Op } \text{Fail}) &= [] \\ & & \text{alg}_{\text{ND}} (\text{Op } (\text{Or } p \ q)) &= p \ ++ \ q \end{aligned}$$

Consider the following example.

```
>>> hND (or (return 1) (or (or (return 2) (return 3)) fail))
[1, 2, 3]
```

3.4 Modular Composition

In order to make a combination of different effects in the style of “Datatypes à la Carte” [146], we require a coproduct of higher-order functors k_1 and k_2 . This coproduct² (denoted by $\oplus$, with separator $\oplus$) works in a similar way as that for functors (Section 2.4).

data $(k_1 \oplus k_2) f a = In (k_1 f a) \mid Out (k_2 f a)$

Thus, we can write a handler for algebraic effects that is modular in terms of other effects³, the latter of which are represented by a higher-order functor k .

$$\begin{aligned} h_{\text{Alg}}^{\oplus} &:: (H\text{Functor } k, \text{Functor } \sigma, \text{Pointed } g) \\ &\Rightarrow (a \rightarrow g b) \rightarrow (\forall x. (K^{\text{Alg}} \sigma \oplus k) g (g x) \rightarrow g x) \\ &\rightarrow \text{Free}_H (K^{\text{Alg}} \sigma \oplus k) a \rightarrow g b \\ h_{\text{Alg}}^{\oplus} &= \text{fold} \end{aligned}$$

The difficulty of this approach is to write an appropriate function to *forward* unknown effects. For algebraic effects, this forwarding can be written generically, either because the result type is required to have a specific format (like in languages such as Eff), or because the forwarding algebra does not depend on the specific signature or result type (like for nondeterminism). In the latter case, forwarding is just defined by *Op*.

For higher-order effects, this generic forwarding is not possible. We adopt the approach of Schrijvers et al. [132], who capture the forwarding in a type class of modular carriers: let’s assume we can write a function *fwd* as part of a type class:

class $H\text{Functor } k \Rightarrow \text{Fwd } k f$ **where**
 $\text{fwd} :: \forall a. k f (f a) \rightarrow f a$

We can now redefine the state handler h_{State} (Section 3.3) to be modular in terms of other higher-order effects. Higher-order functor k represents these

²For the sake of readability, we omit *In* and *Out* from our examples.

³We denote this with a superscript $\oplus$.

“other” effects, and we interpret the effectful program using the state monad transformer $StateT$ [52].

$$\begin{aligned}
h_{State}^{\oplus} &:: (HFunctor\ k, Fwd\ k\ (StateT\ s\ (Free_H\ k))) \\
&\Rightarrow Free_H\ (K^{Alg}\ (State\ s) \oplus k)\ a \rightarrow (s \rightarrow Free_H\ k\ (a, s)) \\
h_{State}^{\oplus} &= runStateT . fold\ return\ (alg_{State}^{\oplus} \oplus fwd) \\
alg_{State}^{\oplus} &:: K^{Alg}\ (State\ s)\ (StateT\ s\ m)\ (StateT\ s\ m\ a) \\
&\rightarrow StateT\ s\ m\ a \\
alg_{State}^{\oplus}\ (Op\ (Get\ k)) &= StateT\ (\lambda s \rightarrow runStateT\ (k\ s)\ s) \\
alg_{State}^{\oplus}\ (Op\ (Put\ s'\ k)) &= StateT\ (\lambda _ \rightarrow runStateT\ k\ s')
\end{aligned}$$

The type class constraint $Fwd\ k\ (StateT\ s\ (Free_H\ k))$ assumes we can write a forwarding function for this type. Unfortunately, we cannot do this generically; the forwarding function is also dependent on the type of higher-order functor k . For example, if k is algebraic, the forwarding looks like this:

instance $Functor\ \sigma \Rightarrow Fwd\ (K^{Alg}\ \sigma)\ (StateT\ s\ (Free_H\ (K^{Alg}\ \sigma)))$ **where**
 $fwd\ (Op\ op) = StateT\ (\lambda s \rightarrow (Op_H\ (Op\ (fmap\ ((\$s) . runStateT)\ op))))$

Similarly, we rewrite h_{ND} (Section 3.3) to be modular. We revisit the example of Section 2.4 to distinguish global state and local state semantics via the order of the handlers. We have an initial state value of 5.

```

>>> h_{ND}^{\oplus} (h_{State}^{\oplus} (or (incr >> return "a") (return "b"))) 5)
[("a", 6), ("b", 5)] -- local state
>>> h_{State}^{\oplus} (h_{ND}^{\oplus} (or (incr >> return "a") (return "b"))) 5
[("a", "b"), 6] -- global state

```

As before, we can also combine state with exceptions in two different ways: one achieves global updates, the other local updates [173]. To do so, we rewrite the handler for exceptions h_{Exc} (Section 3.1) to be modular in terms of composition with other (higher-order) effects. For that, we use the monad transformer $MaybeT$ [52] and write a forwarding clause.

$$\begin{aligned}
h_{Exc}^{\oplus} &:: (HFunctor\ k, Fwd\ k\ (MaybeT\ (Free_H\ k))) \\
&\Rightarrow Free_H\ (Exc \oplus k)\ a \rightarrow Free_H\ k\ (Maybe\ a) \\
h_{Exc}^{\oplus} &= runMaybeT . fold\ return\ (alg_{Exc}^{\oplus} \oplus fwd) \\
alg_{Exc}^{\oplus} &:: HFunctor\ k \Rightarrow Exc\ (MaybeT\ (Free_H\ k))\ (MaybeT\ (Free_H\ k)\ a) \\
&\rightarrow MaybeT\ (Free_H\ k)\ a
\end{aligned}$$

```

alg⊕Exc Throw      = MaybeT (return Nothing)
alg⊕Exc (Catch c k) = MaybeT (runMaybeT c ≫= runMaybeT . k)
instance Functor σ ⇒ Fwd (KAlg σ) (MaybeT (FreeH (KAlg σ))) where
  fwd (Op op) = MaybeT (OpH (Op (fmap runMaybeT op)))

```

The operation `decr` decrements the state if it is positive, and throws an exception otherwise.

```
decr = get ≫= λs → if s > 0 then put (s - 1) else throw
```

Depending on the order of handlers, local updates or global updates can be achieved. We have an initial state value of 2.

```

>>> h⊕Exc (h⊕State (decr ≫ catch (decr ≫ decr) (return ()))) 2)
Just ((), 1)                                     -- local updates
>>> h⊕State (h⊕Exc (decr ≫ catch (decr ≫ decr) (return ()))) 2
Just ((), 0)                                     -- global updates

```

In what follows, we will not set up handlers modularly, because of the verbosity of their forwarding functions. We rather focus on specific combinations of effectful operations to give examples of different higher-order effects.

Chapter 4

Higher-Order Effects & Handlers

This Chapter discusses different classes of higher-order effects and shows how they can all be defined as an instance of the generic framework of Chapter 3. Their definitions in terms of the framework are isomorphic to previous work, if that exists, such as for scoped effects (Section 4.1), parallel effects (Section 4.3), and latent effects (Section 4.4). Furthermore, this Chapter introduces two novel kinds of effects: writer effects (Section 4.2) and bracketing effects (Section 4.5).

4.1 Scoped Effects & Handlers

Definition Scoped effects & handlers model effects that delimit a certain scope, for example, nondeterminism with `once` and state with local variables. In order to retain the separation between syntax and semantics, and to achieve the same modularity as for algebraic effects, the literature [117, 173, 176] proposes to model scoped effects by a recursive datatype *Prog* that captures both algebraic and scoped effects. Algebraic operations are represented by a functor σ , and scoped operations by another functor γ .

```
data Prog  $\sigma$   $\gamma$  a where
  Var  :: a                                 $\rightarrow$  Prog  $\sigma$   $\gamma$  a
  Op   ::  $\sigma$  (Prog  $\sigma$   $\gamma$  a)           $\rightarrow$  Prog  $\sigma$   $\gamma$  a
  Enter ::  $\gamma$  (Prog  $\sigma$   $\gamma$  (Prog  $\sigma$   $\gamma$  a))  $\rightarrow$  Prog  $\sigma$   $\gamma$  a
```

Here, Op corresponds to algebraic effects, and $Enter$ enters a scope, representing scoped effects. $Enter$ can be rewritten (using the co-Yoneda lemma) as a program with (1) a *scoped computation* that represents the program in scope, and (2) a *continuation*, outside the scope.

$$Enter :: \forall b\ c.\ \gamma\ b \rightarrow \underbrace{(b \rightarrow Prog\ \sigma\ \gamma\ c)}_{\text{scoped computation}} \rightarrow \underbrace{(c \rightarrow Prog\ \sigma\ \gamma\ a)}_{\text{continuation}} \rightarrow Prog\ \sigma\ \gamma\ a$$

We focus on scoped effects only with $Free_{Sc}$, to later compose them again with algebraic effects, and others. $Free_{Sc}$ is isomorphic to $Prog$ without Op for algebraic effects.

data $Free_{Sc}\ \gamma\ a$ **where**

$$\begin{aligned} Var &:: a \rightarrow Free_{Sc}\ \gamma\ a \\ Enter &:: \gamma\ (Free_{Sc}\ \gamma\ (Free_{Sc}\ \gamma\ a)) \rightarrow Free_{Sc}\ \gamma\ a \end{aligned}$$

Interpretation Yang et al. [176] have proposed functorial algebras as a structured way of handling scoped effects, in contrast with the more tedious approach of Piróg et al. [117] in terms of indexed algebras. A functorial algebra consists of two parts: an *endo-algebra* (Alg_E) interprets the part of the program in scope, and a *base algebra* (Alg_B) interprets the continuation.

$$\begin{aligned} \textbf{data } Alg_E\ \gamma\ f &= Alg_E\ \{ \\ &\quad return_E :: \forall x.\ x \rightarrow f\ x, \\ &\quad enter_E :: \forall x.\ \gamma\ (f\ (f\ x)) \rightarrow f\ x \} \\ \textbf{data } Alg_B\ \gamma\ f\ a &= Alg_B\ \{ \\ &\quad enter_B :: \gamma\ (f\ a) \rightarrow a \} \end{aligned}$$

We discuss the categorical foundations of functorial algebras in Section 5.4. The structural recursion scheme $fold_{Sc}$ is specialized to interpreting $Free_{Sc}$.

$$\begin{aligned} fold_{Sc} &:: Functor\ \gamma \\ &\Rightarrow (a \rightarrow b) \rightarrow Alg_E\ \gamma\ f \rightarrow Alg_B\ \gamma\ f\ b \rightarrow Free_{Sc}\ \gamma\ a \rightarrow b \\ fold_{Sc}\ gen\ alg_E\ alg_B\ (Var\ x) &= gen\ x \\ fold_{Sc}\ gen\ alg_E\ alg_B\ (Enter\ sc) &= enter_B\ alg_B\ (fmap\ endo\ sc) \\ \textbf{where } endo &= h_{cata} \cdot fmap\ (fold_{Sc}\ gen\ alg_E\ alg_B) \\ h_{cata}\ (Var\ x) &= return_E\ alg_E\ x \\ h_{cata}\ (Enter\ sc) &= (enter_E\ alg_E \cdot fmap\ (h_{cata} \cdot fmap\ h_{cata}))\ sc \end{aligned}$$

The endo- and base algebra almost always have the same implementation. For that reason, we opt for a single algebra in what follows.

Generic Framework Scoped effects & handlers fit our generic framework.

(1) We define a mapping using a functor γ to represent scoped operations.

$$K_{\gamma}^{\text{Sc}} F A = \gamma (F A) \qquad \text{data } K^{\text{Sc}} \gamma f a \text{ where} \\ \text{Enter} :: \gamma (f a) \rightarrow K^{\text{Sc}} \gamma f a$$

(2) This mapping K_{γ}^{Sc} is a higher-order functor.

$$\text{instance Functor } \gamma \Rightarrow H\text{Functor } (K^{\text{Sc}} \gamma) \text{ where} \\ \text{hmap } k (\text{Enter } sc) = \text{Enter } (\text{fmap } k \text{ } sc)$$

(3) The following isomorphism holds (Appendix A):

$$Free_{\text{Sc}} \gamma a \cong Free_{\text{H}} (K^{\text{Sc}} \gamma) a$$

(4) We write a handler in terms of the generic recursion scheme.

$$h_{\text{Sc}} :: (\text{Functor } \gamma, \text{Pointed } g) \\ \Rightarrow (a \rightarrow g b) \rightarrow (\forall x. K^{\text{Sc}} \gamma g (g x) \rightarrow g x) \rightarrow Free_{\text{H}} (K^{\text{Sc}} \gamma) a \rightarrow g b \\ h_{\text{Sc}} = \text{fold}$$

In Appendix A we show an isomorphism between the generic handler and the handler to interpret $Free_{\text{Sc}}$.

$$\text{fold}_{\text{Sc}} \text{ gen alg alg} \cong h_{\text{Sc}} \text{ gen } (\lambda (\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg } sc) \\ \text{with } \text{return}_E \text{ alg} = \eta$$

We reconstruct the *Prog* datatype using the coproduct of the higher-order functors for algebraic and scoped effects, and define a corresponding handler.

$$Prog \sigma \gamma a \cong Free_{\text{H}} (K^{\text{Alg}} \sigma \oplus K^{\text{Sc}} \gamma) a \\ h_{\text{Prog}} :: (\text{Functor } \sigma, \text{Functor } \gamma, \text{Pointed } g) \\ \Rightarrow (a \rightarrow g b) \rightarrow (\forall x. (K^{\text{Alg}} \sigma \oplus K^{\text{Sc}} \gamma) g (g x) \rightarrow g x) \\ \rightarrow Free_{\text{H}} (K^{\text{Alg}} \sigma \oplus K^{\text{Sc}} \gamma) a \rightarrow g b \\ h_{\text{Prog}} = \text{fold}$$

Example: Nondeterminism with Once The operation *once* returns only the first result of a nondeterministic program. It only makes sense to define this

effect (captured by the functor *Once*) in combination with nondeterminism (captured by the functor *Choice*).

data *Once* *a* = *Once* *a*

As before, we interpret nondeterministic programs in terms of a list that retains all found results. The *once*-effect takes the head of that list, i.e., the first-found result.

$$\begin{aligned} h_{\text{Once}} &:: H\text{Functor } k \Rightarrow \text{Free}_H (K^{\text{Alg}} \text{Choice} \oplus K^{\text{Sc}} \text{Once}) \ a \rightarrow [a] \\ h_{\text{Once}} &= \text{fold } \eta \ (\text{alg}_{\text{ND}} \oplus \text{alg}_{\text{Once}}) \\ \text{alg}_{\text{Once}} &:: K^{\text{Sc}} \text{Once } [] \ [a] \rightarrow [a] \\ \text{alg}_{\text{Once}} \ (\text{Enter } (\text{Once } x)) &= \text{head } x \end{aligned}$$

The following example shows the difference between a program with and without scoped operation *once*, which continues the program after the first result.

```
>>> hOnce (once (or (return 1) (return 5)) >>=
      \x → or (return x) (return (x + 1)))
[1, 2]
>>> hOnce (or (return 1) (return 5)) >>=
      \x → or (return x) (return (x + 1)))
[1, 2, 5, 6]
```

Appendix B contains more examples on scoped effects, such as local state, nondeterminism with cut, depth-bounded search and concurrency.

4.2 Writer Effects & Handlers

We use the writer monad as a running example of defining *writer effects* & *handlers*. The writer monad keeps track of both return values and output messages, for example in a log file. The minimal complete definition of the writer effect, according to `MonadWriter`¹, requires three operations: *tell*, *listen* and *pass*. The former is an algebraic operation, producing output messages of type *w*:

¹<https://hackage.haskell.org/package/mtl-2.2.2/docs/Control-Monad-Writer-Class.html>

$\text{tell} :: w \rightarrow m ()$

Both `listen` and `pass` do not fit any of the previously defined effects. In what follows, we explain why and we reformulate their definitions to make them fit our generic framework.

First, `listen` executes a computation and returns a tuple of the resulting value and the output message. It is used to *inspect* what a subcomputation has written to the output.

$$\begin{aligned}
 & \text{listen} :: m\ a \rightarrow m\ (a, w) \\
 \cong & \text{listen} :: m\ a \rightarrow ((a, w) \rightarrow m\ b) \rightarrow m\ b \\
 \cong & \text{listen} :: m\ (w \rightarrow m\ b) \rightarrow m\ b \\
 \cong & \text{listen} :: m\ (\varphi_{\text{listen}}\ (m\ b)) \rightarrow m\ b \text{ with } \varphi_{\text{listen}}\ w = ((\rightarrow)\ w)
 \end{aligned}$$

Second, `pass` executes a computation, resulting in a value and a function, the latter of which is applied to the output message. It is used to *modify* what is written to the output.

$$\begin{aligned}
 & \text{pass} :: m\ (a, w \rightarrow w) \rightarrow m\ a \\
 \cong & \text{pass} :: m\ (\varphi_{\text{pass}}\ a) \rightarrow m\ a \text{ with } \varphi_{\text{pass}}\ w = ((,) (w \rightarrow w))
 \end{aligned}$$

We now abstract over these two definitions of `listen` and `pass`, defining a novel kind of effect and showing that they are a special case of our generic framework.

Definition We dub the effects of `listen` and `pass` *writer effects* and denote them by $\text{FreeWrite } \varphi\ a$, where φ is a functor for writer effect signatures.

data $\text{FreeWrite } \varphi\ a$ **where**

$$\begin{aligned}
 \text{Var} & :: a \rightarrow \text{FreeWrite } \varphi\ a \\
 \text{Exec} & :: \text{FreeWrite } \varphi\ (\varphi\ (\text{FreeWrite } \varphi\ a)) \rightarrow \text{FreeWrite } \varphi\ a
 \end{aligned}$$

Here, Var represents a pure computation and Exec is a writer computation. We can rewrite (using the co-Yoneda lemma) Exec so that it consists of a writer action, of which the result is decorated by some functor φ , and a continuation.

$$\text{Exec} :: \forall\ b. \underbrace{\text{FreeWrite } \varphi\ (\varphi\ b)}_{\text{writer computation}} \rightarrow \underbrace{(b \rightarrow \text{FreeWrite } \varphi\ a)}_{\text{continuation}} \rightarrow \text{FreeWrite } \varphi\ a$$

In particular, for `listen`, this functor φ is $((\rightarrow)\ w)$ as the result type of the writer action is $w \rightarrow m\ a$. Similarly, `pass` is decorated by the functor $((,) (w \rightarrow w))$ as its writer action has result type $(a, w \rightarrow w)$.

Generic Framework We show that the writer effect fits our generic framework.

Step 1 We choose an appropriate mapping K_φ^{Write} to express writer effects in terms of our theoretical model. Notice the similarity with scoped effects & handlers, with the order of the functors reversed.

$$K_\varphi^{\text{Write}} F A = F (\varphi A) \quad \text{data } K_\varphi^{\text{Write}} \varphi f a \text{ where} \\ \text{Exec} :: f (\varphi a) \rightarrow K_\varphi^{\text{Write}} \varphi f a$$

(2) K_φ^{Write} is a higher-order functor:

$$\text{instance Functor } \varphi \Rightarrow H\text{Functor } (K_\varphi^{\text{Write}} \varphi) \text{ where} \\ \text{hmap } k (\text{Exec } x) = \text{Exec } (k x)$$

(3) Furthermore, the following isomorphism holds (Appendix A):

$$\text{Free}_{\text{Write}} \varphi a \cong \text{Free}_H (K_\varphi^{\text{Write}} \varphi) a$$

This implies that also $\text{Free}_{\text{Write}}$ is a free monad.

(4) A handler for writer effects can now be defined in terms of our framework:

$$h_{\text{Wr}} :: (\text{Functor } \varphi, \text{Pointed } g) \\ \Rightarrow (a \rightarrow g b) \rightarrow (\forall x. K_\varphi^{\text{Write}} \varphi g (g x) \rightarrow g x) \\ \rightarrow \text{Free}_H (K_\varphi^{\text{Write}} \varphi) a \rightarrow g b \\ h_{\text{Wr}} = \text{fold}$$

Writing an isomorphism for this handler is not meaningful as no specialized handler for $\text{Free}_{\text{Write}}$ exists.

Example: Resetting the Log The writer monad uses algebraic and writer effects. Monoid w keeps track of the output messages constructed by Tell .

$$\text{data Tell } w a = \text{Tell } w a$$

Both writer operations are represented by a functor corresponding to the argument type of `listen` and `pass`, respectively.

$$\text{type Listen } w = ((\rightarrow) w) \quad \text{type Pass } w = ((,) (w \rightarrow w))$$

The handler for writing is defined in terms of our generic framework:

$$\begin{aligned}
h_{\text{Write}} &:: \text{Monoid } w \\
&\Rightarrow \text{Free}_H (K^{\text{Alg}} (\text{Tell } w) \oplus K^{\text{Write}} (\text{Listen } w) \oplus K^{\text{Write}} (\text{Pass } w)) a \\
&\rightarrow (w, a) \\
h_{\text{Write}} &= \text{fold } ((,) \epsilon) (alg_{\text{Tell}} \oplus (alg_{\text{Listen}} \oplus alg_{\text{Pass}})) \\
alg_{\text{Tell}} &:: \text{Monoid } w \Rightarrow K^{\text{Alg}} (\text{Tell } w) ((,) w) (w, a) \rightarrow (w, a) \\
alg_{\text{Tell}} (\text{Op } (\text{Tell } w k)) &= (fst k \diamond w, snd k) \\
alg_{\text{Listen}} &:: K^{\text{Write}} (\text{Listen } w) ((,) w) (w, a) \rightarrow (w, a) \\
alg_{\text{Listen}} (\text{Exec } (w, f)) &= f w \\
alg_{\text{Pass}} &:: K^{\text{Write}} (\text{Pass } w) ((,) w) (w, a) \rightarrow (w, a) \\
alg_{\text{Pass}} (\text{Exec } (w, (f, x))) &= (f w, snd x)
\end{aligned}$$

We define a function *reset* which resets the log to empty it, with ϵ the neutral element of monoid w .

$$reset = pass (\text{return } ((), \text{const } \epsilon))$$

For example, consider the following program which first logs "pre", then resets the log and then logs "post".

```
>>> hWrite (tell "post" >> reset >> tell "pre")
("post", ())
```

Alternatively, this *reset* function can be written in terms of *cancel*, a derived method in the *MonadWriter* library:

$$\text{cancel} :: (w \rightarrow w) \rightarrow m a \rightarrow m a$$

It modifies the writer using function $w \rightarrow w$, after computation $m a$ has taken place, leaving the return value untouched. *cancel* can either be defined as a scoped effect, or as a special case of *pass* (Appendix B.7).

4.3 Parallel Effects & Handlers

Definition In general, algebraic effects are executed sequentially. However, in several cases, for example for performance reasons, a parallel execution of

effects might be desired. Xie et al. [174] define parallel effects & handlers, with an operational semantics that supports both algebraic and parallel effects. In particular, they represent parallel effects by means of a `for` keyword to iterate over parallelizable computations. Their work comes with an implementation in Dex [111] and in Haskell. We define parallel effects by $Free_{Par} \rho a$, where ρ is an iterable functor to represent parallel operations².

```
data  $Free_{Par} \rho a$  where
   $Var :: a \rightarrow Free_{Par} \rho a$ 
   $For :: \rho (Free_{Par} \rho b) \rightarrow (\rho b \rightarrow Free_{Par} \rho a) \rightarrow Free_{Par} \rho a$ 
```

Here, Var is a pure computation and For a parallelizable computation. For takes two arguments: an iterable structure of computations (indicated by functor ρ), and a continuation.

Xie et al. [174] do not show that this representation of parallel effects is a free monad.

Generic Framework Parallel effects can be expressed in terms of our framework.

(1) We define a higher-order functor K_ρ^{Par} for mapping effect signatures:

$$K_\rho^{Par} F A = \int^B \rho (F B) \times (\rho B \Rightarrow A)$$

```
data  $K^{Par} \rho f a$  where
   $For :: \rho (f b) \rightarrow (\rho b \rightarrow a) \rightarrow K^{Par} \rho f a$ 
```

(2) Indeed, K_ρ^{Par} is a higher-order functor:

```
instance  $Functor \rho \Rightarrow HFunctor (K^{Par} \rho)$  where
   $hmap k (For\ iters\ c) = For\ (fmap\ k\ iters)\ c$ 
```

(3) The following isomorphism holds (Appendix A), which implies that $Free_{Par}$ is a free monad.

$$Free_{Par} \rho a \cong Free_H (K^{Par} \rho) a$$

²This differs from the implementation of Xie et al. [174] in two ways: it omits algebraic effects and uses a generic functor ρ instead of *Lists* to iterate over computations.

(4) A handler for parallel effects can now be defined generically. In Appendix A, we show that a handler in the style of Xie et al. is isomorphic to $h_{\text{Par}} \text{ gen } (\lambda(\text{For } \text{iters } k) \rightarrow h_{\text{For}} \text{ alg } \text{iters } k)$, with h_{For} defined in the same Appendix as part of the algebra for parallel effects.

$$\begin{aligned} h_{\text{Par}} &:: (\text{Functor } \rho, \text{Pointed } g) \\ &\Rightarrow (a \rightarrow g \ b) \rightarrow (\forall x. K^{\text{Par}} \ \rho \ g \ (g \ x) \rightarrow g \ x) \rightarrow \text{Free}_{\text{H}} (K^{\text{Par}} \ \rho) \ a \rightarrow g \ b \\ h_{\text{Par}} &= \text{fold} \end{aligned}$$

Example: Weak Exceptions We revisit the example of Xie et al. [174] to model weak exceptions. If multiple iterations in a for-loop are run in parallel, an exception in one iteration does not disturb the others. These weak exceptions can be modeled using a combination of three effects: exceptions *Exc* (Section 3.4), writing to an output with *Tell* (Section 4.2), and a parallel effect that uses lists.

We write this example modularly using the *WriterT* monad transformer to interpret the *tell* effect, and the *MaybeT* monad transformer for exceptions.

$$\begin{aligned} h_{\text{Tell}}^{\oplus} &:: (\text{Monoid } w, \text{Fwd } k \ (\text{WriterT } w \ (\text{Free}_{\text{H}} \ k))) \\ &\Rightarrow \text{Free}_{\text{H}} (K^{\text{Alg}} (\text{Tell } w) \oplus k) \ a \rightarrow \text{Free}_{\text{H}} \ k \ (a, w) \\ h_{\text{Tell}}^{\oplus} &= \text{runWriterT} . \text{fold } \eta \ (\text{alg}_{\text{Tell}}^{\oplus} \oplus \text{fwd}) \end{aligned}$$

$$\begin{aligned} h_{\text{For}}^{\oplus} &:: (\text{Monoid } w, \text{Fwd } k \ (\text{MaybeT } (\text{WriterT } w \ (\text{Free}_{\text{H}} \ k)))) \\ &\Rightarrow \text{Free}_{\text{H}} (K^{\text{Par}} [] \oplus k) \ (\text{Maybe } a, w) \rightarrow \text{Free}_{\text{H}} \ k \ (\text{Maybe } a, w) \\ h_{\text{For}}^{\oplus} &= \text{runWriterT} . \text{runMaybeT} . \\ &\quad \text{fold } (\text{MaybeT} . \text{WriterT} . \text{return}) \ (\text{alg}_{\text{For}}^{\oplus} \oplus \text{fwd}) \end{aligned}$$

As a result, we have three handlers ($h_{\text{Exc}}^{\oplus}$, $h_{\text{Tell}}^{\oplus}$ and $h_{\text{For}}^{\oplus}$) and forwarding clauses that forward parallel effects in combination with functors *WriterT* and *MaybeT*. For the details of this example, we refer to the implementation.

The program *weak* is the same as in Xie et al.’s work, with a different interpretation of exceptions (in terms of *Maybe* instead of *Either err*). Weak exceptions make sure that an error in an iteration of a loop does not interfere with the other loop iterations: in *weak*, an exception is thrown in the iteration with index 2. The code after the thrown exception is unreachable, but the remaining iterations are not hindered.

```

weak = do tell "start "
      for (fmap ( $\lambda i \rightarrow$  if  $i \equiv 2$ 
                then do tell "!" ; throw;
                tell "unreachable"
                else tell ( $show\ i$ )) [0..4])
      tell " end"; return 0

```

$$\begin{aligned}
& \ggg h_{\text{For}}^{\oplus} (h_{\text{Tell}}^{\oplus} (h_{\text{Exc}}^{\oplus} \text{weak})) \\
& (\text{Nothing}, \text{"start 01!34"})
\end{aligned}$$

Appendix B contains another example on parallel effects, namely the *amb* effect [174].

4.4 Latent Effects & Handlers

Definition In van den Berg et al. [156, 157], we introduce *latent effects & handlers* as a class of effects that have a control-flow in which some computations are deferred for evaluation at a later point in the program interpretation. Examples of latent effects are function abstractions with effectful bodies, call-by-name or call-by-need evaluation strategies [46, 93] and multi-staging annotations for (dynamically staging) and unstaging code [148, 140, 130, 135, 170, 171]. It is theoretically possible [44], but not modular, to define control-flow effects by making this control flow explicit, e.g., by writing interpreters in continuation-passing style.

Intuitively, latent effects & handlers adopt the same ideas as algebraic effects & handlers, but instead of replacing bit by bit the structure of the effect tree by their meaning, latent effects & handlers grow little “pockets of meaning” around the individual nodes remaining in the effect tree, and not just around the root. The latter supports deferred effects because later handlers can still re-arrange the semantic pockets created by earlier handlers.

Latent effects are represented by a signature $\zeta :: * \rightarrow (* \rightarrow *) \rightarrow *$. For example, a signature for memoizing a computation’s result, where *Think* defers the evaluation, and *Force* forces it, is the following:

```

data Thinking v :: *  $\rightarrow$  (*  $\rightarrow$  *)  $\rightarrow$  * where
  Force :: Ptr  $\rightarrow$  Thinking v v   NoSub
  Think ::      Thinking v Ptr (OneSub v)

```

Besides its first parameter v , the type *Thinking v p c* is indexed by two further parameters. Parameter p is the return type of the primitive operations; this is the type they pass to their continuation. *Force* returns a value of type v .

Thunk returns a pointer (**type** $Ptr = Int$) to the environment that keeps track of thunks and memoized values. Parameter $c :: * \rightarrow *$ captures the number and result type of the subcomputations. As *Force* has no subcomputations, they use type *NoSub* to indicate that. *Thunk* has one subcomputation *OneSub* v , whose return type is v . Hence, *OneSub* v has one constructor $One :: OneSub\ v\ v$.

data *NoSub* $:: * \rightarrow *$ **where**

data *OneSub* $v :: * \rightarrow *$
where $One :: OneSub\ v\ v$

Latent effect computations are represented by $Free_{Lat}\ \zeta\ \ell\ a$ where ζ is the effect signature, and ℓ is the functor decorating the result type³.

data $Free_{Lat}\ \zeta\ \ell\ a$ **where**

$Leaf :: a \rightarrow Free_{Lat}\ \zeta\ \ell\ a$

$Node :: \zeta\ p\ c \rightarrow \ell\ () \rightarrow (\forall x. c\ x \rightarrow \ell\ () \rightarrow Free_{Lat}\ \zeta\ \ell\ (\ell\ x))$
 $\rightarrow (\ell\ p \rightarrow Free_{Lat}\ \zeta\ \ell\ a) \rightarrow Free_{Lat}\ \zeta\ \ell\ a$

Here, *Leaf* is a pure computation, and *Node* is an internal node that contains (1) an operation $\zeta\ p\ c$ with result type p and a number of subcomputations c ; (2) the effect state $\ell\ ()$ of the latent effects in the node; (3) a function $(\forall x. c\ x \rightarrow \ell\ () \rightarrow Free_{Lat}\ \zeta\ \ell\ (\ell\ x))$ to interpret a subcomputation c with result type x , given the current effect state; and (4) a continuation $(\ell\ p \rightarrow Free_{Lat}\ \zeta\ \ell\ a)$ to interpret the remainder of the program.

We can create modular constructors for latent effects in the style of “Datatypes à la Carte” [146], using a coproduct ($\pm$) and an empty signature *Ending* as the neutral element:

data $(\zeta_1 \pm \zeta_2)\ p\ c = Inl'\ (\zeta_1\ p\ c) \mid Inr'\ (\zeta_2\ p\ c)$

data *Ending* $:: * \rightarrow (* \rightarrow *) \rightarrow *$

This way, we can compose handlers using function composition. In Appendix C we use this coproduct to construct a library of latent effects.

In van den Berg et al. [156] we do not show that $Free_{Lat}\ \zeta\ \ell\ a$ is a free monad, but we do indicate that $Free_{Lat}\ \zeta\ \ell\ a$ has a similar notion of **return** and $(\gg=)$ as the free monad for algebraic effects.

³This datatype is equivalent to the *Tree* datatype of van den Berg et al. [156]. The code implementing this *Tree* datatype can be found at <https://github.com/birthedvb/Latent-Effect-and-Handlers>.

Generic Framework Latent effects are an instance of our generic free monad.

(1) We choose a mapping⁴ $K_{\zeta, \ell}^{\text{Lat}}$ to represent latent effects & handlers with effect functor ζ and *latent* effect functor ℓ . It contains a **Node** constructor to represent internal nodes, similar to the *Node* constructor of $Free_{\text{Lat}}$.

$$K_{\zeta, \ell}^{\text{Lat}} F A = \int^{P, C} \zeta P C \times \ell 1 \times (\int_X C X \times \ell 1 \Rightarrow F (\ell X)) \times (\ell P \Rightarrow A)$$

data $K^{\text{Lat}} \zeta \ell f a$ **where**

$$\begin{aligned} \text{Node} &:: \zeta p c \rightarrow \ell () \rightarrow (\forall x. c x \rightarrow \ell () \rightarrow f (\ell x)) \\ &\rightarrow (\ell p \rightarrow a) \rightarrow K^{\text{Lat}} \zeta \ell f a \end{aligned}$$

(2) It has a corresponding higher-order functor instance.

instance $H\text{Functor } (K^{\text{Lat}} \zeta \ell)$ **where**

$$\text{hmap } k (\text{Node } \text{sub } \ell \text{ st } c) = \text{Node } \text{sub } \ell (\text{fmap } k . \text{st}) c$$

(3) From this, the following isomorphism holds (Appendix A):

$$Free_{\text{Lat}} \zeta \ell a \cong Free_{\text{H}} (K^{\text{Lat}} \zeta \ell) a$$

This implies that also $Free_{\text{Lat}}$ is a free monad.

(4) We can now write a generic handler for latent effects.

$$\begin{aligned} h_{\text{Lat}} &:: \text{Pointed } g \\ &\Rightarrow (a \rightarrow g b) \rightarrow (\forall x. K^{\text{Lat}} \zeta \ell g (g x) \rightarrow g x) \\ &\rightarrow Free_{\text{H}} (K^{\text{Lat}} \zeta \ell) a \rightarrow g b \\ h_{\text{Lat}} &= \text{fold} \end{aligned}$$

Writing an isomorphism for this handler is not meaningful because, as far as we know, no specialized handler for $Free_{\text{Lat}}$ exists in the literature.

Example: Lazy Evaluation with Memoization We implement call-by-need with memoization and call-by-value. Consider the following program:

$$\text{prog}_{\text{Lazy}} = \text{app } (\text{abs } (\text{return } 3)) (\text{do put } 42; \text{return } 5)$$

⁴Using ends and coends, corresponding to Haskell's universal and existential quantification, respectively.

It applies a constant function to an argument that changes the state of the program. With a call-by-need evaluation strategy, we expect the state to remain unchanged, as the argument is never needed and thus never executed. With an eager strategy, we expect the state of the program to be 42.

We require algebraic, scoped and latent effects to model this example: state *State*, the reader effects *Ask* and *Local* (Appendix B.2), and the above *Thinking* for memoizing results.

For brevity, we omit forwarding and define an expression datatype that contains the above algebraic, scoped and latent effects.

$$\begin{aligned} \text{type Expr } v \ a = & \text{Free}_H \ (K^{\text{Alg}} (\text{State } v) \oplus K^{\text{Alg}} (\text{Ask } [v]) \\ & \oplus K^{\text{Sc}} (\text{Local } [v]) \oplus K^{\text{Lat}} (\text{Thinking } v) \text{Id}) \ a \end{aligned}$$

A handler for lazy evaluation has three environments: (1) the *state* s of type v ; (2) the *evaluation environment* nv of type $[v]$ to know which variables are in scope; and (3) the *environment of thunks and memoized values* th of type $[\text{Thunk } v]$. A thunk is an unevaluated operation.

$$\begin{aligned} \text{type Thunk } v = & \text{Either } (\text{Id } () \rightarrow v \rightarrow [v] \rightarrow [\text{Thunk } v] \\ & \rightarrow \text{State}_\ell (v, [\text{Thunk } v]) \text{Id } v) \ v \end{aligned}$$

The semantic domain of our handler is $\text{State}_\ell \ s \ l \ a$, where, in this case, state s consists of a tuple $(v, [\text{Thunk } v])$ and l is the identity functor⁵.

$$\text{newtype State}_\ell \ s \ l \ a = \text{State}_\ell \ \{ \text{unStateL} :: (s, l \ a) \}$$

With this in place, we can define h_{Lazy} , which lazily interprets a stateful program. For the algebras of state and reader, we refer to Section 3.4 and Appendix B.2.

$$\begin{aligned} h_{\text{Lazy}} :: & \text{Expr } v \ a \rightarrow v \rightarrow [v] \rightarrow [\text{Thunk } v] \rightarrow \text{State}_\ell (v, [\text{Thunk } v]) \text{Id } a \\ h_{\text{Lazy}} \text{ prog } s \ nv \ th = & \text{fold gen } (\text{algState} \oplus \text{algRead} \oplus \text{algLazy}) \ \text{prog} \\ \text{gen } x = & \text{State}_\ell ((s, th), \text{Id } x) \\ \text{alg}_{\text{Lazy}} (\text{Node Thunk } l \ st \ k) = & \lambda s \ nv \ th \rightarrow \\ & k (\text{fmap } (\text{const } (\text{length } th)) \ l) \ s \ nv \ (th \ ++ \ [\text{Left } (st \ \text{One})]) \\ \text{alg}_{\text{Lazy}} (\text{Node } (\text{Force } p) \ l \ st \ k) = & \lambda s \ nv \ th \rightarrow \text{case } (th \ !! \ p) \ \text{of} \\ & \text{Left } t \rightarrow \text{let } \text{State}_\ell ((s', th'), \text{Id } lv) = (t \ l) \ s \ nv \ th \end{aligned}$$

⁵We use a compact datatype **data** $\text{Id } a = \text{Id } \{ \text{unId} :: a \}$ for the identity functor.

$$\begin{aligned} & \text{in } \text{unId } (\text{fmap } (\lambda v \rightarrow (k \text{ } lv) \text{ } s' \text{ } nv \text{ } (\text{replace } p \text{ } (\text{Right } v) \text{ } th')) \text{ } lv) \\ & \text{Right } v \rightarrow k \text{ } (\text{fmap } (\text{const } v) \text{ } l) \text{ } s \text{ } nv \text{ } th \end{aligned}$$

When thinking an operation, we call the continuation and add the current operation to the list of thunks. When forcing an operation, we get the thunk or value from the environment *th*. In case of a thunk we evaluate it, replace it with its value in the list of thunks in order to memoize it, and call the continuation. In case of a value, we immediately call the continuation with this value.

An eager handler swaps the behavior of *Thunk* and *Force*:

$$\begin{aligned} h_{\text{Eager}} &:: \text{Expr } v \text{ } a \rightarrow v \rightarrow [v] \rightarrow [\text{Thunk } v] \rightarrow \text{State}_\ell (v, [\text{Thunk } v]) \text{ } \text{Id } a \\ h_{\text{Eager}} \text{ prog } s \text{ } nv \text{ } th &= \text{fold gen } (\text{algState} \oplus \text{algRead} \oplus \text{algEager}) \\ \text{algEager } (\text{Node } \text{Thunk } l \text{ } st \text{ } k) &= \lambda s \text{ } nv \text{ } th \rightarrow \\ & \quad \text{let } \text{State}_\ell ((s', th'), lw) = (st \text{ } \text{One } l) \text{ } s \text{ } nv \text{ } th \text{ in} \\ & \quad \text{unId } (\text{fmap } (\lambda v \rightarrow k \text{ } (\text{fmap } (\text{const } (\text{length } th')) \text{ } l) \\ & \quad \quad \quad s' \text{ } nv \text{ } (th' \text{ } ++ [\text{Right } v])) \text{ } (\text{unId } lw)) \\ \text{algEager } (\text{Node } (\text{Force } p) \text{ } l \text{ } st \text{ } k) &= \lambda s \text{ } nv \text{ } th \rightarrow \text{case } (th \text{ } !! \text{ } p) \text{ of} \\ & \quad \text{Right } v \rightarrow k \text{ } (\text{fmap } (\text{const } v) \text{ } l) \text{ } s \text{ } nv \text{ } th \end{aligned}$$

We now revisit our example program *prog_{Lazy}*. Values can be either integers or function abstractions:

data *Value* = *Val Int* | *Fun (Expr Value Value)*

We define variables, function abstraction and function application as follows. We thunk the argument of function application and evaluate it.

```
var :: Ptr → Expr Value Value
var x = do nv ← ask; local ([nv !! x]) (force 0)
abs :: Expr Value Value → Expr Value Value
abs body = return (Fun body)
app :: Expr Value Value → Expr Value Value → Expr Value Value
app e1 e2 = do vf ← e1; nv ← ask; p ← thunk e2
               case vf of Fun body → local ([nv !! p]) body
```

We now lazily and eagerly evaluate the above program *prog_{Lazy}* with initial state 0 and empty environments.

```
>>> hLazy progLazy (Val 0) [] []
(0, [Left "thunk"], 3)
>>> hEager progLazy (Val 0) [] []
(42, [Right 5], 3)
```


The first element of the triple is the current state, the second element is the list of thunks or memoized values, and the third element is the program result. Appendix B.8 proposes an implementation for variables, function abstraction and function application inspired by Levy's call-by-push-value [93].

4.5 Bracketing Effects & Handlers

Definition Consider the situation where a (built-in) exception is raised while a resource (such as a file) was being used. It is not always straightforward to determine what has to happen in this scenario, but in each case we want to avoid that our resource is unreachable by code. The function `bracket` [99] acquires a resource, uses it and then safely releases it again. Even if an exception occurs, `bracket` correctly releases the resource and re-raises the exception. This is reflected in its type: $m\ r$ acquires the resource r , the function $r \rightarrow m\ b$ releases the resource, the function $r \rightarrow m\ a$ uses the resource, and $m\ a$ is the result.

$$\begin{aligned} \text{bracket} &:: m\ r \rightarrow (r \rightarrow m\ b) \rightarrow (r \rightarrow m\ a) \rightarrow m\ a \\ &\cong \text{bracket} :: m\ r \rightarrow (r \rightarrow (m\ b, m\ a)) \rightarrow m\ a \\ &\cong \text{bracket} :: m\ (m\ b, m\ a) \rightarrow m\ a \end{aligned}$$

Typically, b is the unit type⁶ `()` and the monad m is the *IO*-monad. For example, consider the program `firstTwo` that opens a file `"foo.txt"`, reads it and prints its first two characters. In case an exception occurs, the file is released (indicated by printing `"released"`).

```
firstTwo = bracket (openFile "foo.txt" ReadMode)
                  (\h → do hClose h; print "released")
                  (\h → do x ← hGetChar h; y ← hGetChar h; print (x, y))
```

The following prints the contents of `"foo.txt"` and then executes `firstTwo`. There are two possible situations.

First, if `"foo.txt"` does contain at least two characters, these first two characters are printed and the file is released.

```
>>> readFile "foo.txt" >> print >> firstTwo
"HELLO, WORLD!"
```

⁶In what follows, we replace this type b by `()`.

```
( 'H', 'E' )
"released"
```

Second, if "foo.txt" does not contain enough characters, an exception occurs and the file is released. Notice that the exception is re-raised after release.

```
>>> readFile "foo.txt" >>= print >> firstTwo
"H"
"released"
"***Exception: foo.txt hGetChar end of file."
```

bracket does not fit any of the previous higher-order effects. For that reason, we define $Free_{Res} a$ to represent the *bracketing effect & handler*.

```
data FreeRes a where
  Var    :: a                      → FreeRes a
  Bracket :: FreeRes (FreeRes ()), FreeRes a → FreeRes a
```

Here, Var represents a pure computation and $Bracket$ has the same structure as `bracket`, rewritten above.

Generic Framework We show that the bracketing effect fits our framework.

Step 1 We choose an appropriate mapping K^{Res} to express bracketing.

```

 $K^{Res} F A = F (F (), F A)$ 
data KRes f a where
  Bracket :: f (f ()), f a → KRes f a
```

(2) K^{Res} is a higher-order functor:

```
instance HFunctor KRes where
  hmap k (Bracket res) = Bracket (k (fmap (λ(r, u) → (k r, k u)) res))
```

(3) Furthermore, the following isomorphism holds (A):

$$Free_{Res} a \cong Free_H K^{Res} a$$

(4) A handler for the bracketing effect can be defined in terms of our framework:

$$\begin{aligned}
h_{\text{Res}} &:: (\text{Pointed } g) \\
&\Rightarrow (a \rightarrow g \, b) \rightarrow (\forall x. K^{\text{Res}} \, g \, (g \, x) \rightarrow g \, x) \rightarrow \text{Free}_{\text{H}} \, K^{\text{Res}} \, a \rightarrow g \, b \\
h_{\text{Res}} &= \text{fold}
\end{aligned}$$

Writing an isomorphism for this handler is not meaningful as no specialized handler for Free_{Res} exists.

Example: Print First Two Characters of File We revisit the above example that prints the first two characters of a file, if possible. To model this example with our framework, we require algebraic effects to open (OpenFile), close (HClose) and read a file (ReadFile), to read a character from a file (HGetChar), and to print it to the standard output (Print). Furthermore, we require the bracketing effect to encode the bracketing behavior for safely releasing resources.

```

data Teletype a = HGetChar Handle (Char → a)
                | Print      String  a
                | ReadFile   FilePath (String → a)
                | OpenFile   FilePath IOMode (Handle → a)
                | HClose     Handle  a

```

The handler for bracketing is defined in terms of our generic framework:

```

hBracket :: FreeH (KAlg Teletype ⊕ KRes) a → IO a
hBracket = fold return (algTele ⊕ algRes)

algTele :: KAlg Teletype IO (IO a) → IO a
algTele (Op (HGetChar h f)) = hGetChar h      >>= f
algTele (Op (Print s io))   = print s         >> io
algTele (Op (ReadFile fp f)) = readFile fp    >>= f
algTele (Op (OpenFile fp mode f)) = openFile fp mode >>= f
algTele (Op (HClose h k))    = hClose h       >> k

algRes :: KRes IO (IO a) → IO a
algRes (Bracket res) = do (r, u) ← res;
                          bracket (return ()) (const r) (const (join u))

```

We define constructors `hGetC`, `prnt`, `brckt`, `readF`, `openF` and `hCls` to revisit the example.

```

firstTwo' = brckt (openF "foo.txt" ReadMode)
               (λh → do hCls h; prnt "released")
               (λh → do x ← hGetC h; y ← hGetC h; prnt (show (x, y)))

```

```
>>> hBracket (readF "foo.txt" >=> prnt >> firstTwo')  
"H"  
"released"  
"***Exception: foo.txt hGetChar end of file"
```

Chapter 5

Categorical Foundations

In what follows, we present the categorical foundations of our framework, starting with some basic definitions in Section 5.1 and Section 5.2. In Section 5.3, we present a generalized adjunction to back our higher-order framework and in Section 5.4 we compare with the established categorical work on scoped effects.

5.1 Categories and Functors

First, we go over some basic definitions, concepts and terminology.

Definition 1 (Category). *A category $\mathbb{C}$ is defined in terms of four parts:*

- *A collection of objects A, B .*
- *For every two objects in $\mathbb{C}$, a set of morphisms between those two objects. This is often called the hom-set, and we denote it by $\mathbb{C}(A, B)$.*
- *For every object an identity morphism: $\forall A \in \mathbb{C} : id_A \in \mathbb{C}(A, A)$*
- *For two morphisms f and g their composition: $\forall f \in \mathbb{C}(A, B), g \in \mathbb{C}(B, C) : f \circ g \in \mathbb{C}(A, C)$*

Furthermore, composition should be associative:

$$\forall A, B, C, D \in \mathbb{C} : \forall f \in \mathbb{C}(A, B), g \in \mathbb{C}(B, C), h \in \mathbb{C}(C, D) : \\ (f \circ g) \circ h \equiv f \circ (g \circ h)$$

and have a left and right unit:

$$\forall A, B \in \mathbb{C} : \forall f \in \mathbb{C}(A, B) : id_B \circ f \equiv f \equiv f \circ id_A$$

□

Definition 2 (Homomorphism). A homomorphism is a morphism that preserves the algebraic structure of its objects. □

Definition 3 (Initial Object). The initial object of a category $\mathbb{C}$ is an object in $\mathbb{C}$ such that for all objects A in $\mathbb{C}$ there exists a unique morphism from that initial object to A . □

Definition 4 (Functor). A functor $F : \mathbb{C} \rightarrow \mathbb{D}$ is a structure-preserving mapping between two categories $\mathbb{C}$ and $\mathbb{D}$. It maps both the objects and the hom-set of category $\mathbb{C}$ into category $\mathbb{D}$, respecting identities and composition: $F id_A = id_{F A}$ and $F(f \circ g) = F f \circ F g$. □

Definition 5 (Natural Transformation). A natural transformation $\alpha : F \rightsquigarrow G$ is a mapping between two functors $F : \mathbb{C} \rightarrow \mathbb{D}$ and $G : \mathbb{C} \rightarrow \mathbb{D}$ such that for any morphism $f : A \rightarrow B$ the following holds:

$$G f \circ \alpha = \alpha \circ F f$$

Definition 6 (Endofunctor). An endofunctor $F : \mathbb{C} \rightarrow \mathbb{C}$ is a functor from a category to itself. The category of endofunctors on $\mathbb{C}$ is denoted by $\mathbb{C}^{\mathbb{C}}$. □

Definition 7 (Pointed Endofunctor). An endofunctor $P : \mathbb{C} \rightarrow \mathbb{C}$ is pointed if it is equipped with a designated natural transformation from the identity functor $\eta^P : Id \rightarrow P$. $\mathbb{P}$ is the category of pointed endofunctors. □

5.2 Adjunctions

Definition 8 (Adjunction). An adjunction $L \dashv R$ is a relation between two functors $L : \mathbb{D} \rightarrow \mathbb{C}$ and $R : \mathbb{C} \rightarrow \mathbb{D}$, with $\mathbb{C}$ and $\mathbb{D}$ categories, so that there exists a natural isomorphism between their hom-sets:

$$\psi : \mathbb{C}(L A, B) \cong \mathbb{D}(A, R B) : \psi^{-1}$$

ψ and ψ^{-1} are two natural transformations witnessing the isomorphism, which should satisfy the requisite roundtrip properties ($\psi \circ \psi^{-1} = \text{id} = \psi^{-1} \circ \psi$). $\square$

An adjunction gives rise to a monad $R \circ L$ in $\mathbb{D}$, that is equipped with a *unit* operator η and a sequencing operator *join*. Here, L and R are called the *left adjoint* and *right adjoint*, respectively.

$$\begin{array}{ccc} \mathbb{C} & \begin{array}{c} \xleftarrow{L} \\ \perp \\ \xrightarrow{R} \end{array} & \mathbb{D} \end{array} \quad \begin{array}{l} \eta : A \rightarrow R(L A) \\ \text{join} : (R \circ L \circ R \circ L) A \rightarrow R(L A) \end{array}$$

Adjunctions can be combined: we can compose $L_1 \dashv R_1$ and $L_2 \dashv R_2$, composing their functors such that $L_2 \circ L_1 \dashv R_1 \circ R_2$.

$$\mathbb{C}_1 \begin{array}{c} \xleftarrow{L_2} \\ \perp \\ \xrightarrow{R_2} \end{array} \mathbb{C}_2 \begin{array}{c} \xleftarrow{L_1} \\ \perp \\ \xrightarrow{R_1} \end{array} \mathbb{C}_3 \cong \mathbb{C}_1 \begin{array}{c} \xleftarrow{L_2 \circ L_1} \\ \perp \\ \xrightarrow{R_1 \circ R_2} \end{array} \mathbb{C}_3$$

Furthermore, we can define the product of adjunctions $L_1 \dashv R_1$ and $L_2 \dashv R_2$, acting between their product categories and taking the product of the adjoints.

$$\mathbb{C}_1 \begin{array}{c} \xleftarrow{L_1} \\ \perp \\ \xrightarrow{R_1} \end{array} \mathbb{C}_2 \text{ and } \mathbb{C}_3 \begin{array}{c} \xleftarrow{L_2} \\ \perp \\ \xrightarrow{R_2} \end{array} \mathbb{C}_4 \text{ then } \mathbb{C}_1 \times \mathbb{C}_3 \begin{array}{c} \xleftarrow{L_1 \times L_2} \\ \perp \\ \xrightarrow{R_1 \times R_2} \end{array} \mathbb{C}_2 \times \mathbb{C}_4$$

Definition 9 (Free-forgetful Adjunction). Consider an endofunctor $\Sigma : \mathbb{C} \rightarrow \mathbb{C}$ in category $\mathbb{C}$. The category on the left is the category of Σ -algebras over $\mathbb{C}$, with as objects pairs $\langle A, \text{alg} \rangle$, where A is called the carrier of the algebra and $\text{alg} : \Sigma A \rightarrow A$ is the action. Homomorphisms $f : A \rightarrow B$ in $\Sigma\text{-Alg}(\mathbb{C})$ map the carriers.

$$\begin{array}{ccc} \Sigma A & \xrightarrow{\Sigma f} & \Sigma B \\ \text{alg} A \downarrow & & \downarrow \text{alg} B \\ A & \xrightarrow{f} & B \end{array}$$

A free-forgetful adjunction consists of a free functor *Free*, left adjoint of a forgetful functor *U*, which forgets the action of the Σ -algebra.

$$\Sigma\text{-Alg}(\mathbb{C}) \begin{array}{c} \xleftarrow{\text{Free}} \\ \perp \\ \xrightarrow{U} \end{array} \mathbb{C}$$

□

Now, category $\Sigma\text{-Alg}(\mathbb{C})$ also has an initial object. Following Lambek's lemma, the action of the initial object is an isomorphism ($\Sigma i \cong i$ with i the initial object). This means that the initial object is the least fixed point of Σ , denoted by Σ^* , where the Kleene star denotes the iterated composition of Σ . We consider the algebra for which $\Sigma^* A$ is the carrier; it has an action $Op: \Sigma(\Sigma^* A) \rightarrow \Sigma^* A$. Furthermore, the definition of initial objects states that there must be a unique homomorphism from the initial object to any other Σ -algebra. We call this unique homomorphism a *fold* and describe the uniqueness by the following two diagrams, which must commute.

$$\begin{array}{ccc} A & \xrightarrow{\text{Var}} & \Sigma^* A \\ \text{gen} \downarrow & \swarrow \text{fold} & \\ B & & \end{array} \qquad \begin{array}{ccc} \Sigma(\Sigma^* A) & \xrightarrow{Op} & \Sigma^* A \\ \Sigma \text{ fold} \downarrow & & \downarrow \text{fold} \\ \Sigma B & \xrightarrow{\text{alg}} & B \end{array}$$

This uniqueness is called the *universal property* of fold [73]. This fold is determined in terms of the generator $gen: A \rightarrow B$, which is a morphism in the category of Σ -algebras, and the action of the target Σ -algebra. The free-forgetful adjunction gives rise to the free monad in $\mathbb{C}$, generated by Σ ; this free monad is often denoted by Σ^* . An alternative notation for a fold with generator gen and algebra alg is $(\downarrow gen, alg)$. Notice the similarity between this adjunction and the free monad definition of Section 2.2, where we denote Σ^* as $Free \sigma$, we use Var as the return of the monad, Op as the action of the initial object, and $\text{fold}_{Alg} gen alg$ as the structural recursion scheme to interpret $\Sigma^* A$ into B , with the implementation matching the above commuting diagrams.

The model of *algebraic effects & handlers* revolves around the free monad definition (effects) and its interpretation in terms of a fold-style recursion scheme (handlers). Operations are captured by an endofunctor $\Sigma: \mathbb{C} \rightarrow \mathbb{C}$ on the base category $\mathbb{C}$. This endofunctor is exactly the functor we have described in the above free-forgetful adjunction. Furthermore, it represents the effect signature and determines the shapes of programs that call Σ -operations, and the shapes of handlers that interpret them.

5.3 A Generalized Adjunction

We translate the perspective of free-forgetful adjunctions to construct a free monad to a more generic setting with higher-order functors. Thus, our operation signatures are no longer endofunctors in $\mathbb{C}$ (as for algebraic effects), but rather higher-order endofunctors. For that reason, it is necessary to make an intermediate step from $\mathbb{C}$ to a custom product category before constructing the free-forgetful adjunction.

Summarized, to get a free monad on base category $\mathbb{C}$, we compose two adjunctions, which this Section discusses in more detail.

$$\begin{array}{ccccc}
 & \xleftarrow{\overline{Free}} & & \xleftarrow{Id \rtimes -} & \\
 H-Alg(\mathbb{P} \rtimes \mathbb{C}) & \xleftarrow{\perp} & \mathbb{P} \rtimes \mathbb{C} & \xleftarrow{\perp} & \mathbb{C} \\
 & \xrightarrow{\overline{U}} & & \xrightarrow{App} &
 \end{array}$$

In what follows we explain the two adjunctions with their corresponding categories and functors before putting them together.

5.3.1 The $Id \rtimes - \dashv App$ Adjunction

We first focus on the rightmost adjunction in the overview diagram.

$$\begin{array}{ccc}
 & \xleftarrow{Id \rtimes -} & \\
 \mathbb{P} \rtimes \mathbb{C} & \xleftarrow{\perp} & \mathbb{C} \\
 & \xrightarrow{App} &
 \end{array}$$

It connects the base category $\mathbb{C}$ with the category $\mathbb{P} \rtimes \mathbb{C}$, the latter of which is defined as a variation on the product of the category $\mathbb{P}$ of pointed endofunctors on $\mathbb{C}$ and the base category $\mathbb{C}$. This category $\mathbb{P} \rtimes \mathbb{C}$ represents the higher-order signatures of effect operations, consisting of an internal computation (in $\mathbb{P}$) and a continuation (in $\mathbb{C}$). The following table summarizes the objects and morphisms of the different categories.

Symbol	Objects	Morphisms
$\mathbb{C}$	A, B	$f : A \rightarrow B$
$\mathbb{P}$	F, G	$\alpha : F \rightarrow G$
$\mathbb{P} \times \mathbb{C}$	$\langle F, A \rangle, \langle G, B \rangle$	$\langle \alpha : F \rightarrow G, f : A \rightarrow B \rangle$
$\mathbb{P} \rtimes \mathbb{C}$	$\langle F, A \rangle, \langle G, B \rangle$	$\langle \alpha : F \rightarrow G, f : F A \rightarrow G B \rangle$

Objects in $\mathbb{P} \rtimes \mathbb{C}$ have the form $\langle F, A \rangle$, where F is a pointed endofunctor on $\mathbb{C}$ and A is an object of $\mathbb{C}$. Morphisms are represented by pairs $\langle \alpha, f \rangle$ of a structure-preserving natural transformation $\alpha : F \rightarrow G$ ($\alpha \circ \eta^F = \eta^G$), and a morphism $f : F A \rightarrow G A$. Composition and identity morphisms are defined componentwise:

$$\langle \beta, g \rangle \circ \langle \alpha, f \rangle = \langle \beta \circ \alpha, g \circ f \rangle \quad id_{\langle F, A \rangle} = \langle id_F, id_{F A} \rangle$$

The left adjoint, functor $Id \rtimes -$, maps objects and morphisms from $\mathbb{C}$ to $\mathbb{P} \rtimes \mathbb{C}$. Furthermore, Id itself is a pointed endofunctor with $\eta^{Id} = id$. The right adjoint, functor App , does the opposite: it maps objects and morphisms from $\mathbb{P} \rtimes \mathbb{C}$ to $\mathbb{C}$, applying the pointed endofunctor on $\mathbb{C}$ to the object in $\mathbb{C}$ and forgetting the natural transformation in the morphism.

$$\begin{aligned} (Id \rtimes -) A &= \langle Id, A \rangle & App \langle F, A \rangle &= F A \\ (Id \rtimes -) f &= \langle \eta^{Id}, Id f \rangle = \langle id, f \rangle & App \langle \alpha, f \rangle &= f \end{aligned}$$

The adjunction is witnessed by the following isomorphism, where the left adjoint ψ and right adjoint ψ^{-1} are defined as follows:

$$\begin{aligned} \psi : \mathbb{P} \rtimes \mathbb{C} \left((Id \rtimes -) A, \langle F, B \rangle \right) &\cong \mathbb{C} \left(A, App \langle F, B \rangle \right) : \psi^{-1} \\ \psi \langle \alpha, f \rangle &= f & \psi^{-1} g &= \langle \eta^F, g \rangle \end{aligned}$$

These witnesses of the isomorphism satisfy the requisite round-trip properties.

5.3.2 The $\overline{Free} \dashv \overline{U}$ Adjunction

We now explain the free-forgetful adjunction on the left side of the diagram.

$$H-Alg(\mathbb{P} \rtimes \mathbb{C}) \begin{array}{c} \xleftarrow{\overline{Free}} \\ \perp \\ \xrightarrow{\overline{U}} \end{array} \mathbb{P} \rtimes \mathbb{C}$$

In fact, category $\mathbb{P} \rtimes \mathbb{C}$ is a special case of the product category, for which the free-forgetful adjunction is an established construction. We exploit this situation to define our free-forgetful adjunction as a special instance of the product category.

$$\begin{array}{ccc}
H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C}) & \xleftarrow{\text{Free}} & \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \\
\uparrow J \text{ (3)} & \xrightarrow{\perp \text{ (2)}} & \uparrow I \text{ (1)} \\
H-Alg(\mathbb{P} \rtimes \mathbb{C}) & \xleftarrow{\text{Free}} & \mathbb{P} \rtimes \mathbb{C} \\
& \xrightarrow{\overline{U}} &
\end{array}$$

In particular, we work in 4 steps: (1) we include category $\mathbb{P} \rtimes \mathbb{C}$ in $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$; (2) we discuss the free-forgetful adjunction of functor $H : \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \rightarrow \mathbb{C}^{\mathbb{C}} \times \mathbb{C}$; (3) we present $H-Alg(\mathbb{P} \rtimes \mathbb{C})$ as a restricted version of the category of H -algebras; and (4) we demonstrate how $\overline{Free} \dashv \overline{U}$ is a composition of $Free \dashv U$ and two inclusion functors expressing the subcategory relations. To determine what the initial algebras of $\mathbb{P} \rtimes \mathbb{C}$ look like, we consider them as initial algebras in $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$, which have already been amply studied in the literature.

Generalizing the $\mathbb{P} \rtimes \mathbb{C}$ Category $\mathbb{P} \rtimes \mathbb{C}$ is isomorphic to a subcategory of the product category $\mathbb{P} \times \mathbb{C}$. Indeed, every object $\langle F, A \rangle$ in $\mathbb{P} \rtimes \mathbb{C}$ corresponds to an object $\langle F, F A \rangle$ in $\mathbb{P} \times \mathbb{C}$. Moreover, $\mathbb{P} \rtimes \mathbb{C}$ is a subcategory¹ of $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ such that every object in $\mathbb{P} \rtimes \mathbb{C}$ is an object in $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$, forgetting F 's pointedness.

$$\begin{aligned}
\mathbb{P} \rtimes \mathbb{C} \left(\langle F, A \rangle, \langle G, B \rangle \right) &= \mathbb{P} \times \mathbb{C} \left(\langle F, F A \rangle, \langle G, G B \rangle \right) \\
&\subseteq \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \left(\langle F, F A \rangle, \langle G, G B \rangle \right)
\end{aligned}$$

By transitively combining these two relations, we conclude that $\mathbb{P} \rtimes \mathbb{C}$ is isomorphic to a subcategory of $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$, captured by an inclusion functor $I : (\mathbb{P} \rtimes \mathbb{C}) \rightarrow (\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ such that

$$\begin{aligned}
I \langle F, A \rangle &= \langle F, F A \rangle \\
I \langle h_1, h_2 \rangle &= \langle h_1, h_2 \rangle
\end{aligned}$$

Generalized Adjunction We shift our focus to the $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ category. Provided that $\mathbb{C}$ has fixpoints of endofunctors, $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ also does. Hence, we can instantiate the free-forgetful adjunction over $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$, based on an endofunctor $H : (\mathbb{C}^{\mathbb{C}} \times \mathbb{C}) \rightarrow (\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$.

$$\begin{array}{ccc}
H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C}) & \xleftarrow{\text{Free}} & \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \\
& \xrightarrow{\perp} & \\
& \xrightarrow{U} &
\end{array}$$

¹No isomorphism since not every natural transformation preserves η .

Objects in $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ are pairs $\langle F, A \rangle, \langle G, B \rangle$ and morphisms are defined by natural transformations $\alpha : F \rightarrow G$ and functions $f : A \rightarrow B$. The category on the left is the category of H -algebras over $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$, with as objects quadruples:

$$\langle F : \mathbb{C}^{\mathbb{C}}, A : \mathbb{C}, alg_{\alpha} : H F \rightarrow F, alg_f : H F A \rightarrow A \rangle$$

Category $H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ also has an initial object: the fixed point of H . Thus, H^* is a free monad, defined as $H^* = U Free$. Moreover, this fixpoint is equipped with a fold recursion scheme.

$$\begin{array}{ccc} F & \xrightarrow{Var} & H^* F \\ \text{gen} \downarrow & \swarrow \text{fold} & \\ G & & \end{array} \qquad \begin{array}{ccc} H(H^* F) & \xrightarrow{Op} & H^* F \\ H \text{ fold} \downarrow & & \downarrow \text{fold} \\ H G & \xrightarrow{alg} & G \end{array}$$

In what follows we specialize this adjunction, by constraining H and the category.

Specializing the Adjunction In what follows we impose five restrictions that make the adjunction act between categories $H-Alg(\mathbb{P} \rtimes \mathbb{C})$ and $\mathbb{P} \rtimes \mathbb{C}$.

- (1) We restrict ourselves to the subcategory of $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ where objects are of the form $\langle F, F A \rangle$.
- (2) We only consider endofunctors H created out of higher-order bifunctor $K : \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \rightarrow \mathbb{C}$ such that $H \langle F, A \rangle = \langle K F \circ F, K F A \rangle$.
Furthermore, algebra actions for carriers $\langle F, F A \rangle$ under $Free : \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \rightarrow H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ have the form $\langle \alpha : K F \circ F \rightarrow F, f : K F (F A) \rightarrow F A \rangle$.
- (3) We restrict ourselves further to the subcategory of algebras where $f = \alpha_A$ and consider only actions $\langle \alpha, \alpha_A \rangle$ that are uniquely determined by their natural transformation α .
- (4) We only study $\mathbb{C}^{\mathbb{C}}$ objects F that are pointed functors, i.e., that are also objects in $\mathbb{P}$ with an associated $\eta^F : Id \rightarrow F$.
- (5) Finally, we consider solely those morphisms $\langle h_1, h_2 \rangle$ where h_1 is a pointed functor homomorphism.

It turns out that under (1) and (2), $H^* \langle F, F A \rangle$ takes the form $\langle Free_H K F, Free_H K F A \rangle$, where H^* is the generalized free monad.

$$Free_H K F = \Lambda X . F X + K (Free_H K F) (Free_H K F X)$$

Hence, $H^*\langle F, F A \rangle$ is part of the subcategory of **(1)**. Furthermore, the algebra action created by *Free* **(2)** also resides in this subcategory **(3)**. Notice that $Free_H K F$ is not a monad on $\mathbb{P} \times \mathbb{C}$, but that it is pointed **(4)** because F is. Also **(5)** holds, as functor *Free* respects it: the first component of the fold with generator $\langle h_1, h_2 \rangle$ and algebra $\langle \alpha, \alpha_A \rangle$ is a pointed functor morphism. From these five restrictions on category $H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$, we can define category $H-Alg(\mathbb{P} \times \mathbb{C})$. *Objects* in $H-Alg(\mathbb{P} \times \mathbb{C})$ are tuples $\langle \langle F, A \rangle, a : K F \circ F \rightarrow F \rangle$, where carrier $\langle F, A \rangle$ is an object in $\mathbb{P} \times \mathbb{C}$ and action a is a morphism in $\mathbb{C}^{\mathbb{C}}$. *Morphisms* $\langle h_1, h_2 \rangle : \langle \langle F, A \rangle, a \rangle \rightarrow \langle \langle G, B \rangle, b \rangle$ in $H-Alg(\mathbb{P} \times \mathbb{C})$ are homomorphisms $\langle h_1, h_2 \rangle : \langle F, A \rangle \rightarrow \langle G, B \rangle$ in $\mathbb{P} \times \mathbb{C}$:

$$\begin{array}{ccc} K F \circ F & \xrightarrow{K h_1 h_1} & K G \circ G \\ a \downarrow & & \downarrow b \\ F & \xrightarrow{h_1} & G \end{array} \qquad \begin{array}{ccc} K F (F A) & \xrightarrow{K h_1 h_2} & K G (G B) \\ a_A \downarrow & & \downarrow b_B \\ F A & \xrightarrow{h_2} & G B \end{array}$$

Observe that objects $\langle \langle F, A \rangle, a \rangle$ in $H-Alg(\mathbb{P} \times \mathbb{C})$ are in one-to-one correspondence with $\langle \langle F, F A \rangle, \langle a, a_A \rangle \rangle$ of $H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ that are subject to the above restrictions. Similarly, morphisms $\langle h_1, h_2 \rangle$ in $H-Alg(\mathbb{P} \times \mathbb{C})$ are also morphisms in $H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ that meet the restrictions. The above two correspondences are witnessed by an inclusion functor $J : H-Alg(\mathbb{P} \times \mathbb{C}) \rightarrow H-Alg(\mathbb{C}^{\mathbb{C}} \times \mathbb{C})$ such that

$$\begin{aligned} J\langle \langle F, A \rangle, a \rangle &= \langle \langle F, F A \rangle, \langle a, a_A \rangle \rangle \\ J\langle h_1, h_2 \rangle &= \langle h_1, h_2 \rangle \end{aligned}$$

The Specialized Adjunction In categorical terms we define the $\overline{Free} \dashv \overline{U}$ adjunction using two equivalences $Free \circ I = J \circ \overline{Free}$ and $I \circ \overline{U} = U \circ J$, where both I and J are inclusion functors.

In particular, the following correspondences hold:

$$\begin{aligned} \forall A : \mathbb{P} \times \mathbb{C}. \quad & \text{exists } B : H-Alg(\mathbb{P} \times \mathbb{C}). Free (I A) = J B \\ \forall B : H-Alg(\mathbb{P} \times \mathbb{C}). & \text{exists } A : \mathbb{P} \times \mathbb{C}. \quad U (J B) = I A \end{aligned}$$

For example, take $\langle F, A \rangle : \mathbb{P} \times \mathbb{C}$.

$$\begin{aligned}
& \text{Free } (I (\langle F, A \rangle)) \\
&= \text{Free } (\langle F, F A \rangle) \\
&= \text{Free } (\langle \langle F, F A \rangle, \langle \alpha : K F . F \rightarrow F, f : K F (F A) \rightarrow F A \rangle \rangle) \\
&= J (\langle \langle F, A \rangle, \alpha \rangle)
\end{aligned}$$

The other derivation goes in a similar way. Thus, the adjunction $\overline{\text{Free}} \dashv \overline{U}$ is a special instance of the $\text{Free} \dashv U$ adjunction. The universal property of Free carries over to $\overline{\text{Free}}$. Hence, $\overline{\text{Free}} \dashv \overline{U}$ is also a free-forgetful adjunction and $\overline{U} \circ \overline{\text{Free}}$ is a free monad on $\mathbb{P} \rtimes \mathbb{C}$.

5.3.3 Composing the Two Adjunctions

When we compose the two adjunctions, the diagram is complete.

$$\begin{array}{ccccc}
H\text{-Alg}(\mathbb{C}^{\mathbb{C}} \times \mathbb{C}) & \xleftarrow[\overline{U}]{\text{Free}} & \mathbb{C}^{\mathbb{C}} \times \mathbb{C} & & \\
\uparrow J & & \uparrow I & & \\
H\text{-Alg}(\mathbb{P} \rtimes \mathbb{C}) & \xleftarrow[\overline{U}]{\overline{\text{Free}}} & \mathbb{P} \rtimes \mathbb{C} & \xleftarrow[\text{App}]{\text{Id} \rtimes -} & \mathbb{C}
\end{array}$$

In order to get a free monad on $\mathbb{C}$, we apply the $\overline{\text{Free}} \circ (\text{Id} \rtimes -) \dashv \text{App} \circ \overline{U}$ adjunction. We start with an object A in $\mathbb{C}$, and transform it to $\langle \text{Id}, A \rangle$ in $\mathbb{P} \rtimes \mathbb{C}$, where the identity functor Id is pointed. Using the free-forgetful adjunction and applying the above restrictions, we get a free monad $\langle \text{Free}_{\mathbb{H}} K \text{Id}, \text{Free}_{\mathbb{H}} K \text{Id} A \rangle$ in $\mathbb{P} \rtimes \mathbb{C}$. If we then apply the App functor, we get a free monad $\text{Free}_{\mathbb{H}} K \text{Id} A$ in $\mathbb{C}$, which is indeed the monad we have used for our instantiations. Notice that $\text{Free}_{\mathbb{H}} K \text{Id}$ of this section is isomorphic to $\text{Free}_{\mathbb{H}} K$ of Section 3.1.

5.4 An Adjunction for Scoped Effects

The categorical model for algebraic effects & handlers is clean and neat. In the same vein, different works have tried to put scoped effects on a similar formal footing. These proposals include modeling scoped effects as handlers [121], using an adjunction with Eilenberg-Moore algebras [176], indexed algebras [117] and functorial algebras [176]. The trade-off to make here is to find a representation that retains the modularity as for algebraic effects, and balances structuredness and simplicity.

5.4.1 Scoped Effects as Handlers are Not Modular

Plotkin and Power [121] have proposed to model generic effects, which include scoped effects, as handlers. The problem is that they cannot be freely used amongst other algebraic operations: when a handler implementing a scoped operation is applied to a computation, the computation is transformed from a syntactic tree of algebraic operations into some semantic model implementing the scoped operation. Consequently, all subsequent operations on the computation can only be given in the particular semantics model rather than as mere syntactic operations, thus nullifying the crucial advantage of modularity when separating syntax and semantics of effects.

For example, when modeling a scoped effect such as `once` as a handler, this handler plays a dual role of modelling the syntax of the operation and giving semantics to it. It is possible to model `once` as a handler once_h :

$$\begin{aligned} \text{once}_h &:: \text{Functor } \sigma \Rightarrow \text{Free } (\text{Choice} + \sigma) \ a \rightarrow \text{Free } (\text{Choice} + \sigma) \ a \\ \text{once}_h &= \text{fold}_{\text{Alg}} \ \text{return} \ (\text{alg } \nabla \text{Op}) \ \text{where} \\ &\quad \text{alg } \text{Fail} \quad = \text{Op } \text{Fail} \\ &\quad \text{alg } (\text{Or } p \ q) = \text{case } \text{once}_h \ p \ \text{of } \{ \text{Op } \text{Fail} \rightarrow \text{once}_h \ q; _ \rightarrow \text{once}_h \ p \} \end{aligned}$$

We construct a program with (global) state and `once` as a handler, and interpret this program with the modular handlers of Section 2.4:

$$\begin{aligned} \text{prog}_{\text{Once}} &= \text{once}_h \ (\text{or } (\text{return } 1) \ (\text{put } 3 \gg \text{return } 5)) \\ &\quad \gg \lambda x \rightarrow \text{or } (\text{return } x) \ (\text{return } (x + 1)) \\ &\gg \gg h_{\text{State}}^+ (h_{\text{ND}}^+ \text{prog}_{\text{Once}}) \ 0 \\ &([1, 2], 0) \end{aligned}$$

The issue is that if we want to change the handler implementation, we also have to reformulate the program it acts on. For example, if we would define `once` as a handler $\text{onceHdl}' :: \text{Functor } \sigma \Rightarrow \text{Free } (\text{Choice} + \sigma) \ a \rightarrow \text{Free } (\text{Choice} + \sigma) \ (\text{Maybe } a)$, that interprets the program in terms of a *Maybe*, we would have to rewrite $\text{prog}_{\text{Once}}$ accordingly, with a case split on x . Thus, the program must be written differently, depending on which handler is used. The issue is that these handlers interpret the operation in different semantic models, and this affects both the value x and the continuation. Therefore, non-algebraic operation `once` modelled as a handler is not as modular as algebraic operations, weakening the advantage of programming with algebraic effects.

5.4.2 Eilenberg-Moore Algebras and Indexed Algebras

Piróg et al. [117] sought to put the ad-hoc approach of Wu et al. [173] to model scoped effects on formal footing. They propose a solution based on a level-indexed category, called *indexed algebras*.

In particular, Piróg et al. present a monad $P : \mathbb{C} \rightarrow \mathbb{C}$ that models the syntax of programs with algebraic operations in Σ and scoped operations in Γ .

$$P A = A + \Sigma (P A) + \int^{X:\mathbb{C}} \coprod_{\mathbb{C}(X, PA)} \Gamma (P X)$$

This monad is equivalent [48] to $P A = A + \Sigma (P A) + \Gamma (P (P A))$ which is exactly the *Prog* $\Sigma \Gamma A$ datatype of Section 4.1.

To handle scoped effects modeled by monad P , we look at Eilenberg-Moore algebras [176] and indexed algebras [117].

Eilenberg-Moore Algebras In standard algebraic effects, handlers are just Σ -algebras for some signature functor $\Sigma : \mathbb{C} \rightarrow \mathbb{C}$, and it is well known that the category of Σ -algebras is equivalent to the category of Eilenberg-Moore algebras of the monad Σ^* . Thus, handlers of algebraic operations are exactly Eilenberg-Moore algebras of the monad Σ^* modelling the syntax of algebraic operations. This observation suggests that we may also use Eilenberg-Moore algebras of the monad P as the notion of handlers for scoped operations: Eilenberg-Moore algebras of P are equivalent to $(\Sigma + \Gamma \circ P)$ -algebras. Thus, we obtain a way of interpreting scoped operations based on the free-forgetful adjunction:

$$(\Sigma + \Gamma \circ P) - Alg \begin{array}{c} \xleftarrow{Free_{EM}} \\ \perp \\ \xrightarrow{U_{EM}} \end{array} \mathbb{C}$$

Interpreting scoped operation with Eilenberg-Moore algebras can be understood as simulating scoped operations with algebraic operations and general recursion: a signature (Σ, Γ) of algebraic-and-scoped operations is simulated by a signature $\Sigma + \Gamma \circ P$ of algebraic operations where P is recursively given by $\Sigma + \Gamma \circ P^*$. In this way, one can simulate scoped operation in languages implementing algebraic effects that allow signatures of operation to be recursive, such as [61, 92, 9], but not the original design by Plotkin and Pretnar [120], which requires signatures of operations to mention only some base types. The downside of this simulating approach is that the denotational semantics of the language becomes more

complex and usually involves solving some domain-theoretic recursive equations, like in [9]. Moreover, this approach typically requires handlers to be defined with general recursion, which obscures the inherent structure of scoped operations, making reasoning about handlers of scoped operations more difficult.

Indexed Algebras An alternative for Eilenberg-Moore algebras are indexed algebras [117], which are based on the following adjunction:

$$\begin{array}{ccccc} & \xleftarrow{\text{Free}_{\text{Ix}}} & & \xleftarrow{\downarrow} & \\ \text{Ix-Alg} & \xrightleftharpoons[\text{U}_{\text{Ix}}]{\perp} & \mathbb{C}^{|\mathbb{N}|} & \xrightleftharpoons[\downarrow]{\perp} & \mathbb{C} \end{array}$$

Here, $\mathbb{C}^{|\mathbb{N}|}$ is the functor category from the discrete category $|\mathbb{N}|$ of natural numbers to the base category $\mathbb{C}$. That is to say, an object in $\mathbb{C}^{|\mathbb{N}|}$ is a family of objects A_i in $\mathbb{C}$ indexed by natural numbers $i \in |\mathbb{N}|$, and a morphism $\tau : A \rightarrow B$ in $\mathbb{C}^{|\mathbb{N}|}$ is a family of morphisms $\tau_i : A_i \rightarrow B_i$ in $\mathbb{C}$ with no coherence conditions between the levels. An endofunctor $\text{Ix} : \mathbb{C}^{|\mathbb{N}|} \rightarrow \mathbb{C}^{|\mathbb{N}|}$ is defined to characterise indexed algebras:

$$\text{Ix } A = \Sigma \circ A + \Gamma \circ (\triangleleft A) + (\triangleright A)$$

where $\triangleleft$ and $\triangleright$ shift indices such that $(\triangleleft A)_i = A_{i+1}$ and $(\triangleright A)_0 = 0$ and $(\triangleright A)_{i+1} = A_i$. Then objects in Ix-Alg are indexed algebras.

The operational intuition is that a carrier A_i at level i interprets the part of syntax enclosed by i layers of scopes, and when interpreting a scoped operation at layer i , the part of syntax outside the scope is first interpreted. Then the indexed algebra provides a way to promote the carrier to the next level ($p : A \rightarrow (\triangleleft A)$). After the inner layer is also interpreted, the indexed algebra provides a way to demote the carrier ($d : \Gamma (\triangleleft A) \rightarrow A$). Additionally a morphism interprets ordinary algebraic operations ($a : \Sigma \circ A \rightarrow A$).

The adjunction $\text{Free}_{\text{Ix}} \dashv \text{U}_{\text{Ix}}$ is the free-forgetful adjunction for Ix on $\mathbb{C}^{|\mathbb{N}|}$. The other adjunction $\downarrow \dashv \downarrow$ is given by $\downarrow A = A_0$, $(\downarrow X)_0 = X$, and $(\downarrow X)_{i+1} = 0$ for all $i \in |\mathbb{N}|$. Piróg et al. [117] show that the monad induced by the adjunction is isomorphic to monad P .

This formalisation in terms of indexed algebras introduces a disparity between syntax and semantics, which makes them not as structured as the programs they interpret. They use an ad-hoc hybrid fold that requires indexing of the handlers, but not for the program syntax. Moreover, indexed algebras are not ideal for widespread implementation as they require dependent typing, in at

least a limited form like GADTs [77]. This approach can be implemented in Haskell with GHC's `DataKinds` extension for type-level natural numbers.

5.4.3 Functorial Algebras

Yang et al. [176] present a more structured way of handling scoped effects by *functorial algebras*. They are more structured than indexed algebras in the sense that the structure of functorial algebras directly follows the abstract syntax of programs with scoped effects. Additionally, this approach can be practically implemented without the need for dependent types or GADTs, making it available for a wider range of programming languages, shown by an implementation in Haskell and OCaml².

Functorial algebras are based on the following adjunction:

$$\begin{array}{ccccc} \text{Fn-Alg} & \xleftarrow{\text{Free}_{\text{Fn}}} & \mathbb{C}^{\mathbb{C}} \times \mathbb{C} & \xleftarrow{\uparrow} & \mathbb{C} \\ & \perp & & \perp & \\ & \xrightarrow{U_{\text{Fn}}} & & \xrightarrow{\downarrow} & \end{array}$$

The first adjunction is given by $\uparrow : \mathbb{C} \rightarrow \mathbb{C}^{\mathbb{C}} \times \mathbb{C}$ such that $\uparrow X = \langle 0, X \rangle$, where $0 : \mathbb{C} \rightarrow \mathbb{C}$ is the initial endofunctor, i.e., the constant functor sending everything to the initial object in $\mathbb{C}$. This functor is left adjoint to the projection functor $\downarrow : \mathbb{C}^{\mathbb{C}} \times \mathbb{C} \rightarrow \mathbb{C}$ of the second component. The resulting adjunction induces a monad Free_{H} isomorphic to P , and it provides a means to interpret scoped operations modelled with the monad P . The second adjunction $\text{Free}_{\text{Fn}} \dashv U_{\text{Fn}}$ is the free-forgetful adjunction for Fn on $\mathbb{C}^{\mathbb{C}} \times \mathbb{C}$. Objects in Fn-Alg are defined by the following quadruple:

$$\langle F : \mathbb{C} \rightarrow \mathbb{C}, X : \mathbb{C}, \alpha : \text{Id} + \Sigma \circ F + \Gamma \circ F \circ F \rightarrow F, f : \Sigma X + \Gamma (F X) \rightarrow X \rangle$$

Functorial algebras differ from our H -algebras in different ways. First, our H -algebras abstract over functors Σ and $\Gamma \circ H$, i.e., we use a functor $K F$ that represents any class of effects. That would change the objects to

$$\langle F : \mathbb{C} \rightarrow \mathbb{C}, X : \mathbb{C}, \alpha : \text{Id} + K F \circ F \rightarrow F, f : K F X \rightarrow X \rangle$$

Second, in $H\text{-Alg}$, we require F to be pointed so that it has a function $\eta : \text{Id} \rightarrow F$, making the identity functor in α redundant:

²This implementation can be found at <https://github.com/birthedvdb/Structured-Handling-of-Scoped-Effects>.

$$\langle F : \mathbb{C} \rightarrow \mathbb{C}, X : \mathbb{C}, \alpha : K F \circ F \rightarrow F, f : K F X \rightarrow X \rangle$$

Third, H -algebras assume that the internal computation is interpreted in the same manner as the continuation. A functorial algebra offers the flexibility that the outermost layer can be interpreted differently from the inner layers. In particular, this means that H -algebras deal with objects X that are of the form $F A$, which implies that $\beta : K F (F A) \rightarrow F A$ is covered by natural transformation α in A .

$$\langle F : \mathbb{C} \rightarrow \mathbb{C}, A : \mathbb{C}, \alpha : K F \circ F \rightarrow F \rangle$$

This triple denotes an object in $H\text{-Alg}$.

Yang et al. show that the expressivity of functorial algebras, indexed algebras, and simulating scoped effects with algebraic operations and recursion are equal. However, although they have the same expressivity in theory, they structure the interpretation of scoped operations in different ways: Eilenberg-Moore algebras impose no constraint on how the part of syntax enclosed by scopes is handled; indexed algebras demand them to be handled layer by layer but impose no coherent conditions between the layers; functorial algebras additionally force all inner layers must be handled in a uniform way by an endofunctor.

On the whole, it is a trade-off between simplicity and structuredness: Eilenberg-Moore algebras are the simplest for implementation, whereas the structuredness of functorial algebras make them easier to reason about.

Chapter 6

A Calculus for Scoped Effects & Handlers

Denotational semantics of effects, as in the previous Chapters, are interesting to understand effects from a conceptual point of view. Operational semantics, however, focus more on the execution of effects, and are often captured in calculi that lay the foundation of libraries or programming languages.

Algebraic effects & handlers, for example, have been studied from many perspectives. Plotkin and Power [121] first introduced algebraic effects, and later, Plotkin and Pretnar [119] have added handlers for interpreting them. Bauer and Pretnar [9, 10] designed an effect system for programming with algebraic effects & handlers. From then on, many research languages for algebraic effects & handlers have been proposed (Eff [124], Frank [95], Effekt [18]), or have been extended to include them (Links [60], Koka [92], Multicore OCaml [141]). Furthermore, several libraries have adopted these effects, such as fused-effects [129], extensible-effects [82] and Eff in OCaml [84].

Scoped effects & handlers are following the same trend: since their first introduction by Wu et al. [173], they have been studied denotationally [117, 178], have been included in Haskell libraries (eff [80], polysemy [97] and fused-effects [129]), and were adopted by Github [149]. However, a calculus that supports scoped effects & handlers as primitives is missing from the literature.

This Chapter presents the design of such a calculus, λ_{sc} , in terms of its syntax, operational semantics and type-and-effect system. We formulate and prove λ_{sc} 's metatheoretical properties. At <https://github.com/thwfhk/lambdaSC>

an interpreter of λ_{sc} is available.

Section 6.1 starts with an introduction to Eff [124], on which λ_{sc} is based. Section 6.2 discusses the most important design decisions and Section 6.3 presents some highlights of λ_{sc} .

6.1 A Calculus for Algebraic Effects

Eff [9, 10, 124] forms the baseline and starting point of our calculus. This Section briefly discusses its syntax and operational semantics.

In Eff, effects are denoted by a name (or *label*) and characterized by a *signature* $A \rightarrow B$, taking a value of type A and producing a value of type B . For example, $\text{or} : () \rightarrow \text{Bool}$ takes a unit value and produces a boolean (e.g., nondeterministically). To invoke effects, we use the form **op** l v ($y . c_1$), i.e., we combine the **op** keyword with an effect l to invoke, a *parameter* v passed to the effect, and a *continuation* ($y . c_1$). For example, c_{ND} revisits the example of Section 4.1.

$$c_{\text{ND}} = \text{do } x \leftarrow \text{op or } () (b . \text{if } b \text{ then return 1 else return 5}); \\ \text{op or } () (b . \text{if } b \text{ then return } x \text{ else return } (x + 1))$$

In accordance with its signature, **or** is passed $()$, and in the continuation b has type *Bool*. Algebraic effects commute with sequencing, captured by the *algebraicity property*:

$$\text{do } x \leftarrow \text{op } l \text{ } v (y . c_1); c_2 \equiv \text{op } l \text{ } v (y . \text{do } x \leftarrow c_1; c_2)$$

For example:

$$c_{\text{ND}} \equiv \text{op or } () (b . \text{do } x \leftarrow \text{if } b \text{ then return 1 else return 5}; \\ \text{op or } () (b . \text{if } b \text{ then return } x \text{ else return } (x + 1)))$$

Handler h_{ND} interprets nondeterministic programs in terms of a list; it has the same implementation as in Section 2.4.

$$h_{\text{ND}} = \text{handler } \{ \text{return } x \mapsto \text{return } [x] \\ , \text{op fail } - \mapsto \text{return } [] \\ , \text{op or } - k \mapsto \text{do } xs \leftarrow k \text{ true}; \text{do } ys \leftarrow k \text{ false}; xs ++ ys \}$$

The handler consists of three clauses. The first is the *return clause*, and is invoked when a value x is returned. This corresponds to the generator of a fold-style handler. The next two are *algebraic effect clauses*, interpreting the nondeterministic effects *fail* and *or*. This handler is equivalent to h_{ND}^+ of Section 2.4. We apply h_{ND} to c_{ND} with the $\star$ -operator to obtain its results:

$$h_{\text{ND}} \star c_{\text{ND}} \rightsquigarrow^* [1, 2, 5, 6]$$

Effects can be composed by combining different primitive effects. For example, computation c_{inc} increments a state using $\text{get} : () \rightarrow \text{Int}$ and $\text{put} : \text{Int} \rightarrow ()$, and h_{State} has the same implementation as in Section 2.4.

$$\begin{aligned} c_{\text{inc}} &= \text{do } s \leftarrow \text{get } () \text{ (} x. \text{return } x \text{); op put (} s + 1 \text{) (} _ . \text{return } () \text{)} \\ h_{\text{State}} &= \text{handler } \{ \text{return } x \mapsto \lambda s \rightarrow \text{return } (x, s), \\ &\quad \text{op get } _ k \mapsto \lambda s \rightarrow k \ s \ s, \\ &\quad \text{op put } s k \mapsto \lambda _ \rightarrow k \ () \ s \} \end{aligned}$$

If we would apply h_{ND} to c_{inc} , which does not contain any nondeterministic effects, it forwards the state effects to be interpreted by the next handler. Just as before, we can modularly compose handlers by applying them one by one, and a different order leads to different semantics.

$$\begin{aligned} c_{\text{ND}, \text{inc}} &= \text{op or } () \text{ (} b. \text{if } b \text{ then do } _ \leftarrow c_{\text{inc}}; \text{return true else return false)} \\ h_{\text{ND}} (h_{\text{State}} c_{\text{ND}, \text{inc}}) 0 &\rightsquigarrow^* [(1, \text{true}), (0, \text{false})] \\ h_{\text{State}} (h_{\text{ND}} c_{\text{ND}, \text{inc}}) 0 &\rightsquigarrow^* (1, [\text{true}, \text{false}]) \end{aligned}$$

6.2 Challenges & Design Choices

This Section discusses the design of λ_{sc} , a novel calculus with support for scoped effects & handlers as built-in features. We present the necessary features and motivate our design decisions.

6.2.1 Eff with Koka-based Row Typing

λ_{sc} is based on Eff [124, 9, 10], an existing calculus for algebraic effects & handlers. We have made some cosmetic changes to the presentation of Eff, and have adopted row-based typing in the style of Koka [92]. In particular,

computations have types of shape $A ! \langle E \rangle$, where A is the type of the value returned by the computation, and E is a collection of effects that *can* occur during its evaluation. For example, $Int ! \langle \text{or} \rangle$ is a type of c_{ND} .

Handlers turn one computation into another. Their type reflects that: handlers of type $A ! \langle E \rangle \Rightarrow B ! \langle F \rangle$ take a computation of type $A ! \langle E \rangle$ and return a computation of type $B ! \langle F \rangle$. For example, h_{ND} handles `fail` and `or`.

$$h_{\text{ND}} : Int ! \langle \text{or}, \text{fail} \rangle \Rightarrow [Int] ! \langle \rangle$$

With subtyping-based effect types, like in *Eff*, each type of computation is decorated with an effect type Δ to represent the set of operations that might be invoked. The subtyping relations are used to extend the effect type Δ with other effects, which makes it possible to compose programs in a modular way. λ_{sc} does not implement subtyping-based effect types, which would require complex subtyping for type operators. As an alternative for subtyping-based effect types, Koka [91] uses row polymorphism that is based on scoped labels [90], and it allows duplicate labels. Rémy style row polymorphism, like in *Links* [60], explicitly shows the absence of labels and each label is restricted to appear at most once. In λ_{sc} , we opt for an effect system similar to Koka's, mainly because of its brevity. We believe that the *Links*-style effect system should also work well with scoped effects.

6.2.2 Scoped Effects as Built-in Syntactic Primitives

Modeling scoped effects as handlers comes at the cost of modularity (Section 5.4). For that reason, we add scoped effects as built-in primitives with a new notation, signalled by the **sc** keyword.

$$\text{sc } \ell^{\text{sc}} v (y . c_1) (z . c_2)$$

Similar to their algebraic counterpart, scoped effects feature a label ℓ^{sc} to identify the effect, a parameter v , and a continuation $(z . c_2)$. Scoped effects differ from algebraic ones by their additional *scoped computation* $(y . c_1)$, i.e., the computation in scope. The dataflow allows for a value y to be passed from the operation to c_1 and a value z from c_1 to c_2 . For example, c_{once} models nondeterminism with *once* from Section 4.1.

$$c_{\text{once}} = \text{sc once } () \ (_ . \text{op or } () \ (b . \text{return } b)) \\ \quad (b . \text{if } b \text{ then return 1 else return 5})$$

h_{Once} is an extension of h_{ND} , where the dots represent its return clause and the algebraic effect clauses for fail and or.

$$h_{\text{Once}} = \text{handler } \{ \dots, \\ \text{sc once } _ p \ k \mapsto \text{do } ts \leftarrow p \ (); \text{do } t \leftarrow \text{head } ts; k \ t \} \\ h_{\text{Once}} \star c_{\text{Once}} \rightsquigarrow^* [1]$$

Adding scoped effects gives rise to a variant of the algebraicity property: scoped effects commute with sequencing in the continuation, but leave the scoped computation intact.

$$\text{do } x \leftarrow \text{sc } \ell^{\text{sc}} v (y . c_1) (z . c_2); c_3 \equiv \text{sc } \ell^{\text{sc}} v (y . c_1) (z . \text{do } x \leftarrow c_2; c_3)$$

But, this novel representation for scoped effects & handlers also adds complexity: the result of the scoped computation is the argument of the continuation. This means that they must agree on a type of this result, which we name the *scoped result type*. For example, c_{Once} has overall type $\text{Int}! \langle \text{once}; \text{or} \rangle$ but its scoped result type is a *Bool*:

$$\text{sc once } () \underbrace{(_ . \text{op or } () (b . \text{return } b))}_{() \rightarrow \text{Bool}! \langle \text{once}; \text{or} \rangle} \underbrace{(b . \text{if } b \text{ then return } 1 \text{ else return } 5)}_{\text{Bool} \rightarrow \text{Int}! \langle \text{once}; \text{or} \rangle}$$

Dealing with the presence of this scoped computation type is tricky, and introduces two complications. First, since the type does not occur in the computation's overall type, *polymorphic handlers* are required to handle scoped effects. Second, the scoped result type describes a dependency between the scoped computation and continuation: if one changes its type, the other must match this. This makes *generic forwarding* impossible: it alters the type of the scoped computation, but does not make up for it in the continuation.

6.2.3 Polymorphic Handlers

Applying a handler to a computation involves recursively applying the handler to the computation's subcomputations as well. In the case of algebraic effects, these subcomputations always have the same type as the operation itself, as witnessed by the algebraicity property. For that reason, calculi with solely algebraic effects & handlers, such as *Eff*, can (and do) type handlers monomorphically, without limitations.

However, typing scoped effect handlers monomorphically *does* limit their implementation freedom: it only allows scoped operations of which the scoped result type matches the operation's overall type. For example, if we type h_{Once} monomorphically, the scoped result type is required to be Int as well.

$$h_{\text{Once}} : \text{Int}! \langle \text{or}; \text{fail}; \text{once} \rangle \Rightarrow [\text{Int}]! \langle \rangle$$

However, c_{once} 's scoped result type is Bool . For that reason, not all scoped computations can be handled by monomorphic handlers. To mitigate this, we allow handlers to abstract over the value type of computations so that they can handle scoped operations with *any* scoped result type. For example, h_{Once} can be typed as follows:

$$h_{\text{Once}} : \forall \alpha . \alpha! \langle \text{or}; \text{fail}; \text{once} \rangle \Rightarrow [\alpha]! \langle \rangle$$

With this polymorphic typing in place, $h_{\text{Once}} \star c_{\text{once}}$ can be evaluated by *polymorphic recursion*. To support this, λ_{sc} features **let**-polymorphism, F_{ω} -style type operators, and requires all handlers for scoped effects to be polymorphic.

6.2.4 Forwarding Unknown Operations

For algebraic effects, forwarding happens generically. For example, consider the forwarding of h_{Once} applied to the algebraic **get**.

$$h_{\text{Once}} \star \mathbf{op\ get\ } () \ (x . \mathbf{return\ } x) \rightsquigarrow \mathbf{op\ get\ } () \ (x . h_{\text{Once}} \star \mathbf{return\ } x)$$

One might hope to forward scoped effects in a similar way. Unfortunately, this does not work because of the discrepancy in the scoped result type: a handler for scoped effects introduces a type operator when handling a computation. h_{Once} , for example, introduces a list. For scoped operations this is problematic: the return type of the scoped computation changes (e.g., $[\text{Int}]$), whereas the continuation still expects the original type (e.g., Int). Thus, scoped effects cannot be forwarded generically. We require every handler to be equipped with an explicit forwarding clause of the form $\mathbf{fwd\ } f \ p \ k \mapsto c$, based on the type operator that applies.

For example, h_{Once} mitigates the discrepancy between $[\text{Int}]$ and Int by settling the scoped result type on $[\text{Int}]$. In the forwarding clause of h_{Once} , *concatMap* (Appendix G) ensures that the transformed continuation now takes a value of type $[\text{Int}]$ as argument.

$$h_{\text{Once}} = \mathbf{handler} \{ \dots, \mathbf{fwd} \ f \ p \ k \mapsto f \ (p, (\lambda z. \mathbf{concatMap} \ z \ k)) \}$$

We implement forwarding by means of a function f , which is a partial application of $\mathbf{sc} \ l \ v$: it takes the pair of a (possibly transformed) scoped computation p' and continuation k' and re-introduces the to-be-forwarded scoped operation with these parameters.

$$f = \lambda(p', k'). \mathbf{sc} \ l \ v \ (y. p' \ y) \ (z. k' \ z)$$

6.3 The λ_{sc} -Calculus

This Section highlights parts of the syntax, operational semantics and type-and-effect system of λ_{sc} . For the full details, we refer to Appendix D. In Appendix B we have worked out a range of other examples of scoped effects in λ_{sc} .

6.3.1 Syntax

Figure 6.1 displays a summary of the syntax of λ_{sc} , with the extensions that are required to support scoped effects & handlers, with respect to Eff, highlighted in gray. Like Eff, we implement fine-grained call-by-value semantics [94], where we split terms in inert values and reducible computations. Unlike Eff, we implement effect types as effect rows using row polymorphism [90] in the style of Koka [92]: rows E are collections of labels ℓ , optionally terminated by a row variable μ . To support polymorphic handlers, we add let-polymorphism, for which we need let-bindings, type variables α , type operators M , and type abstraction and application. Forwarding scoped effects cannot be done generically. For that reason, each handler now requires an explicit forwarding clause $\mathbf{fwd} \ f \ p \ k \mapsto c_f$. Furthermore, scoped effects are primitive computations and the effect clauses are extended with a clause for scoped effects. Finally, we abstract over type- and row variables, giving rise to type schemes σ .

6.3.2 Operational Semantics

Relation $c \rightsquigarrow c'$ denotes that computation c steps to computation c' , with $\rightsquigarrow^*$ its reflexive, transitive closure. We focus on the novel rules for λ_{sc} , and refer to Appendix D for the full semantics.

values v	$::=$	$() \mid (v_1, v_2) \mid x \mid \lambda x. c \mid h$
handlers h	$::=$	handler $\{ \text{return } x \mapsto c_r, cls$ $, \text{fwd } f \ p \ k \mapsto c_f \}$
effect clauses cls	$::=$	$\cdot \mid \text{op } \ell^{\text{op}} \ x \ k \mapsto c, cls$ $\mid \text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, cls$
computations c	$::=$	$\text{return } v \mid v \star c \mid \text{do } x \leftarrow c_1; c_2 \mid v_1 \ v_2$ $\mid \text{op } \ell^{\text{op}} \ v \ (y. c) \mid \text{sc } \ell^{\text{sc}} \ v \ (y. c_1) \ (z. c_2)$ $\mid \text{let } x = v \text{ in } c$
value types A, B, M	$::=$	$() \mid (A, B) \mid A \rightarrow \underline{C} \mid \underline{C} \Rightarrow \underline{D}$ $\mid \underline{\alpha} \mid \Lambda \alpha. A \mid M A$
type schemes σ	$::=$	$A \mid \forall \mu. \sigma \mid \forall \alpha. \sigma$
computation types $\underline{C}, \underline{D}$	$::=$	$A! \langle E \rangle$
effect rows E, F	$::=$	$\cdot \mid \mu \mid \ell; E$
signature contexts Σ	$::=$	$\cdot \mid \Sigma, \ell^{\text{op}} : A \rightarrow B \mid \Sigma, \ell^{\text{sc}} : A \rightarrow B$
type contexts Γ	$::=$	$\cdot \mid \Gamma, x : \sigma \mid \Gamma, \mu \mid \Gamma, \alpha$

Figure 6.1: Syntax λ_{sc} .

Rule E-DoSc embodies the generalization of the algebraicity property for scoped effects:

$$\frac{}{\text{do } x \leftarrow \text{sc } \ell^{\text{sc}} \ v \ (y. c_1) \ (z. c_2); c_3 \rightsquigarrow \text{sc } \ell^{\text{sc}} \ v \ (y. c_1) \ (z. \text{do } x \leftarrow c_2; c_3)} \text{E-DoSc}$$

Rules E-HANDSC and E-FWDSC represent the handling and forwarding of scoped effects, respectively.

If the handler contains a clause for ℓ^{sc} , evaluation switches to the clause's computation c , with v substituted for parameter x . Both the scoped computation and the continuation are replaced by a function that contains the handled computations c_1 and c_2 .

$$\frac{(\text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c) \in h}{h \star \text{sc } \ell^{\text{sc}} \ v \ (y. c_1) \ (z. c_2) \rightsquigarrow c \ [v / x, (\lambda y. h \star c_1) / p, (\lambda z. h \star c_2) / k]} \text{E-HANDSC}$$

If the handler does not contain a clause for ℓ^{sc} , the effect is forwarded via the handler's forwarding clause. Evaluation switches to computation c_f , in which scoped computation p and continuation k are replaced by their handled equivalents. Computation c_f may re-invoke the unknown scoped effect ℓ^{sc} by

means of parameter f , which is replaced for g .

$$\frac{(\mathbf{sc} \ell^{\mathbf{sc}} - -) \notin h \quad (\mathbf{fwd} f p k \mapsto c_f) \in h \quad g = \lambda(p', k'). \mathbf{sc} \ell^{\mathbf{sc}} v (y. p' y) (z. k' z)}{h \star \mathbf{sc} \ell^{\mathbf{sc}} v (y. c_1) (z. c_2) \rightsquigarrow c_f [(\lambda y. h \star c_1) / p, (\lambda z. h \star c_2) / k, g / f]} \text{E-FWDSC}$$

6.3.3 Type-and-Effect System

For the typing of λ_{sc} , we distinguish between value typing, computation typing and handler typing. We highlight some important and novel rules and refer to Appendix D.3 for the full type-and-effect system.

Value Typing Rule T-EQV shows that typing holds up to equivalence of types.

$$\frac{\Gamma \vdash v : A \quad A \equiv B}{\Gamma \vdash v : B} \text{T-EQV}$$

The full type equivalence relation ($A \equiv B$), which also uses the equivalence of rows ($E \equiv_{\langle \rangle} F$), is included in Appendix D.3. However, these relations can be described as the congruence closure of Q-APPABS and R-SWAP.

$$\frac{}{(\Lambda \alpha. A) B \equiv A [B / \alpha]} \text{Q-APPABS} \quad \frac{\ell_1 \neq \ell_2}{\ell_1; \ell_2; E \equiv_{\langle \rangle} \ell_2; \ell_1; E} \text{R-SWAP}$$

Rule Q-APPABS captures type application, following F_{ω} , and R-SWAP captures the insignificance of the order in effect rows, according to Koka's row typing.

Other value typing rules deal with generalization and instantiation of type variables and row variables.

$$\frac{\Gamma \vdash v : \forall \alpha. \sigma \quad \Gamma \vdash A}{\Gamma \vdash v : [A / \alpha] \sigma} \text{T-INST} \quad \frac{\Gamma, \alpha \vdash v : \sigma \quad \alpha \notin \Gamma}{\Gamma \vdash v : \forall \alpha. \sigma} \text{T-GEN}$$

$$\frac{\Gamma \vdash v : \forall \mu. \sigma \quad \Gamma \vdash E}{\Gamma \vdash v : [E / \mu] \sigma} \text{T-INSTEFF} \quad \frac{\Gamma, \mu \vdash v : \sigma \quad \mu \notin \Gamma}{\Gamma \vdash v : \forall \mu. \sigma} \text{T-GENEFF}$$

The definition of well-scopedness for types $\Gamma \vdash \sigma$ and effect rows $\Gamma \vdash E$ is straightforward (Appendix D.3).

Computation Typing Rule T-SC types scoped effects.

$$\frac{(\ell^{\mathbf{sc}} : A_{\mathbf{sc}} \rightarrow B_{\mathbf{sc}}) \in \Sigma \quad \Gamma \vdash v : A_{\mathbf{sc}} \quad \Gamma, y : B_{\mathbf{sc}} \vdash c_1 : B! \langle E \rangle \quad \Gamma, z : B \vdash c_2 : A! \langle E \rangle \quad \ell^{\mathbf{sc}} \in E}{\Gamma \vdash \mathbf{sc} \ell^{\mathbf{sc}} v (y. c_1) (z. c_2) : A! \langle E \rangle} \text{T-SC}$$

Looking up ℓ^{sc} in the signature context yields signature $A_{\text{sc}} \rightarrow B_{\text{sc}}$ where A_{sc} corresponds to the type of the effect's parameter v . Where B_{op} in the algebraic case refers to the *continuation's* argument, B_{sc} describes the *scoped computation's* argument. This leaves the scoped result type undescribed by the signature, but this freedom is exactly what we want. As for the effect rows, T-SC requires the rows of the scoped computation to match.

Handler Typing Rule T-H types handlers with a polymorphic type of the form $\forall \alpha. \alpha! \langle F \rangle \Rightarrow M \alpha! \langle E \rangle$. A handler consists of a **return**-clause, zero or more effect clauses, and a forwarding clause, which should all agree on their result type $M \alpha! \langle E \rangle$. Effect row E contains at least the labels of the algebraic and scoped effect clauses in the handler (computed by the *labels*-function).

$$\frac{\begin{array}{c} \langle F \rangle \equiv_{\langle \rangle} \langle \text{labels}(cls); E \rangle \quad \Gamma, \alpha \vdash \text{return } x \mapsto c_r : M \alpha! \langle E \rangle \\ \Gamma, \alpha \vdash cls : M \alpha! \langle E \rangle \quad \Gamma, \alpha \vdash \text{fwd } f \ p \ k \mapsto c_f : M \alpha! \langle E \rangle \end{array}}{\Gamma \vdash \text{handler } \{ \text{return } x \mapsto c_r, cls, \text{fwd } f \ p \ k \mapsto c_f \} : \forall \alpha. \alpha! \langle F \rangle \Rightarrow M \alpha! \langle E \rangle} \text{ T-H}$$

Rule T-OPRSC types scoped effect clauses.

$$\frac{\begin{array}{c} \Gamma \vdash cls : M A! \langle E \rangle \quad (\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \\ \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle E \rangle, k : \beta \rightarrow M A! \langle E \rangle \vdash c : M A! \langle E \rangle \end{array}}{\Gamma \vdash \text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, cls : M A! \langle E \rangle} \text{ T-OPRSC}$$

We use a type variable β for the scoped result type to allow the necessary freedom. Furthermore, computation c uses the *handled* subcomputations p and k , which makes type operator M occur both in the scoped result type and in the continuation's result type:

$$p : B_{\text{sc}} \rightarrow M \beta! \langle E \rangle \quad k : \beta \rightarrow M A! \langle E \rangle$$

This means that, even though our focus on mitigating the type mismatch between scoped computation and continuation so far has been on forwarding *unknown* scoped effects, the same applies when handling *known* scoped effects, where computation c accounts for this discrepancy.

Rule T-FWD types forwarding clauses.

$$\frac{\begin{array}{c} A_p = \alpha \rightarrow M \beta! \langle E \rangle \quad A'_p = \alpha \rightarrow \gamma! \langle E \rangle \\ A_k = \beta \rightarrow M A! \langle E \rangle \quad A'_k = \gamma \rightarrow \delta! \langle E \rangle \\ \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow \delta! \langle E \rangle \vdash c_f : M A! \langle E \rangle \end{array}}{\Gamma \vdash \text{fwd } f \ p \ k \mapsto c_f : M A! \langle E \rangle} \text{ T-FWD}$$

The forwarding clause abstracts over the effect to not make assumptions about which effect to expect: function f abstracts over possible scoped effect calls $\mathbf{sc} \ell^{\mathbf{sc}} v$; type variable α abstracts over possible argument types $B_{\mathbf{sc}}$ of the scoped computation; and type variable β abstracts over the scoped result type.

$$p : \alpha \rightarrow M \beta! \langle E \rangle \qquad k : \beta \rightarrow M A! \langle E \rangle$$

Notice the difference between the type of p and k and the arguments of function f : it expects as argument (p', k') a transformed version of the scoped computation and continuation so that they agree on the scoped result type. An intuitive solution would be to transform the scoped computation and continuation as follows to agree on type $M \beta$.

$$p' : \alpha \rightarrow M \beta! \langle E \rangle \qquad k' : M \beta \rightarrow M A! \langle E \rangle$$

However, in some situations we require a more general type γ . For example, when our type constructor M deals with state, we require uncurrying of the result type to apply k . A similar reasoning goes for the result type of k' and f . Consequently, the type of p' and k' are as follows:

$$p' : \alpha \rightarrow \gamma! \langle E \rangle \qquad k' : \gamma \rightarrow \delta! \langle E \rangle$$

Transforming the original scoped computation and continuation to agree on this type is the exact purpose of our explicit forwarding clauses.

6.3.4 Metatheory and Type Inference

Appendix D.4 contains a syntax-directed version of λ_{sc} , which we prove type safe, and serves as the specification of our type inference algorithm. The syntax-directed version is obtained by the following three transformations. Firstly, we remove rule T-EQV, which is used, for example, for changing the order of the labels in effect rows. As a consequence, in the syntax directed version we inline T-EQV wherever it is needed. Secondly, we remove the rules dealing with generalization and instantiation (T-INST, T-INSTEFF, T-GEN and T-GENEFF). Instead, whenever rules insist on some kind of polymorphism on a subderivation, we extend the environment with fresh type variables, and generalize over them locally, instead of via auxiliary rules. Thirdly, as dealing with higher-kinded polymorphism is orthogonal to our work (and programming languages like Haskell and OCaml have their solutions [175]), we avoid undecidable higher-order unification by annotating handlers with their type operator: **handler** _{M} $\{ \dots \}$.

The type-and-effect system of λ_{sc} is type safe, which we show by proving Subject Reduction and Progress (Appendix D.5).

Theorem 1 (Subject Reduction). *If $\Gamma \vdash c : \underline{C}$ and $c \rightsquigarrow c'$, then there exists a $\underline{C}'$ such that $\underline{C} \equiv \underline{C}'$ and $\Gamma \vdash c' : \underline{C}'$.*

Theorem 2 (Progress). *If $\cdot \vdash c : \underline{C}$, then either:*

- *there exists a computation c' such that $c \rightsquigarrow c'$, or*
- *c is in normal form, which means it is in one of the following forms:
 $c = \text{return } v$, $c = \text{op } \ell^{op} v (y . c')$, or $c = \text{sc } \ell^{sc} v (y . c_1) (z . c_2)$.*

Appendix D.6 contains an inference algorithm for λ_{sc} , based on the approach of Hindley-Milner [104] and Koka [91]. We prove this type inference algorithm sound and complete with respect to the syntax-directed version of λ_{sc} .

Chapter 7

Summary

This part of the thesis has proposed a generic framework for modeling higher-order effects. By designing such framework, we aim to retain the key properties of algebraic effects & handlers: their separation of concerns and their modular composition using a coproduct. Effects that are not algebraic, and which we call higher-order because of their internal computation, can be modeled using our framework. Modular composition of higher-order effects is challenging as it can no longer be done generically. Forwarding unknown effects to other handlers may be a challenge and depends on the specific (combination of) effects. This framework not only offers a novel denotational view on higher-order effects, it also puts them on formal footing. We show the free-forgetful adjunction that forms the foundation of this denotational semantics. Furthermore, we have formulated the syntax, operational semantics and type-and-effect system for a calculus with algebraic- and scoped effects & handlers as built-in primitives.

In what follows, we motivate our most impactful design choices.

Single Algebra: Concise and Consistent In Section 3.1 we have defined the fold-style recursion scheme for our higher-order free monad in terms of a single algebra. We have opted to reuse this algebra to interpret both the internal computation and the continuation for two reasons: conciseness and consistency.

First, reusing the algebras significantly reduces the handler's complexity. Second, having a consistent interpretation for the internal computation and continuation makes example programs more comprehensible and less verbose.

However, this design choice also implies that some examples of computational

effects are not supported. For example, Yang et al. [176] propose to abstract over evaluation strategies such as depth-first search, breadth-first search or depth-bounded search, employing a different evaluation strategy for the computation in scope as for the continuation.

The choice of a single algebra is thus a trade-off between being encompassing and being concise. We chose the latter.

Pointed Endofunctors Our fold-style interpreter (Section 3.1) works with a single generator $gen_1 :: a \rightarrow g\ b$. The other generator $gen_2 :: \forall x. x \rightarrow g\ x$ is implicitly provided by the fact that functor g is pointed (this is reflected in the type constraints). Two alternative recursion schemes are possible.

First, we could use two generators instead of a pointed endofunctor. This alternative makes the second generator explicit, dropping the requirement of g being pointed. Notice that g *does* have to be a functor.

```
fold :: (HFunctor k, Functor g)
  => (a -> g b) -> (forall x. x -> g x) -> (forall x. k g (g x) -> g x)
  -> (FreeH k a -> g b)
fold gen1 gen2 alg (VarH x)   = gen1 x
fold gen1 gen2 alg (OpH hop) = alg (hmap fold↘
                                     (fmap (fold gen1 gen2 alg) hop))
where fold↘ :: FreeH k ↘ g
      fold↘ (VarH x) = gen2 x
      fold↘ (OpH t)  = alg (hmap fold↘ (fmap fold↘ t))
```

Second, we could include the generator in the algebra by means of an identity functor. This alternative drops the requirement of functor g being pointed, and instead adds an identity functor in the algebra. Thus, the generator function for g is now included in its algebra.

```
type f ∘ g = forall x. f (g x)
fold :: (HFunctor k, Functor g)
  => (a -> g b) -> (forall x. (Id + (k g ∘ g)) x -> g x)
  -> (FreeH k a -> g b)
fold gen alg (VarH x)   = gen x
fold gen alg (OpH hop) = alg (Inr (hmap fold↘ (fmap (fold gen alg) hop)))
where fold↘ :: FreeH k ↘ g
      fold↘ (VarH x) = alg (Inl (Id x))
      fold↘ (OpH t)  = alg (Inr (hmap fold↘ (fmap fold↘ t)))
```

Backwards Compatibility of λ_{sc} Scoped effects require *every* handler in λ_{sc} to be polymorphic and equipped with an explicit forwarding clause. This breaks backwards compatibility: calculi that support only algebraic effects, such as Eff, miss an explicit forwarding clause for scoped operations and allow monomorphic handlers. This problem can be solved by including kinds and kind polymorphism: we add two kinds K_{op} and K_{sc} for effect types, such that $\Gamma \vdash E : K_{op}$ means effect type E only contains algebraic operations, and $\Gamma \vdash E : K_{sc}$ means effect type E may contain some scoped operations. Then, for handlers of type $A!\langle E \rangle \Rightarrow M A!\langle F \rangle$ that lack forwarding clauses, we add the condition $\Gamma \vdash E : K_{op}$ to their typing rules.

Related Work Monad transformers [52] are a well-known alternative for the algebraic effects approach in combination with the modularity achieved from “Datatypes à la Carte” [146]. Jaskelioff [75, 76] has investigated how to make monad transformers modular. Filinski [42] provides an alternative implementation in terms of layered monads. More recently, Schrijvers et al. [132] have described for which conditions the approach of algebraic effects & handlers is better or worse than monad transformers and how the one can be encoded as the other. Monad transformers appear more expressive, but algebraic effects & handlers are more modular.

In their work about scoped effects & handlers, Wu et al. [173] suggest a higher-order syntax for effect signatures, with explicit “weaving” (also sometimes called “forwarding”) of handlers: threading of the handler through the higher-order syntax, distributing it for every signature. Their design is made for scoped effects specifically, and, compared to our framework, requires extra verbosity (the weaving) in order to achieve the desired modularity.

Bach Poulsen and van der Rest [7] propose to *elaborate* higher-order effects, into algebraic effects using so-called *Hefty algebras*. They limit their set of higher-order effects to scoped effects and latent effects.

Part II

Automatic Differentiation

Preamble

This part is mostly based on the following paper:

- BIRTHE VAN DEN BERG, TOM SCHRIJVERS, JAMES MCKINNA, AND ALEXANDER VANDENBROUCKE. (2023). Forward- or Reverse-Mode Automatic Differentiation: What's the Difference? In: Science of Computer Programming, Volume 231, 103010, ISSN 0167-6423. <https://doi.org/10.1016/j.scico.2023.103010>

Section 12.1 and Section 12.2 contain parts from the following paper:

- TOM SCHRIJVERS, BIRTHE VAN DEN BERG, AND FABRIZIO RIGUZZI. (2023). Automatic Differentiation in Prolog. In Theory And Practice Of Logic Programming. Cambridge University Press (CUP). <https://doi.org/10.1017/S1471068423000145>

Section 12.4 contains parts from the following paper:

- BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2022). A Functional Account of Probabilistic Programming with Possible Worlds (Declarative Pearl). In: Hanus, M., Igarashi, A. (eds) *Functional and Logic Programming*. FLOPS 2022. Lecture Notes in Computer Science, vol 13215. Springer, Cham. https://doi.org/10.1007/978-3-030-99461-7_11

Abstract Automatic differentiation (AD) has been a topic of interest for researchers in many disciplines, with increased popularity since its application to machine learning and neural networks. Although many researchers appreciate and know how to apply AD, it remains a challenge to truly understand the underlying processes. From an algebraic point of view, however, AD appears surprisingly natural: it originates from the differentiation laws. In this work we use Algebra of Programming techniques to reason about different AD variants, leveraging Haskell to illustrate our observations. Our findings stem from *three algebraic abstractions*: (1) the notion of semimodule, (2) Nagata's construction of the idealization of a module, and (3) Kronecker's delta function, that together allow us to write a *single-line abstract definition* of AD. From this single-line definition, and by instantiating our algebraic structures in various ways, we derive different AD variants, that have the same *extensional* behavior, but different *intensional* properties, mainly in terms of (asymptotic) computational

complexity. We show the different variants equivalent by means of *Kronecker isomorphisms*, a further elaboration of our infrastructure which guarantees correctness by construction. With this framework in place, this paper seeks to make AD variants more comprehensible, taking an algebraic perspective on the matter.

Overview Chapter 8 shows how symbolic differentiation and evaluation of symbolic expressions can both be seen algebraically as unique semiring homomorphisms from the free semiring. In terms of code, they are both instances of the same fold-like recursion scheme. This algebraic view enables us to fuse the two homomorphisms to obtain basic forward-mode AD, and then to recover symbolic differentiation as an instance of that.

Chapter 9 abstracts the AD algorithm in terms of three algebraic abstractions, whose specification and (equational) properties are realized via Haskell type classes: *semimodules*, *Nagata numbers*, and Kronecker’s *delta* function. New, and correct-by-construction, AD variants, can be devised through Kronecker isomorphisms.

Chapter 10 shows instances of the abstract algorithm, starting with a form of forward-mode AD that computes the whole gradient vector of partial derivatives in one go. Further improvements entail a sparse vector representation, and ultimately some purely functional and imperative variants of reverse-mode AD. We successively improve the time complexity of scalar multiplication and gradient vector addition with representations based on homomorphisms, and finally, we switch to their equivalent mutable representation.

Chapter 11 demonstrates that our approach readily admits common, useful extensions: additional primitive functions, non-commutative semirings, direct computation with Nagata numbers to eliminate the overhead in building and evaluating symbolic expressions, higher derivatives, and **let**-sharing.

Chapter 12 elaborates on applications of the different AD variants, including two case studies on probability learning in the context of the Sum-Product Loop Language and possible-worlds semantics. Another application exploits matrix differentiation for square matrices with this framework.

The code accompanying this part can be found at <https://github.com/birthedvdb/Abstract-AD>.

Personal Contributions I contributed to all parts of the project, from first draft to finished paper, including software and novel research ideas. For the Prolog version of this framework, I contributed benchmarks of the different AD

versions and the implementation of a case study (SPLL). For the possible-worlds semantics, I contributed to all parts of the project.

Acknowledgements I would like to thank James McKinna and Alexander Vandenbroucke for the valuable discussions, fruitful ideas and, critical spirit, but, above all, their perseverance to bring this project to a successful conclusion.

Chapter 8

Background

This Chapter provides a basic executable definition of automatic differentiation (AD) based on a minimal setup. For readers who want a gentle introduction to the methodology used here, we refer to “Domain-Specific Languages of Mathematics” [74].

8.1 A Symbolic Expression Language

In order to focus on the essential algorithmic mechanics, we pare down to a minimal programming language in which functions can be expressed: symbolic expressions *Expr v* that capture polynomials over variables *v*, that vary independently¹

```
data Expr v = Var v | Zero | One
           | Plus (Expr v) (Expr v) | Times (Expr v) (Expr v)
```

For example, for univariate expressions we can define a type *X* with one variable. The symbolic representation of $x \times (x + 1)$ is captured in *example₁*.

```
data X = X
example1 :: Expr X
example1 = Times (Var X) (Plus (Var X) One)
```

¹In Section 11.5 we extend the language of expressions with a **let**-construct to permit variables that may be bound to complex subexpressions.

To interpret this expression language, we write a fold-style *eval* function. The simplest way to interpret our expressions is in terms of integers. The *evaluator* proceeds by structural recursion over the symbolic expressions to map them to their interpretation (integer value), given a mapping $var :: v \rightarrow Int$ for variables.

$$\begin{aligned}
 eval &:: (v \rightarrow Int) \rightarrow Expr \rightarrow Int \\
 eval\ var\ (Var\ x) &= var\ x \\
 eval\ var\ Zero &= 0 \\
 eval\ var\ One &= 1 \\
 eval\ var\ (Plus\ e_1\ e_2) &= eval\ var\ e_1 + eval\ var\ e_2 \\
 eval\ var\ (Times\ e_1\ e_2) &= eval\ var\ e_1 * eval\ var\ e_2
 \end{aligned}$$

This way we can evaluate $example_1$ for $X = 5$.

```
>>> eval (\X → 5) example1
30
```

We can be more generic than this. In fact, it is possible to assign an overloaded meaning to symbolic expressions by mapping them to an algebraic structure that captures integers and more. By means of that, we have the flexibility to interpret expressions in terms of both their plain value and different AD flavors.

8.2 Semirings

The algebraic structure to which we map our symbolic expressions is a (commutative) *semiring*.

Semirings are a structure in abstract algebra that have a notion of addition and multiplication, but not of negation or division. Sometimes, semirings are also called *rigs*, as they are rings without *negatives*. Well-known examples of semirings are

- Integers, with $+$ as addition and $*$ as multiplication;
- Sets, with union as addition and intersection as multiplication;
- Algebraic datatypes, with their sum as addition and product as multiplication;
- Programs, with choice as addition and sequencing as multiplication.

Semirings not only have their algebraic use, but also various applications in computer science problems. Tropical semirings², for instance, are used to compute the length of the shortest path in a graph [33]. Matrices over a semiring can be used for signal flow graph analysis [43].

8.2.1 Commutative Semirings

A commutative semiring is a semiring with commutative multiplication. An example of an algebraic structure that is a semiring but not commutative is that of square matrices, where $A \times B \neq B \times A$ (Section 12.3). In what follows, we impose a commutativity requirement, which is not essential for AD to work, but simplifies the presentation of our algebraic structures. Section 11.3 extends the approach to non-commutative semirings.

More formally, we define a commutative semiring as follows.

Definition 10 (Commutative Semiring). *A commutative semiring is a set, together with two binary operators $\oplus$ (“addition”) and $\otimes$ (“multiplication”), and two elements zero and one. These operators $\oplus$ and $\otimes$ are associative and commutative and have neutral elements zero and one, respectively. Moreover, $\otimes$ distributes over $\oplus$ and has zero as annihilator (Figure 8.1).*

In fact, a commutative semiring D can also be viewed as two commutative monoids: $(D, \oplus)$, a commutative monoid with *zero* as identity element, and $(D, \otimes)$, a commutative monoid with *one* as identity, both obeying the laws of Figure 8.1.

In Haskell, we capture the algebraic concept of a semiring in the *Semiring* type class, with instances for numeric types (e.g., *Int*, *Float*³) that respect the laws.

<pre>class Semiring d where zero :: d one :: d (⊕) :: d → d → d (⊗) :: d → d → d</pre>	<pre>instance Num d ⇒ Semiring d where zero = 0 one = 1 (⊕) = (+) (⊗) = (*)</pre>
---	--

There is one instance of semirings that is of special interest to us: that for symbolic expressions *Expr v*, quotiented by the laws. Quotienting by the laws means that we should consider two (even structurally different) expressions

²A tropical semiring consists of integers, together with ∞ , where addition is defined by the minimum, and multiplication is defined by addition.

³If we ignore the limitations of floating point arithmetic.

identity:	commutativity:
$zero \oplus x = x = x \oplus zero$	$x \oplus y = y \oplus x$
$one \otimes x = x = x \otimes one$	$x \otimes y = y \otimes x$
associativity:	distributivity:
$x \oplus (y \oplus z) = (x \oplus y) \oplus z$	$x \otimes (y \oplus z) = (x \otimes y) \oplus (x \otimes z)$
$x \otimes (y \otimes z) = (x \otimes y) \otimes z$	$(x \oplus y) \otimes z = (x \otimes z) \oplus (y \otimes z)$
annihilator:	
$zero \otimes x = zero = x \otimes zero$	

Figure 8.1: The commutative semiring laws.

equal if the laws say so (e.g., *Plus Zero (Var X) ≡ Var X*). What makes this instance special is that it is the *free* semiring instance: it is the minimally constrained⁴ structure that adheres to the semiring laws (Figure 8.1).

instance *Semiring* (*Expr v*) **where**

zero = *Zero*

one = *One*

$(\oplus)$ = *Plus*

$(\otimes)$ = *Times*

The free semiring gives rise to a fold-style *eval* function. This evaluator proceeds by structural recursion over the symbolic expressions to map them to their interpretation in a semiring *d*, given a mapping *var* :: *v* → *d* for variables.

```

eval :: Semiring d => (v -> d) -> Expr v -> d
eval var (Var x)      = var x
eval var Zero         = zero
eval var One          = one
eval var (Plus e1 e2) = eval var e1 ⊕ eval var e2
eval var (Times e1 e2) = eval var e1 ⊗ eval var e2

```

This way we can evaluate *example₁* for *X* = 5 with the *Int* semiring.

```

>>> eval (λX → 5) example1
30

```

⁴No more terms are equal than those required by the laws.

8.2.2 Semiring Homomorphisms

In the remainder of this work, we encounter different kinds of semirings, which we want to be able to relate, using semiring homomorphisms. Informally, a homomorphism is a function that preserves algebraic structure; in case of a semiring homomorphism, the structure of the commutative semiring is preserved.

Definition 11 (Semiring homomorphism). *A (semiring) homomorphism between two semirings D_1 and D_2 is a function $h :: D_1 \rightarrow D_2$ that preserves the semiring structure:*

$$\begin{aligned} h \text{ zero}_{D_1} &= \text{zero}_{D_2} & h (x \oplus_{D_1} y) &= h x \oplus_{D_2} h y \\ h \text{ one}_{D_1} &= \text{one}_{D_2} & h (x \otimes_{D_1} y) &= h x \otimes_{D_2} h y \end{aligned}$$

Semiring homomorphisms compose: given homomorphisms $h_1 :: D_1 \rightarrow D_2$ and $h_2 :: D_2 \rightarrow D_3$, their composition $h_2 \cdot h_1 :: D_1 \rightarrow D_3$ is also a homomorphism; and the expected laws hold. Furthermore, the identity function $\text{id} :: D \rightarrow D$ is trivially a homomorphism. The function eval var is a semiring homomorphism for any choice of var . In fact, it is the *unique* semiring homomorphism h such that $h (\text{Var } v) = \text{var } v$. From this uniqueness follow two useful properties:

Property 1 (Fusion). *The fusion property allows us to incorporate any homomorphism h into evaluation: $h . \text{eval var} = \text{eval } (h . \text{var})$.*

Property 2 (Reflection). *The reflection property observes that evaluation with Var is the identity: $\text{eval Var} = \text{id}$.*

8.3 Differentiation

Differentiation is the procedure that calculates the derivative of an expression, used, for instance, in optimization problems. Three popular methods exist to compute the derivative.

- Symbolic differentiation transforms a symbolic expression into its symbolic derivative, using textbook differentiation rules such as the sum rule, the product rule, and the chain rule. Unfortunately, it suffers from the so-called expression swell, which blows up the symbolic expression exponentially in the multiplication case⁵, leaving a larger expression:

$$d (f \times g) = df \times g + f \times dg$$

⁵We use Leibniz's notation.

$$\begin{aligned}
\frac{\partial n}{\partial x} &= 0 & \frac{\partial e_1 + e_2}{\partial x} &= \frac{\partial e_1}{\partial x} + \frac{\partial e_2}{\partial x} \\
\frac{\partial y}{\partial x} &= \begin{cases} 1 & (x = y) \\ 0 & (x \neq y) \end{cases} & \frac{\partial e_1 \times e_2}{\partial x} &= e_2 \times \frac{\partial e_1}{\partial x} + e_1 \times \frac{\partial e_2}{\partial x}
\end{aligned}$$

Figure 8.2: Symbolic differentiation rules.

This inefficiency gets even worse for higher derivatives.

- Numerical differentiation estimates the differentiated value of an expression at a point, using the definition:

$$df(x) = \lim_{\varepsilon \rightarrow 0} \left(\frac{f(x+\varepsilon) - f(x)}{\varepsilon} \right)$$

This method computes an *approximation* and may therefore suffer from accuracy issues due to both truncation errors and rounding errors. For higher derivatives, these errors increase.

- Automatic differentiation computes the (partial) derivatives of an expression at a point, combining differentiation and evaluation, and repeatedly applying the chain rule:

$$d(f \circ g) = df \circ g \times dg$$

Automatic differentiation (AD) comes in two flavors: forward-mode and reverse-mode, the latter of which is also referred to as backpropagation.

8.3.1 Symbolic Differentiation

The above evaluator (*eval*) allows us to do symbolic differentiation. We capture its rules (Figure 8.2) in the *derive* function, using the *Semiring* instance of *Expr v*, and exploiting commutativity in the multiplication case.

$$\begin{aligned}
\text{derive} &:: \text{Eq } v \Rightarrow v \rightarrow \text{Expr } v \rightarrow \text{Expr } v \\
\text{derive } x \text{ (Var } y) &= \text{if } x \equiv y \text{ then one else zero} \\
\text{derive } x \text{ Zero} &= \text{zero} \\
\text{derive } x \text{ One} &= \text{zero} \\
\text{derive } x \text{ (Plus } e_1 \ e_2) &= \text{derive } x \ e_1 \oplus \text{derive } x \ e_2 \\
\text{derive } x \text{ (Times } e_1 \ e_2) &= (e_2 \otimes \text{derive } x \ e_1) \oplus (e_1 \otimes \text{derive } x \ e_2)
\end{aligned}$$

These do not fit *eval*'s fold-style recursion scheme on the nose because, in the case for *Times*, e_1 and e_2 appear outside recursive calls (*gray*). Using the so-called “banana-split”-property [73, 103, 102] we can turn this function into the proper shape: we make it return a tuple $(Expr\ v, Expr\ v)$ of the original expression and the symbolic derivative.

$$derive'\ x\ e = (e, derive\ x\ e)$$

We thus obtain the following structurally recursive definition for *derive'*, from which we may then recover *derive* by projection on the second component:

$$\begin{aligned} derive' &:: Eq\ v \Rightarrow v \rightarrow Expr\ v \rightarrow (Expr\ v, Expr\ v) \\ derive'\ x\ (Var\ y) &= (Var\ y, \text{if } x \equiv y \text{ then one else zero}) \\ derive'\ x\ Zero &= (zero, zero) \\ derive'\ x\ One &= (one, zero) \\ derive'\ x\ (Plus\ e_1\ e_2) &= \text{let } (e'_1, de_1) = derive'\ x\ e_1 \\ &\quad (e'_2, de_2) = derive'\ x\ e_2 \\ &\quad \text{in } (e'_1 \oplus e'_2, de_1 \oplus de_2) \\ derive'\ x\ (Times\ e_1\ e_2) &= \text{let } (e'_1, de_1) = derive'\ x\ e_1 \\ &\quad (e'_2, de_2) = derive'\ x\ e_2 \\ &\quad \text{in } (e'_1 \otimes e'_2, (e'_2 \otimes de_1) \oplus (e'_1 \otimes de_2)) \end{aligned}$$

8.3.2 Dual Numbers

Before we write the functionality of *derive'* in terms of *eval*, we generalize $(Expr\ v, Expr\ v)$ to tuples whose components are drawn from an arbitrary semiring. In the AD literature, these are known as *dual numbers*, a notion that predates AD and was first conceived by Clifford [26].

```
data Dual d = D { priD :: d,
                  tanD :: d }
instance Functor Dual where
  fmap :: (a → b) → Dual a → Dual b
  fmap h (D f df) = D (h f) (h df)
```

Dual numbers are functorial; `fmap f` transforms both parameters using *f*. The two components of a dual number $D\ x\ dx$ are respectively known as the *primal* x , which represents an expression's value at a particular point, and the *tangent* dx , its derivative at that point. For a semiring d , the type $Dual\ d$ also admits a semiring structure, where the definition of tangent precisely follows the corresponding cases of *derive*.

```

instance Semiring d  $\Rightarrow$  Semiring (Dual d) where
  zero           = D zero   zero
  one            = D one    zero
  (D f df)  $\oplus$  (D g dg) = D (f  $\oplus$  g) (df  $\oplus$  dg)
  (D f df)  $\otimes$  (D g dg) = D (f  $\otimes$  g) ((g  $\otimes$  df)  $\oplus$  (f  $\otimes$  dg))

```

Now, symbolic differentiation may be seen as another instance of *evaluation*, but into the semiring of dual numbers over symbolic expressions.

```

symbolic :: Eq v  $\Rightarrow$  v  $\rightarrow$  Expr v  $\rightarrow$  Dual (Expr v)
symbolic x = eval gen where gen y = D (Var y) ( $\delta_x$  y)
                       $\delta_x$  y = if x  $\equiv$  y then one else zero

```

8.3.3 Forward-Mode Automatic Differentiation

Automatic differentiation (AD) is the umbrella term for algorithms that programmatically compute both the value *and* the derivative of an expression *at a point* [86, 147, 67]. Thus, if we combine symbolic differentiation with subsequent evaluation into a semiring d , we obtain a specification for forward-mode AD:

```

forwardAD :: (Eq v, Semiring d)  $\Rightarrow$  (v  $\rightarrow$  d)  $\rightarrow$  v  $\rightarrow$  Expr v  $\rightarrow$  Dual d
forwardAD var x = fmap (eval var) . symbolic x

```

We can make this specification more efficient by exploiting *eval*'s fusion property.

```

fmap (eval var) . symbolic x
= {-definition of symbolic -}
  fmap (eval var) . eval gen where gen y = D (Var y) ( $\delta_x$  y)
                       $\delta_x$  y = if x  $\equiv$  y then one else zero
= {-eval fusion (Property 1) -}
  eval (fmap (eval var) . gen) where gen y = D (Var y) ( $\delta_x$  y)
                       $\delta_x$  y = if x  $\equiv$  y then one else zero

```

After inlining `fmap (eval var)` into `gen`, we get the classic definition of forward-mode AD, which calls *eval* only once. Differences compared with *symbolic* are highlighted in gray.

```

forwardAD :: (Eq v, Semiring d)  $\Rightarrow$  (v  $\rightarrow$  d)  $\rightarrow$  Expr v  $\rightarrow$  v  $\rightarrow$  Dual d
forwardAD var e x = eval gen e where gen y = D (var y) ( $\delta_x$  y)
                       $\delta_x$  y = if x  $\equiv$  y then one else zero

```

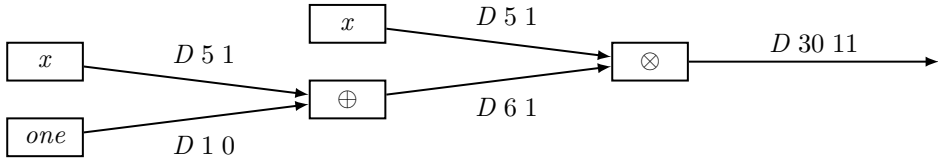



Figure 8.3: Running forward-mode AD on $example_1$ at $X = 5$.

For example, the derivative of $example_1$ at the point $X = 5$ is 11; the process computing the result is depicted in Figure 8.3.

```
>>> forwardAD (λX → 5) example1 X
D 30 11
```

8.3.4 Symbolic AD as Forward AD

The only essential difference between symbolic and automatic differentiation is that the former evaluates into the symbolic $Expr\ v$ semiring, while the latter uses a numeric semiring d . In fact, we can also have $forwardAD$ use the symbolic semiring, interpreting variables with Var .

```
forwardAD Var
= {-AD specification (Specification 8.3.3) -}
  fmap (eval Var) . symbolic
= {-reflection property (Property 2) -}
  fmap id. . symbolic
= {-functor law and id. as unit of composition -}
  symbolic
```

In summary, $symbolic$ is a special case of $forwardAD$:

$$symbolic = forwardAD\ Var$$

Notice that we use the *Leibnizian* notation for understanding both AD and symbolic differentiation as an algebraic operation on a language $Expr\ v$ of (concrete) symbolic expressions. This contrasts with the *Newtonian* view of Elliott [38, 36, 37] and others, who think of AD as an algebraic operation on spaces of (abstract) functions. Baydin et al. even assert [11, Section 2.2, under

What AD Is Not] that AD is *not symbolic differentiation*, but echoes their observation, following Griewank [58], that it has a *two-sided nature that is partly symbolic and partly numerical*.

Chapter 9

Abstract Automatic Differentiation

This Chapter explains how to express the two previous differentiation functions (symbolic differentiation and classic forward-mode AD) as well as several more variants to come later, as the same single-line function. To accomplish this, we have identified three fundamental algebraic structures: semimodules, generalized dual numbers, and the Kronecker delta function. Furthermore, we introduce Kronecker isomorphisms to relate different AD variants.

$e \diamond \epsilon$	$= e$	<i>zero</i>	$\bullet e = \epsilon$
$e \diamond (e' \diamond e'')$	$= (e \diamond e') \diamond e''$		$(d \oplus d') \bullet e = (d \bullet e) \diamond (d' \bullet e)$
$e \diamond e'$	$= e' \diamond e$	<i>one</i>	$\bullet e = e$
			$(d \otimes d') \bullet e = d \bullet (d' \bullet e)$
$d \bullet \epsilon$	$= \epsilon$		
$d \bullet (e \diamond e')$	$= (d \bullet e) \diamond (d \bullet e')$		

Figure 9.1: The laws for a D -module E over a semiring D , where E is a commutative monoid.

9.1 Semimodules

Our first abstraction generalizes the notion of gradient ∇e of an expression e .

$$\nabla e = \begin{bmatrix} \frac{\partial e}{\partial x_1} \\ \vdots \\ \frac{\partial e}{\partial x_n} \end{bmatrix}$$

By analogy with vector spaces (over a field) or modules (over a ring), we consider d -modules over a semiring d , also called *semimodules* [56].

Definition 12 (Semimodule). *Given a semiring D , a D -module E is defined as a commutative monoid E , together with scalar multiplication $(\bullet): D \rightarrow E \rightarrow E$, which distributes over the additive structure of E . This means that E should have a commutative, associative binary operator $(\diamond)$ with neutral element ϵ . Figure 9.1 gives E 's monoid laws and the laws that govern the interaction between the monoid structure of E and the scalar multiplication.*

In Haskell, we capture the concept of a semimodule in the type class *Module*,¹ on top of the *Monoid* type class.

<pre>class Monoid e where ϵ :: e ($\diamond$) :: e $\rightarrow$ e $\rightarrow$ e</pre>	<pre>class (Semiring d, Monoid e) $\Rightarrow$ Module d e e $\rightarrow$ d where ($\bullet$) :: d $\rightarrow$ e $\rightarrow$ e</pre>
--	--

Every semiring d may be viewed trivially as a d -module, by taking $(\bullet) = (\otimes)$. A well-known example that is not itself a semiring is the vector space $\mathbb{R}^3$; it forms a monoid with vector addition and the zero vector $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$, and an $\mathbb{R}$ -module with the usual scalar multiplication.

9.2 Nagata Numbers

The second abstraction generalizes dual numbers of Section 8.3.2 to two components d and e that have *distinct* types, reflecting the idea that the value of an expression (its primal), and that of its derivative (tangent), conceptually live in different spaces. We exploit precisely the generalization that permits primal and tangent *to vary independently*. This concept first appeared in the work of Nagata [108] in the early 1950s, under the name *idealization of a module* [3].

¹We use the functional dependency $e \rightarrow d$ to anchor a d -module e to its underlying semiring d .

However, the application of Nagata's construction to the study of AD appears to be novel. Most works in the AD literature use the standard homogenous dual numbers. Some, like those of Krawiec et al. [86] and Smeding and Vákár [142], do feature various heterogeneous dual numbers, but they do not observe that these are instances of a general structure.

Nagata originally defined his idealization of a module over a ring. Our weakening to consider only *semiring* structure is just cosmetic, and the extension of our methods to consider a (commutative) *ring* d of coefficients is straightforward (Section 11.2).

We call the type $d \ltimes e$ *Nagata numbers*. The type constructor $d \ltimes \cdot$ is functorial in its second parameter and *bifunctorial* in both parameters (see Appendix E).

```
data  $d \ltimes e = N \{ pri^N :: d,$            instance Functor ( $d \ltimes \cdot$ ) where
                     $tan^N :: e \}$            fmap  $h (N f df) = N f (h df)$ 
```

Nagata proved the following fundamental theorem.

Theorem 3 (Nagata's theorem). *Given a d -semimodule e , then $d \ltimes e$ admits a semiring structure.*

The Haskell type class instance below witnesses this theorem.

```
instance Module  $d e \Rightarrow$  Semiring ( $d \ltimes e$ ) where
    zero           =  $N \ zero \quad \epsilon$ 
    one            =  $N \ one \quad \epsilon$ 
    ( $N f df$ )  $\oplus$  ( $N g dg$ ) =  $N (f \oplus g) (df \diamond dg)$ 
    ( $N f df$ )  $\otimes$  ( $N g dg$ ) =  $N (f \otimes g) ((f \bullet dg) \diamond (g \bullet df))$ 
```

9.3 Kronecker's Delta

The final piece of our jigsaw comes from the observation that the function $\delta_x y = \mathbf{if} \ x \equiv y \ \mathbf{then} \ one \ \mathbf{else} \ zero$ is the Kronecker delta function.

$$\delta_x(y) = \begin{cases} 1 & x = y \\ 0 & otherwise \end{cases}$$

This gives rise to our third fundamental algebraic abstraction. In Haskell, we capture a generalized notion of this concept in the *Kronecker* type class.

```
class Module  $d\ e \Rightarrow$  Kronecker  $v\ d\ e$  where
  delta ::  $v \rightarrow e$ 
```

Notice that, at this point, no extra laws are imposed on *Kronecker* $v\ d\ e$; later, we will use isomorphism conditions (Definition 14) to constrain different AD variants. The idea is that *delta* x does not compute $\delta_x(y)$ for just a single variable y , but for *all* variables at once. This results in a value of a d -semimodule e where only the x -component is *one*, and all others are *zero*. A helpful analogy may be to think of v as the type of basis vectors of e . Then, *delta* x is the embedding of basis vector x into e . Continuing Section 9.1’s example of the $\mathbb{R}$ -semimodule $\mathbb{R}^3$, there are usually three variables x, y, z such that

$$\text{delta } x = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{delta } y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{delta } z = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

In the AD literature, these basis vectors, where one entry is *one* and all the other are *zero*, are often referred to as *one-hot vectors*.

9.4 Automatic Differentiation as a One-Liner

With the three algebraic structures in place, we present our abstract definition of differentiation.

```
abstractD :: Kronecker  $v\ d\ e \Rightarrow (v \rightarrow d) \rightarrow \text{Expr } v \rightarrow d \ltimes e$ 
abstractD var = eval gen where gen  $x = N\ (\text{var } x)\ (\text{delta } x)$ 
```

This definition captures the essence of differentiation in a single line!

We show how, by choosing an appropriate type e , this abstract definition can be instantiated to recover different forms of AD. Kronecker isomorphisms relate these different forms.

9.5 Kronecker Isomorphisms

A key concern when devising new AD variants is their correctness. Using *Kronecker* isomorphisms, we can show their functional equivalence to an

established AD baseline, designing them correct by construction.

Definition 13 (Kronecker Homomorphism). *Given a type of variables V and a semiring D , with types E_1 and E_2 such that $\text{Kronecker } V \ D \ E_1$ and $\text{Kronecker } V \ D \ E_2$ hold (and thus also $\text{Monoid } E_1$, $\text{Monoid } E_2$, $\text{Module } D \ E_1$ and $\text{Module } D \ E_2$), then a Kronecker homomorphism is a function $h :: E_1 \rightarrow E_2$ that preserves*

1. the monoid structure:

$$\begin{aligned} h \ \epsilon_{E_1} &= \epsilon_{E_2} \\ h \ (x \diamond_{E_1} y) &= (h \ x) \diamond_{E_2} (h \ y) \end{aligned}$$

2. the semimodule structure:

$$h \ (d \bullet_{E_1} m) = d \bullet_{E_2} (h \ m)$$

3. and the kronecker structure:

$$h \ (\text{delta}_{E_1} \ v) = \text{delta}_{E_2} \ v$$

Definition 14 (Kronecker Isomorphism). *A Kronecker isomorphism is a pair of Kronecker homomorphisms $h :: E_1 \rightarrow E_2$ and $h^{-1} :: E_2 \rightarrow E_1$, that are inverses: $h \cdot h^{-1} = \text{id.}$ and $h^{-1} \cdot h = \text{id.}$*

Thus, when coming up with a new tangent type E' , we also want to find a Kronecker isomorphism $h :: E \rightarrow E'$ to establish its correctness, where E is a known tangent type for AD. This isomorphism is a function pair consisting of a representation function rep and its inverse abs (abstraction), following the terminology of data representation [53, 66]. To streamline our approach, we capture this isomorphism in a type class *CorrectAD*.

```
class Kronecker v d e'  $\Rightarrow$  CorrectAD v d e' where
  rep :: Kronecker v d e  $\Rightarrow$  e  $\rightarrow$  e'
  abs :: Kronecker v d e  $\Rightarrow$  e'  $\rightarrow$  e
```

With the *free* theorem [166] that follows from the parametricity of *abstractD* [126] we can then relate the new instance of the algorithm to the baseline.

Theorem 4 (Correct AD). *Given a CorrectAD $V D E'$ instance, we have that*

$$\begin{aligned} \text{fmap } \text{abs } (\text{abstractD}_{E'} \text{ var } e) &= \text{abstractD}_E \text{ var } e \\ \text{abstractD}_{E'} \text{ var } e &= \text{fmap } \text{rep } (\text{abstractD}_E \text{ var } e) \end{aligned}$$

Given a *Kronecker* $V D E$ structure and a newly desired representation E' , we do not have to devise the *Kronecker* $V D E'$ structure and the Kronecker isomorphism out of the blue. Instead, following the method outlined by Elliott [35], we can constructively derive both from a plain invertible function $h :: E \rightarrow E'$. Indeed, we can create the instances for E' in terms of the instances of E and of h .

instance Monoid E' where

$$\epsilon = h \epsilon$$

$$x \diamond y = h (h^{-1} x \diamond h^{-1} y)$$

instance Module $D E'$ where

$$d \bullet x = h (d \bullet h^{-1} x)$$

instance Kronecker $V D E'$ where

$$\text{delta } v = h (\text{delta } v)$$

instance CorrectAD $V D E'$ where

$$\text{rep} = h$$

$$\text{abs} = h^{-1}$$

By construction, these instances both satisfy all the necessary laws, and ensure that h is a Kronecker isomorphism. For example, we can verify that $x \diamond \epsilon = x$ and that $h x \diamond h y = h (x \diamond y)$ as follows:

$$\begin{aligned} & x \diamond \epsilon \\ = & \{-\text{definitions of } \epsilon \text{ and } \diamond -\} \\ & h (h^{-1} x \diamond h^{-1} (h \epsilon)) \\ = & \{-\text{isomorphism } -\} \\ & h (h^{-1} x \diamond \epsilon) \\ = & \{-\text{monoid identity law } -\} \\ & h (h^{-1} x) \\ = & \{-\text{isomorphism } -\} \\ & x \end{aligned}$$

$$\begin{aligned} & h x \diamond h y \\ = & \{-\text{definition of } \diamond -\} \\ & h (h^{-1} (h x) \diamond h^{-1} (h y)) \\ = & \{-\text{isomorphism } -\} \\ & h (x \diamond y) \end{aligned}$$

Verifying the other laws and homomorphism properties proceeds similarly.

In short, each time we devise a new AD variant, we come up with a new representation E' and an invertible function $h :: E \rightarrow E'$. The rest of the infrastructure then follows mechanically, except for the fact that we can simplify

the definitions, sometimes exploiting further equations, for the sake of optimizing the (algorithmic properties of the) representation.

The invertible function ensures, through the constructed Kronecker isomorphism, that the extensional behavior remains the same, and thus guarantees correctness. At the same time, a well-chosen alternative representation may improve intensional properties like asymptotic runtime.

Chapter 10

Forward-Mode and Reverse-Mode Automatic Differentiation

This Chapter shows how forward-mode and reverse-mode automatic differentiation arise from one and the same definition of abstract differentiation (Chapter 9). Successive refinements improve the intensional behavior of these AD variants, mainly in terms of their (asymptotic) computational complexity.

10.1 Forward-Mode Automatic Differentiation

This Section reformulates forward-mode automatic differentiation (from Section 8.3.3) in terms of the generic definition *abstractD*. By moving from the dense function space to sparse maps, we exploit the fact that many partial derivatives are zero.

10.1.1 Dense Function Space

We recover *forwardAD*—the classical forward-mode AD implementation given in Section 8.3.3—as an instance of *abstractD*, by using the standard function

space $e = v \rightarrow d$. We call this space *dense*¹ to contrast with the *sparse* function space coming up in Section 10.1.2, by analogy with dense/sparse polynomial representations in computer algebra [165].

type *Dense* $v\ d = v \rightarrow d$

Informally, we can think of this structure as a functional representation of a gradient vector, required by least square fitting, gradient descent and many other applications. Just like a gradient vector, it contains one d -value per variable of type v . The actions of its monoid and d -module instances are given pointwise, using the d -as- d -module construction of Section 9.1.

instance *Semiring* $d \Rightarrow \text{Monoid } (\text{Dense } v\ d)$ **where**

$\epsilon = \lambda v \rightarrow \text{zero}$

$f_1 \diamond f_2 = \lambda v \rightarrow f_1\ v \oplus f_2\ v$

instance *Semiring* $d \Rightarrow \text{Module } d\ (\text{Dense } v\ d)$ **where**

$d \bullet f = \lambda v \rightarrow d \bullet (f\ v) = \lambda v \rightarrow d \otimes (f\ v)$

The Kronecker delta function for dense functions is the textbook one:

instance $(Eq\ v, \text{Semiring } d) \Rightarrow \text{Kronecker } v\ d\ (\text{Dense } v\ d)$ **where**

$\text{delta } v = \lambda w \rightarrow \text{if } v \equiv w \text{ then one else zero}$

Using this *Kronecker* instance and generalizing dual numbers into $d \ltimes (\text{Dense } v\ d)$, we obtain the following definition for forward-mode AD.

$\text{fwdAD}_{\text{dense}} :: (Eq\ v, \text{Semiring } d) \Rightarrow (v \rightarrow d) \rightarrow \text{Expr } v \rightarrow d \ltimes (\text{Dense } v\ d)$
 $\text{fwdAD}_{\text{dense}} = \text{abstractD}$

This is *forwardAD* with its parameters re-arranged and *Dual* d generalized:

$\text{pri}^D (\text{forwardAD } \text{var } x\ e) = \text{pri}^N (\text{fwdAD}_{\text{dense}} \text{var } e)\ x$
 $\text{tan}^D (\text{forwardAD } \text{var } x\ e) = \text{tan}^N (\text{fwdAD}_{\text{dense}} \text{var } e)\ x$

The essential difference between *forwardAD* and $\text{fwdAD}_{\text{dense}}$ is intensional. In moving from computing a value in $v \rightarrow (d \ltimes d)$ to one in $d \ltimes (v \rightarrow d)$, we no

¹To avoid clutter, we present *Dense* and later types as **type** synonyms in the text, but our actual implementation uses **newtypes** for unambiguous type class resolution.

longer compute both the primal and tangent for each individual v , but instead only compute the primal *once* for the whole gradient. In other words, we share the computation of the primal among all partial derivatives.

For instance, *example₂* represents expression $(x \times y) + x + 1$ in variables x and y using our expression language. We compute its gradient at the point $(5, 3)$:

```
>>> let { var X = 5; var Y = 3; e = tanN (fwdADdense var example2) }
      in (e X, e Y)
(4, 5)
```

We consider the dense function space as the base case for our Kronecker isomorphisms. In practice, this means that, when coming up with a tangent type e for a different AD variant, we define the *rep* and *abs* functions with respect to *Dense v d*. The *CorrectAD v d e* type class changes to the following:

```
class Kronecker v d e ⇒ CorrectAD v d e where
  rep :: Dense v d → e
  abs :: e → Dense v d
```

10.1.2 Sparse Maps

We switch from the dense function space representation to that of sparse maps. This exploits the fact that many partial derivatives of sub-expressions are zero. Indeed, if a variable x does not occur in an expression e , the partial derivative $\frac{\partial e}{\partial x}$ is zero. Notably, in the base case, where the expression is just a variable, all partial derivatives but one are by definition zero. Thus, whereas the dense function space consists of functions returning a value for *every* variable, the sparse function space consists of (representations of) those functions only where their values are *non-zero*. We avoid explicit representations of zeros—as well as explicit operations with them—with the type of finite key-value maps² *Map v d* instead of a specialized dictionary representation [139].

```
type Sparse v d = Map v d
```

The omitted values in the map are *zero*. This way we recover the *Dense* representation from the sparse one by looking up the variables in the map and defaulting to *zero* when a variable is not present.

²We borrow map-specific features and functions from Haskell’s *Data.Map* library and refer to Appendix G for an explanation of these functions.

```

rep→    :: (Ord v, Bounded v, Enum v, Semiring d, Eq d)
          ⇒ Dense v d → Sparse v d
rep→ f   = fromList [(v, f v) | v ← [minBound..maxBound], f v ≠ zero]
abs→    :: (Ord v, Semiring d) ⇒ Sparse v d → Dense v d
abs→ m   = λv → findWithDefault zero v m

```

Observe that $abs_{\rightarrow}$ and $rep_{\rightarrow}$ are inverse functions under the constraints (provided that for the equality of maps it does not matter whether zero entries are explicit or implicit).

With the new $Sparse\ v\ d$ representation and the two inverse functions we can derive the *Monoid*, *Module*, *Kronecker* and *CorrectAD* instances. This results in the all-zero map being empty. Addition of maps is performed key-wise, but avoided if either or both maps have an implicit zero at a given (variable) key.

```

instance (Ord v, Semiring d) ⇒ Monoid (Sparse v d) where
  ε   = empty
  (◇) = unionWith (⊕)

```

Scalar multiplication is also key-wise. As $d \otimes zero = zero$ (Figure 8.1), we can effortlessly preserve the implicit zero entries.

```

instance (Ord v, Semiring d) ⇒ Module d (Sparse v d) where
  d • m = fmap (d ⊗) m

```

The only non-zero entry for δx is that for x .

```

instance (Ord v, Semiring d) ⇒ Kronecker v d (Sparse v d) where
  delta x = singleton x one

```

Moreover, and crucially in what follows, we observe the following relationship between this *Kronecker* structure and its *Module* and *Monoid* instances:

$$d \bullet \delta v = \text{singleton } v\ d \quad (10.1)$$

$$e \diamond (d \bullet \delta v) = \text{insertWith } (\diamond) v\ d\ e \quad (10.2)$$

Lastly, using the $rep_{\rightarrow}$ and $abs_{\rightarrow}$ isomorphisms and starting from the $Dense\ v\ d$ base case, the *CorrectAD* instance is straightforward.

```

instance (Ord v, Bounded v, Enum v, Semiring d, Eq d)
  ⇒ CorrectAD v d (Sparse v d) where
    rep = rep→
    abs = abs→
    
```

In summary, all that the sparse gradient computation requires, in comparison to the dense computation, is the switch from $d \ltimes (Dense\ v\ d)$ to $d \ltimes (Sparse\ v\ d)$.

```

fwdADsparse :: (Ord v, Semiring d) ⇒ (v → d) → Expr v → d ⋈ Sparse v d
fwdADsparse = abstractD
    
```

Theorem 4 now holds by construction. We can verify this correspondence by revisiting *example₂* at the point (5, 3).

```

>>> let { var X = 5; var Y = 3 } in tanN (fwdADsparse var example2)
{ X ↦ 4, Y ↦ 5 }
    
```

10.2 Reverse-Mode Automatic Differentiation

For an expression with N nodes and V variables, forward-mode automatic differentiation has an $\mathcal{O}(N \times V)$ time complexity: each node is visited once and the most costly operations (scalar multiplication ($\bullet$) and addition ($\diamond$)) are both linear in V . For a small number of variables, forward-mode is a perfectly suited solution, but for a large number of variables, the reverse-mode variant of AD appears more efficient, but also more sophisticated.

Motivated by the quest for improved efficiency, we show how to obtain this reverse-mode AD by means of successively optimizing the essential functions for abstract differentiation with the underlying Nagata numbers semiring: scalar multiplication ($\bullet$), vector addition ($\diamond$) and Kronecker’s δ . We do so by refining the above sparse representation. This way, we become a declarative version of reverse-mode AD, with an $\mathcal{O}(N \times \log V)$ time complexity, and its imperative counterpart, at $\mathcal{O}(N + V)$.

10.2.1 Semimodule Homomorphisms

We start with another function-based representation. Instead of the d -module e , we consider the space of d -module homomorphisms from d to e . It is a

standard result in commutative algebra [5, p.18]³ that these two structures are isomorphic, considered as d -modules. In terms of the *underlying* Haskell types, this amounts to taking the plain function space.

type $d \multimap e = d \rightarrow e$

But we intend this type $d \multimap e$ to represent only the d -semimodule homomorphisms. This is the case for *linear* functions from d to e . A function $f :: d \rightarrow e$ is linear⁴ iff it has the multiplicative *homogeneity* property:

$$f(x \otimes y) = x \bullet f y$$

As the Haskell type $d \rightarrow e$ also includes non-linear functions, we have to make disciplined use of $d \multimap e$ and be careful to avoid the non-linear ones. Intuitively, we can think of $d \multimap e$ as augmenting e with an accumulator for a scalar multiplier of type d . By supplying the scalar *one*, we recover the original semimodule.

$$\begin{array}{ll} \text{rep}_\multimap :: \text{Module } d \ e \Rightarrow e \rightarrow (d \multimap e) & \text{abs}_\multimap :: \text{Module } d \ e \Rightarrow (d \multimap e) \rightarrow e \\ \text{rep}_\multimap \ e = \lambda d \rightarrow d \bullet e & \text{abs}_\multimap \ f = f \ \text{one} \end{array}$$

In particular, as $\text{one} \bullet e = e$ (Figure 9.1), we have that $\text{abs}_\multimap \circ \text{rep}_\multimap = \text{id}$. Moreover, for linear functions also $\text{rep}_\multimap \circ \text{abs}_\multimap = \text{id}$.

Elsman et al. [39] do a functional analysis for differentiation that is also based on linear functions. They show that symbolic and forward-mode derivatives follow from the same rules, but reverse-mode essentially computes the adjoint of a derivative. Adjoints arise by taking a curried form of d -valued inner product structure on vector spaces e , giving rise to a duality between e and the linear function space $e \multimap d$. Our more general setting does not consider such normed/inner product structure on our d -modules e , but we expect that such richer structure may be smoothly incorporated into a general picture, and we leave this as a topic for future work.

Now, we derive *Monoid*, *Module* and *Kronecker* instances that make $\text{rep}_\multimap$ a homomorphism by construction. It follows that $d \multimap e$ inherits its monoidal structure in a pointwise fashion from e . For $e = \text{Dense } v \ d$, the time complexity for additive operations remains linear in the number of variables.

³For the case of commutative *rings* and their modules; the argument for semirings and semimodules is identical, but simpler, because there are fewer equations to check.

⁴Preservation of addition follows from the fact that the result of f is a semimodule.

instance *Monoid* $e \Rightarrow \text{Monoid } (d \multimap e)$ **where**

$$\begin{aligned}\epsilon &= \lambda d \rightarrow \epsilon \\ f \diamond g &= \lambda d \rightarrow f \, d \diamond g \, d\end{aligned}$$

Furthermore, its d -semimodule structure uses the function parameter as an accumulator for the scalar multiplier. This affords a “*strength reduction*”: the expensive scalar multiplication $(\bullet)$ is replaced by the cheaper semiring multiplication $(\otimes)$.

instance *Module* $d \, e \Rightarrow \text{Module } d \, (d \multimap e)$ **where**

$$d' \bullet f = \lambda d \rightarrow f \, (d' \otimes d)$$

This reduces the time complexity of scalar multiplication from $\mathcal{O}(V)$ for the *Sparse* $v \, d$ representation, where V denotes the number of variables with non-zero values in the given sparse map, to $\mathcal{O}(1)$ for that of $d \multimap \text{Sparse } v \, d$. However, the general *Kronecker* instance of $d \multimap e$ still contains the expensive $(\bullet)$ operator.

instance *Kronecker* $v \, d \, e \Rightarrow \text{Kronecker } v \, d \, (d \multimap e)$ **where**

$$\text{delta } v = \lambda d \rightarrow d \bullet \text{delta } v$$

Fortunately, we only require a specialized instance for sparse maps, which operates in constant time, as a consequence of Equation 10.1 for $d \bullet \text{delta } v$.

instance (*Ord* v , *Semiring* d) $\Rightarrow \text{Kronecker } v \, d \, (d \multimap \text{Sparse } v \, d)$ **where**

$$\text{delta} = \text{singleton}$$

Finally, the *CorrectAD* instance for $d \multimap e$ is defined as the composition of $\text{rep}_{\multimap} / \text{abs}_{\multimap}$ with rep / abs of e .

instance *CorrectAD* $v \, d \, e \Rightarrow \text{CorrectAD } v \, d \, (d \multimap e)$ **where**

$$\begin{aligned}\text{rep} &= \text{rep}_{\multimap} . \text{rep} \\ \text{abs} &= \text{abs} . \text{abs}_{\multimap}\end{aligned}$$

In summary, by using $d \ltimes (d \multimap \text{Sparse } v \, d)$ we obtain reverse-mode AD.

$$\begin{aligned}\text{reverseAD} &:: (\text{Ord } v, \text{Semiring } d) \\ &\Rightarrow (v \rightarrow d) \rightarrow \text{Expr } v \rightarrow d \ltimes (d \multimap \text{Sparse } v \, d) \\ \text{reverseAD} &= \text{abstractD}\end{aligned}$$

For instance, consider the more extensive example expression $x \times (x+1) \times (x+x)$, encoded in Haskell as *example₃*.

```
example3 :: Expr X
example3 = Times (Var X) (Times (Plus (Var X) One) (Plus (Var X) (Var X)))
```

Running reverse-mode AD on this expression, the derivative at $X = 5$ is 170.

```
>>> (abs◦ ∘ tanN) (reverseAD (λX → 5) example3)
{X ↦ 170}
```

Figure 10.1 shows that this computation effectively works in multiple passes, which is a key characteristic of reverse-mode AD. During the forward (bottom-up) pass (shown by the black arrows) the algorithm computes Nagata numbers, which contain the value of each sub-expression, and the dependencies between the nodes as function closures $c :: d \multimap \text{Sparse } v \ d$, e.g., $N \ 300 \ c_{\otimes_2}$ for the final outcome. In the backward (top-down) pass (shown by the red left-to-right arrows), we invoke the function closure $c_{\otimes_2}$ on argument *one* (using $\text{abs}_{\multimap}$). It propagates the scalar accumulator downwards. At each downward step, the accumulator changes to reflect how much the overall derivative changes given that the subexpression changes by one. Finally, the results at the leaves are added (shown by the blue right-to-left arrows) to yield the overall derivative. The conventional imperative algorithms typically accomplish this with imperative updates. We derive that form in Section 10.2.3, but first we address the linear cost of addition.

10.2.2 Cayley Representation

We optimize the additive structure of d -modules in much the same way as we have optimized scalar multiplications: by considering a representation of $e \rightarrow e$ functions that have the *additive* homogeneity property:

$$f(x \diamond y) = x \diamond f y$$

This representation is also known as the Cayley representation [65, 16] of e , the most well-known example of which are difference lists [25] or Hughes lists [70]. Again, the Haskell type $e \rightarrow e$ includes functions that do not have the homogeneity property; those are not included in the *Cayley* e type.

```
type Cayley e = e → e
rep◊ :: Monoid e ⇒ e → Cayley e      abs◊ :: Monoid e ⇒ Cayley e → e
rep◊ e = λe' → e' ◊ e                  abs◊ f = f ◊
```

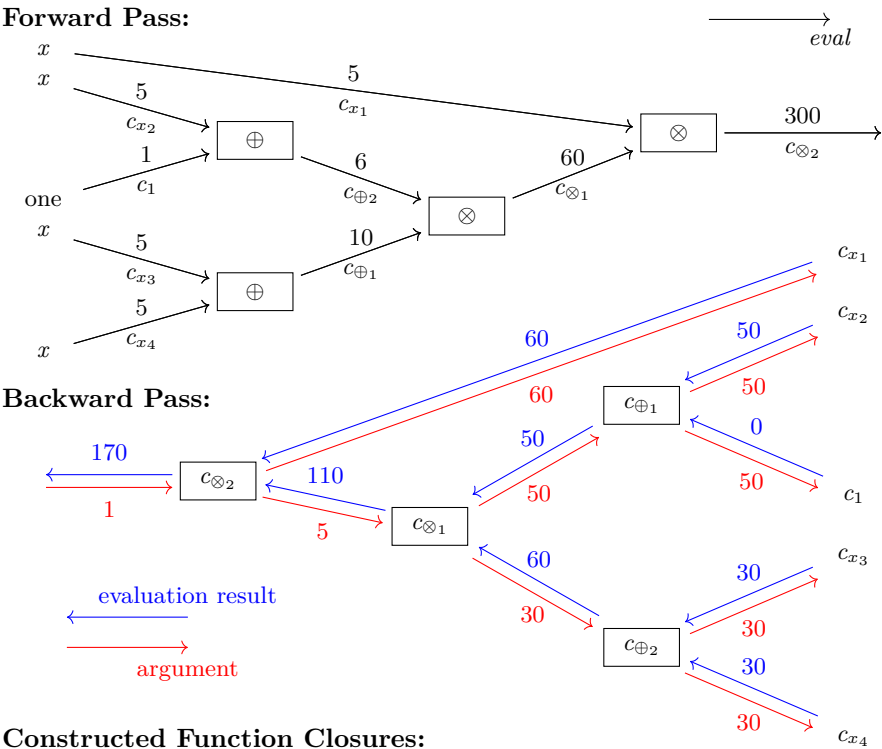


Figure 10.1: Reverse-mode AD of $x \times (x + 1) \times (x + x)$, for $x = 5$.

This representation typically implies another “strength reduction”, namely from e ’s binary operator ($\diamond$) to function composition ($\circ$). We thereby reduce the time complexity of addition from $\mathcal{O}(V)$ for *Sparse v d* to $\mathcal{O}(1)$ for *Cayley (Sparse v d)*.

instance *Monoid* (*Cayley e*) **where**

$$\begin{aligned}\epsilon &= id. \\ f \diamond g &= g \circ f\end{aligned}$$

We derive the *Module* and *Kronecker* instances following our standard recipe. However, we cannot eliminate the expensive operations from the *Cayley e* instances for *Module* and *Kronecker*. Indeed, the *Module* instance requires converting from *Cayley e* to e and back, where notably $rep_\diamond$ re-introduces e ’s expensive $\mathcal{O}(V)$ addition operation ($\diamond$), which we have just managed to eliminate from the *Monoid* instance.

instance *Module* $d\ e \Rightarrow$ *Module* $d\ (Cayley\ e)$ **where**

$$d \bullet f = rep_\diamond\ (d \bullet abs_\diamond\ f)$$

The *Kronecker* instance derived via $rep_\diamond$ from e similarly features e ’s addition operation ($\diamond$).

instance *Kronecker* $v\ d\ e \Rightarrow$ *Kronecker* $v\ d\ (Cayley\ e)$ **where**

$$delta\ v = \lambda e \rightarrow e \diamond delta\ v$$

Fortunately, we can again define a specialized *Kronecker* instance for the type $e = d \multimap Cayley\ (Sparse\ v\ d)$, which avoids appeal to the expensive ($\diamond$)-operation. Indeed, the ($\bullet$)-operation defined on e need not make use of that defined on *Cayley (Sparse v d)*. For the specialized *delta*, we instead calculate:

$$\begin{aligned}\delta\ v &= \{-\text{definition of } \delta \text{ for } d \multimap -\} \\ &\quad \lambda d \rightarrow d \bullet \delta\ v \\ &= \{-\text{definition of } (\bullet) \text{ and } \delta \text{ for } Cayley -\} \\ &\quad \lambda d \rightarrow rep_\diamond\ (d \bullet abs_\diamond\ (\lambda e \rightarrow e \diamond \delta\ v)) \\ &= \{-\text{definition of } rep_\diamond \text{ and } abs_\diamond -\} \\ &\quad \lambda d \rightarrow \lambda e \rightarrow e \diamond (d \bullet (\lambda e \rightarrow e \diamond \delta\ v)\ \epsilon) \\ &= \{-\beta\text{-reduction and unit of monoid (Figure 9.1) -}\} \\ &\quad \lambda d \rightarrow \lambda e \rightarrow e \diamond (d \bullet \delta\ v) \\ &= \{-\text{Equation 10.2 -}\} \\ &\quad \lambda d \rightarrow \lambda e \rightarrow insertWith\ (\oplus)\ v\ d\ e\end{aligned}$$

$$= \{-\eta\text{-reduction -}\} \\ \lambda d \rightarrow \text{insertWith } (\oplus) \ v \ d$$

Hence, we obtain an implementation of *delta* that takes only $\mathcal{O}(\log V)$ time.

instance (*Ord* *v*, *Semiring* *d*) $\Rightarrow$ *Kronecker* *v* *d* (*d* $\multimap$ *Cayley* (*Sparse* *v* *d*))
where *delta* = *insertWith* ($\oplus$)

Moreover, the *Cayley* *e* representation preserves the correctness of *e*.

instance *CorrectAD* *v* *d* *e* $\Rightarrow$ *CorrectAD* *v* *d* (*Cayley* *e*) **where**
rep = *rep* $_{\diamond}$ $\circ$ *rep*
abs = *abs* $\circ$ *abs* $_{\diamond}$

In summary, with type *d* $\multimap$ *Cayley* (*Sparse* *v* *d*), both $(\bullet)$ and $(\diamond)$ take $\mathcal{O}(1)$ and δ takes $\mathcal{O}(\log V)$. As a consequence, the time complexity of *abstractD* for a program with *N* nodes and *V* variables is now $\mathcal{O}(N \times \log V)$.

*reverseAD*_{*cayley*} :: (*Ord* *v*, *Semiring* *d*)
 $\Rightarrow (v \rightarrow d) \rightarrow \text{Expr } v \rightarrow d \times (d \multimap \text{Cayley } (\text{Sparse } v \ d))$
*reverseAD*_{*cayley*} = *abstractD*

For example, as before, the derivative of *example*₃ at *X* = 5 is 170.

```
>>> (abs $_{\diamond}$   $\circ$  abs $_{\multimap}$   $\circ$  tanN) (reverseADcayley ( $\lambda X \rightarrow 5$ ) example3)
{X  $\mapsto$  170}
```

Notice that we can see values of type *Cayley* (*Sparse* *v* *d*) as a functional model for computations that modify a state of type *Sparse* *v* *d*, i.e., immutable maps.

10.2.3 Mutable Arrays

Performance-wise, we can do even better by switching from immutable maps to mutable arrays (from *Data.Array.ST*). Using these to represent the state, instead of the purely functional implementation above, we can perform in-place updates. Furthermore, the array can be provided by an implicit environment (*Control.Monad.Reader*). Hence, instead of type *Cayley* (*Sparse* *v* *d*), we can work with the type *ReaderT* (*STArray* *s* *v* *d*) (*ST* *s*) (). We refer to

Appendix G for an explanation of the functions for mutable arrays that we use. It is convenient to encapsulate this rather wordy type in the type synonym $ST_{Cayley} v d$.

type $ST_{Cayley} v d = \forall s \text{ ReaderT } (STArray s v d) (ST s) ()$

Observe that $ST_{Cayley} v d$ polymorphically quantifies over the state thread parameter s , which is necessary if we want to run the computation using Haskell's built-in primitive $runST :: (\forall s \text{ ST } s a) \rightarrow a$. The isomorphism between $Cayley (Sparse v d)$ and $ST_{Cayley} v d$ essentially converts between the array and map representations.

```
rep>> :: (Ix v, Semiring d) => Cayley (Sparse v d) -> ST_{Cayley} v d
rep>> p = foldMap (\(v, d) -> modifyAt (\oplus d) v) (toList (p \epsilon))

abs>> :: (Ix v, Bounded v, Semiring d) => ST_{Cayley} v d -> Cayley (Sparse v d)
abs>> q = runST (do \to' \leftarrow newArray (minBound, maxBound) zero
                  runReaderT q \to'
                  l \leftarrow getAssocs \to'
                  return (foldMap (\(v, d) -> insertWith (\oplus) v d) l))
```

The $rep>>$ function uses a helper function $modifyAt$ to edit the value at a specific position of the array, which is the counterpart of $insertWith$ on maps.

```
modifyAt :: Ix v => (d -> d) -> v -> ST_{Cayley} v d
modifyAt f v = do \to' \leftarrow ask; x \leftarrow readArray \to' v; writeArray \to' v (f x)
```

Using the methodology of Chapter 9, the *Monoid* instance works out to be the generic instance that is available to any type $m ()$ where m is a monad.

instance *Monoid* ($ST_{Cayley} v d$) **where**

```
\epsilon      = return ()
p \diamond q = p >> q
```

As we work with $d \multimap ST_{Cayley} v d$, the *Module* structure of $ST_{Cayley} v d$ is not used. For that reason, we skip immediately to the *Kronecker* instance where the purpose of using mutable arrays is to reduce the $\mathcal{O}(\log V)$ time complexity of $insertWith$ in δ to the $\mathcal{O}(1)$ time complexity of $modifyAt$.

instance ($Ix v, Semiring d$) => *Kronecker* $v d$ ($ST_{Cayley} v d$) **where**
 $\delta v = modifyAt (\oplus one) v$

In the $d \multimap ST_{Cayley} v d$ combination this becomes:

instance ($Ix v, Semiring d \Rightarrow Kronecker v d (d \multimap ST_{Cayley} v d)$ **where**
 $\delta v = \lambda d \rightarrow modifyAt (\oplus d) v$

The correctness of $ST_{Cayley} v d$ is established in terms of that of $Cayley (Sparse v d)$.

instance ($Ix v, Ord v, Bounded v, Enum v, Semiring d, Eq d$)
 $\Rightarrow CorrectAD v d (ST_{Cayley} v d)$ **where**
 $rep = rep_{\gg} \circ rep$
 $abs = abs \circ abs_{\gg}$

Finally, we get the resulting monadic definition for reverse-mode AD.

$reverseAD_{array} :: (Ix v, Semiring d)$
 $\Rightarrow (v \rightarrow d) \rightarrow Expr v \rightarrow d \ltimes (d \multimap ST_{Cayley} v d)$
 $reverseAD_{array} = abstractD$

We use this efficient version of reverse-mode AD on $example_3$ as follows:

$\gg> (abs_{\circ} \circ abs_{\gg} \circ abs_{\multimap} \circ tan^N) (reverseAD_{array} (\lambda X \rightarrow 5) example_3)$
 $\{X \mapsto 170\}$

10.3 Computational Complexity

The following table summarizes the successive differentiation algorithms we have constructed by instantiating $abstractD$ with semiring types $d \ltimes e$ for different choices of tangent type e . It also shows the worst-case time complexity of each algorithm.

Differentiation Mode	Semiring Type	Complexity
Symbolic	$(Expr v) \ltimes (Expr v)$	$\mathcal{O}(N^2)$
Forward Mode	$d \ltimes (Dense v d)$	$\mathcal{O}(N \times V)$
Forward Mode	$d \ltimes (Sparse v d)$	$\mathcal{O}(N \times V)$
Reverse Mode	$d \ltimes (d \multimap Sparse v d)$	$\mathcal{O}(N \times V)$
Reverse Mode	$d \ltimes (d \multimap Cayley (Sparse v d))$	$\mathcal{O}(N \times \log V)$
Reverse Mode	$d \ltimes (d \multimap ST_{Cayley} v d)$	$\mathcal{O}(N + V)$

Notice that, when V is large, i.e., when there are many variables, reverse-mode AD, with its last two variants in particular, is the most interesting. For few variables, forward-mode AD is preferable as it only takes a single pass.

Chapter 11

Extensions

Our setup allows for various extensions that make AD more useful. Notice that, in these extensions, we often revert to the more naive $fwdAD_{dense}$ with the *Dense* representation for the sake of brevity. In what follows, we discuss the overloading of expressions using semirings (Section 11.1), adding primitives such as transcendental functions (Section 11.2), non-commutative semirings (Section 11.3), higher derivatives (Section 11.4) and sharing subexpressions using **let** (Section 11.5).

11.1 Overloading Initial Expressions

Automatic differentiation avoids the intermediate symbolic derivative when computing precise numeric results. This is shown by the elimination of *eval* in Section 8.3.3, which was invoked twice in the naive implementation. In this section we show that, by overloading operators in the initial expression and thereby avoiding the initial symbolic expression, we can further eliminate the *eval* call and thus further reduce verbosity.

In particular, we assume that the initial expression is not given as a value of type *Expr* v , but as a polymorphic function of type $\forall d \text{ Semiring } d \Rightarrow d \rightarrow \dots \rightarrow d \rightarrow d$ with as many inputs of type d as there are variables v . For example, *example₂* from Section 10.1.1, can be written as a polymorphic function.

$$\begin{aligned} \text{example}'_2 &:: \text{Semiring } d \Rightarrow d \rightarrow d \rightarrow d \\ \text{example}'_2 \ x \ y &= ((x \otimes y) \oplus x) \oplus \text{one} \end{aligned}$$

The mechanism behind deriving $example'_2$ is that of *fold/build fusion* [54], applied to $eval\ var\ example_2$, where $example_2$ is an instance of the builder pattern, building a symbolic expression, and $eval$ is a fold. Fold/build fusion eliminates the intermediate symbolic structure and directly builds the result of evaluation. Gibbons and Wu [51] show how to exploit such structure systematically (generalizing to all algebraic structures defined by Haskell type classes, not only *Semirings*), by relating the deep embedding in terms of symbolic expressions $Expr\ v$ with the corresponding shallow embedding $\forall d\ Semiring\ d \Rightarrow (v \rightarrow d) \rightarrow d$. Here, $eval$ is one half of an isomorphism pair between these two types, with the inverse given by instantiation at Var .

Hence, instead of the verbose call (left), we readily invoke $example'_2$ (right).

```
>>> let { var X = 5; var Y = 3 }      >>> example'_2 5 3
      in eval var example_2           21
21
```

To invoke forward-mode AD, we use Nagata numbers:

```
>>> let { var X = 5; var Y = 3 }      >>> example'_2 (N 5 (delta X))
      in fwdAD_sparse var example_2    (N 3 (delta Y))
N 21 { X ↦ 4, Y ↦ 5 }                 N 21 { X ↦ 4, Y ↦ 5 }
```

Both cases avoid the interpretative overhead of evaluating the symbolic expression. Yet, we can also recover the original symbolic expression.

```
>>> example'_2 (Var X) (Var Y)
Plus (Plus (Times (Var X) (Var Y)) (Var X)) One
```

Finally, we invoke the more efficient reverse-mode AD with mutable arrays:

```
>>> let ex = example'_2 (N 5 (delta X)) (N 3 (delta Y))
      in N (priN res) ((abs◊ ◦ abs≫ ◦ abs◦ ◦ tanN) ex)
N 21 { X ↦ 4, Y ↦ 5 }
```

11.2 Additional Primitives

We have so far focused only on the minimal *Semiring* interface. Yet, it is well-known that dual numbers (and, by extension, Nagata numbers) easily adapt to other primitives, such as trigonometric, exponential and logarithmic functions. We model such functions as subclasses of the *Semiring* type class.

For instance, we can extend the approach from semirings to rings by creating a *Ring* subclass with a *negate* method for additive inverses¹.

```
class Semiring d ⇒ Ring d where
  negate :: d → d
instance (Module d e, Ring d) ⇒ Ring (d ⋈ e) where
  negate ((N f df)) = N (-f) (-one • df)
```

Here, we use the fact that Haskell allows us to write $-x$ for *negate* x . Likewise, we collect trigonometric functions in *Trig*, whose Nagata number instance relies on the well-known chain rule. In general, to enrich d (and hence *Expr* v) with differentiable primitives requires us to know both the operation (*sin*), and its derivative (*cos*). Here, in order to be able to negate *sin* as the derivative of *cos*, d must be a *Ring*.

```
class Trig d where
  sin :: d → d
  cos :: d → d
instance (Module d e, Ring d, Trig d) ⇒ Trig (d ⋈ e) where
  sin (N f df) = N (sin f) ( cos f • df)
  cos (N f df) = N (cos f) (-sin f • df)
```

Observe that, while d must now also have a *Trig*-instance, no additional demands are made of e . Hence, all the AD variations we have covered, which are only based on variations of e , also support additional primitive functions.

Using the overloading of Section 11.1, we define *example*₄ and compute its derivative at point 5.0 as follows:

```
example4 :: (Ring d, Trig d) ⇒ d → d
example4 x = x ⋈ sin (x ⋈ x)
>>> tanN (example4 (N 5.0 (delta X)))
λX → 49.4277888430759
```

We can even extend d further to fields, which means that it has reciprocals.

```
class Ring d ⇒ Field d where
  reciprocal :: d → d
```

¹This deviates from the Haskell Prelude where *negate* is a method in the *Num* type class.

instance (*Module* $d\ e$, *Field* d) $\Rightarrow$ *Field* ($d \ltimes e$) **where**
reciprocal ($N\ f\ df$) = $N\ (\frac{one}{f})\ (-\frac{one}{(f \otimes f)} \bullet df)$

Here, we write $\frac{one}{x}$ for *reciprocal* x . Notice that we do not require a vector space here, where e should be a group. Instead, we want to make no additional demands on e so that all previous AD variants are applicable to this extension as well. For example, we capture exponential and logarithmic functions in type class *Exp*, whose Nagata instance also relies on the chain rule.

class *Exp* d **where**
exp :: $d \rightarrow d$
log :: $d \rightarrow d$
instance (*Module* $d\ e$, *Field* d , *Exp* d) $\Rightarrow$ *Exp* ($d \ltimes e$) **where**
exp ($N\ f\ df$) = $N\ (exp\ f)\ (exp\ f \bullet df)$
log ($N\ f\ df$) = $N\ (log\ f)\ (\frac{one}{f} \bullet df)$

Using the overloading of Section 11.1, we define *example₅* and compute its derivative at point 5.0:

```
example5 :: (Field d, Exp d) => d -> d
example5 x = x ⊗ log x ⊕ x
>>> tanN (example5 (N 5.0 (delta X)))
λX → 3.6094379124341005
```

11.3 Non-Commutative Semirings

We can generalize our approach from commutative semirings to semirings with non-commutative multiplication ($\otimes$), such as (square) matrices over semirings (Section 12.3) and the semiring of formal languages over an alphabet.

The key change in our approach is that rather than starting from the following rule for symbolic differentiation, which exploits the commutativity:

$$derive\ x\ (Times\ e_1\ e_2) = (e_2 \otimes derive\ x\ e_1) \oplus (e_1 \otimes derive\ x\ e_2)$$

where e_1 and e_2 's derivatives are *both scaled on the left*, we instead use the more conventional rule:

$$derive\ x\ (Times\ e_1\ e_2) = (derive\ x\ e_1 \otimes e_2) \oplus (e_1 \otimes derive\ x\ e_2)$$

which scales *derive* $x\ e_2$ on the left and *derive* $x\ e_1$ on the right.

More abstractly, we can model this with d - d -bimodules that feature distinct left-scaling and right-scaling operators.

```
class Semiring  $d \Rightarrow \text{Bimodule } d\ e \mid e \rightarrow d$  where
   $(\vec{\bullet}) :: d \rightarrow e \rightarrow e$ 
   $(\overleftarrow{\bullet}) :: e \rightarrow d \rightarrow e$ 
```

Notice that, for every d whose multiplication is commutative, every *Module* $d\ e$ instance gives rise to a *Bimodule* $d\ e$ instance, by taking the two scaling operations to be the same underlying $(\bullet)$ operation. If multiplication in d is not commutative, nevertheless the distinguished case $e = d$ yields a *Bimodule* structure with $(\vec{\bullet})$ (resp. $(\overleftarrow{\bullet})$) given by *left*-(resp. *right*-)multiplication (see below). However, it is *not* sufficient to take (a restricted subset of) $d \rightarrow e$ as a representative of the set of *Bimodule* homomorphisms from d to e .

Instead, we can avoid introducing this new d - d -bimodule abstraction and reuse the existing *Module*-related infrastructure by means of (d, d) -semimodules. At the core of this idea is the semiring construction for pairs.

```
instance Semiring  $d \Rightarrow \text{Semiring } (d, d)$  where
  zero           = (zero, zero)
  one            = (one, one)
   $(x_l, x_r) \oplus (y_l, y_r) = (x_l \oplus y_l, x_r \oplus y_r)$ 
   $(x_l, x_r) \otimes (y_l, y_r) = (x_l \otimes y_l, y_r \otimes x_r)$ 
```

This is the usual component-wise instance, except that the factors in the multiplication are flipped for the second component. Then d is a (d, d) semimodule as follows:

```
instance Semiring  $d \Rightarrow \text{Module } (d, d)\ d$  where
   $(l, r) \bullet d = l \otimes d \otimes r$ 
```

Here, the first component of the tuple scales from the left and the second component scales from the right. We can make use of the left and right injections below to turn a d -value either into a left or right factor, using the neutral element *one* for the other factor.

```
inl, inr :: Semiring  $d \Rightarrow d \rightarrow (d, d)$ 
inl  $x = (x, \text{one})$ 
inr  $y = (\text{one}, y)$ 
```

More generally, with (d, d) -module e , we obtain a variation on the Nagata numbers:

instance (*Semiring* d , *Module* (d, d) e) $\Rightarrow$ *Semiring* $(d \ltimes e)$ **where**
 $zero = N \ zero \ zero$
 $one = N \ one \ \epsilon$
 $N \ f \ df \oplus N \ g \ dg = N \ (f \oplus g) \ (df \diamond dg)$
 $N \ f \ df \otimes N \ g \ dg = N \ (f \otimes g) \ ((\text{inr} \ g \bullet df) \diamond (\text{inl} \ f \bullet dg))$

This one localized modification seamlessly integrates non-commutative multiplication into our AD variants. In particular, following the earlier recipe we now use the $(d, d) \multimap e$ representation for accumulating multiplications, which is that of $(d, d) \rightarrow e$ functions that take two d accumulators. The scalar multiplication of this representation updates both accumulators.

instance (*Semiring* d , *Module* (d, d) e) $\Rightarrow$ *Module* (d, d) $((d, d) \multimap e)$
where $(d_l', d_r') \bullet f = \lambda(d_l, d_r) \rightarrow f \ (d_l' \otimes d_l, d_r' \otimes d_r')$

This shows that the first accumulator is for multiplications on the left and the other for multiplications on the right.

In short, (d, d) -modules support non-commutative multiplication with minimal upheaval. We use this extension in Section 12.3 for square matrices.

11.4 Higher Derivatives

Higher derivatives are obtained through repeated differentiation. For instance, the second-order derivative is the derivative of the derivative. The naive approach to compute the second-order derivative at a point is through repeated evaluation of the symbolic derivative.

$derive_{2nd} :: (Eq \ v, Semiring \ d)$
 $\Rightarrow (v \rightarrow d) \rightarrow Expr \ v \rightarrow Dense \ v \ (Dense \ v \ d)$
 $derive_{2nd} \ var \ e = \lambda x \ y \rightarrow eval \ var \ (tan^N \ (fwdAD_{dense} \ Var$
 $\quad (tan^N \ (fwdAD_{dense} \ Var \ e) \ x)) \ y)$

The Haskell call $derive_{2nd} \ var \ e \ x \ y$ corresponds to the mathematical notation $\left. \frac{\partial}{\partial y} \left(\frac{\partial}{\partial x} e \right) \right|_{var}$. By using naturality of tan^N and $eval$'s fusion property, we merge the three traversals of $derive_{2nd}$ into a single $eval$ (Appendix E):

$$\begin{aligned} \text{derive}'_{2nd} \text{ var } e &= \lambda x \ y \rightarrow \tan^N (\tan^N (\text{fwdAD}_{dense} \text{ gen } e) x) y \\ \textbf{where } \text{gen } z &= N (\text{var } z) (\text{delta } z) \end{aligned}$$

This instance of *abstractD* uses the $(d \ltimes (Dense \ v \ d))$ -semimodule of nested Nagata numbers of type $(d \ltimes (Dense \ v \ d)) \ltimes (Dense \ v \ (d \ltimes (Dense \ v \ d)))$.

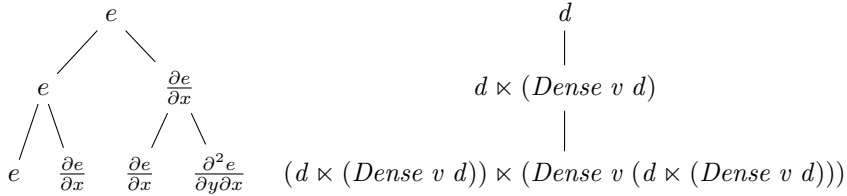


Figure 11.1: 0^{th} - 2^{nd} -order derivatives and their types.

The nested representation (Figure 11.1) of the second-order derivative contains redundancy: the value $\frac{\partial e}{\partial x}$ appears twice. This redundancy gets worse as the order (and thus the level of nesting) increases. In general, for the n th-order derivative we get 2^n values of which only $n + 1$ are distinct. To avoid this, it is more economical to use a compact representation for the Nagata numbers.

$$\textbf{data } N_{2nd} \ v \ d = N_{2nd} \ d \ (Dense \ v \ (d \ltimes (Dense \ v \ d)))$$

The corresponding forward-mode AD algorithm is similar to *abstractD*:

$$\begin{aligned} \text{fwdAD}_{2nd} &:: (Semiring \ d) \Rightarrow (v \rightarrow d) \rightarrow Expr \ v \rightarrow N_{2nd} \ v \ d \\ \text{fwdAD}_{2nd} \text{ var} &= eval \ \text{gen} \ \textbf{where} \ \text{gen } x = N_{2nd} \ (\text{var } x) (\text{delta } x) \end{aligned}$$

Running this on *example₃* gives the evaluation result, first and second derivative:

$$\begin{aligned} &>>> \text{fwdAD}_{2nd} (\lambda X \rightarrow 5) \ \text{example}_3 \\ &N_{2nd} \ 300 \ (\lambda X \rightarrow N \ \{pri^N = 170, \tan^N = \lambda X \rightarrow 64\}) \end{aligned}$$

This generalizes to other higher derivatives.

All Higher-Order Partial Derivatives In the extreme case where $n = \infty$, the economical representation becomes an infinite stream of all successive

derivatives [78]. This stream has the form $x :< dxs$ where x is the value and dxs are all higher-order partial derivatives with respect to all variables v .²

data $Stream\ v\ d = d :< Dense\ v\ (Stream\ v\ d)$

Then, the semiring operations for the infinite stream of successive derivatives are defined as follows.

instance $Semiring\ d \Rightarrow Semiring\ (Stream\ v\ d)$ **where**
 $zero = zero :< zero$
 $one = one :< zero$
 $(x :< xs) \quad \oplus \quad (y :< ys) \quad = \quad (x \oplus y) :< (xs \oplus ys)$
 $xxs @ (x :< xs) \otimes yys @ (y :< ys) = (x \otimes y) :< \lambda v \rightarrow ((xs\ v \otimes yys) \oplus (xxs \otimes ys\ v))$

The *Module* and *Kronecker* instances follow naturally from this definition. They allow us to automatically differentiate a symbolic expression and obtain its stream of derivatives.

$fwdAD_{all} :: (Eq\ v, Semiring\ d) \Rightarrow (v \rightarrow d) \rightarrow Expr\ v \rightarrow Stream\ v\ d$
 $fwdAD_{all}\ var = eval\ gen\ \mathbf{where}\ gen\ y = var\ y :< delta\ y$
 $>>> fwdAD_{all}\ (\lambda X \rightarrow 5)\ example_3$
 $300 :< (\lambda X \rightarrow 170 :< (\lambda X \rightarrow 64 :< (\lambda X \rightarrow 12 :< (\lambda X \rightarrow 0 :< ...$

Hybrid Automatic Differentiation In second-order optimization, it is often preferred to use a hybrid approach, i.e., combine forward-mode and reverse-mode AD to get a more efficient algorithm. For instance, when taking a function's curvature into account, it is necessary to compute the Hessian³ applied to a vector. Pearlmutter [113] introduces this mixture of the two modes. In a forward sweep the first derivative is computed, whereupon the second, reverse sweep computes the second-order derivative at a point. This can be obtained by exploiting the d -semimodule isomorphisms to work with

data $N_H\ v\ d = N_H\ d\ (Dense\ v\ (d \ltimes (d \multimap (Cayley\ (Sparse\ v\ d))))))$

²It differs from Hinze's stream representation [64] in the interposition of *Dense* in the tail.

³The Hessian of a function f in n variables x_i is the $n \times n$ square matrix consisting of all second-order derivatives $\frac{\partial^2 f}{\partial x_i \partial x_j}$.

11.5 Sharing Subexpressions

Our approach so far ignores any sharing in the source expression. Consider for instance $example_4$, which denotes the expression $(x + x) \times (x + x)$, but at the level of Haskell shares the two occurrences of $x + x$ by means of a **let** binding.

$example_4 :: Expr\ X$
 $example_4 = \mathbf{let}\ y = Plus\ (Var\ X)\ (Var\ X)\ \mathbf{in}\ Times\ y\ y$

The *eval* recursion scheme used by *abstractD* does not see the **let** binding and processes both occurrences of $x + x$. In particular, $abstractD\ (\lambda X \rightarrow 5)\ example_4$ reduces to $(N\ 5\ one \oplus N\ 5\ one) \otimes (N\ 5\ one \oplus N\ 5\ one)$ which features two $(\oplus)$ operations instead of one. To avoid this re-computation of intermediate results, we extend the symbolic expression language with a **let**-construct so that we can write $example'_4$ with this new syntax.

data $Expr\ v = \dots \mid Let\ v\ (Expr\ v)\ (Expr\ v)$
 $example'_4 = Let\ Y\ (Plus\ (Var\ X)\ (Var\ X))\ (Times\ (Var\ Y)\ (Var\ Y))$

We adapt the evaluator accordingly with the standard semantics for **let**-binding.

$eval :: (Eq\ v, Semiring\ d) \Rightarrow (v \rightarrow d) \rightarrow Expr\ v \rightarrow d$
 $\dots$
 $eval\ gen\ (Let\ y\ e_1\ e_2) = \mathbf{let}\ d_1 = eval\ gen\ e_1$
 $\quad\quad\quad gen_2\ x = \mathbf{if}\ x \equiv y\ \mathbf{then}\ d_1\ \mathbf{else}\ gen\ x$
 $\quad\quad\quad \mathbf{in}\ eval\ gen_2\ e_2$

Observe that, because e_2 has the additional **let**-bound variable in scope, we evaluate it with the extended generator gen_2 , which maps the free variables to their given *gen* meaning and the new **let**-bound variable y to the meaning d_1 of e_1 . The definition of *abstractD* is as before, now with support for sharing.

Sharing for Sparse Maps Now, $abstractD\ (\lambda X \rightarrow 5)\ example'_4$ reduces to an intermediate form with only one $(\oplus)$ operation. If we commit to the sparse module for the tangent, this reduces further, nicely preserving the sharing.

$\mathbf{let}\ y = N\ 5\ one \oplus N\ 5\ one\ \mathbf{in}\ (y \otimes y)$
 $\equiv \mathbf{let}\ y = N\ 5\ \{X \mapsto 1\} \oplus N\ 5\ \{X \mapsto 1\}\ \mathbf{in}\ (y \otimes y)$
 $\equiv \mathbf{let}\ y = N\ 10\ \{X \mapsto 2\}\ \mathbf{in}\ (y \otimes y)$

$$\begin{aligned}
&\equiv N (10 * 10) (\text{fmap } (10*) \{X \mapsto 2\} \diamond \text{fmap } (10*) \{X \mapsto 2\}) \\
&\equiv N 100 \quad (\{X \mapsto 20\} \diamond \{X \mapsto 20\}) \\
&\equiv N 100 \quad \{X \mapsto 40\}
\end{aligned}$$

Sharing for Functions However, if we use a function-based module representation, such as *Dense* and our three reverse-mode variants, then the $(\oplus)$ is duplicated again later in the evaluation. This happens because the $(\oplus)$ under the function binder is not reduced until the function is applied. When both references of the shared function are applied, the $(\oplus)$ in its body will be evaluated each time. The following shows this for dense functions because it is the simplest of the function-based representations.

$$\begin{aligned}
&\text{let } y = N 5 \text{ one } \oplus N 5 \text{ one in } (y \otimes y) \\
&\equiv \text{let } y = N 5 \quad (\lambda X \rightarrow 1) \oplus N 5 (\lambda X \rightarrow 1) \quad \text{in } (y \otimes y) \\
&\equiv \text{let } y = N 10 (\lambda X \rightarrow (\lambda X' \rightarrow 1) X \oplus (\lambda X' \rightarrow 1) X) \text{ in } (y \otimes y) \\
&\equiv (N 10 \quad (\lambda X \rightarrow (\lambda X' \rightarrow 1) X \oplus (\lambda X' \rightarrow 1) X)) \otimes \\
&\quad (N 10 \quad (\lambda X \rightarrow (\lambda X' \rightarrow 1) X \oplus (\lambda X' \rightarrow 1) X)) \\
&\equiv N (10 * 10) (\lambda X \rightarrow (\lambda X' \rightarrow 10 * (\lambda X'' \rightarrow (\lambda X''' \rightarrow 1) X'' \oplus \\
&\quad \quad \quad (\lambda X''' \rightarrow 1) X'') X') X + \\
&\quad \quad \quad (\lambda X' \rightarrow 10 * (\lambda X'' \rightarrow (\lambda X''' \rightarrow 1) X'' \oplus \\
&\quad \quad \quad (\lambda X''' \rightarrow 1) X'') X') X)
\end{aligned}$$

Fortunately, we can address the problem of sharing for function-representations with a custom symbolic rule for *Let* (Proof in Appendix F)⁴.

Lemma 1 (Let Rule).

$$\frac{\partial(\text{let } y = e_1 \text{ in } e_2)}{\partial x_i} = \frac{\partial e_2}{\partial y} \Big|_{y=e_1} \times \frac{\partial e_1}{\partial x_i} + \frac{\partial e_2}{\partial x_i} \Big|_{y=e_1}$$

For instance, the expression $example'_4$ has partial derivative:

$$\begin{aligned}
\frac{\partial example'_4}{\partial x} &= \frac{\partial y \times y}{\partial y} \Big|_{y=x+x} \times \frac{\partial x + x}{\partial x} + \frac{\partial y \times y}{\partial x} \Big|_{y=x+x} \\
&= ((x + x) + (x + x)) \times (1 + 1) + 0
\end{aligned}$$

⁴We ignore the fact that some primitive functions may have non-differentiable points.

This formula is captured in a *Letin* type class with a method *letin*, which in turn is defined in terms of $abs :: e \rightarrow v \rightarrow d$ of the *CorrectAD* type class, that extracts the v -component of a d -module e .

```
class CorrectAD v d e  $\Rightarrow$  Letin v d e where
  letin :: v  $\rightarrow$  e  $\rightarrow$  e  $\rightarrow$  e
  letin y de1 de2 = ((abs de2 y) • de1)  $\diamond$  de2
```

We specialize the *evaluator* used by *abstractD* to use this formula, also specializing its type d to $d \ltimes e$.

```
eval :: (Eq v, Letin v d e)  $\Rightarrow$  (v  $\rightarrow$  d  $\ltimes$  e)  $\rightarrow$  Expr v  $\rightarrow$  d  $\ltimes$  e
...
eval gen (Let y e1 e2) =
  let N f1 df1 = eval gen e1
      gen2 x    = if x  $\equiv$  y then N f1 (delta y) else gen x
      N f2 df2 = eval gen2 e2
  in N f2 (letin y df1 df2)
```

This evaluator uses the standard substitution behavior for the primal, but defers to the *letin* method of the *Letin* type class for the behavior for the tangent. If we inline and specialize *letin* for $d \multimap \text{Cayley} (\text{Sparse } v \ d)$, we get a tight definition that computes the maps produced by de_1 and de_2 only once.

```
instance (Ord v, Semiring d)
   $\Rightarrow$  Letin v d (d  $\multimap$  Cayley (Sparse v d)) where
  letin y de1 de2 =  $\lambda n \ m \rightarrow$  let m' = de2 n m in de1 (m' ! y) m'
```

With the above definition, the **let**-sharing is properly preserved.

Complications In order to get the main points across, we have ignored two complicating aspects above.

Firstly, the way the *Let* case of *eval* extends the generator *gen* results in a lookup time that is linear in the number (L) of nested **let**-bindings. A more efficient data structure (a map or array) can be used instead to improve the lookup time complexity to $\mathcal{O}(\log L)$ or $\mathcal{O}(1)$.

Secondly, we have implicitly assumed that **let**-bound variables do not shadow other variables, be they free variables or other **let**-bound variables. To support

shadowing, the partial derivative for the shadowed variable y —or at least, as much of it as has been accumulated so far in m —should be saved before running $dx2$ and restored afterwards. Regardless, other than for debugging purposes, one may want to purge the partial derivative for the **let**-bound variable y from m' as it is no longer relevant. Both together are accomplished as follows.

$$\begin{aligned} \text{let } y \text{ de}_1 \text{ de}_2 = \lambda n \ m \rightarrow & \text{let } old = \text{lookup } y \ m \\ & m' = \text{de}_2 \ n \ m \\ \text{in } & \text{de}_1 \ (m' ! y) \ (\text{update } (const \ old) \ y \ m') \end{aligned}$$

Chapter 12

Applications

This Chapter shows that the algebraic approach for AD is usable for different applications. Section 12.1 translates the approach to logic programming, developing the same algorithms in Prolog. Section 12.2 then applies the Prolog version to the probabilistic Sum-Product Loop Language (SPLL) for parameter learning and to benchmark the different versions. Section 12.3 shows how the extension for non-commutative semirings can be applied in a setting with square matrices. Finally, Section 12.4 uses the algebraic approach for learning probabilities of facts in the context of distribution semantics with possible worlds.

12.1 Automatic Differentiation in Prolog

We have developed the same principles for AD in Prolog¹. We demonstrate that Prolog features, such as definite clause grammars (DCGs) and co-routines contribute to the succinct formulation of the algorithm. Co-routines in particular make it easy to express reverse-mode variants in a single pass whereas conventional presentations require two passes. As far as we know, there is little work on AD in the logic programming community. Abdallah [2] implements AD with Constraint Handling Rules (CHR), which involves a different programming style (e.g., using constraints to represent the expression AST and using delay mechanisms much more extensively) and deviates more from the traditional algorithms. Moreover, it presents two algorithms as given, rather than deriving them systematically like we do. In his unfinished blogpost, Gabel [45] provides

¹The code can be found at <https://github.com/birthvdb/ad-prolog-code>.

a Prolog implementation of forward-mode AD that takes a compilation-based approach and produces a sequence of assignments. Algebraic ProbLog [79] uses semirings to support AD in the domain of probabilistic logic programming, which is used by DeepProbLog [98] for gradient-descent optimization in their backpropagation.

12.1.1 Symbolic Expressions and Their Evaluation

As before, we start from a minimal expression language, consisting of literals `lit(N)`, with N a number, variables `var(X)`, with X a variable identifier, addition `add(E1,E2)`, and multiplication `mul(E1,E2)`. The `eval(E,Env,N)` predicate evaluates expressions E to a numerical result N given an environment `Env` that maps the variables to their value.

```
eval(lit(N),      _Env,N) .
eval(var(X),      Env ,N) :- arg(X,Env,N) .
eval(add(E1,E2), Env ,N) :-
    eval(E1,Env,N1), eval(E2,Env,N2), N is N1 + N2 .
eval(mul(E1,E2), Env ,N) :-
    eval(E1,Env,N1), eval(E2,Env,N2), N is N1 * N2 .
```

To allow an $\mathcal{O}(1)$ lookup in the environment (in the `var` case), we use natural numbers in the range $[1,n]$ for the variable identifiers and represent the environment itself as a term `env(N1,...,Nn)` whose arguments are the values of the corresponding variables.

12.1.2 Symbolic Differentiation

Symbolic differentiation of our expressions follows the example of Clocksin and Mellish [27] and the textbook differentiation rules (Figure 8.2). The predicate `symb(E,X,F,DF)` computes *dual numbers*: the original expression (*primal*) $E=F$ and the (symbolic) partial derivative (*tangent*) $\frac{\partial E}{\partial X} = DF$.

```
symb(lit(N),      _X,lit(N),  lit(0)) .
symb(var(Y),      X, var(Y),  lit(DF)) :-
    ( X == Y -> DF = 1 ; DF = 0 ) .
symb(add(E1,E2), X,add(F1,F2),add(DF1,DF2)) :-
    symb(E1,X,F1,DF1), symb(E2,X,F2,DF2) .
symb(mul(E1,E2), X,mul(F1,F2),DF) :-
    symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
    DF = add(mul(F2,DF1),mul(F1,DF2)) .
```

12.1.3 Forward-Mode Automatic Differentiation

AD performs symbolic derivation and numerical evaluation simultaneously, in order to avoid the intermediate symbolic result. This is accomplished by `fwdad(E,X,Env,F,DF)`, where `F` and `DF` form a dual number, evaluated with respect to environment `Env`. This definition is obtained from a naive version that symbolically derives the expression first, and then evaluates it, by unfold/fold transformations [32] (Appendix H).

```
fwdad(lit(N),_X,_Env,N,0).
fwdad(var(Y),X,Env,F,DF) :-
    arg(Y,Env,F), ( X == Y -> DF = 1 ; DF = 0 ).
fwdad(add(E1,E2),X,Env,F,DF) :-
    fwdad(E1,X,Env,F1,DF1), fwdad(E2,X,Env,F2,DF2),
    F is F1 + F2, DF is DF1 + DF2.
fwdad(mul(E1,E2),X,Env,F,DF) :-
    fwdad(E1,X,Env,F1,DF1), fwdad(E2,X,Env,F2,DF2),
    F is F1 * F2, DF is F2 * DF1 + F1 * DF2.
```

Notice that, in the variable case, we find the kronecker delta function. Furthermore, thanks to `symb/4`'s *dual numbers*, the expression is evaluated *incrementally*, reusing intermediate results. This way the runtime remains linear in the size of the expression. When computing the *gradient* ∇E of an expression, we switch from the conventional dual numbers to our Nagata numbers, where the tangent is the whole gradient. Furthermore, we choose a sparse representation, which ignores intermediate results that are zero, using the `assoc` library² (based on AVL trees) as a partial map from variables X to the partial derivative $\frac{\partial E}{\partial X}$.

```
fwdadgrad(lit(N),_Env,N,DF) :-
    empty_assoc(DF).
fwdadgrad(var(Y),Env,F,DF) :-
    arg(Y,Env,F), singleton_assoc(Y,1,DF).
fwdadgrad(add(E1,E2),Env,F,DF) :-
    fwdadgrad(E1,Env,F1,DF1), fwdadgrad(E2,Env,F2,DF2),
    F is F1 + F2, union_with_assoc(plus,DF1,DF2,DF).
fwdadgrad(mul(E1,E2),Env,F,SDF) :-
    fwdadgrad(E1,Env,F1,DF1), fwdadgrad(E2,Env,F2,DF2),
    map_assoc(times(F2),DF1,SDF1), map_assoc(times(F1),DF2,SDF2),
    F is F1 * F2,
    union_with_assoc(plus,SDF1,SDF2,SDF).

plus(A,B,C) :- C is A + B.
times(A,B,C) :- C is A * B.
```

²We have extended the library with `union_with_assoc/4` and `insert_with_assoc/5`: established AVL operations of respectively $\mathcal{O}(n)$ and $\mathcal{O}(n \log n)$ time complexity.

12.1.4 Reverse-Mode Automatic Differentiation

The first step towards efficient reverse-mode AD is to replace the costly scalar multiplication of the gradient vector in the `mul/1` operation with a constant-time multiplication. This amounts to using semimodule homomorphisms for accumulating multiplications.

The accumulator is a scalar factor M , which is propagated down into the subexpressions. At the `var/1` and `lit/0` leaves it becomes trivial to take the factor M into account: in the former case we create a single $M \times 1 = M$ vector entry; in the latter case we ignore it as $M \times 0 = 0$ for all variables and 0 need not be explicitly represented. In the case of nested multiplications, we pass multiple factors $M_1, \dots, M_n$ down a path of the tree, which can be combined into a single factor $M = M_1 \times \dots \times M_n$. In the absence of a factor we take $M = 1$ (the neutral element of multiplication).

Traditional reverse-mode algorithms work in two passes: they compute the factors in a first pass and the derivatives in a second pass. This requires additional bookkeeping to store the appropriate information at different nodes in the tree during the first pass for later use in the second pass. With Prolog's co-routine mechanisms, this problem is much easier to solve; we only need to revise the definitions of `plus/3` and `times/3` to delay the arithmetic operations until their inputs are known. We use `freeze(N,Goal)` for this purpose, which defers the execution of `Goal` until `N` is bound. Co-routines replace the classical approaches of Wengert lists or tapes [172], and function abstractions to defer computations of tangents that depend on primals.

```
revad1(E,Env,F,DF) :- revad1(E,Env,1,F,DF).

revad1(lit(N),_Env,_M,N,DF) :-
    empty_assoc(DF).
revad1(var(Y),Env,M,F,DF) :-
    arg(Y,Env,F), singleton_assoc(Y,M,DF).
revad1(add(E1,E2),Env,M,F,DF) :-
    revad1(E1,Env,M,F1,DF1), revad1(E2,Env,M,F2,DF2),
    F is F1 + F2, union_with_assoc(plus,DF1,DF2,DF).
revad1(mul(E1,E2),Env,M,F,DF) :-
    times(M,F2,M1),times(M,F1,M2),
    revad1(E1,Env,M1,F1,DF1),revad1(E2,Env,M2,F2,DF2),
    F is F1 * F2,union_with_assoc(plus,DF1,DF2,DF).

plus(A,B,C) :- freeze(A,freeze(B,C is A + B)).
times(A,B,C) :- freeze(A,freeze(B,C is A * B)).
```


We now address the remaining costly operation: vector addition, which combines the vectors that are created at the leaves of the expression in a bottom-up fashion. We change this dataflow to one that threads the gradient vector through the computation and inserts elements where they are created (in the `var/1` case). To accomplish the threading we make use of Prolog's DCG notation (with association trees instead of lists), which allows us to implicitly pass the gradient. We initialize this vector with the neutral element of vector addition, i.e., the empty vector.

```
revad2(E,Env,F,DF) :-
    empty_assoc(DF0), revad2(E,Env,1,F,DF0,DF).

revad2(lit(N),_Env,_M,F) -->
    { F = N }.
revad2(var(Y),Env,M,F) -->
    { arg(Y,Env,F) }, insert_with_assoc(plus,Y,M).
revad2(add(E1,E2),Env,M,F) -->
    revad2(E1,Env,M,F1), revad2(E2,Env,M,F2), { F is F1 + F2 }.
revad2(mul(E1,E2),Env,M,F) -->
    { times(M,F2,M1) }, { times(M,F1,M2) },
    revad2(E1,Env,M1,F1), revad2(E2,Env,M2,F2), { F is F1 * F2 }.
```

The resulting algorithm has $\mathcal{O}(N \log V)$ time complexity, where the logarithmic factor is due to `insert_with/3`. We replace the AVL tree representation of the gradient vector with an array to achieve an overall $\mathcal{O}(N + V)$ time complexity. The two operations we need to replace are the initialization of the gradient and the insertion of a new value. Initialization goes from $\mathcal{O}(1)$ for an empty AVL tree to $\mathcal{O}(V)$ to create a new array with an explicit 0 entry for each variable. Luckily, this operation is only performed once, at the start of the algorithm.

```
empty_array(N,Arr) :-
    findall(0,between(1,N,_),List),
    Arr =.. [grad|List].
```

Insertion now acts in constant time due to the `setarg/3` destructive update.

```
insert_with_array(Pred,Var,Value,Arr) :-
    arg(Var,Arr,OldValue),
    call(Pred,OldValue,Value,NewValue),
    setarg(Var,Arr,NewValue).
```

The impact of these representation changes on reverse AD itself is minimal.

```
revad(E,Env,F,DF) :-
    functor(Env,_,N), empty_array(N,DF), revad(E,Env,1,F,DF).
```

```

revad(lit(N),_Env,_M,F,_DF) :-
  F = N.
revad(var(Y),Env,M,F,DF) :-
  arg(Y,Env,F), insert_with_array(plus,Y,M,DF).
revad(add(E1,E2),Env,M,F,DF) :-
  revad(E1,Env,M,F1,DF), revad(E2,Env,M,F2,DF), F is F1 + F2.
revad(mul(E1,E2),Env,M,F,DF) :-
  times(M,F2,M1), times(M,F1,M2),
  revad(E1,Env,M1,F1,DF), revad(E2,Env,M2,F2,DF), F is F1 * F2.

```

12.2 Sum-Product Loop Programming

Pfanschilling et al. [116] have recently developed the probabilistic Sum-Product Loop Language (SPLL) in Prolog. As a case study, we have implemented this language, and its parameter estimation functionality, with our AD algorithms.

12.2.1 Parameter Estimation

An SPLL program defines a probability distribution over its possible outcomes. For instance, `main = Uniform >= Theta[1]` transforms the uniform distribution on the interval $[0, 1]$ into a Bernoulli distribution: $p(\text{false}) = \theta_1$ and $p(\text{true}) = 1 - \theta_1$. Formally, SPLL comes with both a generative and a probabilistic semantics. The former samples the program (i.e., generates a result x) in accordance with its probability distribution, and the latter computes the probability $p(x)$ of a given sample x . The probabilistic semantics can be used to estimate the parameters θ of a program in terms of a set of samples X . Specifically, given a set of samples X and a consistent SPLL program, find the parameterization θ of the program that minimizes the negative log-likelihood $\mathcal{L}$ of all samples: $\mathcal{L} = \sum_{x \in X} -\log p(x|\theta)$.

This problem can be tackled with gradient-based optimization, where the gradient $\frac{\partial \mathcal{L}}{\partial \theta}$ is computed with automatic differentiation. Indeed, given a learning rate λ and an initial guess for the parameters θ , we can iteratively improve the parameters with $\theta := \theta - \lambda \frac{\partial \mathcal{L}}{\partial \theta}$. We have implemented a symbolic version of the probabilistic semantics, which yields for our small example program

$$p(\text{true}) = \int_{\theta_1}^{\infty} \varphi_U(x) dx \qquad p(\text{false}) = 1 - \int_{\theta_1}^{\infty} \varphi_U(x) dx$$

Given an initial guess $\theta_1 = 0.5$, a learning rate of 0.02, and a set X of 3 `false` samples and 7 `true` samples, the parameter converges to 0.3000000000000001 in 13 steps.

12.2.2 **Benchmarking our AD Versions**

To compare the runtime of the different AD versions we have used the following SPLL program, which features six parameters $\theta_1, \dots, \theta_6$.

```
main = if Uniform >= Theta[1]
      then if Uniform >= Theta[2]
            then if Uniform >= Theta[3]
                  then null      else [true]
            else if Uniform >= Theta[4]
                  then [false]   else [true,true]
      else if Uniform >= Theta[5]
            then if Uniform >= Theta[6]
                  then [true,false] else [false,true]
            else [false,false]
```

We have used the same gradient-based optimization of the negative log-likelihood to simultaneously learn the six parameters. Given an initial guess of 0.5 for θ_1 and 0.25 for the others, a learning rate of 0.02 and sets of three examples for each outcome, the parameters converge to the following in 100 iterations:

Parameter	Learned Value	Optimal Value
θ_1	0.4285714285714287	3/7
θ_2	0.5	1/2
θ_3	0.49999999999999994	1/2
θ_4	0.49999999999999994	1/2
θ_5	0.3333333333333333	1/3
θ_6	0.49999999999999994	1/2

We have also measured the runtime of this experiment (repeated 100 times) using our four different AD algorithms. These measurements were performed on a Macbook Pro with M1 chip, 16GB RAM, Ventura 13.1, with SWI-Prolog version 8.4.1 for arm64-darwin.

AD version	fwdadgrad	revad1	revad2	revad
Runtime (s)	9.346	9.098	5.768	4.909

These results demonstrate the improvements achieved by the successive algorithms. Our aim is not to provide competitive performance to popular, highly-optimized machine learning libraries, but rather to take a view on AD that makes it more instructive and accessible for the (logic) programming community.

12.3 Matrices

Square matrices form a non-commutative semiring. Indeed, matrix multiplication is not commutative:

$$\begin{pmatrix} 5 & 5 \\ 10 & 10 \end{pmatrix} \otimes \begin{pmatrix} 5 & 10 \\ 5 & 10 \end{pmatrix} = \begin{pmatrix} 50 & 100 \\ 100 & 200 \end{pmatrix} \neq \begin{pmatrix} 125 & 125 \\ 125 & 125 \end{pmatrix} = \begin{pmatrix} 5 & 10 \\ 5 & 10 \end{pmatrix} \otimes \begin{pmatrix} 5 & 5 \\ 10 & 10 \end{pmatrix}$$

We represent square matrices in Haskell by the datatype *Square n d*, where *n* reflects the dimensions of the matrix ($n \times n$) in its type, and *d* is the type of the matrix elements.

```
data Square n d = Square { dims :: Int, cont :: [[d]] }
```

For example, the matrix $\begin{pmatrix} 5 & 5 \\ 10 & 10 \end{pmatrix}$ is represented in Haskell by:

```
Square 2 [[5,5],[10,10]]
```

The type *Square n d* indeed forms a semiring, if *d* is a semiring³.

```
instance (KnownNat n, Semiring d) => Semiring (Square n d) where
  zero = Square n (replicate n (replicate n zero))
  where n = fromInteger (natVal@n Proxy)
  one = Square n [[if r ≡ c then one else zero | c <- [0..n-1]]
                                | r <- [0..n-1]]
  where n = fromInteger (natVal@n Proxy)
  m1 ⊕ m2 = Square (dims m1)
    (zipWith (zipWith (⊕)) (cont m1) (cont m2))
  m1 ⊗ m2 = Square (dims m1)
```

³For the definition of the functions that are used in this Semiring instance, we refer to Appendix G. For the functionality of type-level numbers (*KnownNat*, *natVal*,...), we refer to Haskell's *GHC.TypeLits* library.

$$[[\text{foldr } (\oplus) \text{ zero } (\text{zipWith } (\otimes) \text{ r c}) \mid c \leftarrow \text{transpose } (\text{cont } m_2)] \\ \mid r \leftarrow (\text{cont } m_1)]]$$

Here, *zero* is the zero matrix $\begin{pmatrix} 0 & 0 \\ 0 & \dots \end{pmatrix}$, *one* is the identity matrix $\begin{pmatrix} 1 & 0 \\ 0 & \dots \end{pmatrix}$, $(\oplus)$ is matrix addition (which comes down to element-wise addition of the underlying semiring d), and $(\otimes)$ is matrix multiplication. Notice that matrix multiplication is defined as follows:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \otimes \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} = \begin{pmatrix} a_{11} \otimes b_{11} \oplus a_{12} \otimes b_{21} & a_{11} \otimes b_{12} \oplus a_{12} \otimes b_{22} \\ a_{21} \otimes b_{11} \oplus a_{22} \otimes b_{21} & a_{21} \otimes b_{12} \oplus a_{22} \otimes b_{22} \end{pmatrix}$$

This is not to be confused with the element-wise product of $(n \times n)$ -matrices, the inner product (which results in a scalar), and the kronecker product (which results in an $(n^2 \times n^2)$ -matrix).

Consider the matrices a and b , with as elements expressions of X .

$$a = \begin{pmatrix} \text{Var } X & \text{Var } X \\ \text{Plus } (\text{Var } X) (\text{Var } X) & \text{Plus } (\text{Var } X) (\text{Var } X) \end{pmatrix}$$

$$b = \begin{pmatrix} \text{Var } X & \text{Plus } (\text{Var } X) (\text{Var } X) \\ \text{Var } X & \text{Plus } (\text{Var } X) (\text{Var } X) \end{pmatrix}$$

To compute the derivative of $\text{Times } a \text{ b} :: \text{Expr } (\text{Square } 2 (\text{Expr } X))$, we work in two steps. First, we evaluate the outer expression of matrices, with non-commutative multiplication, in terms of a matrix. Second, we evaluate the expressions in the matrix, which form a commutative semiring element-wise.

$$\begin{aligned} \text{derive}_{\text{matrix}} &:: (\text{Semiring } d, \text{KnownNat } n) \\ &\Rightarrow (v \rightarrow d) \rightarrow \text{Expr } (\text{Square } n (\text{Expr } v)) \rightarrow \text{Square } n (d \times d) \\ \text{derive}_{\text{matrix}} \text{ var} &= \text{fmap } (\text{eval } \text{gen}) . \text{eval id} \\ &\textbf{where } \text{gen } x = N (\text{var } x) (\text{delta } x) \end{aligned}$$

Here, *delta* is defined by as *const one*. Notice that $\text{fmap } (\text{eval } \text{gen}) :: \text{Square } n (\text{Expr } v) \rightarrow \text{Square } n (d \times d)$ is a semiring homomorphism. Property 1 says we can fuse this homomorphism into *eval*, which gives the following definition for *derive_{matrix}*:

$$\begin{aligned} \text{derive}_{\text{matrix}} &:: (\text{Semiring } d, \text{KnownNat } n) \\ &\Rightarrow (v \rightarrow d) \rightarrow \text{Expr } (\text{Square } n (\text{Expr } v)) \rightarrow \text{Square } n (d \times d) \\ \text{derive}_{\text{matrix}} \text{ var} &= \text{eval } (\text{fmap } (\text{eval } \text{gen})) \\ &\textbf{where } \text{gen } x = N (\text{var } x) (\text{delta } x) \end{aligned}$$

With this fusion, we avoid the intermediate datastructure of *Square n (Expr v)*. Applying this function to *Times a b* with $X = 5$, gives the following result:

```
>>> derivematrix ( $\lambda X \rightarrow 5$ ) (Times a b)
```

$$\begin{pmatrix} N\ 50\ 20 & N\ 100\ 40 \\ N\ 100\ 40 & N\ 200\ 80 \end{pmatrix}$$

```
>>> derivematrix ( $\lambda X \rightarrow 5$ ) (Times b a)
```

$$\begin{pmatrix} N\ 125\ 50 & N\ 125\ 50 \\ N\ 125\ 50 & N\ 125\ 50 \end{pmatrix}$$

12.4 Possible worlds

This Section presents a functional account of a probabilistic logic programming method: possible worlds⁴. We believe that this account is of interest because it combines a typical method of probabilistic logic programming with classic functional programming concepts, enabling further cross-fertilization between the two fields.

Functional Programming vs. (Probabilistic) Logic Programming Although many aspects of functional programming and logic programming are of common interest, both communities have a somewhat different focus, with the former tending more towards processes (“doing”), and the latter more towards models (“being”). This is quite prominent in the functional view of logic programming, which mostly focuses on the operational aspects such as backtracking search [144, 62, 83, 118], least fixed-points of nondeterministic computations [163], and unification algorithms [138, 24]. In contrast, non-operational matters, such as well-founded semantics [161] and stable model semantics [47] have received less attention from the functional programming community. There is a range of languages that combine functional and logic programming, such as Curry [34], DataFun [4], Mercury [143], miniKanren [20], and Toy [96].

Of particular interest here is the subarea of probabilistic programming, which, under the aegis of artificial intelligence, is receiving much attention. Probabilistic programming has been based on the monadic structure of

⁴The accompanying code can be found at <https://github.com/birthevdb/functional-possible-worlds>.

probability distributions [55, 125, 40]. Again, there are many languages that combine elements of both functional and probabilistic programming, such as Anglican [89], Church [57], and Hakaru [109].

However, and as far as we know, there is very little work that combines all three, and examines, for example, functional models of probabilistic logic programming. Gibbons [49] studies the possible interactions between nondeterministic and probabilistic choice. Vandenbroucke and Schrijvers [162] propose an applicative-functor model of ProbLog Kimmig et al. [79] have argued that weighted model counting should be generalized to arbitrary commutative semirings, and Belle and De Raedt [12] have argued that commutative semirings have many artificial intelligence applications.

Distribution Semantics of Programming with Possible Worlds In what follows, we provide a functional programming incarnation of probabilistic logic programming based on Sato’s distribution semantics [131], which is derived from intensionally modeling possible worlds in terms of queries over facts and associating probability distributions with those facts. This approach underlies most contemporary probabilistic logic programming languages, such as Probabilistic Logic Programs [28], Probabilistic Horn Abduction [122], Independent Choice Logic [123], PRISM [131], LPADs [164], ProbLog [41], and PITA [127].

It is not only suited to compute the probability of queries via possible worlds, but also to learn the individual probabilities of the facts.

12.4.1 Facts, Worlds and Queries

Consider the following running example: *Toss a coin and draw a ball from each of two urns. The two urns contain both red and blue balls. You win if you toss heads and draw a red ball or if you draw two balls of the same color.*

Facts and Worlds In logic programming, the subsets W of a finite set B (usually called the Herbrand base) of possible *facts* $A_1, A_2, \dots$ are known as *worlds*.

The facts in our example are *Heads*, *Red₁* and *Red₂*.

data $F = \text{Heads} \mid \text{Red}_1 \mid \text{Red}_2$

We use designated types, which we name fact types, and require that they provide a finite number of facts: the method *facts* of type class *Fact f* enumerates them.

```

class Ord f  $\Rightarrow$  Fact f
  where facts :: [f]

instance Fact F where
  facts = [Heads, Red1, Red2]

```

For n facts, there are 2^n worlds: (sub)sets of values of the fact type.

```

type World f = Set f

```

Among all the worlds, only some may be possible.

Queries The possible worlds are typically expressed intensionally, by means of a logical formula Q , often called a *query*, over the facts. A query expresses those worlds in which the formula holds.

The free monad $Q f a$ captures queries over facts f . It is the nondeterminism monad, supplemented with query literals, which are positive or negative occurrences of a fact.

```

data Q f a where
  Var :: a  $\rightarrow$  Q f a
  Fail :: Q f a
  Or  :: Q f a  $\rightarrow$  Q f a  $\rightarrow$  Q f a
  Lit :: Lit f  $\rightarrow$  Q f a  $\rightarrow$  Q f a

instance Monad (Q f) where
  return      = Var
  Var x  $\gg$  k = k x
  Fail  $\gg$  k = Fail
  Or p q  $\gg$  k = Or (p  $\gg$  k) (q  $\gg$  k)
  Lit f p  $\gg$  k = Lit f (p  $\gg$  k)

```

$Q f a$ is a monad, and *Fail* and *Or* provide an *Alternative* structure.

```

data Lit f = Pos f | Neg f

instance Alternative (Q f)
  where empty = Fail
        (<|>) = Or

```

The monadic structure of $Q f a$ aligns with the conjunctive structure of queries, while the *Fail* and *Or* constructors align with their disjunctive structure. With this, we can write the *win* query as follows (using smart constructors for *heads*, *red₁*, *red₂*, *blue₁* and *blue₂*).

```

win = (heads  $\gg$  (red1 <|> red2)) <|> ((red1  $\gg$  red2) <|> (blue1  $\gg$  blue2))

```


Probabilities Probabilities are introduced at the level of facts. An interpretation function associates each fact with a probability. For example,

```

type Prob = Float
prob :: Lit F → Prob
prob (Pos Heads) = 0.4
prob (Pos Red1)  = 0.3
prob (Pos Red2)  = 0.2
prob (Neg f)      = 1 - prob (Pos f)

```

The probability $P(W)$ of a world W follows from its facts.

$$P(W) = \prod_{A_i \in W} I(A_i) \times \prod_{A_i \notin W} (1 - I(A_i))$$

For example, we compute the probabilities of the worlds:

$$\begin{aligned}
P(W_1) &= (1 - 0.4) \times (1 - 0.3) \times (1 - 0.2) = 0.336 \\
P(W_2) &= (1 - 0.4) \times (1 - 0.3) \times 0.2 = 0.084 \\
P(W_3) &= (1 - 0.4) \times 0.3 \times (1 - 0.2) = 0.144 \\
P(W_4) &= (1 - 0.4) \times 0.3 \times 0.2 = 0.036 \\
P(W_5) &= 0.4 \times (1 - 0.3) \times (1 - 0.2) = 0.224 \\
P(W_6) &= 0.4 \times (1 - 0.3) \times 0.2 = 0.056 \\
P(W_7) &= 0.4 \times 0.3 \times (1 - 0.2) = 0.096 \\
P(W_8) &= 0.4 \times 0.3 \times 0.2 = 0.024
\end{aligned}$$

Likewise, the probability of a query is the sum of the probabilities of the worlds in which it holds.

$$P(Q) = \sum_{W \models Q} P(W)$$

You win the game when you toss heads and draw a red ball, or when you draw two red balls or two blue balls. The worlds that model this query are W_1 , W_4 , W_5 , W_6 , W_7 and W_8 . The probability to win is then computed as follows:

$$\begin{aligned}
P(\text{Win}) &= P(W_1) + P(W_4) + P(W_5) + P(W_6) + P(W_7) + P(W_8) \\
&= 0.336 + 0.036 + 0.224 + 0.056 + 0.096 + 0.024 \\
&= 0.772
\end{aligned}$$

12.4.2 Possible Worlds as Boolean Algebras

As usual with the free monad approach, $Q f a$ queries do not respect any particular laws other than the monad laws on the nose. Instead, the interpretation function (handler) ensures that the interpretations that they produce respect further laws. To enable this, we provide a structural recursion scheme:

$$\begin{aligned}
 fold_Q &:: BooleanAlgebra\ b \Rightarrow (a \rightarrow b) \rightarrow (f \rightarrow b) \rightarrow Q\ f\ a \rightarrow b \\
 fold_Q\ gen\ inj\ (Var\ x) &= gen\ x \\
 fold_Q\ gen\ inj\ Fail &= \perp \\
 fold_Q\ gen\ inj\ (Or\ p\ q) &= fold_Q\ gen\ inj\ p \vee fold_Q\ gen\ inj\ q \\
 fold_Q\ gen\ inj\ (Lit\ (Pos\ f)\ p) &= inj\ f \quad \wedge\ fold_Q\ gen\ inj\ p \\
 fold_Q\ gen\ inj\ (Lit\ (Neg\ f)\ p) &= (\neg)\ (inj\ f) \quad \wedge\ fold_Q\ gen\ inj\ p
 \end{aligned}$$

This recursion scheme enforces that queries are interpreted in terms of a boolean algebra [15], which generalizes the notion of booleans and thus provide a natural framework for the interpretation of logical queries. Boolean algebras are captured in a type class and are subject to the laws in Figure 12.1.

Commutativity	Distributivity
$a \wedge b \equiv b \wedge a$	$a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c)$
$a \vee b \equiv b \vee a$	$a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$
Universal Bounds	Associativity
$\perp \wedge a \equiv \perp$	$a \wedge (b \wedge c) \equiv (a \wedge b) \wedge c$
$\perp \vee a \equiv a$	$a \vee (b \vee c) \equiv (a \vee b) \vee c$
$\top \wedge a \equiv a$	Idempotence
$\top \vee a \equiv \top$	$a \wedge a \equiv a \vee a \equiv a$
Complementation	Absorption
$a \wedge \neg a \equiv \perp$	$a \wedge (a \vee b) \equiv a \vee (a \wedge b) \equiv a$
$a \vee \neg a \equiv \top$	

Figure 12.1: Laws of boolean algebras.

```

class BooleanAlgebra b where
  ⊥ :: b
  ⊤ :: b
  (¬) :: b → b

```

$$\begin{aligned}
(\wedge) &:: b \rightarrow b \rightarrow b \\
(\vee) &:: b \rightarrow b \rightarrow b
\end{aligned}$$

The standard instance for boolean algebras is that of *Bool*, but also the function type $w \rightarrow b$ is a boolean algebra, provided that b is a boolean algebra. Of particular interest is the boolean algebra of powersets (i.e., subsets of a set) of facts, because then the set represents a world. The *BooleanAlgebra* instance uses functions from Haskell's *Data.Set* library (Appendix G).

```

instance Fact f  $\Rightarrow$  BooleanAlgebra (Set (World f)) where
   $\perp$  = Set.empty
   $\top$  = Set.fromList [Set.fromList l | l  $\leftarrow$  subsequences facts]
  ( $\neg$ ) = Set.difference  $\top$ 
  ( $\wedge$ ) = Set.intersection
  ( $\vee$ ) = Set.union

```

With this boolean algebra, we can compute the possible worlds of a query:

```

possibleWorlds :: Fact f  $\Rightarrow$  Q f a  $\rightarrow$  Set (World f)
possibleWorlds = foldQ (const  $\top$ ) lit where lit f = Set.filter (elem f)  $\top$ 

```

```

>>> possibleWorlds win
{ {}, { Heads }, { Heads, Red1 }, { Heads, Red1, Red2 },
  { Heads, Red2 }, { Red1, Red2 } }

```

12.4.3 Probabilities as Commutative Semirings

The typical interpretation of queries in probabilistic logic programming, however, is their probability, given the independent probabilities of the facts. Unfortunately, probabilities do not form a boolean algebra with $(\times)$ for $(\wedge)$ and $(+)$ for $(\vee)$. Indeed, several of the boolean algebra laws are not satisfied: idempotence, absorption, $\top \vee a \equiv \top$, and $a \wedge \neg a \equiv \perp$.

Hence, the probability of queries cannot be expressed compositionally in terms of a fold. This makes sense at a conceptual level where we know, for example, that in general it holds that $P(Q_1 \vee Q_2) = P(Q_1) + P(Q_2) - P(Q_1 \wedge Q_2)$ which contains the non-compositional summand $-P(Q_1 \wedge Q_2)$. Also, in general we compute $P(Q_1 \wedge Q_2) = P(Q_1) \times P(Q_2|Q_1)$ where we read $P(Q_2|Q_1)$ as the probability of Q_2 given Q_1 .

Restricted forms of queries do admit compositional computation of probabilities:

Determinism: Two queries Q_1 and Q_2 are *mutually exclusive* when no world makes both queries true. This means, we get the compositional form

$$P(Q_1 \vee Q_2) = P(Q_1) + P(Q_2)$$

Here, indeed, $P(Q_1 \wedge Q_2)$ equals zero. A query in which all branches of or-nodes are mutually exclusive is known as *deterministic*.

Decomposability Similarly, the probability of the conjunction of two queries Q_1 and Q_2 is compositional if they are independent. Then

$$P(Q_1 \wedge Q_2) = P(Q_1) \times P(Q_2)$$

Thus, in this case, $P(Q_2|Q_1) = P(Q_2)$, making Q_2 independent of Q_1 . A query in which this is true for all children of and-nodes is known as *decomposable*.

Queries that are deterministic, decomposable and also only feature negations on facts are said to be in *deterministic, decomposable negation normal form (d-DNNF)* [29, 30]. For example, the query on the left is equivalent to its deterministic, decomposable negation normal form on the right:

$$(heads \wedge red_1) \vee (red_1 \wedge red_2) \equiv ((tails \wedge red_2) \vee heads) \vee red_1$$

Such d-DNNF queries do have a well-defined mapping from possible worlds to probabilities, and, more generally, to any other interpretation that satisfies the commutative semiring laws. Probabilities are indeed a special case of commutative semirings.

The main advantage of the d-DNNF is that we can interpret queries in linear time. An example of a d-DNNF is the possible-worlds representation, which is essentially isomorphic to a query with the d-DNNF structure. That is, it corresponds to a disjunction of mutually exclusive worlds. Each world is a conjunction of independent facts. In this form, $\top$ and $\perp$ are absent, and negations are in normal form: they only appear on facts.

We define a valuation function *valuate* that interprets possible worlds in terms of a commutative semiring. Given an interpretation of the literals in terms of a commutative semiring, it valuates a set of possible worlds over those literals.

$$\begin{aligned} &valuate :: (Fact\ f, Semiring\ d) \Rightarrow (Lit\ f \rightarrow d) \rightarrow Set\ (World\ f) \rightarrow d \\ &valuate\ gen = sum.\ map\ (prod.\ lits).\ Set.toList\ \mathbf{where} \\ &prod = foldr\ (\otimes)\ one \end{aligned}$$

```

sum = foldr (⊕) zero
lits s = map lit facts where
  lit f | Set.member f s = gen (Pos f)
        | otherwise = gen (Neg f)

```

Interpreting a query means computing its possible worlds and valuating them:

```

>>> valuate prob (possibleWorlds win)
0.7720001

```

Notice that the *valuate* function allows additional interpretations beyond probabilities. The possible-worlds representation is not the only d-DNNF; several compilers exist that use heuristics to efficiently compute compact d-DNNFs for a query, such as c2d [31], dsharp [107] and d4 [87].

Notice that, while the boolean algebra interpretation is naturally compositional, that is not the case with the commutative semiring interpretation. Indeed, the normalization procedure is a whole program transformation. This means that there is no room for variables as non-terminals with arbitrary interpretations. Thus, although the interpretation of a query in terms of a commutative semiring is more expressive and efficient than that in terms of a boolean algebra, it also requires an additional normalization step, which forces us to use a trivial implementation for interpreting variables. Ongoing research, especially on so-called lifted inference techniques [159, 160], develops additional optimization heuristics to exploit common structure in subqueries.

12.4.4 Automatic Differentiation to Learn Probabilities

We can also *learn* the facts' probabilities using a gradient-descent style learning, based on the observed probability of a query.

Given a set $\mathcal{S}$ of 10 samples of the *win* query, where 7 are true and 3 are false, we compute the negative log-likelihood of all samples as follows:

$$\begin{aligned} \mathcal{L} = \sum_{x \in \mathcal{S}} -\log p(s|gen) &= -7 \times \log (valuate \ gen \ (possibleWorlds \ win)) \\ &\quad - 3 \times \log (1 - valuate \ gen \ (possibleWorlds \ win)) \end{aligned}$$

Here, *gen* maps the positive and negative literals to variable expressions.

```

gen :: Lit F → Expr F
gen (Pos Heads) = Var Heads

```

$$\begin{aligned} \textit{gen}(\textit{Pos Red}_1) &= \textit{Var Red}_1 \\ \textit{gen}(\textit{Pos Red}_2) &= \textit{Var Red}_2 \\ \textit{gen}(\textit{Neg } f) &= \textit{Plus One}(\textit{Minus}(\textit{gen}(\textit{Pos } f))) \end{aligned}$$

To use our abstract automatic differentiation algorithm, we have to extend our expression language with logarithms and negatives (Section 11.2). Thus, instead of with commutative semirings, we are working with fields.

We take an initial guess for the probabilities of the facts, and update them after each iteration with a learning rate of 0.05. The results are summarized in the following table.

	<i>Heads</i>	<i>Red</i> ₁	<i>Red</i> ₂	<i>win</i>
initial guess	0.5	0.25	0.25	0.8125
update 1	0.36153847	0.3423077	0.3423077	0.7125223
update 2	0.3477752	0.34846267	0.3484627	0.7038424
update 3	0.34359017	0.35028455	0.35028458	0.7012216
...	...	...	...	...

The probability of our *win* query approaches 70%, which is what we expect with a sample of 7 out of 10 positive examples.

Chapter 13

Summary

Having identified three algebraic abstractions, we can write symbolic differentiation, forward-mode and reverse-mode AD as different instances of one and the same abstract algorithm. A significant contribution is the observation that we can generalize the well-known dual numbers into Nagata numbers $d \ltimes e$ for any d -module e , which allows the e parameter to vary independently of the semiring d of coefficients, and that we can use abstract Kronecker deltas to represent partial derivatives. Furthermore, guided by structure-preserving maps in the form of Kronecker isomorphisms, we have taken successive steps in optimizing the representation, in a correct-by-construction manner, from basic symbolic differentiation via forward-mode AD to the (typically) much more efficient reverse-mode AD. We have discussed extensions to rings, fields, and have included let-sharing to support richer expressions. Furthermore, we have laid the foundations for higher derivatives and matrix differentiation with square matrices. We have shown how to exploit these ideas for probability learning in the Sum-Product Loop Language and for possible-worlds semantics.

A Note on Performance Many machine learning libraries come with highly optimized automatic differentiation implementations (e.g., with tailored memory management techniques and GPU leverage), such as Tensorflow’s [1] and PyTorch’s [110] Autograd, which targets Python code. Google’s JAX [19] extends Autograd with just-in-time compilation, inspired by Tensorflow’s XLA (accelerated linear algebra). The probabilistic programming language Stan [22, 23] leverages C++ to implement automatic differentiation. Wang et al. [169] show that the two passes of reverse-mode AD can be conveniently implemented using delimited control in an impure functional setting like Scala. Kmett’s `ad`

library [85] is a Haskell implementation based on Pearlmutter and Siskind’s work [114], which explained how to incorporate AD as a first-class function in a functional language. The library exploits Haskell’s abstraction and overloading mechanisms to support different AD modes.

These and others easily outperform our implementations, whose aim is not to be competitive in terms of performance. Instead, we provide an algebraic account of AD that aims to be comprehensible, reasoning about intensional behavior.

Related Work In recent years, much effort has been devoted to establishing the correctness of different AD variants, and to extending the supported language features. Most works focus on the operational semantics of formal calculi they define, and reason about the correctness of their AD approaches in operational terms. Wang et al. [169] derive forward-mode and reverse-mode AD from symbolic differentiation, but do not elaborate on the connection between the two; we believe our work is complementary to this. Smeding and Vákár [142] derive several variants of reverse-mode AD. They use “well-known program transformations”, reasoning mostly in algorithmic terms.

Krawiec et al. [86] also go through several steps of refinement, from a standard forward-mode AD to an efficient reverse-mode variant. They use a defunctionalized form of our $d \multimap STCayley\ v\ d$ representation that resembles the *Wengert list* [172]. They cover additional language features, which we believe can also be incorporated into our approach, such as support for higher-order functions¹, sum and product types. A virtue of their paper is in identifying criteria for a (successful!) approach to AD: that it should demonstrate efficient reverse-mode AD; that it be higher-order; that it be asymptotically efficient; and that it be provably correct. Our approach, thanks to our careful algebraic treatment, ensures that (functional) correctness is more or less immediate via our use of Kronecker type-class isomorphisms; efficiency relative to the computation of the primal follows from our use of a single abstract computation *abstractD*; and our ability to capture all the various modes of AD may be seen as arising by careful instantiation of Nagata’s fundamental construction. Krawiec et al. [86] give only informal arguments for the relation between different variants, but the correctness proof of their final version constitutes a non-trivial effort involving a Kripke logical relation, inspired by earlier work of Huot et al. [71, 72] on a logical-relation-based correctness proof of a basic forward-mode algorithm for a higher-order language.

¹Observe that features like higher-order functions are parametric with respect to the semiring type and can thus be used as is.

Chapter 14

Conclusion

This concluding Chapter summarizes the work of this thesis, discusses future work and the elegance and generality of abstract algebra.

14.1 Global Summary

We have summarized the two parts of this work in their respective conclusions. The overall goal of this thesis is to combine abstract algebra and functional modeling to reveal novel, algebraic perspectives on computational effects and automatic differentiation.

Part I presents a generic framework for higher-order effects & handlers based on a free-forgetful adjunction. It shows that different (existing and novel) classes of effects are instances of this framework. We provide a range of examples and an implementation in Haskell in order to facilitate the adoption of these effects in programming languages and libraries.

Part II provides an algebraic perspective on automatic differentiation and shows that different variants can be written as instances of a single-line definition. Semimodules, Nagata numbers and Kronecker's delta function form the core of this one-liner. We show various extensions and applications and aim to show that AD is surprisingly simple from this algebraic point of view.

14.2 Future Work

This thesis facilitates various strains of possible future work that could be pursued from here.

A Calculus for Higher-Order Effects In line with the calculus we have proposed in Chapter 6 for modeling algebraic- and scoped effects & handlers as built-in syntactic primitives, and with the insight that scoped effects are a subset of higher-order effects, we could define a calculus for algebraic and higher-order effects & handlers. We have laid the first bricks by extending λ_{sc} to also support parallel effects, following the work of Xie et al. [174], the latter of which is untyped. To distinguish these different kinds of effects, we use different keywords, e.g., **sc** and **for**. Alternatively, this distinction can be made by smart management of labels. Particularly challenging is the forwarding of unknown effects. Whereas for scoped effects, this forwarding was ad-hoc but relatively intuitive in most cases, forwarding higher-order effects is a challenge. Indeed, we are not even sure that writing a forwarding clause is possible for each combination of effects.

Generic Forwarding Generically forwarding higher-order effects in their current form seems not possible. A forwarding function $\text{fwd} :: k\ f\ (f\ a) \rightarrow f\ a$ is dependent both on the specific higher-order functor k and functor f . For specific combinations of k and f , such as for the examples in this work, a forwarding function can be defined. However, we cannot confidently say that this is possible for *all* combinations of k and f .

For that reason, we assume in our handler definitions that a forwarding function for the combination of effects at hand exists. We do so by capturing the forwarding functionality in a type class, and in each modular handler, we add this type class as a constraint. This approach is inspired by the modular carriers of Schrijvers et al. [132].

Future work could investigate under which conditions or for which effects a more generic forwarding clause can be written, if that is even possible.

Reasoning about Higher-Order Effects Yang and Wu [178] explain how they can leverage fusion to reason about effect interactions for algebraic operations. They already hint that a similar approach for scoped effects would be interesting to look at, and in their follow-up work on handling scoped effects [176], they present a fusion law for functorial algebras. In the future, we would like to investigate how we can reason about higher-order effects using fusion.

A possible way to start this strain of work is to look at generalized folds [14] for nested datatypes. Our free monad $Free_H k a$ forms a nested datatype [14], in contrast with $Free \sigma a$ for algebraic effects, which is a regular parametric datatype. The regular fold for nested datatypes has polymorphic parameters (in our case the algebra), and generalized folds overcome this. Such generalized fold has a uniqueness property and thus allows reasoning about nested datatypes. Hinze [63] and Martin et al. [100] have proposed more efficient and disciplined variations of these generalized folds.

Automatic Differentiation for Matrices with Arbitrary Dimensions Matrix differentiation forms the backbone of machine learning, whose models involve a large number of parameters, often weights, structured as matrices. Backpropagation, a special case of automatic differentiation is then used to optimize these weights. In Section 12.3 we have shown that square matrices form a (non-commutative) semiring and that our framework applies. However, supporting matrices of arbitrary dimensions is non-trivial, in particular when reflecting these dimensions in the types, and would be interesting to investigate further in the future.

Different Applications of Semirings and Dual Numbers In Section 8.2, we have already listed various different semirings and applications. Dual numbers form a commutative semiring that has applications not only in automatic differentiation, but also in robotics [59] and physics [168].

An interesting track of future work could be to investigate whether the abstractions and algebraic perspectives described in this work translate to different contexts and applications of semirings and dual numbers.

Competitive Performance Common libraries that use automatic differentiation or backpropagation in Haskell [85, 88, 21, 145] employ different kinds of optimizations, such as lenses, foreign function interfaces to C, and so on. Although the main focus of this work has been on the algebraic abstractions and their difference in computational complexities, it would be interesting to also investigate their performances in comparison with these libraries, and the impact of similar optimization techniques.

14.3 Abstract Algebra is Elegant and General

In the introduction of this work, we have argued the following:

By combining the elegance and generality of abstract algebra with the descriptive power of functional modeling, we can unlock novel perspectives on well-known and lesser-known concepts in the design and application of programming languages.

In Part I, we have used algebraic structures and category theory to construct a framework that generalizes higher-order effects. This framework structures and unifies the zoo of effects & handler dialects that have arisen over time in a single framework. It is one application of where abstract algebra provides an elegant solution to a complex matter, which, when abstracted over, seems not so complex anymore.

In Part II, we have been able to capture the essence of automatic differentiation in a single line, by means of three abstract algebraic structures. Again, this is an application of where abstract algebra helps to gain insights into a topic that is sometimes perceived as “mystical” [169]. Furthermore, these insights also have an important didactic added value from the subject.

In conclusion, by applying abstract algebra to computational effects and automatic differentiation, two popular topics of study in computer science, we believe we have succeeded in providing novel perspectives on these matters that are both elegant and general.

Bibliography

- [1] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., KUDLUR, M., LEVENBERG, J., MONGA, R., MOORE, S., MURRAY, D. G., STEINER, B., TUCKER, P. A., VASUDEVAN, V., WARDEN, P., WICKE, M., YU, Y., AND ZHENG, X. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016* (2016), K. Keeton and T. Roscoe, Eds., USENIX Association, pp. 265–283.
- [2] ABDALLAH, S. Automatic differentiation using constraint handling rules in Prolog. <https://arxiv.org/abs/1706.00231>, 2017.
- [3] ANDERSON, D. D., AND WINDERS, M. Idealization of a module. *Journal of Commutative Algebra* 1, 1 (2009), 3–56.
- [4] ARNTZENIUS, M., AND KRISHNASWAMI, N. R. Datafun: a functional datalog. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016* (2016), J. Garrigue, G. Keller, and E. Sumii, Eds., ACM, pp. 214–227.
- [5] ATIYAH, M. F., AND G. MACDONALD, I. *Introduction to commutative algebra*. Addison-Wesley-Longman, 1969.
- [6] AWODEY, S. *Category Theory*, 2nd ed. Oxford University Press, Inc., USA, 2010.
- [7] BACH POULSEN, C., AND VAN DER REST, C. Hefty algebras: Modular elaboration of higher-order algebraic effects. *Proc. ACM Program. Lang.* 7, POPL (jan 2023).
- [8] BARENDREGT, H. P. *The lambda calculus - its syntax and semantics*, vol. 103 of *Studies in logic and the foundations of mathematics*. North-Holland, 1985.

- [9] BAUER, A., AND PRETNAR, M. An effect system for algebraic effects and handlers. *Log. Methods Comput. Sci.* 10, 4 (2014).
- [10] BAUER, A., AND PRETNAR, M. Programming with algebraic effects and handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015), 108–123. Special Issue: The 23rd Nordic Workshop on Programming Theory (NWPT 2011) Special Issue: Domains X, International workshop on Domain Theory and applications, Swansea, 5-7 September, 2011.
- [11] BAYDIN, A. G., PEARLMUTTER, B. A., RADUL, A. A., AND SISKIND, J. M. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research* 18, 153 (2018), 1–43.
- [12] BELLE, V., AND RAEDT, L. D. Semiring programming: A framework for search, inference and learning. *CoRR abs/1609.06954* (2016).
- [13] BIRD, R., AND DE MOOR, O. *Algebra of Programming*. Prentice-Hall, Inc., USA, 1997.
- [14] BIRD, R., AND PATERSON, R. Generalised folds for nested datatypes. *Form. Asp. Comput.* 11, 2 (sep 1999), 200–222.
- [15] BIRKHOFF, G., AND MAC LANE, S. *A Survey of Modern Algebra*. Taylor & Francis, 1997.
- [16] BOISSEAU, G., AND GIBBONS, J. What you needa know about Yoneda: profunctor optics and the Yoneda lemma (Functional Pearl). *Proc. ACM Program. Lang.* 2, ICFP (2018), 84:1–84:27.
- [17] BOSMAN, R., VAN DEN BERG, B., TANG, W., AND SCHRIJVERS, T. A calculus for scoped effects and handlers, 2023.
- [18] BRACHTHÄUSER, J. I., SCHUSTER, P., AND OSTERMANN, K. Effects as capabilities: Effect handlers and lightweight effect polymorphism. *Proc. ACM Program. Lang.* 4, OOPSLA (Nov. 2020).
- [19] BRADBURY, J., FROSTIG, R., HAWKINS, P., JOHNSON, M. J., LEARY, C., MACLAURIN, D., NECULA, G., PASZKE, A., VANDERPLAS, J., WANDERMAN-MILNE, S., AND ZHANG, Q. JAX: composable transformations of Python+NumPy programs, 2018.
- [20] BYRD, W. E. *Relational programming in miniKanren: techniques, applications, and implementations*. PhD thesis, Indiana University, 2009.
- [21] CAMPBELL, H. Grenade: practical deep learning in haskell. <https://github.com/HuwCampbell/grenade>, 2016-2017.

- [22] CARPENTER, B., GELMAN, A., HOFFMAN, M. D., LEE, D., GOODRICH, B., BETANCOURT, M., BRUBAKER, M., GUO, J., LI, P., AND RIDDELL, A. Stan: A probabilistic programming language. *Journal of statistical software* 76, 1 (2017).
- [23] CARPENTER, B., HOFFMAN, M. D., BRUBAKER, M., LEE, D., LI, P., AND BETANCOURT, M. The stan math library: Reverse-mode automatic differentiation in c++. *arXiv preprint arXiv:1509.07164* (2015).
- [24] CLAESSEN, K., AND LJUNGLÖF, P. Typed logical variables in haskell. *Electron. Notes Theor. Comput. Sci.* 41, 1 (2000), 37.
- [25] CLARK, K. L., AND TÄRNLUND, S.-Å. A first order theory of data and programs. *Information Processing*, 77 (1977).
- [26] CLIFFORD, W. K. Preliminary sketch of bi-quaternions. In *Proceedings of the London Mathematical Society* (1873), vol. 4, pp. 381—395.
- [27] CLOCKSIN, W. F., AND MELLISH, C. S. *Programming in Prolog*, 5 ed. Springer, Berlin, 2003.
- [28] DANTSIN, E. Probabilistic logic programs and their semantics. In *Logic Programming, First Russian Conference on Logic Programming, Irkutsk, Russia, September 14-18, 1990 - Second Russian Conference on Logic Programming, St. Petersburg, Russia, September 11-16, 1991, Proceedings* (1991), A. Voronkov, Ed., vol. 592 of *Lecture Notes in Computer Science*, Springer, pp. 152–164.
- [29] DARWICHE, A. On the tractable counting of theory models and its application to belief revision and truth maintenance. *CoRR cs.AI/0003044* (2000).
- [30] DARWICHE, A. A compiler for deterministic, decomposable negation normal form. In *Eighteenth National Conference on Artificial Intelligence* (USA, 2002), American Association for Artificial Intelligence, p. 627–634.
- [31] DARWICHE, A. New advances in compiling cnf to decomposable negation normal form. In *Proceedings of the 16th European Conference on Artificial Intelligence* (NLD, 2004), ECAI’04, IOS Press, p. 318–322.
- [32] DEBRAY, S. K. Unfold/fold transformations and loop optimization of logic programs. In *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation* (New York, NY, USA, 1988), PLDI ’88, p. 297–307.
- [33] DOLAN, S. Fun with semirings: A functional pearl on the abuse of linear algebra. *SIGPLAN Not.* 48, 9 (sep 2013), 101–110.

- [34] DYLUSS, S., CHRISTIANSEN, J., AND TEEGEN, F. Implementing a library for probabilistic programming using non-strict non-determinism. *Theory and Practice of Logic Programming* 20, 1 (2020), 147–175.
- [35] ELLIOTT, C. Denotational design with type class morphisms (extended version). Tech. Rep. 2009-01, LambdaPix, March 2009.
- [36] ELLIOTT, C. The simple essence of automatic differentiation. *Proc. ACM Program. Lang.* 2, ICFP (2018), 70:1–70:29.
- [37] ELLIOTT, C. Symbolic and automatic differentiation of languages. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–18.
- [38] ELLIOTT, C. M. Beautiful differentiation. In *Proceeding of the 14th ACM SIGPLAN international conference on Functional programming, ICFP 2009, Edinburgh, Scotland, UK, August 31 - September 2, 2009* (2009), G. Hutton and A. P. Tolmach, Eds., ACM, pp. 191–202.
- [39] ELSMAN, M., HENGLEIN, F., KAARSGAARD, R., MATHIESEN, M. K., AND SCHENCK, R. Combinatory adjoints and differentiation. In *Proceedings Ninth Workshop on Mathematically Structured Functional Programming, MSFP@ETAPS 2022, Munich, Germany, 2nd April 2022* (2022), J. Gibbons and M. S. New, Eds., vol. 360 of *EPTCS*, pp. 1–26.
- [40] ERWIG, M., AND KOLLMANSBERGER, S. Functional pearls: Probabilistic functional programming in haskell. *J. Funct. Program.* 16, 1 (2006), 21–34.
- [41] FIERENS, D., VAN DEN BROECK, G., RENKENS, J., SHTERIONOV, D., GUTMANN, B., THON, I., JANSSENS, G., AND DE RAEDT, L. Inference and learning in probabilistic logic programs using weighted boolean formulas. *Theory and Practice of Logic Programming* 15, 3 (2015), 358–401.
- [42] FILINSKI, A. Representing layered monads. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (New York, NY, USA, 1999), POPL '99, Association for Computing Machinery, p. 175–188.
- [43] FONG, B., AND SPIVAK, D. I. *An invitation to applied category theory: seven sketches in compositionality*. Cambridge University Press, 2019.
- [44] FORSTER, Y., KAMMAR, O., LINDLEY, S., AND PRETNAR, M. On the expressive power of user-defined effects: Effect handlers, monadic reflection, delimited control. *CoRR abs/1610.09161* (2016).

- [45] GABEL, P. L. Automatic differentiation. <http://petergabel.info/blog/AutoDiff/#Backward-Mode-Automatic-Differentiation-1>, 2020.
- [46] GARCIA, R., LUMSDAINE, A., AND SABRY, A. Lazy evaluation and delimited control. *Log. Methods Comput. Sci.* 6, 3 (2010).
- [47] GELFOND, M., AND LIFSCHITZ, V. The stable model semantics for logic programming. In *Logic Programming, Proceedings of the Fifth International Conference and Symposium, Seattle, Washington, USA, August 15-19, 1988 (2 Volumes)* (1988), R. A. Kowalski and K. A. Bowen, Eds., MIT Press, pp. 1070–1080.
- [48] GHANI, N., AND UUSTALU, T. Explicit substitutions and higher-order syntax. In *Proceedings of the 2003 ACM SIGPLAN Workshop on Mechanized Reasoning about Languages with Variable Binding* (New York, NY, USA, 2003), MERLIN '03, Association for Computing Machinery, p. 1–7.
- [49] GIBBONS, J. Unifying theories of programming with monads. In *Unifying Theories of Programming, 4th International Symposium, UTP 2012, Paris, France, August 27-28, 2012, Revised Selected Papers* (2012), B. Wolff, M. Gaudel, and A. Feliachi, Eds., vol. 7681 of *Lecture Notes in Computer Science*, Springer, pp. 23–67.
- [50] GIBBONS, J., AND HINZE, R. Just do it: Simple monadic equational reasoning. *SIGPLAN Not.* 46, 9 (sep 2011), 2–14.
- [51] GIBBONS, J., AND WU, N. Folding domain-specific languages: deep and shallow embeddings (Functional Pearl). In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (2014), pp. 339–347.
- [52] GILL, A. mtl: Monad transformer library, 2008. <https://hackage.haskell.org/package/mtl-1.1.0.2>.
- [53] GILL, A., AND HUTTON, G. The worker/wrapper transformation. *Journal of Functional Programming* 19, 2 (2009), 227–251.
- [54] GILL, A. J., LAUNCHBURY, J., AND JONES, S. L. P. A short cut to deforestation. In *Proceedings of the conference on Functional programming languages and computer architecture, FPCA 1993, Copenhagen, Denmark, June 9-11, 1993* (1993), J. Williams, Ed., ACM, pp. 223–232.
- [55] GIRY, M. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis* (Berlin, Heidelberg, 1982), B. Banaschewski, Ed., Springer Berlin Heidelberg, pp. 68–85.

- [56] GOLAN, J. S. *Semimodules over Semirings*. Springer Netherlands, Dordrecht, 1999, pp. 149–161.
- [57] GOODMAN, N. D., MANSINGHKA, V. K., ROY, D., BONAWITZ, K., AND TENENBAUM, J. B. Church: A language for generative models. In *Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence* (Arlington, Virginia, USA, 2008), UAI’08, AUAI Press, p. 220–229.
- [58] GRIEWANK, A. A mathematical view of automatic differentiation. *Acta Numerica* 12 (2003), 321–398.
- [59] GU, Y.-L., AND LUH, J. Dual-number transformation and its applications to robotics. *IEEE Journal on Robotics and Automation* 3, 6 (1987), 615–623.
- [60] HILLERSTRÖM, D., AND LINDLEY, S. Liberating effects with rows and handlers. In *Proceedings of the 1st International Workshop on Type-Driven Development* (New York, NY, USA, 2016), TyDe 2016, Association for Computing Machinery, p. 15–27.
- [61] HILLERSTRÖM, D., AND LINDLEY, S. Shallow effect handlers. In *Programming Languages and Systems* (Cham, 2018), S. Ryu, Ed., Springer International Publishing, pp. 415–435.
- [62] HINZE, R. Prological features in a functional setting axioms and implementation. In *Third Fuji International Symposium on Functional and Logic Programming, FLOPS 1998, Kyoto, Japan, April 2-4, 1998* (1998), M. Sato and Y. Toyama, Eds., World Scientific, Singapore, pp. 98–122.
- [63] HINZE, R. Efficient generalized folds. In *Proceedings of the Second Workshop on Generic Programming, WGP 2000, Ponte de Lima, Portugal, 6th July 2000* (2000), J. Jeuring, Ed.
- [64] HINZE, R. Functional pearl: Streams and unique fixed points. In *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming* (New York, NY, USA, 2008), ICFP ’08, Association for Computing Machinery, p. 189–200.
- [65] HINZE, R. Kan extensions for program optimisation or: Art and Dan explain an old trick. In *Mathematics of Program Construction - 11th International Conference, MPC 2012, Madrid, Spain, June 25-27, 2012. Proceedings* (2012), J. Gibbons and P. Nogueira, Eds., vol. 7342 of *Lecture Notes in Computer Science*, Springer, pp. 324–362.
- [66] HOARE, C. A. R. Proof of correctness of data representations. *Acta Informatica* 1, 4 (1972), 271–281.

- [67] HOFFMANN, P. H. A hitchhiker's guide to automatic differentiation. *Numerical Algorithms* 72, 3 (2016), 775–811.
- [68] HOOBERGS, J., VAN DEN BERG, B., AND SCHRIJVERS, T. eTeacher: A pilot in flemish secondary education, 2023-07-11.
- [69] HUDAK, P., MAKUCEVICH, T., GADDE, S., AND WHONG, B. Haskore music notation – an algebra of music –. *Journal of Functional Programming* 6, 3 (1996), 465–484.
- [70] HUGHES, J. A novel representation of lists and its application to the function reverse. *Inf. Process. Lett.* 22, 3 (1986), 141–144.
- [71] HUOT, M., STATON, S., AND VÁKÁR, M. Correctness of automatic differentiation via diffeologies and categorical gluing. In *Foundations of Software Science and Computation Structures - 23rd International Conference, FOSSACS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings* (2020), J. Goubault-Larrecq and B. König, Eds., vol. 12077 of *Lecture Notes in Computer Science*, Springer, pp. 319–338.
- [72] HUOT, M., STATON, S., AND VÁKÁR, M. Higher order automatic differentiation of higher order functions. *Log. Methods Comput. Sci.* 18, 1 (2022).
- [73] HUTTON, G. A tutorial on the universality and expressiveness of fold. *J. Funct. Program.* 9, 4 (1999), 355–372.
- [74] JANSSON, P., IONESCU, C., AND BERNARDY, J.-P. *Domain-Specific Languages of Mathematics*, vol. 24 of *Texts in Computing*. College Publications, jan 2022.
- [75] JASKELIOFF, M. Monatron: An extensible monad transformer library. In *Implementation and Application of Functional Languages - 20th International Symposium, IFL 2008, Hatfield, UK, September 10-12, 2008. Revised Selected Papers* (2008), S. Scholz and O. Chitil, Eds., vol. 5836 of *Lecture Notes in Computer Science*, Springer, pp. 233–248.
- [76] JASKELIOFF, M. Modular monad transformers. In *Programming Languages and Systems, 18th European Symposium on Programming, ESOP 2009, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2009, York, UK, March 22-29, 2009. Proceedings* (2009), G. Castagna, Ed., vol. 5502 of *Lecture Notes in Computer Science*, Springer, pp. 64–79.

- [77] JOHANN, P., AND GHANI, N. Foundations for structured programming with gads. *SIGPLAN Not.* 43, 1 (jan 2008), 297–308.
- [78] KARCZMARCZUK, J. Functional differentiation of computer programs. In *Proceedings of the third ACM SIGPLAN International Conference on Functional Programming (ICFP '98), Baltimore, Maryland, USA, September 27-29, 1998* (1998), M. Felleisen, P. Hudak, and C. Queinnec, Eds., ACM, pp. 195–203.
- [79] KIMMIG, A., VAN DEN BROECK, G., AND DE RAEDT, L. An algebraic prolog for reasoning about possible worlds. In *Twenty-Fifth AAAI Conference on Artificial Intelligence* (2011).
- [80] KING, A. eff – screaming fast extensible effects for less, 2019. <https://github.com/hasura/eff>.
- [81] KISELYOV, O., AND ISHII, H. Freer monads, more extensible effects. *SIGPLAN Not.* 50, 12 (aug 2015), 94–105.
- [82] KISELYOV, O., SABRY, A., SWORDS, C., AND FOPPA, B. extensible-effects: An alternative to monad transformers, 2019. <https://hackage.haskell.org/package/extensible-effects>.
- [83] KISELYOV, O., SHAN, C., FRIEDMAN, D. P., AND SABRY, A. Backtracking, interleaving, and terminating monad transformers: (functional pearl). In *Proceedings of the 10th ACM SIGPLAN International Conference on Functional Programming, ICFP 2005, Tallinn, Estonia, September 26-28, 2005* (2005), O. Danvy and B. C. Pierce, Eds., ACM, pp. 192–203.
- [84] KISELYOV, O., AND SIVARAMAKRISHNAN, K. Eff directly in ocaml. *Electronic Proceedings in Theoretical Computer Science* 285 (Dec 2018), 23–58.
- [85] KMETT, E. ad: automatic differentiation. <https://github.com/ekmett/ad>, 2021.
- [86] KRAWIEC, F., JONES, S. P., KRISHNASWAMI, N., ELLIS, T., EISENBERG, R. A., AND FITZGIBBON, A. W. Provably correct, asymptotically efficient, higher-order reverse-mode automatic differentiation. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–30.
- [87] LAGNIEZ, J.-M., AND MARQUIS, P. An improved decision-dnnf compiler. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence* (2017), IJCAI'17, AAAI Press, p. 667–673.

- [88] LE, J. **Backprop**: heterogeneous automatic differentiation. <https://github.com/mstkg/backprop>, 2018.
- [89] LE, T. A., BAYDIN, A. G., AND WOOD, F. Inference compilation and universal probabilistic programming. In *20th International Conference on Artificial Intelligence and Statistics, April 20–22, 2017, Fort Lauderdale, FL, USA* (2017).
- [90] LEIJEN, D. Extensible records with scoped labels. In *Revised Selected Papers from the Sixth Symposium on Trends in Functional Programming, TFP 2005, Tallinn, Estonia, 23-24 September 2005* (2005), M. C. J. D. van Eekelen, Ed., vol. 6 of *Trends in Functional Programming*, Intellect, pp. 179–194.
- [91] LEIJEN, D. Koka: Programming with row polymorphic effect types. *Electronic Proceedings in Theoretical Computer Science* 153 (06 2014).
- [92] LEIJEN, D. Type directed compilation of row-typed algebraic effects. *SIGPLAN Not.* 52, 1 (jan 2017), 486–499.
- [93] LEVY, P. B. Call-by-push-value. *ACM SIGLOG News* 9, 2 (may 2022), 7–29.
- [94] LEVY, P. B., POWER, J., AND THIELECKE, H. Modelling environments in call-by-value programming languages. *Inf. Comput.* 185, 2 (2003), 182–210.
- [95] LINDLEY, S., MCBRIDE, C., AND MCCLAUGHLIN, C. Do be do be do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (New York, NY, USA, 2017), POPL 2017, Association for Computing Machinery, p. 500–514.
- [96] LÓPEZ FRAGUAS, F. J., AND SÁNCHEZ HERNÁNDEZ, J. Toy: A multiparadigm declarative system. In *International Conference on Rewriting Techniques and Applications* (1999), Springer, pp. 244–247.
- [97] MAGUIRE, S. polysemy: Higher-order, low-boilerplate free monads, 2019. <https://hackage.haskell.org/package/polysemy>.
- [98] MANHAEVE, R., DUMANCIC, S., KIMMIG, A., DEMEESTER, T., AND DE RAEDT, L. Deepproblog: Neural probabilistic logic programming. *advances in neural information processing systems* 31 (2018).
- [99] MARLOW, S., JONES, S. P., MORAN, A., AND REPPY, J. Asynchronous exceptions in haskell. *SIGPLAN Not.* 36, 5 (may 2001), 274–285.

- [100] MARTIN, C., GIBBONS, J., AND BAYLEY, I. Disciplined, efficient, generalised folds for nested datatypes. *Formal Aspects of Computing* 16, 1 (2004), 19–35.
- [101] MCCARTHY, J. A basis for a mathematical theory of computation, preliminary report. In *Papers Presented at the May 9-11, 1961, Western Joint IRE-AIEE-ACM Computer Conference* (New York, NY, USA, 1961), IRE-AIEE-ACM '61 (Western), Association for Computing Machinery, p. 225–238.
- [102] MEIJER, E. *Calculating Compilers*. PhD thesis, Katholieke Universiteit Nijmegen, 1992.
- [103] MEIJER, E., FOKKINGA, M., AND PATERSON, R. Functional programming with bananas, lenses, envelopes and barbed wire. In *Proceedings of the 5th ACM Conference on Functional Programming Languages and Computer Architecture* (Berlin, Heidelberg, 1991), Springer-Verlag, p. 124–144.
- [104] MILNER, R. A theory of type polymorphism in programming. *J. Comput. Syst. Sci.* 17, 3 (1978), 348–375.
- [105] MOGGI, E. An abstract view of programming languages. Tech. Rep. ECS-LFCS-90-113, Edinburgh University, Department of Computer Science, June 1989.
- [106] MOGGI, E. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55 – 92. Selections from 1989 IEEE Symposium on Logic in Computer Science.
- [107] MUISE, C., MCILRAITH, S. A., BECK, J. C., AND HSU, E. I. Dsharp: Fast d-dnnf compilation with sharpsat. In *Proceedings of the 25th Canadian Conference on Advances in Artificial Intelligence* (Berlin, Heidelberg, 2012), Canadian AI'12, Springer-Verlag, p. 356–361.
- [108] NAGATA, M. *Local rings*. Wiley Interscience, 1962.
- [109] NARAYANAN, P., CARETTE, J., ROMANO, W., SHAN, C., AND ZINKOV, R. Probabilistic inference by program transformation in hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings* (2016), Springer, pp. 62–79.
- [110] PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., DESMAISON, A., KOPF, A., YANG, E., DEVITO, Z., RAISON, M.,

- TEJANI, A., CHILAMKURTHY, S., STEINER, B., FANG, L., BAI, J., AND CHINTALA, S. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035.
- [111] PASZKE, A., JOHNSON, D. D., DUVENAUD, D., VYTINIOTIS, D., RADUL, A., JOHNSON, M. J., RAGAN-KELLEY, J., AND MACLAURIN, D. Getting to the point: Index sets and parallelism-preserving autodiff for painful array programming. *Proc. ACM Program. Lang.* 5, ICFP (aug 2021).
- [112] PAUWELS, K., SCHRIJVERS, T., AND MU, S.-C. Handling local state with global state. In *Mathematics of Program Construction* (Cham, 2019), G. Hutton, Ed., Springer International Publishing, pp. 18–44.
- [113] PEARLMUTTER, B. A. Fast exact multiplication by the hessian. *Neural Comput.* 6, 1 (Jan. 1994), 147–160.
- [114] PEARLMUTTER, B. A., AND SISKIND, J. M. Reverse-mode AD in a functional framework: Lambda the ultimate backpropagator. *ACM Trans. Program. Lang. Syst.* 30, 2 (2008), 7:1–7:36.
- [115] PEYTON JONES, S., EBER, J.-M., AND SEWARD, J. Composing contracts: An adventure in financial engineering (functional pearl). In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming* (New York, NY, USA, 2000), ICFP '00, Association for Computing Machinery, p. 280–292.
- [116] PFANSCHILLING, V., SHINDO, H., DHAMI, D. S., AND KERSTING, K. Sum-product loop programming: From probabilistic circuits to loop programming. In *Proceedings of KR* (2022).
- [117] PIRÓG, M., SCHRIJVERS, T., WU, N., AND JASKELIOFF, M. Syntax and semantics for operations with scopes. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science* (New York, NY, USA, 2018), LICS '18, Association for Computing Machinery, p. 809–818.
- [118] PIRÓG, M., AND STATON, S. Backtracking with cut via a distributive law and left-zero monoids. *J. Funct. Program.* 27 (2017), e17.
- [119] PLOTKIN, G., AND PRETNAR, M. Handlers of algebraic effects. In *Programming Languages and Systems* (Berlin, Heidelberg, 2009), G. Castagna, Ed., Springer Berlin Heidelberg, pp. 80–94.
- [120] PLOTKIN, G., AND PRETNAR, M. Handling algebraic effects. *Logical Methods in Computer Science* 9, 4 (dec 2013).

- [121] PLOTKIN, G. D., AND POWER, J. Algebraic operations and generic effects. *Appl. Categorical Struct.* 11, 1 (2003), 69–94.
- [122] POOLE, D. Logic programming, abduction and probability: A top-down anytime algorithm for estimating prior and posterior probabilities. In *Selected Papers of International Conference on Fifth Generation Computer Systems 92* (Berlin, Heidelberg, 1993), Springer-Verlag, p. 377–400.
- [123] POOLE, D. The independent choice logic for modelling multiple agents under uncertainty. *Artif. Intell.* 94, 1–2 (jul 1997), 7–56.
- [124] PRETNAR, M. An introduction to algebraic effects and handlers. invited tutorial paper. *Electronic Notes in Theoretical Computer Science* 319 (2015), 19–35. The 31st Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXI).
- [125] RAMSEY, N., AND PFEFFER, A. Stochastic lambda calculus and monads of probability distributions. In *Conference Record of POPL 2002: The 29th SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, OR, USA, January 16-18, 2002* (2002), J. Launchbury and J. C. Mitchell, Eds., ACM, pp. 154–165.
- [126] REYNOLDS, J. C. Types, abstraction and parametric polymorphism. In *Information Processing 83, Proceedings of the IFIP 9th World Computer Congress, Paris, France, September 19-23, 1983* (1983), R. E. A. Mason, Ed., North-Holland/IFIP, pp. 513–523.
- [127] RIGUZZI, F., AND SWIFT, T. The PITA system: Tabling and answer subsumption for reasoning under uncertainty. *Theory Pract. Log. Program.* 11, 4-5 (2011), 433–449.
- [128] RIVAS, E., AND JASKELIOFF, M. Notions of computation as monoids. *Journal of Functional Programming* 27 (2017), e21.
- [129] RIX, R., THOMSON, P., WU, N., AND SCHRIJVERS, T. fused-effects: A fast, flexible, fused effect system, 2018. <https://hackage.haskell.org/package/fused-effects>.
- [130] ROMPF, T., AND ODERSKY, M. Lightweight modular staging: A pragmatic approach to runtime code generation and compiled dsls. *SIGPLAN Not.* 46, 2 (oct 2010), 127–136.
- [131] SATO, T. A statistical learning method for logic programs with distribution semantics. In *Logic Programming, Proceedings of the Twelfth International Conference on Logic Programming, Tokyo, Japan, June 13-16, 1995* (1995), L. Sterling, Ed., MIT Press, pp. 715–729.

- [132] SCHRIJVERS, T., PIRÓG, M., WU, N., AND JASKELIOFF, M. Monad transformers and modular algebraic effects: What binds them together. In *Proceedings of the 12th ACM SIGPLAN International Symposium on Haskell* (New York, NY, USA, 2019), Haskell 2019, Association for Computing Machinery, p. 98–113.
- [133] SCHRIJVERS, T., STUCKEY, P., AND WADLER, P. Monadic constraint programming. *Journal of Functional Programming* 19, 6 (2009), 663–697.
- [134] SCHRIJVERS, T., VAN DEN BERG, B., AND RIGUZZI, F. Automatic differentiation in prolog, 2023.
- [135] SCHUSTER, P., BRACHTHÄUSER, J. I., AND OSTERMANN, K. Compiling effect handlers in capability-passing style. *Proc. ACM Program. Lang.* 4, ICFP (aug 2020).
- [136] SCOTT, D. Outline of a mathematical theory of computation. Tech. Rep. PRG02, OUCL, November 1970.
- [137] SCOTT, D., AND STRACHEY, C. Towards a mathematical semantics for computer languages. *Proceedings of the Symposium on Computers and Automata 21* (01 1971).
- [138] SERES, S., SPIVEY, J. M., AND HOARE, C. A. R. Algebra of logic programming. In *Logic Programming: The 1999 International Conference, Las Cruces, New Mexico, USA, November 29 - December 4, 1999* (1999), D. D. Schreye, Ed., MIT Press, pp. 184–199.
- [139] SHAIKHHA, A., HUOT, M., SMITH, J., AND OLTEANU, D. Functional collection programming with semi-ring dictionaries. *Proc. ACM Program. Lang.* 6, OOPSLA1 (apr 2022).
- [140] SHEARD, T., AND JONES, S. P. Template meta-programming for haskell. In *Proceedings of the 2002 ACM SIGPLAN Workshop on Haskell* (New York, NY, USA, 2002), Haskell '02, Association for Computing Machinery, p. 1–16.
- [141] SIVARAMAKRISHNAN, K., DOLAN, S., WHITE, L., KELLY, T., JAFFER, S., AND MADHAVAPEDDY, A. Retrofitting effect handlers onto ocaml. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (New York, NY, USA, 2021), PLDI 2021, Association for Computing Machinery, p. 206–221.
- [142] SMEDING, T. J., AND VÁKÁR, M. I. L. Efficient dual-numbers reverse ad via well-known program transformations. *Proc. ACM Program. Lang.* 7, POPL (jan 2023).

- [143] SOMOGYI, Z., HENDERSON, F. J., AND CONWAY, T. C. Mercury, an efficient purely declarative logic programming language. *Australian Computer Science Communications* 17 (1995), 499–512.
- [144] SPIVEY, J. M. Algebras for combinatorial search. *J. Funct. Program.* 19, 3-4 (2009), 469–487.
- [145] STITES, S., AND HUANG, A. **Hasktorch**: tensors and neural networks in haskell. <https://github.com/hasktorch/hasktorch>, 2018.
- [146] SWIERSTRA, W. Data types à la carte. *J. Funct. Program.* 18, 4 (jul 2008), 423–436.
- [147] SZIRMAY-KALOS, L. Higher order automatic differentiation with dual numbers. *Periodica Polytechnica Electrical Engineering and Computer Science* 65, 1 (2021), 1–10.
- [148] TAHA, W., AND SHEARD, T. Multi-stage programming with explicit annotations. *SIGPLAN Not.* 32, 12 (dec 1997), 203–217.
- [149] THOMSON, P., RIX, R., WU, N., AND SCHRIJVERS, T. Fusing industry and academia at github (experience report). *Proc. ACM Program. Lang.* 6, ICFP (aug 2022).
- [150] VAN DEN BERG, B., DEDECKER, P., AND SCHRIJVERS, T. A dsl for fluorescence microscopy, 2020-02-13.
- [151] VAN DEN BERG, B., DEN EYNDE, R. V., AMOUROUX, B., MÜLLER, M., DEDECKER, P., AND VANDENBERG, W. A modular approach to active focus stabilization for fluorescence microscopy. *bioRxiv* (2023).
- [152] VAN DEN BERG, B., AND SCHRIJVERS, T. A functional account of probabilistic programming with possible worlds: Declarative pearl. In *Functional and Logic Programming: 16th International Symposium, FLOPS 2022, Kyoto, Japan, May 10–12, 2022, Proceedings* (Berlin, Heidelberg, 2022), Springer-Verlag, p. 186–204.
- [153] VAN DEN BERG, B., AND SCHRIJVERS, T. A framework for higher-order effects and handlers, 2023.
- [154] VAN DEN BERG, B., SCHRIJVERS, T., AND DEDECKER, P. Fluo: A domain-specific language for experiments in fluorescence microscopy (application paper). In *Practical Aspects of Declarative Languages* (Cham, 2023), M. Hanus and D. Incezan, Eds., Springer Nature Switzerland, pp. 73–82.

- [155] VAN DEN BERG, B., SCHRIJVERS, T., MCKINNA, J., AND VANDENBROUCKE, A. Forward- or reverse-mode automatic differentiation: What's the difference? *Science of Computer Programming* 231 (2024), 103010.
- [156] VAN DEN BERG, B., SCHRIJVERS, T., POULSEN, C. B., AND WU, N. Latent effects for reusable language components. In *Programming Languages and Systems* (Cham, 2021), H. Oh, Ed., Springer International Publishing, pp. 182–201.
- [157] VAN DEN BERG, B., SCHRIJVERS, T., POULSEN, C. B., AND WU, N. Latent effects for reusable language components: Extended version. *CoRR abs/2108.11155* (2021).
- [158] VAN DEN BERG, B., VAN BREMEN, T., DERKINDEREN, V., KIMMIG, A., SCHRIJVERS, T., AND DE RAEDT, L. From probabilistic netkat to problog: New algorithms for inference and learning in probabilistic networks, 2021-10-21.
- [159] VAN DEN BROECK, G., KERSTING, K., NATARAJAN, S., AND POOLE, D. *An Introduction to Lifted Probabilistic Inference*. MIT Press, 2021.
- [160] VAN DEN BROECK, G., TAGHIPOUR, N., MEERT, W., DAVIS, J., AND DE RAEDT, L. Lifted probabilistic inference by first-order knowledge compilation. In *Proceedings of the 22th International Joint Conference on Artificial Intelligence (IJCAI)* (2011).
- [161] VAN GELDER, A., ROSS, K. A., AND SCHLIPF, J. S. The well-founded semantics for general logic programs. *J. ACM* 38, 3 (jul 1991), 619–649.
- [162] VANDENBROUCKE, A., AND SCHRIJVERS, T. ProbLog and applicative probabilistic programming, Jan. 2017. Workshop on Probabilistic Programming Semantics (PPS), Paris, France, 17/1/2017 - 17/1/2017.
- [163] VANDENBROUCKE, A., SCHRIJVERS, T., AND PIESENS, F. Fixing non-determinism. In *Proceedings of the 27th Symposium on the Implementation and Application of Functional Programming Languages, IFL '15, Koblenz, Germany, September 14-16, 2015* (2015), R. Lämmel, Ed., ACM, pp. 5:1–5:12.
- [164] VENNEKENS, J., VERBAETEN, S., AND BRUYNOOGHE, M. Logic programs with annotated disjunctions. In *Logic Programming, 20th International Conference, ICLP 2004, Saint-Malo, France, September 6-10, 2004, Proceedings* (2004), B. Demoen and V. Lifschitz, Eds., vol. 3132 of *Lecture Notes in Computer Science*, Springer, pp. 431–445.

- [165] VON ZUR GATHEN, J., AND GERHARD, J. *Modern Computer Algebra, 3rd edition*. Cambridge University Press, 2013.
- [166] WADLER, P. Theorems for free! In *Proceedings of the fourth international conference on Functional programming languages and computer architecture, FPCA 1989, London, UK, September 11-13, 1989* (1989), J. E. Stoy, Ed., ACM, pp. 347–359.
- [167] WADLER, P. Monads for functional programming. In *Advanced Functional Programming, First International Spring School on Advanced Functional Programming Techniques-Tutorial Text* (Berlin, Heidelberg, 1995), Springer-Verlag, pp. 24–2.
- [168] WALKER, M. W., SHAO, L., AND VOLZ, R. A. Estimating 3-d location parameters using dual number quaternions. *CVGIP Image Underst.* 54 (1991), 358–367.
- [169] WANG, F., ZHENG, D., DECKER, J., WU, X., ESSERTEL, G. M., AND ROMPF, T. Demystifying differentiable programming: Shift/reset the penultimate backpropagator. *Proc. ACM Program. Lang.* 3, ICFP (jul 2019).
- [170] WEI, G., BRAČEVAC, O., TAN, S., AND ROMPF, T. Compiling symbolic execution with staging and algebraic effects. *Proc. ACM Program. Lang.* 4, OOPSLA (nov 2020).
- [171] WEI, G., BRAČEVAC, O., TAN, S., AND ROMPF, T. Compiling symbolic execution with staging and algebraic effects. *Proc. ACM Program. Lang.* 4, OOPSLA (nov 2020).
- [172] WENGERT, R. E. A simple automatic derivative evaluation program. *Commun. ACM* 7, 8 (Aug. 1964), 463–464.
- [173] WU, N., SCHRIJVERS, T., AND HINZE, R. Effect handlers in scope. *SIGPLAN Not.* 49, 12 (sep 2014), 1–12.
- [174] XIE, N., JOHNSON, D. D., MACLAURIN, D., AND PASZKE, A. Parallel algebraic effect handlers. *CoRR abs/2110.07493* (2021).
- [175] YALLOP, J., AND WHITE, L. Lightweight higher-kinded polymorphism. In *Functional and Logic Programming* (Cham, 2014), M. Codish and E. Sumii, Eds., Springer International Publishing, pp. 119–135.
- [176] YANG, Z., PAVIOTTI, M., WU, N., VAN DEN BERG, B., AND SCHRIJVERS, T. Structured handling of scoped effects. In *Programming Languages and Systems* (Cham, 2022), I. Sergey, Ed., Springer International Publishing, pp. 462–491.

- [177] YANG, Z., PAVIOTTI, M., WU, N., VAN DEN BERG, B., AND SCHRIJVERS, T. Structured handling of scoped effects: Extended version, 2022.
- [178] YANG, Z., AND WU, N. Reasoning about effect interaction by fusion. *Proc. ACM Program. Lang.* 5, ICFP (aug 2021).

List of Publications

Effects & Handlers

- [156] BIRTHE VAN DEN BERG, TOM SCHRIJVERS, CASPER BACH POULSEN, AND NICOLAS WU. (2021). Latent Effects for Reusable Language Components. In *Programming Languages and Systems: 19th Asian Symposium, APLAS 2021, Chicago, IL, USA, October 17–18, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 182–201. https://doi.org/10.1007/978-3-030-89051-3_11
- [157] BIRTHE VAN DEN BERG, TOM SCHRIJVERS, CASPER BACH POULSEN, AND NICOLAS WU. (2021). Latent Effects for Reusable Language Components: Extended Version. <https://arxiv.org/abs/2108.11155>
- [176] ZHIXUAN YANG, MARCO PAVIOTTI, NICOLAS WU, BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2022). Structured Handling of Scoped Effects. In: Sergey, I. (eds) *Programming Languages and Systems*. ESOP 2022. Lecture Notes in Computer Science, vol 13240. Springer, Cham. https://doi.org/10.1007/978-3-030-99336-8_17
- [177] ZHIXUAN YANG, MARCO PAVIOTTI, NICOLAS WU, BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2022). Structured Handling of Scoped Effects: Extended Version. <https://arxiv.org/abs/2201.10287>
- [153] BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2023). A Framework for Higher-Order Effects & Handlers. <https://ssrn.com/abstract=4383536>
This paper is currently under review at the Science of Computer Programming journal.
- [17] ROGER BOSMAN*, BIRTHE VAN DEN BERG*, WENHAO TANG*, AND TOM SCHRIJVERS. (2023). A Calculus for Scoped Effects & Handlers. <https://arxiv.org/abs/2304.09697>

This paper is currently under review at the Logical Methods of Computer Science journal.

Automatic Differentiation

- [155] BIRTHE VAN DEN BERG, TOM SCHRIJVERS, JAMES MCKINNA, AND ALEXANDER VANDENBROUCKE. (2023). Forward- or Reverse-Mode Automatic Differentiation: What's the Difference? In: Science of Computer Programming, Volume 231, 103010, ISSN 0167-6423. <https://doi.org/10.1016/j.scico.2023.103010>
- [134] TOM SCHRIJVERS, BIRTHE VAN DEN BERG, AND FABRIZIO RIGUZZI. (2023). Automatic Differentiation in Prolog. In Theory And Practice Of Logic Programming. Cambridge University Press (CUP). <https://doi.org/10.1017/S1471068423000145>

Other

- [150] BIRTHE VAN DEN BERG, PETER DEDECKER, AND TOM SCHRIJVERS. (2020). A DSL for Fluorescence Microscopy. In *Trends in Functional Programming*. TFP 2020, Krakow, Poland.
- [158] BIRTHE VAN DEN BERG, TIMOTHY VAN BREMEN, VINCENT DERKINDEREN, ANGELIKA KIMMIG, TOM SCHRIJVERS, AND LUC DE RAEDT. (2021). From Probabilistic NetKAT to ProbLog: New Algorithms for Inference and Learning in Probabilistic Networks. In *International Conference on Probabilistic Programming*. PROBPROG 2021, Online.
- [152] BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2022). A Functional Account of Probabilistic Programming with Possible Worlds (Declarative Pearl). In: Hanus, M., Igarashi, A. (eds) *Functional and Logic Programming*. FLOPS 2022. Lecture Notes in Computer Science, vol 13215. Springer, Cham. https://doi.org/10.1007/978-3-030-99461-7_11
- [151] BIRTHE VAN DEN BERG, ROBIN VAN DEN EYNDE, BAPTISTE AMOUROUX, MARCEL MUELLER, PETER DEDECKER, AND WIM VANDENBERG. (2023). A Modular Approach to Active Focus Stabilization for Fluorescence Microscopy. *bioRxiv*. <https://doi.org/10.1101/2020.09.22.308197>
- [154] BIRTHE VAN DEN BERG, TOM SCHRIJVERS, AND PETER DEDECKER. (2023). Fluo: A Domain-Specific Language for Experiments in

- Fluorescence Microscopy (Application Paper). In: Hanus, M., Inclezan, D. (eds) *Practical Aspects of Declarative Languages*. PADL 2023. Lecture Notes in Computer Science, vol 13880. Springer, Cham. https://doi.org/10.1007/978-3-031-24841-2_5
- [68] JESSE HOOBERGS, BIRTHE VAN DEN BERG, AND TOM SCHRIJVERS. (2023). eTeacher: A Pilot in Flemish Secondary Education. In *ACM conference on Innovation and Technology in Computer Science Education*. ITiCSE 2023, Turku, Finland. <https://doi.org/10.1145/3587103.3594152>

Appendix A

Isomorphisms

This Appendix presents isomorphisms between definitions in terms of our generic framework and definitions from the literature for different classes of effects. Furthermore, we show that the isomorphisms satisfy the required roundtrip property. If a handler is defined in the literature for some class of effects, we show its equivalence in terms of our generic fold.

A.1 Algebraic Effects & Handlers

First, we define the isomorphisms:

$$\begin{aligned} iso_1 &:: Functor\ \sigma \Rightarrow Free\ \sigma\ a \rightarrow Free_H\ (K^{Alg}\ \sigma)\ a \\ iso_1\ (Var\ x) &= Var_H\ x \\ iso_1\ (Op\ op) &= Op_H\ (Op\ (fmap\ iso_1\ op)) \\ iso_2 &:: Functor\ \sigma \Rightarrow Free_H\ (K^{Alg}\ \sigma)\ a \rightarrow Free\ \sigma\ a \\ iso_2\ (Var_H\ x) &= Var\ x \\ iso_2\ (Op_H\ (Op\ op)) &= Op\ (fmap\ iso_2\ op) \end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 . iso_2 = id = iso_2 . iso_1$.

$$\begin{aligned} (iso_1 . iso_2)\ (Var_H\ x) \\ &\equiv iso_1\ (Var\ x) \\ &\equiv Var_H\ x \end{aligned}$$

$$\begin{aligned}
& (iso_1 . iso_2) (Op_H (Op\ op)) \\
& \equiv iso_1 (Op\ (fmap\ iso_2\ op)) \\
& \equiv Op_H (Op\ (fmap\ iso_1\ (fmap\ iso_2\ op))) \\
& \equiv Op_H (Op\ (fmap\ (iso_1 . iso_2)\ op)) \\
& \equiv Op_H (Op\ op)
\end{aligned}$$

$$\begin{aligned}
& (iso_2 . iso_1) (Var\ x) \\
& \equiv iso_2 (Var_H\ x) \\
& \equiv Var\ x \\
& (iso_2 . iso_1) (Op\ op) \\
& \equiv iso_2 (Op_H (Op\ (fmap\ iso_1\ op))) \\
& \equiv Op\ (fmap\ iso_2\ (fmap\ iso_1\ op)) \\
& \equiv Op\ (fmap\ (iso_2 . iso_1)\ op) \\
& \equiv Op\ op
\end{aligned}$$

Furthermore, the handlers for these two recursive datatypes satisfy the following equivalence: $fold_{Alg}\ gen\ alg = h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op) . iso_1$.

$$\begin{aligned}
& h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op)\ (iso_1\ (Var\ x)) \\
& \equiv h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op)\ (Var_H\ x) \\
& \equiv gen\ x \\
& \equiv fold_{Alg}\ gen\ alg\ (Var\ x)
\end{aligned}$$

$$\begin{aligned}
& h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op)\ (iso_1\ (Op\ x)) \\
& \equiv h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op)\ (Op_H (Op\ (fmap\ iso_1\ x))) \\
& \equiv (\lambda(Op\ op) \rightarrow alg\ op)\ (hmap\ fold_{\rightsquigarrow} \\
& \quad (fmap\ (h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op))\ (Op\ (fmap\ iso_1\ x)))) \\
& \equiv (\lambda(Op\ op) \rightarrow alg\ op)\ (hmap\ fold_{\rightsquigarrow} \\
& \quad (Op\ (fmap\ (h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op)\ (fmap\ iso_1\ x)))))) \\
& \equiv (\lambda(Op\ op) \rightarrow alg\ op) \\
& \quad (Op\ (fmap\ (h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op))\ (fmap\ iso_1\ x))) \\
& \equiv alg\ (fmap\ (h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op))\ (fmap\ iso_1\ x)) \\
& \equiv alg\ (fmap\ (h_{Alg}\ gen\ (\lambda(Op\ op) \rightarrow alg\ op) . iso_1)\ x) \\
& \equiv alg\ (fmap\ (fold_{Alg}\ gen\ alg)\ x) \\
& \equiv fold_{Alg}\ gen\ alg\ (Op\ x)
\end{aligned}$$

A.2 Scoped Effects & Handlers

First, we define the isomorphisms:

$$\begin{aligned}
 iso_1 &:: \text{Functor } \gamma \Rightarrow \text{Free}_{\text{Sc}} \gamma \ a \rightarrow \text{Free}_{\text{H}} (K^{\text{Sc}} \gamma) \ a \\
 iso_1 (Var \ x) &= Var_{\text{H}} \ x \\
 iso_1 (Enter \ sc) &= Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . \text{fmap } iso_1) \ sc)) \\
 iso_2 &:: \text{Functor } \gamma \Rightarrow \text{Free}_{\text{H}} (K^{\text{Sc}} \gamma) \ a \rightarrow \text{Free}_{\text{Sc}} \gamma \ a \\
 iso_2 (Var_{\text{H}} \ x) &= Var \ x \\
 iso_2 (Op_{\text{H}} (\text{Enter} \ sc)) &= Enter (\text{fmap } (iso_2 . \text{fmap } iso_2) \ sc)
 \end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 . iso_2 = \text{id} = iso_2 . iso_1$.

$$\begin{aligned}
 (iso_1 . iso_2) (Var_{\text{H}} \ x) \\
 &\equiv iso_1 (Var \ x) \\
 &\equiv Var_{\text{H}} \ x
 \end{aligned}$$

$$\begin{aligned}
 (iso_1 . iso_2) (Op_{\text{H}} (\text{Enter} \ sc)) \\
 &\equiv iso_1 (Enter (\text{fmap } (iso_2 . \text{fmap } iso_2) \ sc)) \\
 &\equiv Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . \text{fmap } iso_1)) (\text{fmap } (iso_2 . \text{fmap } iso_2) \ sc)) \\
 &\equiv Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . \text{fmap } iso_1 . iso_2 . \text{fmap } iso_2) \ sc)) \\
 &\equiv Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . iso_2 . \text{fmap } iso_1 . \text{fmap } iso_2) \ sc)) \\
 &\equiv Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . iso_2 . \text{fmap } (iso_1 . iso_2)) \ sc)) \\
 &\equiv Op_{\text{H}} (\text{Enter} \ sc)
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 . iso_1) (Var \ x) \\
 &\equiv iso_2 (Var_{\text{H}} \ x) \\
 &\equiv Var \ x
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 . iso_1) (Enter \ sc) \\
 &\equiv iso_2 (Op_{\text{H}} (\text{Enter} (\text{fmap } (iso_1 . \text{fmap } iso_1) \ sc))) \\
 &\equiv Enter (\text{fmap } (iso_2 . \text{fmap } iso_2) (\text{fmap } (iso_1 . \text{fmap } iso_1) \ sc)) \\
 &\equiv Enter (\text{fmap } (iso_2 . \text{fmap } iso_2 . iso_1 . \text{fmap } iso_1) \ sc) \\
 &\equiv Enter (\text{fmap } (iso_2 . iso_1 . \text{fmap } iso_2 . \text{fmap } iso_1) \ sc) \\
 &\equiv Enter (\text{fmap } (iso_2 . iso_1 . \text{fmap } (iso_2 . iso_1)) \ sc) \\
 &\equiv Enter \ sc
 \end{aligned}$$

Furthermore, the handlers for these two recursive datatypes satisfy the following equivalence: $\text{fold}_{\text{Sc}} \text{ gen } alg_E \ alg_E = h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter} \ sc) \rightarrow \text{enter}_E \ alg_E \ sc) . iso_1$ with $\text{return}_E \ alg_E = \eta$.

$$\begin{aligned}
& h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) (\text{iso}_1 (\text{Var } x)) \\
& \equiv h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) (\text{Var}_H x) \\
& \equiv \text{gen } x \\
& \equiv \text{fold}_{\text{Sc}} \text{ gen } \text{alg}_E \text{ alg}_E (\text{Var } x) \\
\\
& h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) (\text{iso}_1 (\text{Enter } sc)) \\
& \equiv h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) \\
& \quad (\text{Op}_H (\text{Enter } (\text{fmap } (\text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc}))) \\
& \equiv (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})) \\
& \quad \quad (\text{Enter } (\text{fmap } (\text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc})))) \\
& \equiv (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{Enter } (\text{fmap } (\text{fmap } (h_{\text{Sc}} \text{ gen } \\
& \quad \quad (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})))) \\
& \quad \quad (\text{fmap } (\text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc}))) \\
& \equiv (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{Enter } (\text{fmap} \\
& \quad \quad (\text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})). \\
& \quad \quad \text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc}))) \\
& \equiv (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc}) \\
& \quad (\text{Enter } (\text{fmap fold}_{\rightsquigarrow} \\
& \quad \quad (\text{fmap } (\text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})). \\
& \quad \quad \text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc}))) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap fold}_{\rightsquigarrow} (\text{fmap} \\
& \quad (\text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})). \text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc})) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap} \\
& \quad (\text{fold}_{\rightsquigarrow} . \text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})) \\
& \quad \quad . \text{iso}_1 . \text{fmap } \text{iso}_1) \text{ sc}) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap} \\
& \quad (\text{fold}_{\rightsquigarrow} . \text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})) \\
& \quad \quad . \text{fmap } \text{iso}_1 . \text{iso}_1) \text{ sc}) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap} \\
& \quad (\text{fold}_{\rightsquigarrow} . \text{fmap } (h_{\text{Sc}} \text{ gen } (\lambda(\text{Enter } sc) \rightarrow \text{enter}_E \text{ alg}_E \text{ sc})). \text{iso}_1) \\
& \quad \quad . \text{iso}_1) \text{ sc}) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap } (\text{fold}_{\rightsquigarrow} . \text{fmap } (\text{fold}_{\text{Sc}} \text{ gen } \text{alg}_E \text{ alg}_E) . \text{iso}_1) \text{ sc}) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap } (\text{fold}_{\rightsquigarrow} . \text{iso}_1 . \text{fmap } (\text{fold}_{\text{Sc}} \text{ gen } \text{alg}_E \text{ alg}_E)) \text{ sc}) \\
& \equiv \text{enter}_E \text{ alg}_E (\text{fmap } (h_{\text{cata}} \text{ alg}_E . \text{fmap } (\text{fold}_{\text{Sc}} \text{ gen } \text{alg}_E \text{ alg}_E)) \text{ sc}) \\
& \equiv \text{fold}_{\text{Sc}} \text{ gen } \text{alg}_E \text{ alg}_E (\text{Enter } sc)
\end{aligned}$$

We need a helper lemma that states that $h_{\text{cata}} \text{ alg}_E = \text{fold}_{\rightsquigarrow} . \text{iso}_1$ with $\text{return}_E \text{ alg}_E = \eta$.

$$\begin{aligned}
h_{cata} \text{ alg}_E (Var\ x) & \\
&\equiv \text{return}_E \text{ alg}_E\ x \\
&\equiv \eta\ x \\
&\equiv \text{fold}_{\rightsquigarrow} (Var_H\ x) \\
&\equiv \text{fold}_{\rightsquigarrow} (iso_1 (Var\ x))
\end{aligned}$$

$$\begin{aligned}
h_{cata} \text{ alg}_E (Enter\ sc) & \\
&\equiv (enter_E \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E)))\ sc \\
&\equiv (enter_E \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E)))\ sc \\
&\equiv (\text{alg} . \text{Enter} . \text{fmap} (h_{cata} \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E)))\ sc \\
&\equiv \text{alg} (\text{Enter} (\text{fmap} (h_{cata} \text{ alg}_E . \text{fmap} (h_{cata} \text{ alg}_E))\ sc)) \\
&\equiv \text{alg} (\text{Enter} (\text{fmap} (\text{fold}_{\rightsquigarrow} . iso_1 . \text{fmap} (h_{cata} \text{ alg}_E))\ sc)) \\
&\equiv \text{alg} (\text{Enter} (\text{fmap} (\text{fold}_{\rightsquigarrow} . \text{fmap} (h_{cata} \text{ alg}_E) . iso_1)\ sc)) \\
&\equiv \text{alg} (\text{Enter} (\text{fmap} \text{fold}_{\rightsquigarrow} (\text{fmap} (\text{fmap} (h_{cata} \text{ alg}_E) . iso_1)\ sc))) \\
&\equiv \text{alg} (\text{hmap} \text{fold}_{\rightsquigarrow} (\text{Enter} (\text{fmap} (\text{fmap} (h_{cata} \text{ alg}_E) . iso_1)\ sc))) \\
&\equiv \text{alg} (\text{hmap} \text{fold}_{\rightsquigarrow} (\text{Enter} (\text{fmap} (\text{fmap} (\text{fold}_{\rightsquigarrow} . iso_1) . iso_1)\ sc))) \\
&\equiv \text{alg} (\text{hmap} \text{fold}_{\rightsquigarrow} (\text{Enter} (\text{fmap} (\text{fmap} \text{fold}_{\rightsquigarrow} . iso_1 . \text{fmap} iso_1)\ sc))) \\
&\equiv \text{alg} (\text{hmap} \text{fold}_{\rightsquigarrow} (\text{Enter} (\text{fmap} (\text{fmap} \text{fold}_{\rightsquigarrow}) \\
&\quad (\text{fmap} (iso_1 . \text{fmap} iso_1)\ sc)))) \\
&\equiv \text{alg} (\text{hmap} \text{fold}_{\rightsquigarrow} (\text{fmap} \text{fold}_{\rightsquigarrow} (\text{Enter} (\text{fmap} (iso_1 . \text{fmap} iso_1)\ sc)))) \\
&\equiv \text{fold}_{\rightsquigarrow} (Op_H (\text{Enter} (\text{fmap} (iso_1 . \text{fmap} iso_1)\ sc))) \\
&\equiv \text{fold}_{\rightsquigarrow} (iso_1 (Enter\ sc))
\end{aligned}$$

A.3 Parallel Effects & Handlers

First, we define the isomorphisms:

$$\begin{aligned}
iso_1 &:: \text{Functor } \rho \Rightarrow \text{Free}_{\text{Par}} \rho\ a \rightarrow \text{Free}_H (K^{\text{Par}} \rho)\ a \\
iso_1 (Var\ x) &= Var_H\ x \\
iso_1 (For\ iters\ k) &= Op_H (\text{For} (\text{fmap } iso_1\ iters) (\text{fmap } iso_1\ k)) \\
iso_2 &:: \text{Functor } \rho \Rightarrow \text{Free}_H (K^{\text{Par}} \rho)\ a \rightarrow \text{Free}_{\text{Par}} \rho\ a \\
iso_2 (Var_H\ x) &= Var\ x \\
iso_2 (Op_H (\text{For } iters\ k)) &= For (\text{fmap } iso_2\ iters) (\text{fmap } iso_2\ k)
\end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 . iso_2 = \text{id} = iso_2 . iso_1$.

$$\begin{aligned}
(iso_1 . iso_2) (Var_H\ x) & \\
&\equiv iso_1 (Var\ x) \\
&\equiv Var_H\ x
\end{aligned}$$

$$\begin{aligned}
& (iso_1 . iso_2) (Op_H (For\ iters\ k)) \\
& \equiv iso_1 (For\ (fmap\ iso_2\ iters)\ (fmap\ iso_2\ k)) \\
& \equiv Op_H (For\ (fmap\ iso_1\ (fmap\ iso_2\ iters))\ (fmap\ iso_1\ (fmap\ iso_2\ k))) \\
& \equiv Op_H (For\ (fmap\ (iso_1 . iso_2)\ iters)\ (fmap\ (iso_1 . iso_2)\ k)) \\
& \equiv Op_H (For\ iters\ k)
\end{aligned}$$

$$\begin{aligned}
& (iso_2 . iso_1) (Var\ x) \\
& \equiv iso_2 (Var_H\ x) \\
& \equiv Var\ x
\end{aligned}$$

$$\begin{aligned}
& (iso_2 . iso_1) (For\ iters\ k) \\
& \equiv iso_2 (Op_H (For\ (fmap\ iso_1\ iters)\ (fmap\ iso_1\ k))) \\
& \equiv For\ (fmap\ iso_2\ (fmap\ iso_1\ iters))\ (fmap\ iso_2\ (fmap\ iso_1\ k)) \\
& \equiv For\ (fmap\ (iso_2 . iso_1)\ iters)\ (fmap\ (iso_2 . iso_1)\ k) \\
& \equiv For\ iters\ k
\end{aligned}$$

We define a handler for $Free_{Par}\ \rho\ a$ as follows:

$$\begin{aligned}
\mathbf{data}\ AlgPar\ \rho\ f &= AlgPar\ \{h_{Var} :: \forall a\ .\ a \rightarrow f\ a, \\
&\quad h_{For}^\oplus :: \forall a\ b.\ \rho\ (f\ b) \rightarrow (\rho\ b \rightarrow f\ a) \rightarrow f\ a\} \\
fold_{Par} &:: (Functor\ \rho, Pointed\ f) \\
&\quad \Rightarrow (a \rightarrow f\ b) \rightarrow AlgPar\ \rho\ f \rightarrow Free_{Par}\ \rho\ a \rightarrow f\ b \\
fold_{Par}\ gen\ alg\ (Var\ x) &= gen\ x \\
fold_{Par}\ gen\ alg\ (For\ iters\ k) &= h_{For}^\oplus\ alg\ (fmap\ (fold_{Par}\ (h_{Var}\ alg)\ alg)\ iters) \\
&\quad (fold_{Par}\ gen\ alg.\ k)
\end{aligned}$$

Furthermore, we can prove that there is an isomorphism between the handlers h_{Par} and $fold_{Par}$: $fold_{Par}\ gen\ alg = h_{Par}\ gen\ (\lambda(For\ iters\ k) \rightarrow h_{For}^\oplus\ alg\ iters\ k) . iso_1$.

$$\begin{aligned}
& h_{Par}\ gen\ (\lambda(For\ iters\ k) \rightarrow h_{For}^\oplus\ alg\ iters\ k)\ (iso_1\ (Var\ x)) \\
& \equiv h_{Par}\ gen\ (\lambda(For\ iters\ k) \rightarrow h_{For}^\oplus\ alg\ iters\ k)\ (Var_H\ x) \\
& \equiv gen\ (Var_H\ x) \\
& \equiv fold_{Par}\ gen\ alg\ (Var\ x)
\end{aligned}$$

$$\begin{aligned}
& h_{Par}\ gen\ (\lambda(For\ iters\ k) \rightarrow h_{For}^\oplus\ alg\ iters\ k)\ (iso_1\ (For\ iters\ k)) \\
& \equiv h_{Par}\ gen\ (\lambda(For\ iters\ k) \rightarrow h_{For}^\oplus\ alg\ iters\ k)
\end{aligned}$$

$$\begin{aligned}
& (Op_H (\text{For} (\text{fmap } iso_1 \text{ iters}) (iso_1 . k))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{fmap} (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k)) \\
& \quad \quad (\text{For} (\text{fmap } iso_1 \text{ iters}) (iso_1 . k)))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{For} (\text{fmap } iso_1 \text{ iters}) \\
& \quad \quad (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) . iso_1 . k)))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{For} (\text{fmap fold}_{\rightsquigarrow} (\text{fmap } iso_1 \text{ iters})) \\
& \quad \quad (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) . iso_1 . k))) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap fold}_{\rightsquigarrow} (\text{fmap } iso_1 \text{ iters})) \\
& \quad (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) . iso_1 . k) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap} (\text{fold}_{\rightsquigarrow} . iso_1) \text{ iters}) \\
& \quad (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) . iso_1 . k) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap} (\text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg}) \text{ iters}) \\
& \quad (\text{fold } gen \ (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) . iso_1 . k) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap} (\text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg}) \text{ iters}) \\
& \quad (\text{fold}_{\text{Par}} \text{ gen } \text{ alg} . k) \\
& \equiv \text{fold}_{\text{Par}} \text{ gen } \text{ alg} (\text{For } iters \ k)
\end{aligned}$$

We need a helper lemma that states that $\text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg} = \text{fold}_{\rightsquigarrow} . iso_1$.

$$\begin{aligned}
& \text{fold}_{\rightsquigarrow} (iso_1 (\text{Var } x)) \\
& \equiv \text{fold}_{\rightsquigarrow} (\text{Var}_H x) \\
& \equiv \eta \ x
\end{aligned}$$

$$\begin{aligned}
& \text{fold}_{\rightsquigarrow} (iso_1 (\text{For } iters \ k)) \\
& \equiv \text{fold}_{\rightsquigarrow} (Op_H (\text{For} (\text{fmap } iso_1 \text{ iters}) (\text{fmap } iso_1 \ k))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{fmap fold}_{\rightsquigarrow} (\text{For} (\text{fmap } iso_1 \text{ iters}) (iso_1 . k)))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{hmap fold}_{\rightsquigarrow} (\text{For} (\text{fmap } iso_1 \text{ iters}) (\text{fold}_{\rightsquigarrow} . iso_1 . k)))) \\
& \equiv (\lambda(\text{For } iters \ k) \rightarrow h_{\text{For}}^{\oplus} \text{ alg } iters \ k) \\
& \quad (\text{For} (\text{fmap fold}_{\rightsquigarrow} (\text{fmap } iso_1 \text{ iters})) (\text{fold}_{\rightsquigarrow} . iso_1 . k))) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap fold}_{\rightsquigarrow} (\text{fmap } iso_1 \text{ iters})) (\text{fold}_{\rightsquigarrow} . iso_1 . k) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap} (\text{fold}_{\rightsquigarrow} . iso_1) \text{ iters}) (\text{fold}_{\rightsquigarrow} . iso_1 . k) \\
& \equiv h_{\text{For}}^{\oplus} \text{ alg} (\text{fmap} (\text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg}) \text{ iters}) (\text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg} . k) \\
& \equiv \text{fold}_{\text{Par}} (h_{\text{Var}} \text{ alg}) \text{ alg} (\text{For } iters \ k)
\end{aligned}$$

A.4 Writer Effect & Handler

First, we define the isomorphisms:

$$\begin{aligned}
 iso_1 &:: \text{Functor } \varphi \Rightarrow \text{Free}_{\text{Write}} \varphi a \rightarrow \text{Free}_{\text{H}} (K^{\text{Write}} \varphi) a \\
 iso_1 (Var\ x) &= Var_{\text{H}}\ x \\
 iso_1 (Exec\ op) &= Op_{\text{H}} (\text{Exec } ((iso_1 \cdot \text{fmap } (\text{fmap } iso_1))\ op)) \\
 iso_2 &:: \text{Functor } \varphi \Rightarrow \text{Free}_{\text{H}} (K^{\text{Write}} \varphi) a \rightarrow \text{Free}_{\text{Write}} \varphi a \\
 iso_2 (Var_{\text{H}}\ x) &= Var\ x \\
 iso_2 (Op_{\text{H}} (\text{Exec } op)) &= Exec ((iso_2 \cdot \text{fmap } (\text{fmap } iso_2))\ op)
 \end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 \cdot iso_2 = \text{id} = iso_2 \cdot iso_1$.

$$\begin{aligned}
 (iso_1 \cdot iso_2) (Var_{\text{H}}\ x) & \\
 &\equiv iso_1 (Var\ x) \\
 &\equiv Var_{\text{H}}\ x
 \end{aligned}$$

$$\begin{aligned}
 (iso_1 \cdot iso_2) (Op_{\text{H}} (\text{Exec } op)) & \\
 &\equiv iso_1 (Exec ((iso_2 \cdot \text{fmap } (\text{fmap } iso_2))\ op)) \\
 &\equiv Op_{\text{H}} (\text{Exec } ((iso_1 \cdot \text{fmap } (\text{fmap } iso_1)) ((iso_2 \cdot \text{fmap } (\text{fmap } iso_2))\ op))) \\
 &\equiv Op_{\text{H}} (\text{Exec } ((iso_1 \cdot \text{fmap } (\text{fmap } iso_1) \cdot iso_2 \cdot \text{fmap } (\text{fmap } iso_2))\ op)) \\
 &\equiv Op_{\text{H}} (\text{Exec } ((iso_1 \cdot iso_2 \cdot \text{fmap } (\text{fmap } iso_1 \cdot \text{fmap } iso_2))\ op)) \\
 &\equiv Op_{\text{H}} (\text{Exec } ((iso_1 \cdot iso_2 \cdot \text{fmap } (\text{fmap } (iso_1 \cdot iso_2)))\ op)) \\
 &\equiv Op_{\text{H}} (\text{Exec } op)
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 \cdot iso_1) (Var\ x) & \\
 &\equiv iso_2 (Var_{\text{H}}\ x) \\
 &\equiv Var\ x
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 \cdot iso_1) (Exec\ op) & \\
 &\equiv iso_2 (Op_{\text{H}} (\text{Exec } ((iso_1 \cdot \text{fmap } (\text{fmap } iso_1))\ op))) \\
 &\equiv Exec ((iso_2 \cdot \text{fmap } (\text{fmap } iso_2)) (iso_1 \cdot \text{fmap } (\text{fmap } iso_1))\ op) \\
 &\equiv Exec ((iso_2 \cdot \text{fmap } (\text{fmap } iso_2) \cdot iso_1 \cdot \text{fmap } (\text{fmap } iso_1))\ op) \\
 &\equiv Exec ((iso_2 \cdot iso_1 \cdot \text{fmap } (\text{fmap } iso_2 \cdot \text{fmap } iso_1))\ op) \\
 &\equiv Exec ((iso_2 \cdot iso_1 \cdot \text{fmap } (\text{fmap } (iso_2 \cdot iso_1)))\ op) \\
 &\equiv Exec\ op
 \end{aligned}$$

A.5 Latent Effects & Handlers

First, we define the isomorphisms:

$$\begin{aligned}
 iso_1 &:: Free_{Lat} \zeta \ell a \rightarrow Free_H (K^{Lat} \zeta \ell) a \\
 iso_1 (Leaf x) &= Var_H x \\
 iso_1 (Node op l sub k) &= Op_H (Node op l (fmap (fmap iso_1) sub) \\
 &\quad (fmap iso_1 k)) \\
 iso_2 &:: Free_H (K^{Lat} \zeta \ell) a \rightarrow Free_{Lat} \zeta \ell a \\
 iso_2 (Var_H x) &= Leaf x \\
 iso_2 (Op_H (Node op l sub k)) &= Node op l (fmap (fmap iso_2) sub) \\
 &\quad (fmap iso_2 k)
 \end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 . iso_2 = id = iso_2 . iso_1$.

$$\begin{aligned}
 (iso_1 . iso_2) (Var_H x) \\
 &\equiv iso_1 (Leaf x) \\
 &\equiv Var_H x
 \end{aligned}$$

$$\begin{aligned}
 (iso_1 . iso_2) (Op_H (Node op l sub k)) \\
 &\equiv iso_1 (Node op l (fmap (fmap iso_2) sub) (fmap iso_2 k)) \\
 &\equiv Op_H (Node op l (fmap (fmap iso_1) (fmap (fmap iso_2) sub)) \\
 &\quad (fmap iso_1 (fmap iso_2 k))) \\
 &\equiv Op_H (Node op l (fmap (fmap iso_1 . iso_2) sub) \\
 &\quad (fmap (iso_1 . iso_2) k)) \\
 &\equiv Op_H (Node op l sub k)
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 . iso_1) (Leaf x) \\
 &\equiv iso_2 (Var_H x) \\
 &\equiv Leaf x
 \end{aligned}$$

$$\begin{aligned}
 (iso_2 . iso_1) (Node op l sub k) \\
 &\equiv iso_2 (Op_H (Node op l (fmap (fmap iso_1) sub) (fmap iso_1 k))) \\
 &\equiv Node op l (fmap (fmap iso_2) (fmap (fmap iso_1) sub)) \\
 &\quad (fmap iso_2 (fmap iso_1 k)) \\
 &\equiv Node op l (fmap (fmap (iso_2 . iso_1)) sub) \\
 &\quad (fmap (iso_2 . iso_1) k) \\
 &\equiv Node op l sub k
 \end{aligned}$$

A.6 Bracketing Effect & Handler

First, we define the isomorphisms:

$$\begin{aligned}
 iso_1 &:: Free_{Res} a \rightarrow Free_H K^{Res} a \\
 iso_1 (Var x) &= Var_H x \\
 iso_1 (Bracket res) &= Op_H (Bracket (iso_1 (fmap (\lambda(x, y) \rightarrow \\
 &\quad (iso_1 x, return (iso_1 y))) res))) \\
 iso_2 &:: Free_H K^{Res} a \rightarrow Free_{Res} a \\
 iso_2 (Var_H x) &= Var x \\
 iso_2 (Op_H (Bracket res)) &= Bracket (iso_2 (fmap (\lambda(x, y) \rightarrow \\
 &\quad (iso_2 x, iso_2 (join y))) res))
 \end{aligned}$$

And next, we show that the requisite roundtrip properties hold, i.e., $iso_1 . iso_2 = id = iso_2 . iso_1$.

$$\begin{aligned}
 (iso_1 . iso_2) (Var_H x) & \\
 &\equiv iso_1 (Var x) \\
 &\equiv Var_H x \\
 \\
 (iso_1 . iso_2) (Op_H (Bracket res)) & \\
 &\equiv iso_1 (Bracket (iso_2 (fmap (\lambda(x, y) \rightarrow (iso_2 x, iso_2 (join y))) res))) \\
 &\equiv Op_H (Bracket (iso_1 (fmap (\lambda(x, y) \rightarrow (iso_1 x, return (iso_1 y))) \\
 &\quad (iso_2 (fmap (\lambda(x, y) \rightarrow (iso_2 x, iso_2 (join y))) res)))))) \\
 &\equiv Op_H (Bracket ((iso_1 . fmap (\lambda(x, y) \rightarrow (iso_1 x, return (iso_1 y))) . \\
 &\quad iso_2 . fmap (\lambda(x, y) \rightarrow (iso_2 x, iso_2 (join y)))) res)) \\
 &\equiv Op_H (Bracket ((iso_1 . iso_2 . fmap (\lambda(x, y) \rightarrow \\
 &\quad ((iso_1 . iso_2) x, (return . iso_1 . iso_2 . join) y)))) res) \\
 &\equiv Op_H (Bracket ((fmap (\lambda(x, y) \rightarrow (x, (return . join) y)))) res) \\
 &\equiv Op_H (Bracket (fmap (\lambda(x, y) \rightarrow (x, y))) res) \\
 &\equiv Op_H (Bracket res) \\
 \\
 (iso_2 . iso_1) (Var x) & \\
 &\equiv iso_2 (Var_H x) \\
 &\equiv Var x \\
 \\
 (iso_2 . iso_1) (Bracket res) & \\
 &\equiv iso_2 (Op_H (Bracket (iso_1 (fmap (\lambda(x, y) \rightarrow
 \end{aligned}$$

$$\begin{aligned}
& (iso_1 \ x, \text{return } (iso_1 \ y)) \ res)))) \\
\equiv & \text{Bracket } (iso_2 \ (\text{fmap } (\lambda(x, y) \rightarrow (iso_2 \ x, iso_2 \ (\text{join } y))) \\
& \quad (iso_1 \ (\text{fmap } (\lambda(x, y) \rightarrow (iso_1 \ x, \text{return } (iso_1 \ y))) \ res)))))) \\
\equiv & \text{Bracket } ((iso_2 \ . \text{fmap } (\lambda(x, y) \rightarrow (iso_2 \ x, iso_2 \ (\text{join } y)))) \ . \\
& \quad iso_1 \ . \text{fmap } (\lambda(x, y) \rightarrow (iso_1 \ x, \text{return } (iso_1 \ y)))) \ res) \\
\equiv & \text{Bracket } ((iso_2 \ . iso_1 \ . \text{fmap} \\
& \quad (\lambda(x, y) \rightarrow ((iso_2 \ . iso_1) \ x, (iso_2 \ . \text{join} \ . \text{return} \ . iso_1) \ y))) \ res) \\
\equiv & \text{Bracket } ((\text{fmap } (\lambda(x, y) \rightarrow (x, (iso_2 \ . iso_1) \ y))) \ res) \\
\equiv & \text{Bracket } (\text{fmap } (\lambda(x, y) \rightarrow (x, y)) \ res) \\
\equiv & \text{Bracket } res
\end{aligned}$$

Appendix B

Examples of Higher-Order Effects

This Appendix provides examples of different higher-order effects. For some of the scoped effects in this Chapter, we also give their syntax in terms of λ_{sc} .

To enhance readability of the examples in λ_{sc} , we use syntactic sugar following Eff's conventions: we use top-level definitions, coalesce values and computations, implicitly sequence steps and insert `return` where needed. Furthermore, we drop trivial `return` continuations of operations:

$$\begin{aligned}\mathbf{op} \ell^{\mathbf{op}} x &\equiv \mathbf{op} \ell^{\mathbf{op}} x (y . \mathbf{return} \ y) \\ \mathbf{sc} \ell^{\mathbf{sc}} x (y . c_1) &\equiv \mathbf{sc} \ell^{\mathbf{sc}} x (y . c_1) (z . \mathbf{return} \ z)\end{aligned}$$

B.1 Catch as a Scoped Effect

Until now, we have defined exceptions with *Throw* and *Catch* as higher-order effects with a specialized higher-order signature. However, in the literature, *Catch* is often modeled as a scoped effect. We provide such implementation here, and also extend *Throw* to *Raise e* to support exceptions of type e .

Catch using the Higher-Order Framework

We have defined exceptions in Section 3.1 using a higher-order signature. However, we can also model exceptions using scoped effects:

```
data Raise e a = Raise e
data Catch a  = Catch a a
```

Raise is an algebraic operation, like *Throw*, but it throws an error of type *e*. *Catch* is now modeled as a scoped effect.

```
hCatch :: FreeH (KAlg (Raise e) ⊕ KSc Catch) a → Either e a
hCatch = fold η (algRaise ⊕ algCatch)
algRaise :: KAlg (Raise e) (Either e) (Either e a) → Either e a
algRaise (Op (Raise e)) = Left e
algCatch :: KSc Catch (Either e) (Either e a) → Either e a
algCatch (Enter (Catch p k)) = case p of { Left e' → join k; _ → join p }
```

For example, *c_{Catch}* increments a number *x*, but throws an error if this integer is greater than 5.

```
cCatch x = catch (if x > 5 then raise "Error!" else return (x + 1)) (return x)

>>> hCatch (cCatch 5)           >>> hCatch (cCatch 10)
Right 6                          Left "Error!"
```

Catching Exceptions in λ_{sc}

The scoped computation's true branch is the program that may *raise* exceptions, while the false branch *deals with* the exception. Our handler interprets exceptions in terms of a sum type $\alpha + \beta = \text{left } \alpha \mid \text{right } \beta$, where *left* *v* denotes an exception and *right* *v* a result. This corresponds to the above *Either e a* datatype.

```
hCatch : ∀ α μ . α ! ⟨raise; catch; μ⟩ ⇒ String + α ! ⟨μ⟩
hCatch = handler
  { return x      ↦ right x
  , op raise e _  ↦ left e
  , sc catch _ p k ↦ do x ← p true;
                     case x of left e → mapCatch (p false) k
```

$$\text{, fwd } f \ p \ k \quad \mapsto f \ (p, \lambda z. \text{map}_{\text{Catch}} \ z \ k) \}$$

The return clause and algebraic effect clause for **raise** construct a return value and raise an exception e by calling the *right* and *left* constructors, respectively. The scoped effect clause for **catch** catches an exception e . If the scoped computation in p *true* raises an exception e , it is caught by **catch** and replaced by the scoped computation (p *false*). Otherwise, it continues with p *true* and its results are passed to the continuation k . For forwarding we essentially return the exception if z fails (*left* e), and we apply the continuation k to z if z succeeds (*right* x).

$$\begin{aligned} \text{map}_{\text{Catch}} &: \forall \alpha \beta \mu. \text{String} + \beta \rightarrow^\mu (\beta \rightarrow^\mu \text{String} + \alpha) \rightarrow^\mu \text{String} + \alpha \\ \text{map}_{\text{Catch}} \ z \ k &= \text{case } z \text{ of } \text{left } e \rightarrow \text{left } e \\ &\quad \text{right } x \rightarrow k \ x \end{aligned}$$

For example, we revisit the same computation as above in terms of λ_{sc} :

$$\begin{aligned} c_{\text{Catch}} &= \lambda x \rightarrow \text{sc catch } () \ (b. \text{if } b \text{ then} \\ &\quad (\text{if } x > 5 \quad \text{then op raise "Error!" } (y. \text{absurd } y) \text{ else return } (x + 1)) \\ &\quad \text{else return } x) \ (x. \text{return } x) \end{aligned}$$

$$c_{\text{Catch}} \ 5 \rightsquigarrow^* \text{right } 6 \qquad c_{\text{Catch}} \ 10 \rightsquigarrow^* \text{left "Error!"}$$

B.2 Reader as a Scoped Effect

This effect is based on Haskell's `MonadReader`¹. The *Reader* effect consists of two operations:

$$\begin{aligned} \text{ask} &:: m \ r \\ \text{local} &:: (r \rightarrow r) \rightarrow m \ a \rightarrow m \ a \end{aligned}$$

The former is used to read the environment, and the latter locally modifies the environment r using a function $f :: r \rightarrow r$.

Reader in λ_{sc}

The handler for reading with **ask** and **local** is defined as follows:

¹<https://hackage.haskell.org/package/mtl-2.3/docs/Control-Monad-Reader.html>

$$\begin{aligned}
h_{\text{Read}} &: \forall \alpha \mu . \alpha ! \langle \text{ask}; \text{local}; \mu \rangle \Rightarrow (Int \rightarrow^\mu (\alpha, Int))! \langle \mu \rangle \\
h_{\text{Read}} = \text{handler} \{ &\text{return } x \quad \mapsto \lambda m . (x, m) \\
&, \text{op ask } _ k \quad \mapsto \lambda m . k \ m \ m \\
&, \text{sc local } f \ p \ k \mapsto \lambda m . \text{do } (x, _) \leftarrow p \ () \ (f \ m); k \ x \ m \\
&, \text{fwd } f \ p \ k \quad \mapsto \lambda m . f \ (\lambda y . p \ y \ m, \lambda(z, m') . k \ z \ m') \}
\end{aligned}$$

Computation c_{local} asks the state and locally doubles it. We return a triple with r_1 the initial state, r_2 the local state (doubled), and r_3 the state in the continuation.

$$\begin{aligned}
c_{\text{local}} = &\text{do } r_1 \leftarrow \text{op ask } (); \\
&\text{sc local } (*2) (\lambda _ . \text{do } r_2 \leftarrow \text{op ask } (); \text{return } r_2) \\
&(\lambda r_2 . \text{do } r_3 \leftarrow \text{op ask } (); \text{return } (r_1, r_2, r_3))
\end{aligned}$$

For example,

$$\text{do } c \leftarrow h_{\text{Read}} \star c_{\text{local}}; c \ 1 \rightsquigarrow^* (1, 2, 1)$$

Reader using the Higher-Order Framework

The signatures for these effects are the following:

$$\begin{aligned}
\text{data Ask } r \ a &= \text{Ask } (r \rightarrow a) \\
\text{data Local } r \ a &= \text{Local } (r \rightarrow r) \ a
\end{aligned}$$

Ask is an algebraic operation, that behaves similar to *Get* from the *State* effect. *Local*, however, contains a scope, so it is modeled as a scoped effect. We define a handler and algebras for this reader effect as follows:

$$\begin{aligned}
h_{\text{Read}} &:: \text{Free}_H (K^{\text{Alg}} (\text{Ask } r) \oplus K^{\text{Sc}} (\text{Local } r)) \ a \rightarrow r \rightarrow a \\
h_{\text{Read}} &= \text{fold } \eta \ (\text{alg}_{\text{Ask}} \oplus \text{alg}_{\text{Local}}) \\
\text{alg}_{\text{Ask}} &:: K^{\text{Alg}} (\text{Ask } r) \ ((\rightarrow) \ r) \ (r \rightarrow a) \rightarrow r \rightarrow a \\
\text{alg}_{\text{Ask}} \ (\text{Op } (\text{Ask } k)) &= \lambda r \rightarrow k \ r \ r \\
\text{alg}_{\text{Local}} &:: K^{\text{Sc}} (\text{Local } r) \ ((\rightarrow) \ r) \ (r \rightarrow a) \rightarrow r \rightarrow a \\
\text{alg}_{\text{Local}} \ (\text{Enter } (\text{Local } f \ k)) &= \lambda r \rightarrow k \ (f \ r) \ r
\end{aligned}$$

For example,

$$\begin{aligned}
>>> h_{\text{Read}} \ (\text{do } r_1 \leftarrow \text{ask}; r_2 \leftarrow \text{local } (*2) \ \text{ask}; r_3 \leftarrow \text{ask}; \text{return } (r_1, r_2, r_3)) \ 1 \\
(1, 2, 1)
\end{aligned}$$

B.3 Nondeterminism with Cut

The algebraic operation `cut` provides a flavor of pruning nondeterminism and has its origin as a Prolog primitive. The idea is that `cut` prunes all remaining branches and only allows the current branch to continue. Typically, we want to keep the effect of `cut` local. This is achieved with a scoped operation `call`. This example is inspired by the cut-and-call example of Wu et al. [173].

Cut in λ_{sc}

To handle `cut` : $() \rightarrow ()$ and `call` : $() \rightarrow ()$, we use the *CutList* datatype [118].

data *CutList* $\alpha = \text{ret } [\alpha] \mid \text{flag } [\alpha]$

We can think of *ret* v as a list that may be extended and *flag* v as a list that may not be extended with further elements. This intention is captured in the *append* function, which discards the second list if the constructor of the first list is *flag*.

$$\begin{aligned} \text{append} &: \forall \alpha \mu. \text{CutList } \alpha \rightarrow^\mu \text{CutList } \alpha \rightarrow^\mu \text{CutList } \alpha \\ \text{append } (\text{ret } xs) & (\text{ret } ys) = \text{ret } (xs ++ ys) \\ \text{append } (\text{ret } xs) & (\text{flag } ys) = \text{flag } (xs ++ ys) \\ \text{append } (\text{flag } xs) & _ = \text{flag } xs \end{aligned}$$

The handler for nondeterminism with cut is defined as follows:

$$\begin{aligned} h_{\text{Cut}} &: \forall \alpha \mu. \alpha ! \langle \text{or}; \text{fail}; \text{cut}; \text{call}; \mu \rangle \Rightarrow \text{CutList } \alpha ! \langle \mu \rangle \\ h_{\text{Cut}} = \text{handler } \{ & \text{return } x \quad \mapsto \text{ret } [x] \\ & , \text{op fail } _ \mapsto \text{ret } [] \\ & , \text{op or } x \ k \mapsto \text{append } (k \ \text{true}) (k \ \text{false}) \\ & , \text{op cut } _ \ k \mapsto \text{close } (k \ ()) \\ & , \text{sc call } _ \ p \ k \mapsto \text{concatMap}_{\text{Cut}} (\text{open } (p \ ())) \ k \\ & , \text{fwd } f \ p \ k \mapsto f \ (p, \lambda z. \text{concatMap}_{\text{Cut}} \ z \ k) \} \end{aligned}$$

The operation clause for `cut` closes the cutlist and the clause for `call` (re-)opens it when coming out of the scope.

$$\begin{aligned} \text{close} &: \forall \alpha \mu. \text{CutList } \alpha \rightarrow^\mu \text{CutList } \alpha \\ \text{close } (\text{flag } as) & = \text{flag } as \end{aligned}$$

$$\begin{aligned}
\text{close } (\text{ret } as) &= \text{flag } as \\
\text{open} : \forall \alpha \mu. \text{CutList } \alpha \rightarrow^\mu \text{CutList } \alpha \\
\text{open } (\text{flag } as) &= \text{ret } as \\
\text{open } (\text{ret } as) &= \text{ret } as
\end{aligned}$$

The function $\text{concatMap}_{\text{Cut}}$ is the cutlist counterpart of concatMap for lists which takes the extensibility of CutList (signalled by ret and flag) into account when concatenating.

$$\begin{aligned}
\text{concatMap}_{\text{Cut}} : \forall \alpha \beta \mu. \text{CutList } \beta \rightarrow^\mu (\beta \rightarrow^\mu \text{CutList } \alpha) \rightarrow^\mu \text{CutList } \alpha \\
\text{concatMap}_{\text{Cut}} (\text{ret } []) \quad f &= \text{return } (\text{ret } []) \\
\text{concatMap}_{\text{Cut}} (\text{flag } []) \quad f &= \text{return } (\text{flag } []) \\
\text{concatMap}_{\text{Cut}} (\text{ret } (b : bs)) \quad f &= \text{do } as \leftarrow f \ b; \\
&\quad as' \leftarrow \text{concatMap}_{\text{Cut}} (\text{ret } bs) \ f; \\
&\quad \text{append } as \ as' \\
\text{concatMap}_{\text{Cut}} (\text{flag } (b : bs)) \quad f &= \text{do } as \leftarrow f \ b; \\
&\quad as' \leftarrow \text{concatMap}_{\text{Cut}} (\text{flag } bs) \ f; \\
&\quad \text{append } as \ as'
\end{aligned}$$

In what follows, we give an example usage of `cut` to improve parsers.

Parsers in λ_{sc}

A parser effect can be achieved by combining the nondeterminism-with-cut effect and a token-consuming effect [173]. The latter features the algebraic operation $\text{token} : \text{Char} \rightarrow \text{Char}$ where $\text{op token } t$ consumes a single character from the implicit input string; if it is t , it is passed on to the continuation; otherwise the operation fails. The token handler has result type $\text{String} \rightarrow^{\langle \text{fail}; \mu \rangle} (\alpha, \text{String})$: it threads through the remaining part of the input string. Observe that the function type signals it may fail, in case the token does not match.

$$\begin{aligned}
h_{\text{Token}} : \forall \alpha \mu. \alpha ! \langle \text{token}; \text{fail}; \mu \rangle &\Rightarrow (\text{String} \rightarrow^{\langle \text{fail}; \mu \rangle} (\alpha, \text{String})) ! \langle \text{fail}; \mu \rangle \\
h_{\text{Token}} &= \text{handler} \\
\{ \text{return } x &\mapsto \lambda s. (x, s) \\
, \text{op token } x \ k &\mapsto \lambda s. \text{case } s \text{ of } [] \rightarrow \text{op fail } () \ (y. \text{absurd } y) \\
&\quad (x' : xs) \rightarrow \text{if } x \equiv x' \text{ then } k \ x \ xs \\
&\quad \text{else op fail } () \ (y. \text{absurd } y) \\
, \text{fwd } f \ p \ k &\mapsto \lambda s. f \ (\lambda y. p \ y \ s, \lambda (t, s). k \ t \ s) \}
\end{aligned}$$

We give an example parser for a small expression language, in the typical parser combinator style, built on top of the token-consumer and nondeterminism. For convenience, it uses the syntactic sugar $x \diamond y \equiv \text{op or } (b.\text{if } b \text{ then } x \text{ else } y)$.

```

digit    :  $\forall \mu. () \rightarrow \text{Char} ! \langle \text{token}; \text{or}; \mu \rangle$ 
digit _  = op token '0'  $\diamond$  op token '1'  $\diamond \dots \diamond$  op token '9'
many     :  $\forall \alpha \mu. ( () \rightarrow^\mu \alpha ) \rightarrow^\mu \text{List } \alpha$ 
many p   = do  $i \leftarrow p ()$ ; do  $as \leftarrow \text{many } p \diamond \text{return } []$ ; return ( $a : as$ )
expr'    :  $\forall \mu. () \rightarrow \text{Int} ! \langle \text{token}; \text{or}; \mu \rangle$ 
expr' _  = do  $i \leftarrow \text{term } ()$ ; do op token '+'; do  $j \leftarrow \text{expr}' ()$ ;
           return ( $i + j$ )  $\diamond$  do  $i \leftarrow \text{term } ()$ ; return  $i$ 
term     :  $\forall \mu. () \rightarrow \text{Int} ! \langle \text{token}; \text{or}; \mu \rangle$ 
term _   = do  $i \leftarrow \text{factor } ()$ ; do op token '*'; do  $j \leftarrow \text{term } ()$ ;
           return ( $i * j$ )  $\diamond$  do  $i \leftarrow \text{factor } ()$ ; return  $i$ 
factor   :  $\forall \mu. () \rightarrow \text{Int} ! \langle \text{token}; \text{or}; \mu \rangle$ 
factor _ = do  $ds \leftarrow \text{many digit}$ ; return ( $\text{read } ds$ )
            $\diamond$  do op token '('; do  $i \leftarrow \text{expr}' ()$ ; do op token ')'; return  $i$ 

```

The `expr'` parser is naive and can be improved by two types of refactoring: (1) factoring out the common prefix in the two branches, and (2) pruning the second branch when the first branch successfully consumes a `+`.

```

expr     :  $\forall \mu. () \rightarrow \text{Int} ! \langle \text{token}; \text{or}; \text{cut}; \mu \rangle$ 
expr _   = do  $i \leftarrow \text{term } ()$ ;
           sc call  $() \text{ } (-. (\text{do } \text{op token '+'}; \text{op cut } ());$ 
                $j \leftarrow \text{expr } (); \text{return } (i + j)) \diamond i$ 

```

Here is how we invoke the parser on an example input.

```

hCut  $\star$  (hToken  $\star$  expr ()) "(2+5)*8"  $\rightsquigarrow^* \text{ret } [(56, ""), (7, "*8")]$ 

```

There are two results in the cutlist. Usually we are only interested in the full parsers, i.e., those that have consumed the entire input string.

Cut using the Higher-Order Framework

We revisit the above example, and define *CutList* as follows:

```

data CutList a = Ret [a] | Flag [a]

```

Futhermore, we need helper functions to *append* *CutList*'s, to *open* them and to *close* them.

$$\begin{aligned} \text{append} &:: \text{CutList } a \rightarrow \text{CutList } a \rightarrow \text{CutList } a \\ \text{append } (\text{Ret } xs) \ (\text{Ret } ys) &= \text{Ret } (xs \text{ ++ } ys) \\ \text{append } (\text{Ret } xs) \ (\text{Flag } ys) &= \text{Flag } (xs \text{ ++ } ys) \\ \text{append } (\text{Flag } xs) \ _ &= \text{Flag } xs \end{aligned}$$

$$\begin{array}{ll} \text{open} :: \text{CutList } a \rightarrow \text{CutList } a & \text{close} :: \text{CutList } a \rightarrow \text{CutList } a \\ \text{open } (\text{Flag } xs) = \text{Ret } xs & \text{close } (\text{Flag } xs) = \text{Flag } xs \\ \text{open } (\text{Ret } xs) = \text{Ret } xs & \text{close } (\text{Ret } xs) = \text{Flag } xs \end{array}$$

The signatures for *Cut* and *Call* are the following:

$$\mathbf{data} \text{ Cut } a = \text{Cut } a \qquad \mathbf{data} \text{ Call } k = \text{Call } k$$

Cut is an algebraic operation. *Call* keeps track of the scope of the *Cut* effect, so it is modelled as a scoped effect. We define a handler and algebras for this combination of *Cut* and *Call* with nondeterminism as follows:

$$\begin{aligned} h_{\text{Cut}} &:: \text{Free}_H (K^{\text{Alg}} \text{Choice} \oplus (K^{\text{Alg}} \text{Cut} \oplus K^{\text{Sc}} \text{Call})) \ a \rightarrow \text{CutList } a \\ h_{\text{Cut}} &= \text{fold } \eta \ (alg_{\text{ND,Cut}} \oplus (alg_{\text{Cut}} \oplus alg_{\text{Call}})) \\ alg_{\text{ND,Cut}} &:: K^{\text{Alg}} \text{Choice} \ \text{CutList } (\text{CutList } a) \rightarrow \text{CutList } a \\ alg_{\text{ND,Cut}} \ (\text{Op } \text{Fail}) &= \text{Ret } [] \\ alg_{\text{ND,Cut}} \ (\text{Op } (\text{Or } p \ q)) &= \text{append } p \ q \\ alg_{\text{Cut}} &:: K^{\text{Alg}} \text{Cut} \ \text{CutList } (\text{CutList } a) \rightarrow \text{CutList } a \\ alg_{\text{Cut}} \ (\text{Op } (\text{Cut } k)) &= \text{close } k \\ alg_{\text{Call}} &:: K^{\text{Sc}} \text{Call} \ \text{CutList } (\text{CutList } a) \rightarrow \text{CutList } a \\ alg_{\text{Call}} \ (\text{Enter } (\text{Call } k)) &= \text{join } (\text{open } k) \end{aligned}$$

For example, we apply the above handler to a computation that cuts the program in its first branch. *call* delimits the scope of the cut.

```
>>> h_Cut (or (call (or (cut >> return 1) (return 2))) (return 3))
Ret [1,3]
```

B.4 Depth-Bounded Search

The handlers for nondeterminism defined up till now implement the *depth-first search* (DFS) strategy. However, with scoped effects & handlers we can implement other search strategies, such as *depth-bounded search* (DBS), which uses the scoped operation **depth** to bound the depth of the branches in the scoped computation. This example is inspired by Yang et al. [176].

Depth-Bounded Search in λ_{sc}

We model the operation **depth** : $Int \rightarrow ()$ as a scoped effect. The handler uses return type $Int \rightarrow^\mu [(\alpha, Int)]$. Here, the Int parameter is the current depth bound, and the result is a list of (α, Int) pairs, where α denotes the result and Int reflects the remaining global depth bound.²

$$\begin{aligned}
 h_{\text{Depth}} &: \forall \alpha \mu. \alpha ! \langle \text{or}; \text{fail}; \text{depth}; \mu \rangle \Rightarrow (Int \rightarrow^\mu [(\alpha, Int)]) ! \langle \mu \rangle \\
 h_{\text{Depth}} &= \text{handler} \\
 &\quad \{ \text{return } x \quad \mapsto \lambda d. [(x, d)] \\
 &\quad , \text{op fail } - \quad \mapsto \lambda -. [] \\
 &\quad , \text{op or } x \ k \quad \mapsto \lambda d. \text{if } d \equiv 0 \text{ then } [] \\
 &\quad \quad \quad \quad \quad \quad \quad \text{else } k \text{ true } (d - 1) \text{ ++ } k \text{ false } (d - 1) \\
 &\quad , \text{sc depth } d' \ p \ k \mapsto \lambda d. \text{concatMap } (p \ () \ d') \ (\lambda (v, -). k \ v \ d) \\
 &\quad , \text{fwd } f \ p \ k \quad \mapsto \lambda d. f \ (\lambda y. p \ y \ d, \lambda vs. \text{concatMap } vs \ (\lambda (v, d). k \ v \ d)) \}
 \end{aligned}$$

For the **depth** operation, we locally use the given depth bound d' for the scoped computation p and go back to using the global depth bound d for the continuation k . In case of an unknown scoped operation, the forwarding clause just threads the depth bound through, first into the scoped computation and from there into the continuation. For example, the following program (Figure B.1) has a local depth bound of 1 and a global depth bound of 2. It discards the results 2 and 3 in the scoped computation as they appear after the second **or** operation, and similarly, the results 5 and 6 in the continuation are ignored.

$$\begin{aligned}
 c_{\text{Depth}} &= \text{sc depth } 1 \\
 &\quad (_ . \text{do } b_1 \leftarrow \text{op or } (); \text{if } b_1 \text{ then return } 1 \text{ else} \\
 &\quad \quad \text{do } b_2 \leftarrow \text{op or } (); \text{if } b_2 \text{ then return } 2 \text{ else return } 3) \\
 &\quad (x . \text{do } b_1 \leftarrow \text{op or } (); \text{if } b_1 \text{ then return } x \text{ else}
 \end{aligned}$$

²These pairs (α, Int) differ from Yang et al.'s [176]'s α in order to make the forwarding clause work.

```

do  $b_2 \leftarrow \text{op or } (); \text{if } b_2 \text{ then return 4 else}$ 
do  $b_3 \leftarrow \text{op or } (); \text{if } b_3 \text{ then return 5 else return 6}$ 

```

For example,

$$(h_{\text{Depth}} \star c_{\text{Depth}}) 2 \rightsquigarrow^* [(1, 1), (4, 0)]$$

The result is $[(1, 1), (4, 0)]$, where the tuple's second parameter represents the global depth bound. Notice that **or** operations in the scoped computation **depth** do not consume the global depth bound in the handler.

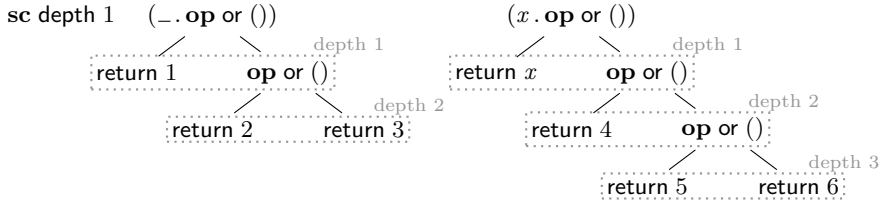


Figure B.1: Visual representation of c_{Depth} .

Depth-Bounded Search using the Higher-Order Framework

The signature of the *Depth* effect is the following:

```
data Depth a = Depth Int a
```

It is modeled as a scoped effect, as we want a different depth bound locally and globally. We also require an algebra that differs from alg_{ND} , as we need to keep track of the depth in the nondeterministic computations. We require a depth carrier DC that represents the result type of h_{Depth} as a functor.

```
newtype DC a = DC { unDC :: Int → [(a, Int)] }
```

The handler for this effect is defined as follows:

```

hDepth :: FreeH (KAlg Choice ⊕ KSc Depth) a → Int → [(a, Int)]
hDepth = unDC . fold η (algND, Depth ⊕ algDepth)

```

```

algND,Depth :: KAlg Choice DC (DC a) → DC a
algND,Depth (Op Fail)      = DC (const [])
algND,Depth (Op (Or p q)) = DC (λi →
  if i ≡ 0 then [] else unDC p (i - 1) ++ unDC q (i - 1))
algDepth :: KSc Depth DC (DC a) → DC a
algDepth (Enter (Depth i k)) = DC (λi' → concatMap fst
  (unDC (fmap (($i'). unDC) k) i))

```

The following example has a local depth bound of 1 (in `depth`), and a global depth bound of 2 (in `hDepth`).

```

cDepth = depth 1 (or (return 1) (or (return 2) (return 3)))
  >>= λx → (or (return x) (or (return 4) (or (return 5) (return 6))))
>>> hDepth cDepth 2
[(1,1), (4,0)]

```

B.5 Concurrency

The following example is inspired by Piròg et al. [117]. It models concurrent computations using scoped effects.

The *Act m* signature consists of two parts: one that carries out an action (*Act c*) and one that terminates the process (*Kill*). The *Con* signature, which we will model as a scoped effect, can spawn a thread to run the two programs concurrently (*Spawn c₁ c₂*), and can define an atomic computation (*Atomic c*), which should not be intertwined with any other computations.

data *Act m a* = *Act (m a) | Kill* **data** *Con a* = *Spawn a a | Atomic a*

Following Piròg et al, we define some helper functions and data structures for modeling resumptions (*Res*), flattening resumptions (*retraction*), and composing two concurrent resumptions (*interleave*).

```

data Res m a = More (m (Res m a)) | Done a
retraction :: Monad m ⇒ Res m a → m a
retraction (More m) = m >>= retraction
retraction (Done x) = return x
interleave :: Monad m ⇒ Res m a → Res m b → Res m a
interleave (Done x) r = fmap (const x) r

```

$$\begin{aligned}
& \text{interleave } r \quad (\text{Done } _) = r \\
& \text{interleave } (\text{More } m_1) (\text{More } m_2) = \text{More } (\text{do } r_1 \leftarrow m_1; r_2 \leftarrow m_2; \\
& \quad \text{return } (\text{interleave } r_1 \ r_2))
\end{aligned}$$

We define a handler for concurrent programming as follows:

$$\begin{aligned}
h_{\text{Conc}} &:: \text{Monad } m \Rightarrow \text{Free}_H (K^{\text{Alg}} (\text{Act } m) \oplus K^{\text{Sc}} \text{Con}) a \rightarrow \text{Res } m \ a \\
h_{\text{Conc}} &= \text{fold Done } (\text{alg}_{\text{Act}} \oplus \text{alg}_{\text{Con}}) \\
\text{alg}_{\text{Act}} &:: K^{\text{Alg}} (\text{Act } m) (\text{Res } m) (\text{Res } m \ a) \rightarrow \text{Res } m \ a \\
\text{alg}_{\text{Act}} (\text{Op } (\text{Act } m)) &= \text{More } m \\
\text{alg}_{\text{Act}} (\text{Op Kill}) &= \text{Done } (\text{error "main process killed"}) \\
\text{alg}_{\text{Con}} &:: \text{Monad } m \Rightarrow K^{\text{Sc}} \text{Con} (\text{Res } m) (\text{Res } m \ a) \rightarrow \text{Res } m \ a \\
\text{alg}_{\text{Con}} (\text{Enter } (\text{Atomic } r)) &= \text{More } (\text{retraction } r) \\
\text{alg}_{\text{Con}} (\text{Enter } (\text{Spawn } r_1 \ r_2)) &= \text{join } (\text{interleave } r_1 \ r_2)
\end{aligned}$$

We revisit the same examples as in Piròg et al.

$$\begin{aligned}
& \text{say} = \text{act.putStr} \\
& \text{test}_{\text{conc},1} = \text{spawn } (\text{say "hello " } \gg \text{say "world "}) \\
& \quad (\text{say "goodbye " } \gg \text{say "cruel " } \gg \text{say "world "}) \\
& \text{test}_{\text{conc},2} = \text{spawn } (\text{atomic } (\text{spawn } (\text{mapM say } ["a", "b", "c"]))) \\
& \quad (\text{mapM say } ["A", "B", "C"]))) \\
& \quad (\text{atomic } (\text{spawn } (\text{mapM say } ["1", "2", "3"]))) \\
& \quad (\text{mapM say } ["-", "-", "-"])))
\end{aligned}$$

The computation *retraction* ($h_{\text{Conc}} \text{test}_{\text{conc},1}$) prints

"hello goodbye world cruel world",

and the computation *retraction* ($h_{\text{Conc}} \text{test}_{\text{conc},2}$) prints "aAbBcC1-2-3-".

B.6 Amb

Our example of the *Amb* effect [101] is inspired by that of Xie et al. [174] for modeling parallel effects. This effect takes a list of values and chooses one nondeterministically.

$$\text{data Amb } r = \text{Amb } [r]$$

We define a handler for this effect in combination with a parallel list as follows:


```

 $h_{\text{Amb}} :: \text{Free}_H (K^{\text{Alg}} \text{Amb} \oplus K^{\text{Par}} []) a \rightarrow [a]$ 
 $h_{\text{Amb}} = \text{fold return } (\text{alg}_{\text{Amb}} \oplus \text{alg}_{\text{For}})$ 
 $\text{alg}_{\text{Amb}} :: K^{\text{Alg}} \text{Amb} [] [a] \rightarrow [a]$ 
 $\text{alg}_{\text{Amb}} (\text{Op } (\text{Amb } r)) = \text{concat } r$ 
 $\text{alg}_{\text{For}} :: K^{\text{Par}} [] [] [a] \rightarrow [a]$ 
 $\text{alg}_{\text{For}} (\text{For } \text{iters } k) = \text{concat } (\text{fmap } k (\text{foldr } (\times) [[]] \text{iters}))$ 
 $(\times) :: [a] \rightarrow [[a]] \rightarrow [[a]]$ 
 $(\times) \text{xs yss} = [x : \text{ys} \mid x \leftarrow \text{xs}, \text{ys} \leftarrow \text{yss}]$ 

```

For example, *coin* gives all combinations of "H" and "T" of length 3.

```

 $\text{coin} = \text{do } \text{chars} \leftarrow \text{for } (\text{replicate } 3 (\text{amb } [\text{return "H"}, \text{return "T"}]));$ 
 $\text{return } (\text{concat } \text{chars})$ 
 $\gg\gg h_{\text{Amb}} \text{coin}$ 
 $["HHH", "HHT", "HTH", "HTT", "THH", "THT", "TTH", "TTT"]$ 

```

Similarly, we revisit the example of Xie et al. that introduces parallel code into sequential code. To do so, we require an *amb* carrier *AC*.

```

newtype  $AC \text{ } w \text{ } a = AC \{ \text{unAmb} :: [(a, w)] \}$ 

```

Furthermore, we extend the above handler to contain the algebraic part of a writer, which can write to the log using *tell*.

```

 $h'_{\text{Amb}} :: \text{Monoid } w \Rightarrow \text{Free}_H (K^{\text{Alg}} \text{Amb} \oplus (K^{\text{Alg}} (\text{Tell } w) \oplus K^{\text{Par}} [])) a \rightarrow [(a, w)]$ 
 $h'_{\text{Amb}} = \text{unAmb} . \text{fold } \eta (\text{alg}'_{\text{Amb}} \oplus (\text{alg}_{\text{Tell,Amb}} \oplus \text{alg}_{\text{For,Amb}}))$ 
 $\text{alg}'_{\text{Amb}} :: K^{\text{Alg}} \text{Amb} (AC \text{ } w) (AC \text{ } w \text{ } a) \rightarrow AC \text{ } w \text{ } a$ 
 $\text{alg}'_{\text{Amb}} (\text{Op } (\text{Amb } r)) = AC (\text{concat } (\text{fmap } \text{unAmb } r))$ 
 $\text{alg}_{\text{Tell,Amb}} :: \text{Monoid } w \Rightarrow K^{\text{Alg}} (\text{Tell } w) (AC \text{ } w) (AC \text{ } w \text{ } a) \rightarrow AC \text{ } w \text{ } a$ 
 $\text{alg}_{\text{Tell,Amb}} (\text{Op } (\text{Tell } w \text{ } k)) = AC (\text{do } (x, w') \leftarrow \text{unAmb } k;$ 
 $\text{return } (x, w \diamond w'))$ 
 $\text{alg}_{\text{For,Amb}} :: \text{Monoid } w \Rightarrow K^{\text{Par}} [] (AC \text{ } w) (AC \text{ } w \text{ } a) \rightarrow AC \text{ } w \text{ } a$ 
 $\text{alg}_{\text{For,Amb}} (\text{For } \text{iters } k) = AC (\text{aggr } (\text{unAmb} . k)$ 
 $(\text{foldr } (\times) [[]] (\text{fmap } \text{unAmb } \text{iters})))$ 
 $\text{aggr} :: \text{Monoid } w \Rightarrow ([b] \rightarrow [(a, w)]) \rightarrow [[(b, w)]] \rightarrow [(a, w)]$ 
 $\text{aggr } k = \text{concat} . \text{fmap } ((\lambda (bs, w) \rightarrow \text{fmap } (\text{fmap } (\diamond) w) (k \text{ } bs)) . \text{bimap})$ 

```

For example, *digit* models all digits that add up to 13.

```

digit = do
  d1 ← amb (fmap return [0..9])
  d2 ← amb (fmap return [0..9])
  if d1 + d2 ≡ 13 then tell [(d1, d2)] else return ()
>>> concat (fmap snd (h'_{Amb} digit))
[(4, 9), (5, 8), (6, 7), (7, 6), (8, 5), (9, 4)]

```

B.7 Two Ways of Modifying the Log with censor

We can write `censor` in terms of `pass`. The difference between these two operations is that `censor` gets the modifying function separately, whereas in `pass`, it is part of the computation result. Thus, we can define `censor` in terms of `pass`:

```

censor :: (w → w) → m a → m a
censor f k = pass (do x ← k; return (x, f))

```

From this definition, it follows that we can rewrite our `reset` example in terms of `censor`.

```

reset = censor (const ε) (return ())

```

The result remains the same

```

>>> h_{Write} (tell "post" >> reset >> tell "pre")
("post", ())

```

Alternatively, we can think of this function as the dual of `local` for reading from an environment, where `local` first modifies the environment and then executes a computation. Dually, `censor` executes a computation and then modifies the output. Both `local` and `censor` can be modeled as a scoped computation. For example, we can extend the above handler with scoped effect *Censor* *w*.

```

data Censor w a = Censor (w → w) a

```

Furthermore, we extend our handler accordingly:

$$h_{\text{Censor}} :: \text{Monoid } w \Rightarrow \text{FreeH } (K^{\text{Alg}} (\text{Tell } w) \oplus K^{\text{Sc}} (\text{Censor } w) \oplus K^{\text{Write}} (\text{Listen } w) \oplus K^{\text{Write}} (\text{Pass } w)) a$$

$$\rightarrow (w, a)$$

$$h_{\text{Censor}} = \text{fold } \eta \ (alg_{\text{Tell}} \oplus alg_{\text{Censor}} \oplus alg_{\text{Listen}} \oplus alg_{\text{Pass}}) \ \mathbf{where}$$

$$alg_{\text{Censor}} :: K^{\text{Sc}} \ (Censor \ w) \ ((,) \ w) \ (w, a) \rightarrow (w, a)$$

$$alg_{\text{Censor}} \ (\text{Enter} \ (Censor \ f \ (_, (w, x)))) = (f \ w, x)$$

The example computation gives the same result as before.

```
>>> h_Censor (tell "post" >> reset >> tell "pre")
("post", ())
```

B.8 Lambda by Call-by-Push-Value

This Section proposes a different implementation of lambda abstraction and application by means of the call-by-push-value approach of Levy [93]. This implementation is inspired by the implementation and the ideas of Bach Poulsen and van der Rest [7].

We model lambda's, variables and application using *Lam th fun f a*:

data *Lam th fun f a where*

$$\begin{aligned} Lam &:: (th \ b \rightarrow f \ c) \rightarrow (fun \ (th \ b) \ c \rightarrow a) \rightarrow Lam \ th \ fun \ f \ a \\ Var &:: (th \ b) \rightarrow (b \rightarrow a) \rightarrow Lam \ th \ fun \ f \ a \\ App &:: (fun \ (th \ b) \ c) \rightarrow (f \ b) \rightarrow (c \rightarrow a) \rightarrow Lam \ th \ fun \ f \ a \end{aligned}$$

Here, *th* represents the type of thunks, and *fun* represent a function type. For example, *fun* could be of type *Fun m*.

newtype *Fun m b a = Fun { fun :: b → m a }*

Consider an example in which we combine state and the above *Lam* effect.

```
prog = do put 1; f ← abs (λx → return ()); app f (put 3)
```

We implement the call-by-name approach as follows, where the type of thunks is represented by the identity functor *Id*.

$$\begin{aligned}
h_{\text{CBN}} &:: \text{Free}_H (\text{Lam } \text{Id } (\text{Fun } (\text{StateT } s \text{ Id})) \oplus K^{\text{Alg}} (\text{State } s)) \ a \\
&\rightarrow s \rightarrow (a, s) \\
h_{\text{CBN}} &= \text{fmap } \text{unId} . \text{runStateT} . \text{fold } \eta \ (alg_{\text{CBN}} \oplus alg_{\text{State}}^{\oplus}) \\
alg_{\text{CBN}} &:: \text{Monad } m \Rightarrow \text{Lam } \text{Id } (\text{Fun } m) \ m \ (m \ a) \rightarrow m \ a \\
alg_{\text{CBN}} (\text{Lam } \text{body } k) &= k \ (\text{Fun } \text{body}) \\
alg_{\text{CBN}} (\text{Var } x \ k) &= k \ (\text{unId } x) \\
alg_{\text{CBN}} (\text{App } f \ x \ k) &= \text{do } v \leftarrow x; \text{fun } f \ (\text{Id } v) \gg= k
\end{aligned}$$

Similary, we implement a call-by-value strategy, where we replace the type of thunks by that of *StateT s Id*.

$$\begin{aligned}
h_{\text{CBV}} &:: \text{Free}_H (\text{Lam } (\text{StateT } s \text{ Id}) (\text{Fun } (\text{StateT } s \text{ Id})) \oplus K^{\text{Alg}} (\text{State } s)) \ a \\
&\rightarrow s \rightarrow (a, s) \\
h_{\text{CBV}} &= \text{fmap } \text{unId} . \text{runStateT} . \text{fold } \eta \ (alg_{\text{CBV}} \oplus alg_{\text{State}}^{\oplus}) \\
alg_{\text{CBV}} &:: \text{Monad } m \Rightarrow \text{Lam } m \ (\text{Fun } m) \ m \ (m \ a) \rightarrow m \ a \\
alg_{\text{CBV}} (\text{Lam } \text{body } k) &= k \ (\text{Fun } \text{body}) \\
alg_{\text{CBV}} (\text{Var } x \ k) &= x \gg= k \\
alg_{\text{CBV}} (\text{App } f \ x \ k) &= \text{fun } f \ x \gg= k
\end{aligned}$$

When we apply both evaluation strategies to our *prog* example, we get the following:

$ \begin{aligned} &>>> h_{\text{CBN}} \text{ prog } 0 \\ &((), 3) \end{aligned} $	$ \begin{aligned} &>>> h_{\text{CBV}} \text{ prog } 0 \\ &((), 1) \end{aligned} $
--	--

Appendix C

An Effect Library of Latent Effects

This Appendix contains a library of latent effects, defined in terms of $Free_{Lat} \zeta \ell a$, and composed using the coproduct ($\pm$) of ζ . We forward unknown effects. The code can be found at <https://github.com/birthevdb/Latent-Effect-and-Handlers>.

The following two helper functions are useful in the definitions of these constructors.

$$\begin{aligned} noSub &:: \zeta_1 v \ NoSub \rightarrow Free_{Lat} (\zeta_1 \pm \zeta_2) \ Id \ v \\ noSub \ n &= Node \ (Inl' \ n) \ (Id \ ()) \ (\lambda x \rightarrow \mathbf{case} \ x \ \mathbf{of} \) \ (Leaf \ . \ unId) \\ oneSub &:: \zeta_1 v_1 \ (OneSub \ v_2) \rightarrow Free_{Lat} (\zeta_1 \pm \zeta_2) \ Id \ v_2 \\ &\rightarrow Free_{Lat} (\zeta_1 \pm \zeta_2) \ Id \ v_1 \\ oneSub \ n \ t &= Node \ (Inl' \ n) \ (Id \ ()) \ (\lambda One \ _ \rightarrow fmap \ Id \ t) \ (Leaf \ . \ unId) \end{aligned}$$

For each of the effects, we define their signature, handler and constructors for their operations.

C.1 Reading

The *Reading* effect provides two operations: `ask` retrieves the current environment, and `local` passes down an updated environment to a subtree.

```

data Reading r :: * → (* → *) → * where
  Local :: (r → r) → Reading r a (OneSub a)
  Ask   ::           Reading r r NoSub

hRead :: Functor ℓ
  ⇒ r → FreeLat (Reading r ± ζ) ℓ (ℓ a) → FreeLat ζ ℓ (ℓ a)
hRead r (Leaf x) = Leaf x
hRead r (Node (Inl' (Local f)) l st k) =
  hRead (f r) (st One l) ≫= hRead r . k
hRead r (Node (Inl' Ask) l _ k) = hRead r (k (fmap (const r) l))
hRead r (Node (Inr' hop) l st k) =
  Node hop l (λ c2 → hRead r . st c2) (hRead r . k)

local :: (r → r) → FreeLat (Reading r ± ζ) Id a
  → FreeLat (Reading r ± ζ) Id a
local f = oneSub (Local f)
ask   :: FreeLat (Reading r ± ζ) Id r
ask   = noSub Ask

```

C.2 Mutating the State

The *Mutating* effect provides the two standard operations for state: *get* and *put*.

```

data Mutating v :: * → (* → *) → * where
  Get ::           Mutating v v NoSub
  Put :: v → Mutating v () NoSub

hState :: Functor ℓ
  ⇒ s → FreeLat (Mutating s ± ζ) ℓ a
  → FreeLat ζ (Stateℓ s ℓ) (Stateℓ s Id a)
hState s (Leaf x) = Leaf (Stateℓ s, Id x)
hState s (Node (Inl' Get) l _ k) = hState s (k (fmap (λ _ → s) l))
hState _ (Node (Inl' (Put s)) l _ k) = hState s (k l)
hState s (Node (Inr' c) l st k) =
  Node c (Stateℓ s, l)

```

$$\begin{aligned}
& (\lambda c \ (State_\ell \ (s', l')) \rightarrow \text{fmap } trans_{St} \ (h_{State} \ s' \ (st \ c \ l'))) \\
& (\lambda (State_\ell \ (s', lv')) \rightarrow h_{State} \ s' \ (k \ lv'))
\end{aligned}$$

$$\begin{aligned}
trans_{St} &:: State_\ell \ s \ Id \ (\ell \ a) \rightarrow State_\ell \ s \ \ell \ a \\
trans_{St} \ (State_\ell \ (s, x)) &= State_\ell \ (s, unId \ x)
\end{aligned}$$

$$\begin{aligned}
\text{get} &:: Free_{Lat} \ (Mutating \ s \pm \zeta) \ Id \ s \\
\text{get} &= no_{Sub} \ Get \\
\text{put} &:: s \rightarrow Free_{Lat} \ (Mutating \ s \pm \zeta) \ Id \ () \\
\text{put } s &= no_{Sub} \ (Put \ s)
\end{aligned}$$

C.3 Failing

The *Failing* effect has a single operation `err` for constructing an error.

$$\begin{aligned}
\text{data } Failing \ v &:: * \rightarrow (* \rightarrow *) \rightarrow * \text{where} \\
Err &:: v \rightarrow Failing \ v \ Void \ NoSub
\end{aligned}$$

$$\text{newtype } Either_\ell \ \text{left} \ \ell \ a = Either_\ell \ \{ unEitherL :: Either \ \text{left} \ (\ell \ a) \}$$

$$\begin{aligned}
h_{Err} &:: Functor \ \ell \\
&\Rightarrow Free_{Lat} \ (Failing \ v \pm \zeta) \ \ell \ a \\
&\rightarrow Free_{Lat} \ \zeta \ (Either_\ell \ v \ \ell) \ (Either_\ell \ v \ Id \ a) \\
h_{Err} \ (Leaf \ x) &= Leaf \ (Either_\ell \ (Right \ (Id \ x))) \\
h_{Err} \ (Node \ (Inl' \ (Err \ x)) \ _ \ _) &= Leaf \ (Either_\ell \ (Left \ x)) \\
h_{Err} \ (Node \ (Inr' \ c) \ l \ st \ k) &= \\
&\quad Node \ c \ (Either_\ell \ \$ \ Right \ l) \\
&\quad (\lambda z \rightarrow either_\ell \ (Leaf \ . Either_\ell \ . Left) \ (\text{fmap } trans_E \ . h_{Err} \ . st \ z)) \\
&\quad (either_\ell \ (Leaf \ . Either_\ell \ . Left) \ (h_{Err} \ . k))
\end{aligned}$$

$$\begin{aligned}
either_\ell &:: (v \rightarrow c) \rightarrow (\ell \ a \rightarrow c) \rightarrow Either_\ell \ v \ \ell \ a \rightarrow c \\
either_\ell \ f \ - \ (Either_\ell \ (Left \ x)) &= f \ x \\
either_\ell \ - \ g \ (Either_\ell \ (Right \ y)) &= g \ y
\end{aligned}$$

$$\begin{aligned}
trans_E &:: Either_\ell v \text{ Id } (\ell a) \rightarrow Either_\ell v \ell a \\
trans_E (Either_\ell (Left\ x)) &= Either_\ell (Left\ x) \\
trans_E (Either_\ell (Right\ y)) &= Either_\ell (Right\ (unId\ y))
\end{aligned}$$

$$\begin{aligned}
err &:: v \rightarrow Free_{Lat} (Failing\ v \pm \zeta) \text{ Id } a \\
err\ x &= Node\ (Inl'\ (Err\ x))\ (\text{Id } ()) \\
&\quad (\lambda x \rightarrow \text{case } x \text{ of })\ (\lambda x \rightarrow \text{case } x \text{ of })
\end{aligned}$$

C.4 Throwing and Catching Exceptions

The *Throwing* effect provides the two well-known operations for exceptions: *throw* and *catch*.

$$\begin{aligned}
\text{data } Throwing\ x\ v &:: * \rightarrow (* \rightarrow *) \rightarrow * \text{ where} \\
Throw &:: x \rightarrow Throwing\ x\ v \text{ Void NoSub} \\
Catch &:: Throwing\ x\ v \rightarrow (MaybeSub\ x\ v)
\end{aligned}$$

Operation *catch* has two subcomputations, so it uses *MaybeSub* to model this.

$$\begin{aligned}
\text{data } MaybeSub\ x\ v &:: * \rightarrow * \text{ where} \\
JustS &:: x \rightarrow MaybeSub\ x\ v\ v \\
NothingS &:: MaybeSub\ x\ v\ v
\end{aligned}$$

$$\begin{aligned}
h_{Exc} &:: Functor\ \ell \\
&\Rightarrow Free_{Lat} (Throwing\ x\ v \pm \zeta) \ell a \\
&\rightarrow Free_{Lat} \zeta (Either_\ell (\ell\ (), x) \ell) (Either_\ell (\ell\ (), x) \text{ Id } a) \\
h_{Exc} (Leaf\ x) &= Leaf\ (Either_\ell (Right\ (\text{Id } x))) \\
h_{Exc} (Node\ (Inl'\ (Throw\ x))\ l\ _) &= Leaf\ (Either_\ell (Left\ (l, x))) \\
h_{Exc} (Node\ (Inl'\ Catch)\ l\ st\ k) &= h_{Exc} (st\ NothingS\ l) \\
&\gg= either_\ell (\lambda(l', x) \rightarrow h_{Exc} (st\ (JustS\ x)\ l')) \\
&\gg= either_\ell (Leaf\ . Either_\ell . Left)\ (h_{Exc} . k . unId) \\
&\quad (h_{Exc} . k . unId) \\
h_{Exc} (Node\ (Inr'\ c)\ l\ st\ k) &= \\
Node\ c\ (Either_\ell (Right\ l)) & \\
(\lambda z \rightarrow either_\ell (Leaf\ . Either_\ell . Left)\ (fmap\ trans_E . h_{Exc} . st\ z)) & \\
(either_\ell (Leaf\ . Either_\ell . Left)\ (h_{Exc} . k)) &
\end{aligned}$$


```

throw    :: x → FreeLat (Throwing x v ± ζ) Id v
throw e  = Node (Inl' (Throw e)) (Id ())
          (λx _ → case x of ) (λx → case x of )

catch    :: FreeLat (Throwing x v ± ζ) Id v
          → (x → FreeLat (Throwing x v ± ζ) Id v)
          → FreeLat (Throwing x v ± ζ) Id v
catch p h = Node (Inl' Catch) (Id ())
            (λz _ → case z of
              JustS x   → fmap Id (h x)
              NothingS → fmap Id p)
            (Leaf . unId)

```

C.5 Adding Arithmetics

The *Adding* effect provides operations for adding natural numbers: `add` and `lit`.

```

data Adding v :: * → (* → *) → * where
  Lit  :: Integer → Adding v v NoSub
  Plus :: v → v   → Adding v v NoSub

hAdd :: (Functor ℓ, Num v)
       ⇒ FreeLat (Adding v ± ζ) ℓ a → FreeLat ζ ℓ a
hAdd (Leaf x) = Leaf x
hAdd (Node (Inl' (Lit n)) l _ k) =
  hAdd (k (fmap (const (fromInteger n)) l))
hAdd (Node (Inl' (Plus n1 n2)) l _ k) =
  hAdd (k (fmap (const (n1 + n2)) l))
hAdd (Node (Inr' c) l st k) =
  Node c l (λs → hAdd . st s) (hAdd . k)

add      :: v → v → FreeLat (Adding v ± ζ) Id v
add n1 n2 = noSub (Plus n1 n2)

lit      :: Integer → FreeLat (Adding v ± ζ) Id v
lit n    = noSub (Lit n)

```

C.6 Printing

The *Printing* effect has a single operation for printing a string: `print`.

```
data Printing v :: * → (* → *) → * where
  Print :: String → Printing v NoSub
```

```
hPrint :: Functor ℓ
  ⇒ FreeLat (Printing String ± ζ) ℓ a → FreeLat ζ ℓ a
hPrint (Leaf x) = Leaf x
hPrint (Node (Inl' (Print s)) l − k) = hPrint (k (fmap (const s) l))
hPrint (Node (Inr' c) l st k) =
  Node c l (λs → hPrint . st s) (hPrint . k)
```

```
print :: String → FreeLat (Printing v ± ζ) Id v
print s = noSub (Print s)
```

C.7 Delaying

The *Delaying* effect provides two operations: **suspend** delays the evaluation of a recursive subtree, and **enact** forces the evaluation of the delayed subtree without memoizing the result.

```
data Delaying v :: * → (* → *) → * where
  Suspend :: Delaying v Ptr (OneSub v)
  Enact :: Ptr → Delaying v v NoSub
```

```
type Delay ζ ℓ v = ℓ () → FreeLat (Delaying v ± ζ) ℓ (ℓ v)
data Delayed r = Delayed Ptr r deriving (Functor)
```

```
hDelay :: Functor ℓ
  ⇒ [Delay ζ ℓ v] → FreeLat (Delaying v ± ζ) ℓ (ℓ a)
  → FreeLat ζ (Stateℓ [Delay ζ ℓ v] ℓ) (Stateℓ [Delay ζ ℓ v] ℓ a)
hDelay rs (Leaf x) = Leaf (Stateℓ (rs, x))
hDelay rs (Node (Inl' Suspend) l st k) =
  hDelay (rs ++ [st One]) (k (fmap (const (length rs)) l))
hDelay rs (Node (Inl' (Enact p)) l − k) = do
  Stateℓ (rs', lv) ← hDelay rs ((rs !! p) l); hDelay rs' (k lv)
hDelay rs (Node (Inr' op) l st k) =
```

$$\begin{aligned} \text{Node } \mathbf{op} \ (State_\ell \ (rs, l)) \ (\lambda c_2 \ (State_\ell \ (rs', l')) \rightarrow h_{\text{Delay}} \ rs' \ (st \ c_2 \ l')) \\ (\lambda \ (State_\ell \ (rs', lw')) \rightarrow h_{\text{Delay}} \ rs' \ (k \ lw')) \end{aligned}$$

$\text{suspend} :: Free_{\text{Lat}} \ (Delaying \ v \pm \zeta) \ Id \ v$
 $\quad \rightarrow Free_{\text{Lat}} \ (Delaying \ v \pm \zeta) \ Id \ Ptr$
 $\text{suspend} = one_{\text{Sub}} \ Suspend$
 $\text{enact} \quad :: Ptr \rightarrow Free_{\text{Lat}} \ (Delaying \ v \pm \zeta) \ Id \ v$
 $\text{enact } p = no_{\text{Sub}} \ (Enact \ p)$

C.8 Abstracting

The *Abstracting* effect has three operators: *var* for representing a variable, *app* for function application, and *abs* for function abstraction.

data *Abstracting* $v :: * \rightarrow (* \rightarrow *) \rightarrow *$ **where**
 $Var :: Int \rightarrow Abstracting \ v \ v \ NoSub$
 $App :: v \rightarrow v \rightarrow Abstracting \ v \ v \ NoSub$
 $Abs :: Abstracting \ v \ v \ (OneSub \ v)$

data *Closure* v **where**
 $Clos :: Ptr \rightarrow [v] \rightarrow Closure \ v$
class $v_1 <<< v_2$ **where**
 $injV \quad :: v_1 \rightarrow v_2$
 $projV \quad :: v_2 \rightarrow Maybe \ v_1$

For this effect we can define two handlers: one for call-side handling of effects and another for definition-side handling.

type $R_{\text{CS}} \ \zeta \ \ell \ v = \ell \ () \rightarrow Free_{\text{Lat}} \ (Abstracting \ v \pm \zeta) \ \ell \ (\ell \ v)$
 $h_{\text{CS}} :: (Closure \ v <<< v, Functor \ \ell)$
 $\quad \Rightarrow [v] \rightarrow [R_{\text{CS}} \ \zeta \ \ell \ v] \rightarrow Free_{\text{Lat}} \ (Abstracting \ v \pm \zeta) \ \ell \ a$
 $\quad \rightarrow Free_{\text{Lat}} \ \zeta \ (State_\ell \ ([R_{\text{CS}} \ \zeta \ \ell \ v]) \ \ell) \ ([R_{\text{CS}} \ \zeta \ \ell \ v], a)$
 $h_{\text{CS}} _ r \ (Leaf \ x) = Leaf \ (r, x)$
 $h_{\text{CS}} \ nv \ r \ (Node \ (Inl' \ Abs) \ l \ st \ k) = \mathbf{let} \ v = injV \ (Clos \ (\text{length } r) \ nv)$
 $\quad \mathbf{in} \ h_{\text{CS}} \ nv \ (r \mathrel{+} [st \ One]) \ (k \ (\mathbf{fmap} \ (const \ v) \ l))$
 $h_{\text{CS}} \ nv \ r \ (Node \ (Inl' \ (App \ v_1 \ v_2)) \ l \ _ \ k) = \mathbf{case} \ projV \ v_1 \ \mathbf{of}$
 $\quad Just \ (Clos \ fp \ nv') \rightarrow \mathbf{do}$

```

    (r', v) ← hCS (v2 : nv') r ((r !! fp) l)
    hCS nv r' (k v)
    Nothing → error "application error"
hCS nv r (Node (Inl' (Var n)) l _ k) =
    hCS nv r (k (fmap (const (nv !! n)) l))
hCS nv r (Node (Inr' op) l st k) =
    Node op (Stateℓ (r, l))
    (λc (Stateℓ (r', l)) → fmap Stateℓ (hCS nv r' (st c l)))
    (λ(Stateℓ (r', lw)) → hCS nv r' (k lw))

```

```

type RDS ζ ℓ v = FreeLat (Abstracting v ± ζ) ℓ (ℓ v)
hDS :: (Closure v <<< v, Functor ℓ)
    ⇒ [v] → [RDS ζ ℓ v] → FreeLat (Abstracting v ± ζ) ℓ a
    → FreeLat ζ (Stateℓ ([RDS ζ ℓ v]) ℓ) ([RDS ζ ℓ v], a)
hDS _ r (Leaf x) = Leaf (r, x)
hDS nv r (Node (Inl' Abs) l st k) = let v = injV (Clos (length r) nv)
    in hDS nv (r ++ [st One l]) (k (fmap (const v) l))
hDS nv r (Node (Inl' (App v1 v2)) _ k) =
    case projV v1 of
        Just (Clos fp nv') → do
            (r', v) ← hDS (v2 : nv') r (r !! fp)
            hDS nv r' (k v)
        Nothing → error "application error"
hDS nv r (Node (Inl' (Var n)) l _ k) =
    hDS nv r (k (fmap (const (nv !! n)) l))
hDS nv r (Node (Inr' op) l st k) =
    Node op (Stateℓ (r, l))
    (λc (Stateℓ (r', l)) → fmap Stateℓ (hDS nv r' (st c l)))
    (λ(Stateℓ (r', lw)) → hDS nv r' (k lw))

```

```

var      :: Int → FreeLat (Abstracting v ± ζ) Id v
var n     = noSub (Var n)
app      :: v → v → FreeLat (Abstracting v ± ζ) Id v
app v1 v2 = noSub (App v1 v2)
abs      :: FreeLat (Abstracting v ± ζ) Id v
    → FreeLat (Abstracting v ± ζ) Id v
abs      = oneSub Abs

```

Appendix D

λ_{sc} : Full Version

This Appendix presents the full version of λ_{sc} . It mainly consists of Figures that contain the syntax (Section D.1), semantics (Section D.2) and typing (Section D.3) of λ_{sc} . Section D.4 gives a syntax-directed version of the type-and-effect system, which is necessary to formulate and prove some metatheoretical properties of λ_{sc} . Section D.5 shows that this type-and-effect system is type safe by proving subject reduction and progress. Section D.6 presents a type inference algorithm for λ_{sc} and shows that it is sound and complete with respect to the syntax-directed rules.

D.1 Syntax

values v	$::=$	$() \mid (v_1, v_2) \mid x \mid \lambda x. c \mid h$	
handlers h	$::=$	handler $\{ \text{return } x \mapsto c_r$	return clause
		$, cls$	effect clauses
		$, \text{fwd } f \ p \ k \mapsto c_f \}$	forwarding clause
effect clauses cls	$::=$	$\cdot$	
		$\mid \text{op } \ell^{\text{op}} \ x \ k \mapsto c, cls$	algebraic effect clauses
		$\mid \text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, cls$	scoped effect clauses
computations c	$::=$	return v	return value
		$\mid \text{op } \ell^{\text{op}} \ v \ (y. c)$	algebraic effect
		$\mid \text{sc } \ell^{\text{sc}} \ v \ (y. c_1) \ (z. c_2)$	scoped effect
		$\mid v \star c$	handle
		$\mid \text{do } x \leftarrow c_1; c_2$	do-statement
		$\mid v_1 \ v_2$	application
		$\mid \text{let } x = v \text{ in } c$	let-statement
value types A, B, M	$::=$	$() \mid (A, B) \mid A \rightarrow \underline{C} \mid \underline{C} \Rightarrow \underline{D}$	
		$\mid \alpha$	type variable
		$\mid \Lambda \alpha. A$	type operator abstraction
		$\mid M \ A$	type application
type schemes σ	$::=$	$A \mid \forall \mu. \sigma \mid \forall \alpha. \sigma$	
computation types $\underline{C}, \underline{D}$	$::=$	$A! \langle E \rangle$	
effect rows E, F	$::=$	$\cdot \mid \mu \mid \ell; E$	
signature contexts Σ	$::=$	$\cdot \mid \Sigma, \ell^{\text{op}} : A \rightarrow B \mid \Sigma, \ell^{\text{sc}} : A \rightarrow B$	
type contexts Γ	$::=$	$\cdot \mid \Gamma, x : \sigma \mid \Gamma, \mu \mid \Gamma, \alpha$	

Figure D.1: Syntax λ_{sc} .

D.2 Operational Semantics

$$\boxed{c \rightsquigarrow c'}$$

$$\frac{}{(\lambda x. c) \rightsquigarrow c [v / x]} \text{E-APPABS}$$

$$\frac{}{\text{let } x = v \text{ in } c \rightsquigarrow c [v / x]} \text{E-LET}$$

$$\frac{c_1 \rightsquigarrow c'_1}{\text{do } x \leftarrow c_1; c_2 \rightsquigarrow \text{do } x \leftarrow c'_1; c_2} \text{E-DO}$$

$$\frac{}{\text{do } x \leftarrow \text{return } v; c_2 \rightsquigarrow c_2 [v / x]} \text{E-DORET}$$

$$\frac{}{\text{do } x \leftarrow \text{op } \ell^{\text{op}} v (y. c_1); c_2 \rightsquigarrow \text{op } \ell^{\text{op}} v (y. \text{do } x \leftarrow c_1; c_2)} \text{E-DOOP}$$

$$\frac{}{\text{do } x \leftarrow \text{sc } \ell^{\text{sc}} v (y. c_1) (z. c_2); c_3 \rightsquigarrow \text{sc } \ell^{\text{sc}} v (y. c_1) (z. \text{do } x \leftarrow c_2; c_3)} \text{E-DOsc}$$

$$\frac{c \rightsquigarrow c'}{h \star c \rightsquigarrow h \star c'} \text{E-HAND}$$

$$\frac{(\text{return } x \mapsto c_r) \in h}{h \star \text{return } v \rightsquigarrow c_r [v / x]} \text{E-HANDRET}$$

$$\frac{(\text{op } \ell^{\text{op}} x k \mapsto c) \in h}{h \star \text{op } \ell^{\text{op}} v (y. c_1) \rightsquigarrow c [v / x, (\lambda y. h \star c_1) / k]} \text{E-HANDOP}$$

$$\frac{(\text{op } \ell^{\text{op}} - -) \notin h}{h \star \text{op } \ell^{\text{op}} v (y. c_1) \rightsquigarrow \text{op } \ell^{\text{op}} v (y. h \star c_1)} \text{E-FWDOp}$$

$$\frac{(\text{sc } \ell^{\text{sc}} x p k \mapsto c) \in h}{h \star \text{sc } \ell^{\text{sc}} v (y. c_1) (z. c_2) \rightsquigarrow c [v / x, (\lambda y. h \star c_1) / p, (\lambda z. h \star c_2) / k]} \text{E-HANDSc}$$

$$\frac{(\text{sc } \ell^{\text{sc}} - - -) \notin h \quad (\text{fwd } f p k \mapsto c_f) \in h \quad g = \lambda(p', k'). \text{sc } \ell^{\text{sc}} v (y. p' y) (z. k' z)}{h \star \text{sc } \ell^{\text{sc}} v (y. c_1) (z. c_2) \rightsquigarrow c_f [(\lambda y. h \star c_1) / p, (\lambda z. h \star c_2) / k, g / f]} \text{E-FWDSc}$$

Figure D.2: Operational semantics of λ_{sc} .

D.3 Type-and-Effect System

Type- and Row Equivalence

$$\boxed{\sigma_1 \equiv \sigma_2}$$

$$\begin{array}{c}
\frac{}{\sigma \equiv \sigma} \text{Q-REFL} \qquad \frac{\sigma_1 \equiv \sigma_2}{\sigma_2 \equiv \sigma_1} \text{Q-SYMM} \\
\\
\frac{\sigma_1 \equiv \sigma_2 \quad \sigma_2 \equiv \sigma_3}{\sigma_1 \equiv \sigma_3} \text{Q-TRANS} \qquad \frac{A_1 \equiv A_2 \quad B_1 \equiv B_2}{(A_1, B_1) \equiv (A_2, B_2)} \text{Q-PAIR} \\
\\
\frac{A \equiv B \quad \underline{C} \equiv \underline{D}}{A \rightarrow \underline{C} \equiv B \rightarrow \underline{D}} \text{Q-FUN} \qquad \frac{\underline{C}_1 \equiv \underline{D}_1 \quad \underline{C}_2 \equiv \underline{D}_2}{\underline{C}_1 \Rightarrow \underline{C}_2 \equiv \underline{D}_1 \Rightarrow \underline{D}_2} \text{Q-HAND} \\
\\
\frac{\sigma_1 \equiv \sigma_2}{\forall \alpha. \sigma_1 \equiv \forall \alpha. \sigma_2} \text{Q-ALLTY} \qquad \frac{\sigma_1 \equiv \sigma_2}{\forall \mu. \sigma_1 \equiv \forall \mu. \sigma_2} \text{Q-ALLROW} \\
\\
\frac{A \equiv B}{\Lambda \alpha. A \equiv \Lambda \alpha. B} \text{Q-ABS} \qquad \frac{M_1 \equiv M_2 \quad A \equiv B}{M_1 A \equiv M_2 B} \text{Q-APP} \\
\\
\frac{}{(\Lambda \alpha. A) B \equiv A [B / \alpha]} \text{Q-APPABS} \qquad \frac{A \equiv B \quad E \equiv_{\langle} F}{A! \langle E \rangle \equiv B! \langle F \rangle} \text{Q-COMP}
\end{array}$$

Figure D.3: Type equivalence of λ_{sc} .

$$\boxed{E \equiv_{\langle} F}$$

$$\begin{array}{c}
\frac{}{E \equiv_{\langle} E} \text{R-REFL} \qquad \frac{E \equiv_{\langle} F}{F \equiv_{\langle} E} \text{R-SYMM} \qquad \frac{E \equiv_{\langle} F}{\ell; E \equiv_{\langle} \ell; F} \text{R-HEAD} \\
\\
\frac{E_1 \equiv_{\langle} E_2 \quad E_2 \equiv_{\langle} E_3}{E_1 \equiv_{\langle} E_3} \text{R-TRANS} \qquad \frac{\ell_1 \neq \ell_2}{\ell_1; \ell_2; E \equiv_{\langle} \ell_2; \ell_1; E} \text{R-SWAP}
\end{array}$$

Figure D.4: Row equivalence of λ_{sc} .

Well-Scopedness

$$\boxed{\Gamma \vdash \sigma} \quad \boxed{\Gamma \vdash M} \quad \boxed{\Gamma \vdash E} \quad \boxed{\Gamma \vdash \underline{C}}$$

$$\begin{array}{c}
\frac{}{\Gamma \vdash ()} \text{W-UNIT} \quad \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash (A, B)} \text{W-PAIR} \quad \frac{\Gamma, \alpha \vdash A}{\Gamma \vdash \forall \alpha. A} \text{W-ALL} \\
\\
\frac{\Gamma \vdash \underline{C} \quad \Gamma \vdash \underline{D}}{\Gamma \vdash \underline{C} \Rightarrow \underline{D}} \text{W-HAND} \quad \frac{\Gamma \vdash A \quad \Gamma \vdash E}{\Gamma \vdash A! \langle E \rangle} \text{W-COMP} \quad \frac{}{\Gamma \vdash \cdot} \text{W-EMPTY} \\
\\
\frac{\Gamma, \alpha \vdash A}{\Gamma \vdash \Lambda \alpha. A} \text{W-ABS} \quad \frac{\Gamma \vdash M \quad \Gamma \vdash A}{\Gamma \vdash M A} \text{W-APP} \quad \frac{\Gamma \vdash E}{\Gamma \vdash \ell; E} \text{W-EXT} \\
\\
\frac{\Gamma \vdash A \quad \Gamma \vdash \underline{C}}{\Gamma \vdash A \rightarrow \underline{C}} \text{W-FUN} \quad \frac{\alpha \in \Gamma}{\Gamma \vdash \alpha} \text{W-VAR} \quad \frac{\mu \in \Gamma}{\Gamma \vdash \mu} \text{W-ROW}
\end{array}$$

Figure D.5: Well-scopedness rules of λ_{sc} .

Value Typing

$$\boxed{\Gamma \vdash v : \sigma}$$

$$\begin{array}{c}
\frac{(x : \sigma) \in \Gamma}{\Gamma \vdash x : \sigma} \text{T-VAR} \quad \frac{}{\Gamma \vdash () : ()} \text{T-UNIT} \quad \frac{\Gamma \vdash v_1 : A \quad \Gamma \vdash v_2 : B}{\Gamma \vdash (v_1, v_2) : (A, B)} \text{T-PAIR} \\
\\
\frac{\Gamma, x : A \vdash c : \underline{C}}{\Gamma \vdash \lambda x. c : A \rightarrow \underline{C}} \text{T-ABS} \quad \frac{\Gamma \vdash v : A \quad A \equiv B}{\Gamma \vdash v : B} \text{T-EQV} \\
\\
\boxed{\frac{\Gamma \vdash v : \forall \alpha. \sigma \quad \Gamma \vdash A}{\Gamma \vdash v : [A / \alpha] \sigma} \text{T-INST}} \quad \boxed{\frac{\Gamma, \alpha \vdash v : \sigma \quad \alpha \notin \Gamma}{\Gamma \vdash v : \forall \alpha. \sigma} \text{T-GEN}} \\
\\
\boxed{\frac{\Gamma \vdash v : \forall \mu. \sigma \quad \Gamma \vdash E}{\Gamma \vdash v : [E / \mu] \sigma} \text{T-INSTEFF}} \quad \boxed{\frac{\Gamma, \mu \vdash v : \sigma \quad \mu \notin \Gamma}{\Gamma \vdash v : \forall \mu. \sigma} \text{T-GENEFF}}
\end{array}$$

Figure D.6: Value typing.

Computation Typing

$$\boxed{\Gamma \vdash c : \underline{C}}$$

$$\frac{\Gamma \vdash v_1 : A \rightarrow \underline{C} \quad \Gamma \vdash v_2 : A}{\Gamma \vdash v_1 \ v_2 : \underline{C}} \text{ T-APP} \qquad \frac{\Gamma \vdash c : \underline{C} \quad \underline{C} \equiv \underline{D}}{\Gamma \vdash c : \underline{D}} \text{ T-EQC}$$

$$\frac{\Gamma \vdash c_1 : A! \langle E \rangle \quad \Gamma, x : A \vdash c_2 : B! \langle E \rangle}{\Gamma \vdash \mathbf{do} \ x \leftarrow c_1; c_2 : B! \langle E \rangle} \text{ T-DO} \qquad \frac{\Gamma \vdash v : A \quad \Gamma \vdash E}{\Gamma \vdash \mathbf{return} \ v : A! \langle E \rangle} \text{ T-RET}$$

$$\frac{\Gamma \vdash v : \sigma \quad \Gamma, x : \sigma \vdash c : \underline{C}}{\Gamma \vdash \mathbf{let} \ x = v \ \mathbf{in} \ c : \underline{C}} \text{ T-LET} \qquad \frac{\Gamma \vdash v : \underline{C} \Rightarrow \underline{D} \quad \Gamma \vdash c : \underline{C}}{\Gamma \vdash v \star c : \underline{D}} \text{ T-HAND}$$

$$\frac{(\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \Gamma \vdash v : A_{\text{op}} \quad \Gamma, y : B_{\text{op}} \vdash c : A! \langle E \rangle \quad \ell^{\text{op}} \in E}{\Gamma \vdash \mathbf{op} \ \ell^{\text{op}} \ v \ (y . c) : A! \langle E \rangle} \text{ T-OP}$$

$$\frac{(\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \Gamma \vdash v : A_{\text{sc}} \quad \Gamma, y : B_{\text{sc}} \vdash c_1 : B! \langle E \rangle \quad \Gamma, z : B \vdash c_2 : A! \langle E \rangle \quad \ell^{\text{sc}} \in E}{\Gamma \vdash \mathbf{sc} \ \ell^{\text{sc}} \ v \ (y . c_1) \ (z . c_2) : A! \langle E \rangle} \text{ T-SC}$$

Figure D.7: Computation typing.

Handler Typing

$$\boxed{\Gamma \vdash h : \forall \alpha . \alpha! \langle E \rangle \Rightarrow M \ \alpha! \langle F \rangle}$$

$$\text{ T-H } \frac{\langle F \rangle \equiv_{\emptyset} \langle \text{labels}(\text{cls}); E \rangle \quad \Gamma, \alpha \vdash \mathbf{return} \ x \mapsto c_r : M \ \alpha! \langle E \rangle \quad \Gamma, \alpha \vdash \text{cls} : M \ \alpha! \langle E \rangle \quad \Gamma, \alpha \vdash \mathbf{fwd} \ f \ p \ k \mapsto c_f : M \ \alpha! \langle E \rangle}{\Gamma \vdash \mathbf{handler} \ \{ \mathbf{return} \ x \mapsto c_r, \text{cls}, \mathbf{fwd} \ f \ p \ k \mapsto c_f \} : \forall \alpha . \alpha! \langle F \rangle \Rightarrow M \ \alpha! \langle E \rangle}$$

Figure D.8: Handler typing.

$\Gamma \vdash \text{return } x \mapsto c_r : M \ A! \langle E \rangle$	$\Gamma \vdash \text{cls} : M \ A! \langle E \rangle$	$\Gamma \vdash \text{fwd } f \ p \ k \mapsto c_f : M \ A! \langle E \rangle$
---	---	--

$$\begin{array}{c}
\frac{\Gamma, x : A \mapsto c_r : M \ A! \langle E \rangle}{\Gamma \vdash \text{return } x \mapsto c_r : M \ A! \langle E \rangle} \text{T-RETURN} \qquad \frac{}{\Gamma \vdash \cdot : M \ A! \langle E \rangle} \text{T-EMPTY} \\
\\
\frac{\Gamma \vdash \text{cls} : M \ A! \langle E \rangle \quad (\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \ A! \langle E \rangle \vdash c : M \ A! \langle E \rangle}{\Gamma \vdash \text{op } \ell^{\text{op}} \ x \ k \mapsto c, \text{cls} : M \ A! \langle E \rangle} \text{T-OPROP} \\
\\
\frac{\Gamma \vdash \text{cls} : M \ A! \langle E \rangle \quad (\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \ \beta! \langle E \rangle, k : \beta \rightarrow M \ A! \langle E \rangle \vdash c : M \ A! \langle E \rangle}{\Gamma \vdash \text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, \text{cls} : M \ A! \langle E \rangle} \text{T-CLSSC} \\
\\
\frac{\begin{array}{c} A_p = \alpha \rightarrow M \ \beta! \langle E \rangle \quad A'_p = \alpha \rightarrow \gamma! \langle E \rangle \\ A_k = \beta \rightarrow M \ A! \langle E \rangle \quad A'_k = \gamma \rightarrow \delta! \langle E \rangle \end{array} \quad \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow \delta! \langle E \rangle \vdash c_f : M \ A! \langle E \rangle}{\Gamma \vdash \text{fwd } f \ p \ k \mapsto c_f : M \ A! \langle E \rangle} \text{T-FWD}
\end{array}$$

Figure D.9: Typing of handler clauses.

D.4 Syntax-Directed Type-and-Effect System

This section describes the syntax-direction version of λ_{sc}, which we prove type safe in Appendix D.5. Furthermore, it serves as the specification of our type inference algorithm as described in Appendix D.6.

The syntax-directed rules can be found in Figure D.10 for value typing, Figure D.11 for computation and Figure D.12 for handler typing.

$$\boxed{\Gamma \vdash v : A}$$

$$\begin{array}{c}
 \frac{(x : \sigma) \in \Gamma \quad \sigma \leq A \quad \Gamma \vdash A}{\Gamma \vdash x : A} \text{SD-VAR} \qquad \frac{}{\Gamma \vdash () : ()} \text{SD-UNIT} \\
 \\
 \frac{\Gamma \vdash v_1 : A \quad \Gamma \vdash v_2 : B}{\Gamma \vdash (v_1, v_2) : (A, B)} \text{SD-PAIR} \qquad \frac{\Gamma, x : A \vdash c : \underline{C}}{\Gamma \vdash \lambda x. c : A \rightarrow \underline{C}} \text{SD-ABS} \\
 \\
 \frac{\begin{array}{c} F \equiv_{\langle \rangle} \text{labels}(cls); E \quad \alpha \notin \Gamma \\ \Gamma, \alpha \vdash_M \text{return } x \mapsto c_r : M \alpha! \langle E \rangle \quad \Gamma, \alpha \vdash_M cls : M \alpha! \langle E \rangle \\ \Gamma, \alpha \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \alpha! \langle E \rangle \quad \Gamma \vdash A \end{array}}{\Gamma \vdash \text{handler}_M \{ \text{return } x \mapsto c_r, cls, \text{fwd } f \ p \ k \mapsto c_f \} : A! \langle F \rangle \Rightarrow M \ A! \langle E \rangle} \text{SD-H}
 \end{array}$$

Figure D.10: Syntax-directed value typing.

The syntax-directed system is obtained through incorporating the regular typing rules (Appendix D.3) into the syntax-directed typing where needed. In particular, we inline the equivalence rules (T-EQV and T-EQC) into the syntax-directed rules that mention the same type or row twice in their assumptions (e.g., SD-APP, SD-DO). Similarly, we inline the rules T-INST, T-INSTEFF, T-GEN and T-GENEFF for instantiating and generalizing type and row variables. The generalization is incorporated into the rule for let-bindings (T-LET). Instantiation is built into the variable rule (T-VAR) using $\sigma \leq A$ (Figure D.13).

Instantiation is also incorporated into the handler rule: we implicitly instantiate α with an arbitrary type A , which results in a monomorphically typed handler. However, since SD-H retains its polymorphic handler clauses, we can still handle scoped effects by polymorphic recursion.

Below, we display declarative and syntax-directed typing derivations for both inline handler application ($h \star c$) as well as let-bound handlers. In the case of inline handler application, the declarative system derives a polymorphically typed handler, which is instantiated.

$$\boxed{\Gamma \vdash c : \underline{C}}$$

$$\begin{array}{c}
\frac{\Gamma \vdash v_1 : A_1 \rightarrow \underline{C} \quad \Gamma \vdash v_2 : A_2 \quad A_1 \equiv A_2}{\Gamma \vdash v_1 v_2 : \underline{C}} \text{SD-APP} \\
\\
\frac{\Gamma \vdash c_1 : A! \langle E_1 \rangle \quad \Gamma, x : A \vdash c_2 : B! \langle E_2 \rangle \quad E_1 \equiv E_2}{\Gamma \vdash \mathbf{do} \ x \leftarrow c_1 ; c_2 : B! \langle E_2 \rangle} \text{SD-DO} \\
\\
\frac{\Gamma \vdash v : A}{\Gamma \vdash \mathbf{return} \ v : A! \langle E \rangle} \text{SD-RET} \\
\\
\frac{\Gamma, \bar{\alpha}, \bar{\mu} \vdash v : A \quad (\bar{\alpha} \notin \Gamma) \quad (\bar{\mu} \notin \Gamma) \quad \Gamma, x : \overline{\forall \alpha}. \overline{\forall \mu}. A \vdash c : \underline{C}}{\Gamma \vdash \mathbf{let} \ x = v \ \mathbf{in} \ c : \underline{C}} \text{SD-LET} \\
\\
\frac{\Gamma \vdash v : \underline{C}_1 \Rightarrow \underline{D}_1 \quad \Gamma \vdash c : \underline{C}_2 \quad \underline{C}_1 \equiv \underline{C}_2 \quad \underline{D}_1 \equiv \underline{D}_2}{\Gamma \vdash v \star c : \underline{D}_2} \text{SD-HAND} \\
\\
\frac{\Gamma \vdash v : A_1 \quad A_{\text{op}} \equiv A_1 \quad (\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \Gamma, y : B_{\text{op}} \vdash c : A! \langle E \rangle \quad \ell^{\text{op}} \in E}{\Gamma \vdash \mathbf{op} \ \ell^{\text{op}} \ v \ (y . c) : A! \langle E \rangle} \text{SD-OP} \\
\\
\frac{\Gamma \vdash v : A_1 \quad A_{\text{sc}} \equiv A_1 \quad (\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \Gamma, y : B_{\text{sc}} \vdash c_1 : B! \langle E_1 \rangle \quad \Gamma, z : B \vdash c_2 : A! \langle E_2 \rangle \quad E_1 \equiv E_2 \quad \ell^{\text{sc}} \in E_2}{\Gamma \vdash \mathbf{sc} \ \ell^{\text{sc}} \ v \ (y . c_1) \ (z . c_2) : A! \langle E_2 \rangle} \text{SD-SC}
\end{array}$$

Figure D.11: Syntax-directed computation typing.

$$\frac{\frac{\Gamma, \alpha \vdash \text{cls} : M \ \alpha! \langle E \rangle}{\Gamma \vdash h : \forall \alpha . \alpha! \langle F \rangle \Rightarrow M \ \alpha! \langle E \rangle} \text{T-H} \quad \Gamma \vdash A}{\Gamma \vdash h : A! \langle F \rangle \Rightarrow M \ A! \langle E \rangle} \text{T-INST} \quad \Gamma \vdash c : A! \langle F \rangle}{\Gamma \vdash h \star c : M \ A! \langle E \rangle} \text{T-HAND}$$

The syntax-directed system essentially combines these steps.

$$\frac{\frac{\Gamma, \alpha \vdash_M \text{cls} : M \ \alpha! \langle E \rangle \quad \Gamma \vdash A}{\Gamma \vdash (hT \ (M)) : A! \langle F \rangle \Rightarrow M \ A! \langle E \rangle} \text{SD-H} \quad \Gamma \vdash c : A! \langle F \rangle}{\Gamma \vdash (hT \ (M)) \star c : M \ A! \langle E \rangle} \text{SD-HAND}$$

In case of a let-bound handler, the declarative system keeps the polymorphic handler type as-is.

$$\boxed{\Gamma \vdash_M \text{return } x \mapsto c_r : M \ A! \langle E \rangle} \quad \boxed{\Gamma \vdash_M \text{cls} : M \ A! \langle E \rangle} \quad \boxed{\Gamma \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \ A! \langle E \rangle}$$

$$\frac{\Gamma, x : A_1 \vdash c_r : M \ A_2! \langle E \rangle \quad A_1 \equiv A_2}{\Gamma \vdash_M \text{return } x \mapsto c_r : M \ A! \langle E_2 \rangle} \text{SD-RETURN}$$

$$\frac{}{\Gamma \vdash_M \cdot : M \ A! \langle E \rangle} \text{SD-EMPTY}$$

$$\frac{\Gamma \vdash_M \text{cls} : M \ A_1! \langle E_1 \rangle \quad (\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \ A_1! \langle E_1 \rangle \vdash c : M \ A_2! \langle E_2 \rangle \quad M \ A_1! \langle E_1 \rangle \equiv M \ A_2! \langle E_2 \rangle}{\Gamma \vdash_M \text{op } \ell^{\text{op}} \ x \ k \mapsto c, \text{cls} : M \ A_2! \langle E_2 \rangle} \text{SD-OPROP}$$

$$\frac{\Gamma \vdash_M \text{cls} : M \ A_1! \langle E_1 \rangle \quad (\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \beta \notin \Gamma \quad \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \ \beta! \langle E_1 \rangle, k : \beta \rightarrow M \ A_1! \langle E_1 \rangle \vdash c : M \ A_2! \langle E_2 \rangle \quad M \ A_1! \langle E_1 \rangle \equiv M \ A_2! \langle E_2 \rangle}{\Gamma \vdash_M \text{sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, \text{cls} : M \ A_2! \langle E_2 \rangle} \text{SD-OPRSC}$$

$$\frac{\alpha, \beta, \gamma, \delta \notin \Gamma \quad A_p = \alpha \rightarrow M \ \beta! \langle E_1 \rangle \quad A'_p = \alpha \rightarrow \gamma! \langle E_1 \rangle \quad A_k = \beta \rightarrow M \ A_1! \langle E_1 \rangle \quad A'_k = \gamma \rightarrow \delta! \langle E_1 \rangle \quad \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow \delta! \langle E_1 \rangle \vdash c_f : M \ A_2! \langle E_2 \rangle \quad M \ A_1! \langle E_1 \rangle \equiv M \ A_2! \langle E_2 \rangle \quad \Gamma \vdash A_2}{\Gamma \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \ A_2! \langle E_2 \rangle} \text{SD-FWD}$$

Figure D.12: Syntax-directed handler typing.

$$\boxed{\sigma \leq A}$$

$$\frac{}{A \leq A} \sigma\text{-INST-BASE} \quad \frac{[B / \alpha] \sigma \leq A}{\forall \alpha. \sigma \leq A} \sigma\text{-INST-}\alpha \quad \frac{[E / \mu] \sigma \leq A}{\forall \mu. \sigma \leq A} \sigma\text{-INST-}\mu$$

Figure D.13: σ -instantiation.

$$\frac{\Gamma, \alpha \vdash \text{cls} : M \ \alpha! \langle E \rangle}{\Gamma \vdash h : \forall \alpha. \alpha! \langle F \rangle \Rightarrow M \ \alpha! \langle E \rangle} \text{T-H} \quad \frac{\Gamma, x : \forall \alpha. \alpha! \langle F \rangle \Rightarrow M \ \alpha! \langle E \rangle \vdash c : \underline{C}}{\Gamma \vdash \text{let } x = h \text{ in } c : \underline{C}} \text{T-LET}$$

The syntax-directed system, however, instantiates and then immediately generalizes handlers.

$$\frac{\frac{\Gamma, \alpha, \beta \vdash_M \text{cls} : M \beta! \langle E \rangle}{\Gamma, \alpha \vdash \alpha} \quad \Gamma \vdash (hT(M)) : \alpha! \langle F \rangle \Rightarrow M \alpha! \langle E \rangle}{\Gamma, x : \forall \alpha. \alpha! \langle F \rangle \Rightarrow M \alpha! \langle E \rangle \vdash c : \underline{C}} \text{SD-H} \quad \text{SD-LET}$$

$$\frac{}{\Gamma \vdash \text{let } x = (hT(M)) \text{ in } c : \underline{C}}$$

The other rules of the declarative system are syntax-directed and remain unchanged.

D.5 Metatheory and Proofs

Lemmas

Lemma 2 (Canonical forms). *If $\cdot \vdash v : A \rightarrow \underline{C}$ then v is of shape $\lambda x . c$. If $\cdot \vdash v : \underline{C} \Rightarrow \underline{D}$ then v is of shape h .*

Lemma 3 (Generalization-equivalence). *If $\sigma_1 \leq A_1$ and $\sigma_1 \equiv \sigma_2$, then there exists an A_2 such that $A_1 \equiv A_2$ and $\sigma_2 \leq A_2$.*

Lemma 4 (Generalization-instantiation). *If $\Gamma, \bar{\alpha}, \bar{\mu} \vdash v : A$ and $\forall \bar{\alpha} \forall \bar{\mu} . A \leq B$, then $\Gamma \vdash v : B$.*

Lemma 5 (Unused binding insertion). *If $\Gamma_1, \Gamma_2 \vdash c : \underline{C}$ and $x \notin c$ then $\Gamma_1, x : A, \Gamma_2 \vdash c : \underline{C}$.*

Lemma 6 (Handlers are polymorphic). *If $\Gamma \vdash h : A! \langle F \rangle \Rightarrow M \ A! \langle E \rangle$ and $\Gamma \vdash B$, then $\Gamma \vdash h : B! \langle F \rangle \Rightarrow M \ B! \langle E \rangle$.*

Lemma 7 (Op membership). *If $\Gamma \vdash cls : \underline{C}$ and $\text{op } \ell^{\text{op}} x k \mapsto c \in cls$, then there exists cls_1 and cls_2 such that $cls = cls_1, \text{op } \ell^{\text{op}} k \mapsto c, cls_2$ and $\Gamma \vdash \text{op } \ell^{\text{op}} x k \mapsto c, cls_2 : \underline{C}$.*

Lemma 8 (Sc membership). *If $\Gamma \vdash cls : \underline{C}$ and $(\text{sc } \ell^{\text{sc}} x p k \mapsto c) \in h$, then there exists cls_1 and cls_2 such that $cls = cls_1, \text{sc } \ell^{\text{sc}} x p k \mapsto c, cls_2$ and $\Gamma \vdash \text{sc } \ell^{\text{sc}} x p k \mapsto c, cls_2 : \underline{C}$.*

Lemma 9 (Preservation of types under type substitution). *If $\Gamma_1, \alpha, \Gamma_2 \vdash c : \underline{C}$ and $\Gamma_1 \vdash B$, then $\Gamma_1, [B / \alpha] \Gamma_2 \vdash c : [B / \alpha] \underline{C}$.*

Lemma 10 (Preservation of types under term substitution). *Given $\Gamma_1, \bar{\alpha}, \bar{\mu} \vdash v : A_1$ and $A_1 \equiv A_2$ we have that:*

- *If $\Gamma_1, x : \forall \bar{\alpha} . \forall \bar{\mu} . A_2, \Gamma_2 \vdash c : \underline{C}_1$, then there exists a $\underline{C}_2$ such that $\underline{C}_1 \equiv \underline{C}_2$ and $\Gamma_1, \Gamma_2 \vdash [v / x] c : \underline{C}_2$.*
- *If $\Gamma_1, x : \forall \bar{\alpha} . \forall \bar{\mu} . A_2, \Gamma_2 \vdash v : B_1$, then there exists a B_2 such that $B_1 \equiv B_2$ and $\Gamma_1, \Gamma_2 \vdash [v / x] v : B_2$.*

Proof. By mutual induction on the typing derivations. The only interesting case, SD-VAR, requires us to show that, given $\Gamma_1, x : \forall \bar{\alpha}. \forall \bar{\mu}. A_2, \Gamma_2 \vdash y : B_1$, there exists a B_2 such that $B_1 \equiv B_2$ and $\Gamma_1, \Gamma_2 \vdash [v / x] y : B_2$. If $x \neq y$, it is trivial. If $x = y$, then $\forall \bar{\alpha}. \forall \bar{\mu}. A_2 \leq B_1$, which means by Lemma 3 there exists a B_2 such that $B_1 \equiv B_2$ and $\forall \bar{\alpha}. \forall \bar{\mu}. A_1 \leq B_2$, which means the result follows from Lemma 4. $\square$

Subject reduction

Theorem 5 (Subject Reduction). *If $\Gamma \vdash c : \underline{C}$ and $c \rightsquigarrow c'$, then there exists a $\underline{C}'$ such that $\underline{C} \equiv \underline{C}'$ and $\Gamma \vdash c' : \underline{C}'$.*

Proof. Assume, without loss of generality, that $\underline{C} = B! \langle F \rangle$ for some B, F . Proceed by induction on the derivation $c \rightsquigarrow c'$.

- E-APPABS: Inversion on $\Gamma \vdash (\lambda x. c) v : B! \langle F \rangle$ (SD-APP) gives $\Gamma \vdash \lambda x. c : A_1 \rightarrow B! \langle F \rangle$ (1), $\Gamma \vdash v : A_2$ (2), and $A_1 \equiv A_2$ (3). Inversion on fact 1 (SD-ABS) gives $\Gamma, x : A_1 \vdash c : B! \langle F \rangle$ (4), which means the goal follows from facts 2 and 4 and Lemma 10.
- E-LET: Inversion on $\Gamma \vdash \text{let } x = v \text{ in } c : B! \langle F \rangle$ (SD-LET) gives $\Gamma \vdash v : A$ (1), $\sigma = \text{gen}(A, \Gamma)$ (2), and $\Gamma, x : \sigma \vdash c : B! \langle F \rangle$ (3), which means the goal follows from facts 1 and 3 and Lemma 10.
- E-DO: Follows from the IH.
- E-DORET: Inversion on $\Gamma \vdash \text{do } x \leftarrow \text{return } v \text{ in } c : B! \langle F_2 \rangle$ (SD-DO) gives $\Gamma \vdash \text{return } v : A! \langle F_1 \rangle$ (1) and $\Gamma, x : A \vdash c : B! \langle F_2 \rangle$ (2). Inversion on (1) (SD-RET) gives $\Gamma \vdash v : A$ (3). The case follows from facts 2 and 4 and Lemma 10.
- E-DOOP: Similar to E-DOSc. By inversion on $\Gamma \vdash \text{do } x \leftarrow \text{op } \ell^{\text{op}} v(y. c_1) \text{ in } c_2 : B! \langle F_2 \rangle$ (SD-DO) we have that $\Gamma \vdash \text{op } \ell^{\text{op}} v(y. c_1) : A! \langle F_1 \rangle$ (1), $\Gamma, x : A \vdash c_2 : B! \langle F_2 \rangle$ (2), and $F_1 \equiv F_2$ (3). From inversion on fact 1 (SD-OP) it follows that $\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}} \in \Sigma$ (4), $\Gamma \vdash v : A_1$ (5), $A_{\text{op}} \equiv A_1$ (6), $\Gamma, y : B_{\text{op}} \vdash c_1 : A! \langle F_1 \rangle$ (7), and $\ell^{\text{op}} \in F_1$ (8). Lemma 5 on (2) gives us $\Gamma, y : B_{\text{op}}, x : A \vdash c_2 : B! \langle F_2 \rangle$ (9). Facts 3, 7 and 9 and rule SD-DO give us $\Gamma, y : B_{\text{op}} \vdash \text{do } x \leftarrow c_1 \text{ in } c_2 : B! \langle F_2 \rangle$ (10). Our goal then follows from facts 4, 5, 6, 8, and 10 and rule SD-OP.

- **E-DoSc**: Similar to **E-DoOp**. By inversion on $\Gamma \vdash \mathbf{do} \ x \leftarrow \mathbf{sc} \ \ell^{\mathbf{sc}} \ v \ (y. c_1) \ (z. c_2) \ \mathbf{in} \ c_3 : B! \langle F_3 \rangle$ (**SD-Do**) we have that $\Gamma \vdash \mathbf{sc} \ \ell^{\mathbf{sc}} \ v \ (y. c_1) \ (z. c_2) : A! \langle F_2 \rangle$ (1), $\Gamma, x : A \vdash c_3 : B! \langle F_3 \rangle$ (2), and $F_2 \equiv F_3$ (2.1). From inversion on fact 1 (**SD-Sc**) it follows that $\ell^{\mathbf{sc}} : A_{\mathbf{sc}} \rightarrow B_{\mathbf{sc}} \in \Sigma$ (3), $\Gamma \vdash v : A_1$ (4), $A_{\mathbf{sc}} \equiv A_1$ (5), $\Gamma, y : B_{\mathbf{sc}} \vdash c_1 : B'! \langle F_1 \rangle$ (6), $\Gamma, z : B' \vdash c_2 : A! \langle F_2 \rangle$ (7), $F_1 \equiv F_2$ (8), and $\ell^{\mathbf{sc}} \in F_2$ (9). Lemma 5 on (2) gives us $\Gamma, z : B', x : A \vdash c_3 : B! \langle F_3 \rangle$ (10), which means facts 2.1, 7 and 10 and rule **SD-Do** give us $\Gamma, z : B' \vdash \mathbf{do} \ x \leftarrow c_2 \ \mathbf{in} \ c_3 : B! \langle F_3 \rangle$ (11). Our goal then follows from facts 3, 4, 5, 6, 8, 9, and 11 and rule **SD-Sc**.
- **E-HAND**: Follows from the **IH**.
- **E-HANDRET**: By inversion on $\Gamma \vdash h \star \mathbf{return} \ v : B! \langle F_2 \rangle$ (**SD-HAND**) we have that $\Gamma \vdash h : \underline{C}_1 \Rightarrow B! \langle F_2 \rangle$ (1), $\Gamma \vdash \mathbf{return} \ v : \underline{C}_2$ (2), and $\underline{C}_1 \equiv \underline{C}_2$ (3). Inversion on fact 1 (**SD-H**) gives $B = M \ A_2$, $\underline{C}_1 = A_2! \langle E \rangle$, and $\Gamma, \alpha \vdash_M \mathbf{return} \ x \mapsto c_r : M \ \alpha! \langle F_2 \rangle$ (4). Based on fact (3) we get that $\underline{C}_2 = A'_2! \langle E' \rangle$, $A_2 \equiv A'_2$ (4), and $E \equiv E'$ (5). Inversion on fact 4 (**SD-RETURN**) gives $\Gamma, \alpha, x : A_1 \vdash c_r : M \ \alpha! \langle F_2 \rangle$ (5) and $A_1 \equiv A_2$ (6). Inversion on fact 2 (**SD-RET**) gives $\Gamma \vdash v : A'_2$ (7). From facts 4-8 and Lemma 10, we get that $\Gamma, \alpha \vdash [v\lambda x] \ c_r : M \ \alpha! \langle F_2 \rangle$ (8). We obtain our goal from fact 8 and Lemma 9.
- **E-HANDOP**: By inversion on $\Gamma \vdash h \star \mathbf{op} \ \ell^{\mathbf{op}} \ v \ (y. c_1) : B! \langle F_2 \rangle$ (**SD-HAND**) we have that $\Gamma \vdash h : \underline{C}_1 \Rightarrow B! \langle F_2 \rangle$ (1), $\Gamma \vdash \mathbf{op} \ \ell^{\mathbf{op}} \ v \ (y. c_1) : \underline{C}_2$ (2), and $\underline{C}_1 \equiv \underline{C}_2$ (3). Inversion on fact 1 (**SD-H**) gives $B = M \ A_2$, $\underline{C}_1 = A_2! \langle E \rangle$, and $\Gamma, \alpha \vdash_M \mathbf{cls} : M \ \alpha! \langle F_2 \rangle$ (4). Based on fact (3) we get that $\underline{C}_2 = A'_2! \langle E' \rangle$, $A_2 \equiv A'_2$ (4), and $E \equiv E'$ (5). Inversion on fact 2 (**SD-OP**) gives us $\ell^{\mathbf{op}} : A_{\mathbf{op}} \rightarrow B_{\mathbf{op}} \in \Sigma$ (6), $\Gamma \vdash v : A_1$ (7), $A_{\mathbf{op}} \equiv A_1$ (8), $\Gamma, y : B_{\mathbf{op}} \vdash c_1 : A'_2! \langle E' \rangle$ (9), and $\ell^{\mathbf{op}} \in E'$ (10). By Lemma 7 we get that $\Gamma, \alpha \vdash_M \mathbf{op} \ \ell^{\mathbf{op}} \ x \ k \mapsto c, \mathit{oprs2} : M \ \alpha! \langle F_2 \rangle$ (11). Inversion on fact 11 (**SD-OPROP**) gives that $\Gamma, \alpha \vdash_M \mathbf{cls} : M \ \alpha! \langle F_1 \rangle$ (12), $(\ell^{\mathbf{op}} : A_{\mathbf{op}} \rightarrow B_{\mathbf{op}}) \in \Sigma$ (13), $\Gamma, \alpha, x : A_{\mathbf{op}}, k : B_{\mathbf{op}} \rightarrow M \ \alpha! \langle F_1 \rangle \vdash c : M \ \alpha! \langle F_2 \rangle$ (14), and $F_1 \equiv F_2$ (15). Facts 1, 4 and 9 in combination with constructors **SD-ABS** and **ST-HAND** gives us that $\Gamma \vdash \lambda y. h \star c_1 : B_{\mathbf{op}} \rightarrow M \ A_2! \langle F_2 \rangle$ (16). The goal follows from facts 7, 8, 14 and 16 and Lemma 10 and 9.
- **E-FWDOP**: By inversion on $\Gamma \vdash h \star \mathbf{op} \ \ell^{\mathbf{op}} \ v \ (y. c_1) : B! \langle F_2 \rangle$ (**SD-HAND**) we have that $\Gamma \vdash h : \underline{C}_1 \Rightarrow B! \langle F_2 \rangle$ (1), $\Gamma \vdash \mathbf{op} \ \ell^{\mathbf{op}} \ v \ (y. c_1) : \underline{C}_2$ (2), and $\underline{C}_1 \equiv \underline{C}_2$ (3). Inversion on fact 1 (**SD-H**) gives $B = M \ A_2$, and $\underline{C}_1 = A_2! \langle E \rangle$. Based on fact (3) we get that $\underline{C}_2 = A'_2! \langle E' \rangle$, $A_2 \equiv A'_2$ (4), and $E \equiv E'$ (5). Inversion on fact 2 (**SD-OP**) gives us $\ell^{\mathbf{op}} : A_{\mathbf{op}} \rightarrow B_{\mathbf{op}} \in \Sigma$ (6), $\Gamma \vdash v : A_1$ (7), $A_{\mathbf{op}} \equiv A_1$ (8), $\Gamma, y : B_{\mathbf{op}} \vdash c_1 : A'_2! \langle E' \rangle$ (9), and $\ell^{\mathbf{op}} \in E'$ (10). The goal follows from facts 1, 4, 5, 6, 7, 8, 10, constructors **SD-HAND** and **SD-OP**, and Lemma 5.

- **E-HANDSC**: By inversion on $\Gamma \vdash h \star \mathbf{op} \ell^{\mathbf{op}} v (y . c_1) : B! \langle F_2 \rangle$ (SD-HAND) we have that $\Gamma \vdash h : \underline{C}_1 \Rightarrow B! \langle F_2 \rangle$ (1), $\Gamma \vdash \ell^{\mathbf{sc}} v (y . c_1) (z . c_2) : \underline{C}_2$ (2), and $\underline{C}_1 \equiv \underline{C}_2$ (3). Inversion on fact 1 (SD-H) gives $B = M A_2$, $\underline{C}_1 = A_2! \langle E_1 \rangle$, and $\Gamma, \alpha \vdash_M \text{cls} : M \alpha! \langle F_2 \rangle$ (4). Based on fact (3) we get that $\underline{C}_2 = A_2'! \langle E_2 \rangle$, $A_2 \equiv A_2'$ (5), and $E_1 \equiv E_2$ (6). Inversion on fact 2 (SD-SC) gives us $\ell^{\mathbf{sc}} : A_{\mathbf{sc}} \rightarrow B_{\mathbf{sc}} \in \Sigma$ (7), $\Gamma \vdash v : A_1$ (8), $A_{\mathbf{sc}} \equiv A_1$ (9), $\Gamma, y : B_{\mathbf{sc}} \vdash c_1 : A_3! \langle E_3 \rangle$ (10), $\Gamma, z : A_3 \vdash c_2 : A_2'! \langle E_2 \rangle$ (11), $E_3 \equiv E_2$ (12), and $\ell^{\mathbf{sc}} \in E_2$ (13). Lemma 7 we get that $\Gamma, \alpha \vdash_M \mathbf{op} \ell^{\mathbf{op}} x k \mapsto c, \text{cls}_2 : M \alpha! \langle F_2 \rangle$ (13.1). Inversion on fact 13.1 (SD-OPRSC) gives $\ell^{\mathbf{sc}} \in \Sigma$ (14), $\beta \text{ fresh}$ (15), $\Gamma, \alpha, \beta, x : A_{\mathbf{sc}}, p : B_{\mathbf{sc}} \rightarrow M \beta! \langle F_3 \rangle, k : \beta \rightarrow M \alpha! \langle F_3 \rangle \vdash c : M \alpha! \langle F_2 \rangle$ (16), and $F_2 \equiv F_3$ (17). Facts 1, 5, 6, 10 and 12, constructors SD-ABS and SD-HAND and Lemma 5 and 6 give us that $\Gamma, \beta \vdash \lambda y . h \star c_1 : p : B_{\mathbf{sc}} \rightarrow M \beta! \langle F_2 \rangle$ (18). Facts 1, 5, 6 and 11 and constructors SD-ABS and SD-HAND and Lemma 5 give us that $\Gamma, \beta \vdash \lambda z . h \star c_2 : \beta \rightarrow M A_2! \langle F_2 \rangle$ (19). The goal now follows from facts 8, 9, 16, 17 and 18 and Lemma 10 and 10.
- **E-FWDSC** By inversion on $\Gamma \vdash h \star \mathbf{op} \ell^{\mathbf{op}} v (y . c_1) : B! \langle F_2 \rangle$ (SD-HAND) we have that $\Gamma \vdash h : \underline{C}_1 \Rightarrow B! \langle F_2 \rangle$ (1), $\Gamma \vdash \ell^{\mathbf{sc}} v (y . c_1) (z . c_2) : \underline{C}_2$ (2), and $\underline{C}_1 \equiv \underline{C}_2$ (3). Inversion on fact 1 (SD-H) gives $B = M A_2$, $\underline{C}_1 = A_2! \langle E_1 \rangle$, and $\Gamma, \alpha \vdash_M \text{fwd } f p k \mapsto c_f : M \alpha! \langle F_2 \rangle$ (4). Based on fact (3) we get that $\underline{C}_2 = A_2'! \langle E_2 \rangle$, $A_2 \equiv A_2'$ (5), and $E_1 \equiv E_2$ (6). Inversion on fact 2 (SD-SC) gives us $\ell^{\mathbf{sc}} : A_{\mathbf{sc}} \rightarrow B_{\mathbf{sc}} \in \Sigma$ (7), $\Gamma \vdash v : A_1$ (8), $A_{\mathbf{sc}} \equiv A_1$ (9), $\Gamma, y : B_{\mathbf{sc}} \vdash c_1 : A_3! \langle E_3 \rangle$ (10), $\Gamma, z : A_3 \vdash c_2 : A_2'! \langle E_2 \rangle$ (11), $E_3 \equiv E_2$ (12), and $\ell^{\mathbf{sc}} \in E_2$ (13). Inversion on fact 4 (SD-FWD) gives $A_p = \alpha' \rightarrow M \beta! \langle F_1 \rangle$, $A'_p = \alpha' \rightarrow \gamma! \langle F_1 \rangle$, $A_k = \beta \rightarrow M A_4! \langle F_1 \rangle$, $A'_k = \gamma \rightarrow \delta! \langle F_1 \rangle$, $\Gamma, \alpha, \alpha', \beta, p : A_p, k : A_k, f : \forall \gamma \delta . (A'_p, A'_k) \rightarrow \delta! \langle F_1 \rangle \vdash c_f : M \alpha! \langle F_2 \rangle$ (14) and $M A_1! \langle F_1 \rangle \equiv M \alpha! \langle F_2 \rangle$ (15). Facts 1, 6, 10 and 12, constructors SD-ABS and SD-HAND and Lemma 5 and 6 give us that $\Gamma, \alpha, y : B_{\mathbf{sc}} \vdash h \star c_1 : M A_3! \langle F_2 \rangle$ (16) Facts 1, 6, 10 and 12, constructors SD-ABS and SD-HAND and Lemma 5 give us that $\Gamma, \alpha \vdash \lambda z . h \star c_2 : A_3 \rightarrow M A_4! \langle F_2 \rangle$ (17) Facts 7, 8, 9, 13, the fact that $\ell^{\mathbf{sc}} \notin \text{labels}(\text{oprs})$, constructors SD-ABS, SD-APP and SD-VAR and Lemma 5 give us that $\Gamma, \alpha, (p', k') : \forall \gamma \delta . (B_{\mathbf{sc}} \rightarrow \gamma! \langle F_1 \rangle, \gamma \rightarrow \delta! \langle F_1 \rangle) \vdash \mathbf{sc} \ell^{\mathbf{sc}} v (y . p' y) (z . k' z) : \delta! \langle F_1 \rangle$ (18). Our goal then follows from facts 14, 15, 16, 17, 18 and Lemma 10.

□

Progress

Theorem 6 (Progress). *If $\cdot \vdash c : \underline{C}$, then either:*

- *there exists a computation c' such that $c \rightsquigarrow c'$, or*
- *c is in normal form, which means it is in one of the following forms:
 $c = \text{return } v$, $c = \text{op } \ell^{\text{op}} v (y . c')$, or $c = \text{sc } \ell^{\text{sc}} v (y . c_1) (z . c_2)$.*

Proof. By induction on the typing derivation $\cdot \vdash c : \underline{C}$.

- SD-APP: Here, $\cdot \vdash v_1 v_2$. Since v_1 has type $A \rightarrow B! \langle F \rangle$, by Lemma 2 it rowst be of shape $\lambda x . c$, which means we can step by rule E-APPABS.
- SD-DO: Here, $\cdot \vdash \text{do } x \leftarrow c_1 \text{ in } c_2 : \underline{C}$. By the induction hypothesis, c_1 can either step (in which case we can step by E-DO), or it is a computation result. Every possible form has a corresponding reduction: if $c_1 = \text{return } v$ we can step by E-DORET, if $c_1 = \text{op } \ell^{\text{op}} v (y . c)$ we can step by E-DOOP, and if $\text{sc } \ell^{\text{sc}} v (y . c'_1) (z . c'_2)$ we can step by E-DOSC.
- SD-LET: Here, $\cdot \vdash \text{let } x = v \text{ in } c : \underline{C}$, which means we can step by E-LET.
- SD-RET, SD-OP, and SD-SC: all of these are computation results (forms 1, 2, and 3, resp.).
- SD-HAND: Here $\cdot \vdash v \star c : M A! \langle F \rangle$. By Lemma 2, v is of shape h . By the induction hypothesis, c can either step (in which case we can step by E-HAND), or it is in a normal form. Proceed by case split on the three forms.
 1. Case $c = \text{return } v$. Since $\cdot \vdash h : \underline{C} \Rightarrow \underline{D}$, there rowst be some $(\text{return } x \mapsto c_r) \in h$ which means we can step by rule E-HANDRET.
 2. Case $c = \text{op } \ell^{\text{op}} v (y . c')$. Depending on $(\text{op } \ell^{\text{op}} x k \mapsto c) \in h$ we can step by E-HANDOP or E-FWDOP.
 3. Case $c = \text{sc } \ell^{\text{sc}} v (y . c_1) (z . c_2)$. If $(\text{sc } \ell^{\text{sc}} x p k \mapsto c) \in h$, we can step by E-HANDSC. If not, since $\cdot \vdash h : \underline{C} \Rightarrow \underline{D}$, there rowst be some $(\text{fwd } f p k \mapsto c_f) \in h$ which means we can step by rule E-FWDSC.

□

D.6 Type Inference

Algorithmic Syntax

For type inference, we follow the approach of Koka [91]. The syntax for types in our algorithmic system can be found in Figure D.14. We add unification variables $\hat{\alpha}$ for types and $\hat{\mu}$ for rows to our syntax. The hat on types $\hat{A}, \hat{C}$, rows $\hat{E}$ and contexts $\hat{\Gamma}$ indicates that they may contain unification variables. Furthermore, we add unification worklists U to represent the collection of types or rows that need to be unified.

value types $\hat{A}, \hat{B}$	$::=$	$() \mid (\hat{A}, \hat{B}) \mid \hat{A} \rightarrow \hat{C} \mid \hat{C} \Rightarrow \hat{D}$
		$\mid \alpha \mid \hat{\alpha} \mid \Lambda \alpha. \hat{A} \mid M \hat{A}$
type schemes $\hat{\sigma}$	$::=$	$\hat{A} \mid \forall \mu. \hat{\sigma} \mid \forall \alpha. \hat{\sigma} \mid \forall \hat{\mu}. \hat{\sigma} \mid \forall \hat{\alpha}. \hat{\sigma}$
computation types $\hat{C}, \hat{D}$	$::=$	$\hat{A}! \langle \hat{E} \rangle$
effect rows $\hat{E}$	$::=$	$\cdot \mid \mu \mid \hat{\mu} \mid \ell; \hat{E}$
signature contexts Σ	$::=$	$\cdot \mid \Sigma, \ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}} \mid \ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}$
type contexts $\hat{\Gamma}$	$::=$	$\cdot \mid \hat{\Gamma}, x : \hat{A} \mid \hat{\Gamma}, \mu \mid \hat{\Gamma}, \alpha$
unification worklists U	$::=$	$\cdot \mid \hat{A} \sim \hat{B}, U \mid \hat{E} \sim \hat{F}, U \mid \hat{C} \sim \hat{D}, U$

Figure D.14: Types for algorithmic system.

Type equivalence for algorithmic types is a trivial extension of type equivalence for the declarative system (Figure D.3). Well-scopedness is also a trivial extension of Figure D.5. We do not record unification variables in the contexts, so they are not checked in the well-scopedness rules. Finally, we do not allow free type variables in the annotation of type operators M .

Algorithmic Rules

Our type inference algorithm consists of algorithmic typing rules that output a type $\hat{A}$ and a substitution θ . The judgment $\hat{\Gamma} \vdash v : \hat{A} \dashv \theta$ states that under $\hat{\Gamma}$, a value v has algorithmic type $\hat{A}$, giving a substitution θ . This algorithm is based on Hindley-Milner's algorithm W [104], assigning types or unification variables to each (sub)term and generating substitutions by solving unification constraints, originating from algorithmic rules.

The algorithmic type inference rules can be found in Figure D.15 for values, Figure D.16 for computations, and Figure D.17 for handlers. In these rules, we use unification $\hat{A} \sim \hat{B} : \theta$, which states that two types $\hat{A}$ and $\hat{B}$ can be unified, giving rise to a substitution θ . The same holds for the unification of

rows $\widehat{E} \sim \widehat{F} : \theta$ and the unification of computation types $\widehat{C} \sim \widehat{D} : \theta$. We discuss the unification algorithm in more detail in Appendix D.6.

$$\boxed{\widehat{\Gamma} \vdash v : \widehat{A} \dashv \theta}$$

$$\begin{array}{c}
\frac{}{\widehat{\Gamma} \vdash () : () \dashv \emptyset} \text{A-UNIT} \qquad \frac{(x : \widehat{\sigma}) \in \widehat{\Gamma} \quad \widehat{\sigma} \leq \widehat{A} \quad \widehat{\Gamma} \vdash \widehat{A}}{\widehat{\Gamma} \vdash x : \widehat{A} \dashv \emptyset} \text{A-VAR} \\
\\
\frac{\widehat{\alpha} \text{ fresh} \quad \widehat{\Gamma}, x : \widehat{\alpha} \vdash c : \widehat{C} \dashv \theta}{\widehat{\Gamma} \vdash \lambda x. c : \theta \widehat{\alpha} \rightarrow \widehat{C} \dashv \theta} \text{A-ABS} \qquad \frac{\widehat{\Gamma} \vdash v_1 : \widehat{A} \dashv \theta_1 \quad \theta_1 \widehat{\Gamma} \vdash v_2 : \widehat{B} \dashv \theta_2}{\widehat{\Gamma} \vdash (v_1, v_2) : (\theta_2 \widehat{A}, \widehat{B}) \dashv \theta_{2..1}} \text{A-PAIR} \\
\\
\frac{\begin{array}{l} \widehat{\Gamma} \vdash_M \text{return } x \mapsto c_r : M \widehat{A}_1! \langle \widehat{E}_1 \rangle \dashv \theta_1 \\ \widehat{A}_1 = \widehat{\alpha}_1 \quad \theta_1 \widehat{\Gamma} \vdash_M \text{cls} : M \widehat{A}_2! \langle \widehat{E}_2 \rangle \dashv \theta_2 \\ \widehat{A}_2 = \widehat{\alpha}_2 \quad \theta_2(\widehat{\alpha}_1! \langle \widehat{E}_1 \rangle) \sim \widehat{\alpha}_2! \langle \widehat{E}_2 \rangle : \theta_3 \\ \theta_{3..1} \widehat{\Gamma} \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \widehat{A}_3! \langle \widehat{E}_3 \rangle \dashv \theta_4 \\ \widehat{A}_3 = \widehat{\alpha}_3 \quad \theta_{4..3}(\widehat{\alpha}_2! \langle \widehat{E}_2 \rangle) \sim \widehat{\alpha}_3! \langle \widehat{E}_3 \rangle : \theta_5 \\ \widehat{\alpha}_4 = \theta_5 \widehat{\alpha}_3 \quad \widehat{\alpha}_4 \notin \text{fv}(\theta_{5..1} \widehat{\Gamma}) \quad \langle \widehat{F} \rangle = \langle \text{labels}(\text{cls}); \theta_5 \widehat{E}_3 \rangle \end{array}}{\widehat{\Gamma} \vdash \text{handler}_M \{ \text{return } x \mapsto c_r, \text{cls}, \text{fwd } f \ p \ k \mapsto c_f \} : \widehat{\alpha}_4! \langle \widehat{F} \rangle \Rightarrow M \widehat{\alpha}_4! \langle \theta_5 \widehat{E}_3 \rangle \dashv \theta_{5..1}} \text{A-H}
\end{array}$$

Figure D.15: Type inference rules for values.

The most interesting case is the algorithmic handler rule. Here, we derive (unification) types for each of the subterms and require them to be a computation type with a type variable (e.g., $\widehat{A}_1 = \widehat{\alpha}_1$). Furthermore, all derived types of the subterms should be equivalent. We express this by unifying them (e.g., $\theta_2 \widehat{\alpha}_1! \langle \widehat{E}_1 \rangle \sim \widehat{\alpha}_2! \langle \widehat{E}_2 \rangle : \theta_3$). Then, we implicitly instantiate (Figure D.18) the resulting type using some fresh unification variable $\widehat{\alpha}_4$.

In these inference rules, we use $uv(\widehat{A})$ to represent the set of unification variables in $\widehat{A}$. Furthermore, we use $\text{rng}(\theta)$ to indicate the range of the substitution θ .

$$\boxed{\hat{\Gamma} \vdash c : \hat{\underline{C}} \dashv \theta}$$

$$\begin{array}{c}
\frac{\hat{\Gamma} \vdash v_1 : \hat{A}_1 \dashv \theta_1 \quad \theta_1 \hat{\Gamma} \vdash v_2 : \hat{A}_2 \dashv \theta_2 \quad \hat{\alpha}, \hat{\mu} \text{ fresh} \quad \theta_2 \hat{A}_1 \sim (\hat{A}_2 \rightarrow \hat{\alpha}! \langle \hat{\mu} \rangle) : \theta_3}{\hat{\Gamma} \vdash v_1 v_2 : \theta_3(\hat{\alpha}! \langle \hat{\mu} \rangle) \dashv \theta_{3..1}} \text{A-APP} \\
\\
\frac{\hat{\Gamma} \vdash c_1 : \hat{A}! \langle \hat{E} \rangle \dashv \theta_1 \quad \theta_1 \hat{\Gamma}, x : \hat{A} \vdash c_2 : \hat{B}! \langle \hat{F} \rangle \dashv \theta_2 \quad \theta_2 \hat{E} \sim \hat{F} : \theta_3}{\hat{\Gamma} \vdash \mathbf{do} \ x \leftarrow c_1; c_2 : \theta_3(\hat{B}! \langle \hat{F} \rangle) \dashv \theta_{3..1}} \text{A-DO} \\
\\
\frac{\hat{\Gamma}, \bar{\alpha}, \bar{\mu} \vdash v : \hat{A} \dashv \theta_1 \quad \bar{\alpha}, \bar{\mu} \notin \hat{\Gamma} \quad \bar{\alpha} \bar{\mu} = uv(\hat{A}) - uv(\hat{\Gamma}) \quad \theta_1 \hat{\Gamma}, x : \forall \bar{\alpha}. \forall \bar{\mu}. \forall \bar{\alpha}. \forall \bar{\mu}. \hat{A} \vdash c : \hat{\underline{C}} \dashv \theta_2}{\hat{\Gamma} \vdash \mathbf{let} \ x = v \ \mathbf{in} \ c : \hat{\underline{C}} \dashv \theta_{2..1}} \text{A-LET} \\
\\
\frac{\hat{\Gamma} \vdash v : \hat{A} \dashv \theta \quad \hat{\mu} \text{ fresh}}{\hat{\Gamma} \vdash \mathbf{return} \ v : \hat{A}! \langle \hat{\mu} \rangle \dashv \theta} \text{A-RET} \\
\\
\frac{\hat{\Gamma} \vdash v : \hat{A} \dashv \theta_1 \quad \theta_1 \hat{\Gamma} \vdash c : \hat{\underline{C}} \dashv \theta_2 \quad \hat{\alpha}, \hat{\mu} \text{ fresh} \quad \theta_2 \hat{A} \sim \hat{\underline{C}} \Rightarrow \hat{\alpha}! \langle \hat{\mu} \rangle : \theta_3}{\hat{\Gamma} \vdash v \star c : \theta_3 \hat{\alpha}! \langle \hat{\mu} \rangle \dashv \theta_{3..1}} \text{A-HAND} \\
\\
\frac{(\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \hat{\Gamma} \vdash v : \hat{A} \dashv \theta_1 \quad A_{\text{op}} \sim \hat{A} \dashv \theta_2 \quad \theta_{2..1} \hat{\Gamma}, y : B_{\text{op}} \vdash c : \hat{A}'! \langle \hat{E} \rangle \dashv \theta_3 \quad \hat{\mu} \text{ fresh} \quad \hat{E} \sim \langle \ell^{\text{op}}; \hat{\mu} \rangle : \theta_4}{\hat{\Gamma} \vdash \mathbf{op} \ \ell^{\text{op}} \ v \ (y. c) : \theta_4(\hat{A}'! \langle \hat{E} \rangle) \dashv \theta_{4..1}} \text{A-OP} \\
\\
\frac{(\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \hat{\Gamma} \vdash v : \hat{A} \dashv \theta_1 \quad A_{\text{sc}} \sim \hat{A} : \theta_2 \quad \theta_{2..1} \hat{\Gamma}, y : B_{\text{sc}} \vdash c_1 : \hat{B}! \langle \hat{E} \rangle \dashv \theta_3 \quad \hat{\mu} \text{ fresh} \quad \hat{E} \sim \langle \ell^{\text{sc}}; \hat{\mu} \rangle : \theta_4 \quad \theta_{4..1} \hat{\Gamma}, z : \theta_4 \hat{B} \vdash c_2 : \hat{A}'! \langle \hat{F} \rangle \dashv \theta_5 \quad \hat{F} \sim \theta_{5..4} \hat{E} : \theta_6}{\hat{\Gamma} \vdash \mathbf{sc} \ \ell^{\text{sc}} \ v \ (y. c_1) \ (z. c_2) : \theta_6 \hat{A}'! \langle \hat{F} \rangle \dashv \theta_{6..1}} \text{A-SC}
\end{array}$$

Figure D.16: Type inference rules for computations.

$$\boxed{\widehat{\Gamma} \vdash_M \text{ return } x \mapsto c_r : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta} \quad \boxed{\widehat{\Gamma} \vdash_M \text{ cls} : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta} \\
\boxed{\widehat{\Gamma} \vdash_M \text{ fwd } f \ p \ k \mapsto c : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta}$$

$$\frac{\widehat{\alpha}, \widehat{\mu} \text{ fresh} \quad \widehat{\Gamma}, x : \widehat{\alpha} \vdash c_r : \widehat{C} \dashv \theta_1 \quad \underline{C} \sim \theta_1(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) : \theta_2}{\widehat{\Gamma} \vdash_M \text{ return } x \mapsto c_r : \theta_{2..1} (M \widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{2..1}} \text{A-RETURN}$$

$$\frac{\widehat{\alpha}, \widehat{\mu} \text{ fresh}}{\widehat{\Gamma} \vdash_M \cdot : M \widehat{\alpha}! \langle \widehat{\mu} \rangle \dashv \emptyset} \text{A-EMPTY}$$

$$\frac{(\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma \quad \widehat{\Gamma} \vdash_M \text{ cls} : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta_1 \quad \theta_1 \widehat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \widehat{A}! \langle \widehat{E} \rangle \vdash c : \widehat{C} \dashv \theta_2 \quad \theta_2(M \widehat{A}! \langle \widehat{E} \rangle) \sim \widehat{C} : \theta_3}{\widehat{\Gamma} \vdash_M \text{ op } \ell^{\text{op}} \ x \ k \mapsto c, \text{ cls} : \theta_{3..2}(M \widehat{A}! \langle \widehat{E} \rangle) \dashv \theta_{3..1}} \text{A-OPROP}$$

$$\frac{(\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma \quad \widehat{\Gamma} \vdash_M \text{ cls} : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta_1 \quad \beta \text{ fresh} \quad \theta_1 \widehat{\Gamma}, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle \widehat{E} \rangle, k : \beta \rightarrow M \widehat{A}! \langle \widehat{E} \rangle \vdash c : \widehat{C} \dashv \theta_2 \quad \theta_2(M \widehat{A}! \langle \widehat{E} \rangle) \sim \widehat{C} : \theta_3 \quad \beta \notin \text{rng}(\theta_{3..1})}{\widehat{\Gamma} \vdash_M \text{ sc } \ell^{\text{sc}} \ x \ p \ k \mapsto c, \text{ cls} : \theta_{3..2}(M \widehat{A}! \langle \widehat{E} \rangle) \dashv \theta_{3..1}} \text{A-OPRSC}$$

$$\frac{\alpha, \beta, \gamma, \delta, \widehat{\alpha}, \widehat{\mu} \text{ fresh} \quad \widehat{A}_p = \alpha \rightarrow M \beta! \langle \widehat{\mu} \rangle \quad \widehat{A}'_p = \alpha \rightarrow \gamma! \langle \widehat{\mu} \rangle \quad \widehat{A}_k = \beta \rightarrow M \widehat{\alpha}! \langle \widehat{\mu} \rangle \quad \widehat{A}'_k = \gamma \rightarrow \delta! \langle \widehat{\mu} \rangle \quad \widehat{\Gamma}, \alpha, \beta, p : \widehat{A}_p, k : \widehat{A}_k, f : \forall \gamma \delta. (\widehat{A}'_p, \widehat{A}'_k) \rightarrow \delta! \langle \widehat{\mu} \rangle \vdash c_f : \widehat{C} \dashv \theta_1 \quad \theta_1(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) \sim \widehat{C} : \theta_2 \quad \alpha, \beta, \gamma, \delta \notin \text{rng}(\theta_{2..1})}{\widehat{\Gamma} \vdash_M \text{ fwd } f \ p \ k \mapsto c_f : \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{2..1}} \text{A-FWD}$$

Figure D.17: Type inference rules for handlers.

$$\boxed{\widehat{\sigma} \leq \widehat{A}}$$

$$\frac{}{\widehat{A} \leq \widehat{A}} \widehat{\sigma}\text{-INST-BASE}$$

$$\frac{\widehat{\alpha} \text{ fresh} \quad [\widehat{\alpha} / \alpha] \widehat{\sigma} \leq \widehat{A}}{\forall \alpha. \widehat{\sigma} \leq \widehat{A}} \widehat{\sigma}\text{-INST-}\widehat{\alpha} \quad \frac{\widehat{\mu} \text{ fresh} \quad [\widehat{\mu} / \mu] \widehat{\sigma} \leq \widehat{A}}{\forall \mu. \widehat{\sigma} \leq \widehat{A}} \widehat{\sigma}\text{-INST-}\widehat{\mu}$$

$$\frac{\widehat{\alpha}' \text{ fresh} \quad [\widehat{\alpha}' / \widehat{\alpha}] \widehat{\sigma} \leq \widehat{A}}{\forall \widehat{\alpha}. \widehat{\sigma} \leq \widehat{A}} \widehat{\sigma}\text{-INST-}\widehat{\alpha}' \quad \frac{\widehat{\mu}' \text{ fresh} \quad [\widehat{\mu}' / \widehat{\mu}] \widehat{\sigma} \leq \widehat{A}}{\forall \widehat{\mu}. \widehat{\sigma} \leq \widehat{A}} \widehat{\sigma}\text{-INST-}\widehat{\mu}'$$

Figure D.18: $\widehat{\sigma}$ -instantiation.

Unification Algorithm

Our unification algorithm works in two steps, as shown in Figure D.19.

$$\boxed{\hat{A} \sim \hat{B} : \theta} \quad \boxed{\hat{E} \sim \hat{F} : \theta} \quad \boxed{\hat{C} \sim \hat{D} : \theta}$$

$$\frac{\emptyset \vdash \text{reduce}(\hat{A}) \sim \text{reduce}(\hat{B}), \cdot \longrightarrow^* \theta \vdash \cdot}{\hat{A} \sim \hat{B} : \theta} \text{U-VAL}$$

$$\frac{\emptyset \vdash \hat{E} \sim \hat{F}, \cdot \longrightarrow^* \theta \vdash \cdot}{\hat{E} \sim \hat{F} : \theta} \text{U-Row}$$

$$\frac{\emptyset \vdash \text{reduce}(\hat{C}) \sim \text{reduce}(\hat{D}), \cdot \longrightarrow^* \theta \vdash \cdot}{\hat{C} \sim \hat{D} : \theta} \text{U-COMP}$$

Figure D.19: Main unification judgments.

First, we β -reduce the value types and computation types (D.20).

$$\boxed{\text{reduce}(\hat{A}) = \hat{B}} \quad \boxed{\text{reduce}(\hat{C}) = \hat{D}}$$

$$\begin{aligned}
 \text{reduce}() &= () \\
 \text{reduce}((\hat{A}, \hat{B})) &= (\text{reduce}(\hat{A}), \text{reduce}(\hat{B})) \\
 \text{reduce}(\hat{A} \rightarrow \hat{C}) &= \text{reduce}(\hat{A}) \rightarrow \text{reduce}(\hat{C}) \\
 \text{reduce}(\hat{C} \Rightarrow \hat{D}) &= \text{reduce}(\hat{C}) \Rightarrow \text{reduce}(\hat{D}) \\
 \text{reduce}(\alpha) &= \alpha \\
 \text{reduce}(\hat{\alpha}) &= \hat{\alpha} \\
 \text{reduce}((\Lambda \alpha. \hat{A}) \hat{B}) &= \text{reduce}([\hat{B} / \alpha] \hat{A}) \\
 \text{reduce}(\hat{A}! \langle \hat{E} \rangle) &= \text{reduce}(\hat{A})! \langle \hat{E} \rangle
 \end{aligned}$$

Figure D.20: Reduction rules.

Then, we unify types or rows that occur in the unification worklist step by step (D.21).

The unification algorithm $\longrightarrow$ transforms a unification worklist U_1 and an initial substitution θ_1 into a new unification worklist U_2 and substitution θ_2 . It can be split in three parts. The first part deals with computation types. The unification of two computation types boils down to separately unifying their value types and row types. The second part concerns the unification of value types. We first pattern match on unit types, handler types, function types and

$$\boxed{\theta_1 \vdash U_1 \longrightarrow \theta_2 \vdash U_2}$$

$$\begin{array}{ll}
\theta \vdash \hat{A}_1! \langle \hat{E}_1 \rangle \sim \hat{A}_2! \langle \hat{E}_2 \rangle, U & \longrightarrow \theta \vdash \hat{A}_1 \sim \hat{A}_2, \hat{E}_1 \sim \hat{E}_2, U \\
\theta \vdash () \sim (), U & \longrightarrow \theta \vdash U \\
\theta \vdash \hat{C}_1 \Rightarrow \hat{D}_1 \sim \hat{C}_2 \Rightarrow \hat{D}_2, U & \longrightarrow \theta \vdash \hat{C}_1 \sim \hat{C}_2, \hat{D}_1 \sim \hat{D}_2, U \\
\theta \vdash (\hat{A}_1 \rightarrow \hat{C}_1) \sim (\hat{A}_2 \rightarrow \hat{C}_2), U & \longrightarrow \theta \vdash \hat{A}_1 \sim \hat{A}_2, \hat{C}_1 \sim \hat{C}_2, U \\
\theta \vdash (\hat{A}_1, \hat{A}_2) \sim (\hat{A}_3, \hat{A}_4), U & \longrightarrow \theta \vdash \hat{A}_1 \sim \hat{A}_3, \hat{A}_2 \sim \hat{A}_4, U \\
\theta \vdash \alpha \sim \alpha, U & \longrightarrow \theta \vdash U \\
\theta \vdash \hat{\alpha} \sim \hat{\alpha}, U & \longrightarrow \theta \vdash U \\
\theta \vdash \hat{\alpha} \sim \hat{A}, U & \longrightarrow [\hat{A} / \hat{\alpha}] \theta \vdash [\hat{A} / \hat{\alpha}] U \text{ with } \hat{\alpha} \notin \hat{A} \\
\theta \vdash \hat{A} \sim \hat{\alpha}, U & \longrightarrow [\hat{A} / \hat{\alpha}] \theta \vdash [\hat{A} / \hat{\alpha}] U \text{ with } \hat{\alpha} \notin \hat{A} \\
\theta \vdash \langle \rangle \sim \langle \rangle, U & \longrightarrow \theta \vdash U \\
\theta \vdash \langle \ell; \hat{E} \rangle \sim \hat{F}, U & \longrightarrow \theta' \theta \vdash \theta' U \text{ with } (_, \theta') = \text{find}_\ell \hat{F} \text{ and } \text{tail } \hat{E} \notin \theta' \\
\theta \vdash \hat{F} \sim \langle \ell; \hat{E} \rangle, U & \longrightarrow \theta' \theta \vdash \theta' U \text{ with } (_, \theta') = \text{find}_\ell \hat{F} \text{ and } \text{tail } \hat{E} \notin \theta' \\
\theta \vdash \mu \sim \mu, U & \longrightarrow \theta \vdash U \\
\theta \vdash \hat{\mu} \sim \hat{\mu}, U & \longrightarrow \theta \vdash U \\
\theta \vdash \hat{\mu} \sim \hat{E}, U & \longrightarrow [\hat{E} / \hat{\mu}] \theta \vdash [\hat{E} / \hat{\mu}] U \text{ with } \hat{\mu} \notin \hat{E} \\
\theta \vdash \hat{E} \sim \hat{\mu}, U & \longrightarrow [\hat{E} / \hat{\mu}] \theta \vdash [\hat{E} / \hat{\mu}] U \text{ with } \hat{\mu} \notin \hat{E}
\end{array}$$

Figure D.21: Unification rules.

tuples. Two (unification) type variables with the same name are considered equal. Unifying a unification variable with a type $\hat{A}$ means substituting this unification variable by $\hat{A}$ in the given substitution as well as in all occurrences in the remaining unification workload.

The reasoning for unifying two rows is similar. The function $\text{find}_\ell \hat{E}$ extracts the label ℓ from $\hat{E}$ and returns the remaining part:

$$\begin{array}{ll}
\text{find}_\ell \langle \rangle & = \text{error} \\
\text{find}_\ell \mu & = (\mu', [\langle \ell; \mu' \rangle / \mu]) \text{ where } \mu' \text{ is a fresh variable} \\
\text{find}_\ell \langle \ell'; \hat{E} \rangle & = (\hat{E}, \emptyset) \text{ where } \ell = \ell' \\
\text{find}_\ell \langle \ell'; \hat{E} \rangle & = (\langle \ell'; \hat{E}' \rangle, \theta) \text{ where } \ell \neq \ell' \text{ and } (\hat{E}', \theta) = \text{find}_\ell \hat{E}
\end{array}$$

The function $\text{tail } \hat{E}$ returns the last row variable if $\hat{E}$ is open, and returns $\langle \rangle$ if $\hat{E}$ is closed.

Lemmas for Proof

In order to prove the soundness and completeness of our type inference algorithm with respect to the declarative type system, we require the following helper lemmas. Figure D.22 displays instantiation of contexts.

$$\boxed{\widehat{\Gamma} \rightsquigarrow \Gamma; \theta}$$

$$\frac{}{\Gamma \rightsquigarrow \Gamma; \emptyset} \Gamma\text{-BASE} \qquad \frac{\Gamma, \bar{\alpha} \vdash A : * \quad \widehat{\alpha} \in fv(\widehat{\Gamma}) \quad \Gamma, \bar{\alpha}, [A / \widehat{\alpha}] \widehat{\Gamma} \rightsquigarrow \Gamma_{out}; \theta}{\Gamma, \widehat{\Gamma} \rightsquigarrow \Gamma_{out}; [A / \widehat{\alpha}] \theta} \Gamma\text{-VAR}$$

Figure D.22: Reduction rules.

Lemma 11 (Instantiation instantiates bindings). *If $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta$ and $x : \widehat{\sigma} \in \widehat{\Gamma}$ then there exists some $\sigma = \theta \widehat{\sigma}$ such that $x : \sigma \in \Gamma$.*

Lemma 12 (Inverse of Instantiation instantiates bindings). *If $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta$ and $x : \sigma \in \Gamma$ then there exists $x : \widehat{\sigma} \in \widehat{\Gamma}$ such that $\sigma = \theta \widehat{\sigma}$.*

Lemma 13 (Moving substitutions to instantiation).

One can move instantiations θ_1 from being applied to $\widehat{\Gamma}$ to being generated by environment instantiation and back. That is, $\theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_2 \iff \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_2 \theta_1$.

Lemma 14 (Inferred substitutions are domain-restricted).

- For all $\widehat{\Gamma} \vdash v : \widehat{A} \dashv \theta$, $\theta = \theta|_{\widehat{\Gamma}} \theta|_{\widehat{A}}$.
- For all $\widehat{\Gamma} \vdash c : \widehat{C} \dashv \theta$, $\theta = \theta|_{\widehat{\Gamma}} \theta|_{\widehat{C}}$.

The notation $\theta|_{\widehat{\Gamma}}$ represents the substitution generated by removing all substitution of unification variables that are not in $\widehat{\Gamma}$ from θ . Similarly,

the notation $\theta|_{\widehat{A}} (\theta|_{\widehat{C}})$ represents the substitution generated by removing all substitution of unification variables that are not in $\widehat{A}$ ($\widehat{C}$) from θ .

Lemma 15 (Substitution split w.r.t domain restriction). *If $\theta_1 = \theta_2|_{\widehat{F}}$, then there exists θ such that $\theta_2 = \theta\theta_1$.*

Lemma 16 (Unification unifies).

After unification, substituted terms are equivalent. That is:

- $\widehat{A} \sim \widehat{B} : \theta$ implies $\theta\widehat{A} \equiv \theta\widehat{B}$
- $\widehat{C} \sim \widehat{D} : \theta$ implies $\theta\widehat{C} \equiv \theta\widehat{D}$
- $\widehat{E} \sim \widehat{F} : \theta$ implies $\theta\widehat{E} \equiv \theta\widehat{F}$

Lemma 17 (Unification gives a principal unifier). *The unification algorithm gives a principal unifier. That is:*

- $\theta\widehat{A} \equiv \theta\widehat{B}$ implies there exists θ_1, θ_2 such that $\widehat{A} \sim \widehat{B} : \theta_1$, $\theta = \theta_2\theta_1$.
- $\theta\widehat{E} \equiv \theta\widehat{F}$ implies there exists θ_1, θ_2 such that $\widehat{E} \sim \widehat{F} : \theta_1$, $\theta = \theta_2\theta_1$.
- $\theta\widehat{C} \equiv \theta\widehat{D}$ implies there exists θ_1, θ_2 such that $\widehat{C} \sim \widehat{D} : \theta_1$, $\theta = \theta_2\theta_1$.

Lemma 18 (Algorithmic to declarative instantiation).

For all $\widehat{\sigma} \leq \widehat{A}$ there exists a θ such that $\theta\widehat{\sigma} \leq \theta\widehat{A}$.

Proof. By induction on $\widehat{\sigma} \leq \widehat{A}$, using Lemma 19 and Lemma 20. □

Lemma 19 (Substitution/instantiation inlining). *If $\alpha \notin \theta$ then $\text{app}(\theta) ([\widehat{\alpha} / \alpha] \widehat{\sigma}) = [\theta\widehat{\alpha} / \alpha] (\text{app}(\theta) (\widehat{\sigma}))$.*

Lemma 20 (Substitution/instantiation inlining – Effects). *If $\mu \notin \theta$ then $\text{app}(\theta) ([\widehat{\mu} / \mu] \widehat{\sigma}) = [\theta\widehat{\mu} / \mu] (\text{app}(\theta) (\widehat{\sigma}))$.*

Theorem 7 (Declarative to algorithmic instantiation). *For all $\sigma \leq A$ and $\theta\widehat{\sigma} = \sigma$, there exists $\widehat{A}$ and θ' such that $\widehat{\sigma} \leq \widehat{A}$ and $\theta'\theta\widehat{A} = A$*

Proof. Mutual induction on $\sigma \leq A$ and $\hat{\sigma} \leq \hat{A}$. □

Soundness

Soundness states that all algorithmic typing judgments have a declarative counterpart. For proving this property, we ignore the differences in the domain of substitutions, which implies that the notion of equality is implicitly restricted to the substitution domain.

Theorem 8 (Soundness). *All algorithmic typing judgments have a declarative counterpart:*

$\forall \hat{\Gamma}, v, \hat{A}, \theta_{inf} : \text{if } \hat{\Gamma} \vdash v : \hat{A} \dashv \theta_{inf} \text{ then } \forall \Gamma, \theta_{inst} : \text{if } \theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst} \text{ and } \forall \theta : \Gamma \vdash \theta \text{ and } \theta \theta_{inst} \hat{A} = A \text{ then } \Gamma \vdash v : A.$
 $\forall \hat{\Gamma}, c, \hat{C}, \theta_{inf} : \text{if } \hat{\Gamma} \vdash c : \hat{C} \dashv \theta_{inf} \text{ then } \forall \Gamma, \theta_{inst} : \text{if } \theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst} \text{ and } \forall \theta : \Gamma \vdash \theta \text{ and } \theta \theta_{inst} \hat{C} = C \text{ then } \Gamma \vdash c : C.$
 $\forall \hat{\Gamma}, M, x, c_r, \hat{A}, \hat{E}, \theta_{inf} : \text{if } \hat{\Gamma} \vdash_M \text{ return } x \mapsto c_r : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf} \text{ then } \forall \Gamma, \theta_{inst} : \text{if } \theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst} \text{ and } \forall \theta : \Gamma \vdash \theta \text{ and } \theta \theta_{inst} (M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle \text{ then } \Gamma \vdash \text{ return } x \mapsto c_r : M A! \langle E \rangle.$
 $\forall \hat{\Gamma}, M, cls, \hat{A}, \hat{E}, \theta_{inf} : \text{if } \hat{\Gamma} \vdash_M cls : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf} \text{ then } \forall \Gamma, \theta_{inst} : \text{if } \theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst} \text{ and } \forall \theta : \Gamma \vdash \theta \text{ and } \theta \theta_{inst} (M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle \text{ then } \Gamma \vdash cls : M A! \langle E \rangle.$
 $\forall \hat{\Gamma}, M, f, p, k, c, \hat{A}, \hat{E}, \theta_{inf} : \text{if } \hat{\Gamma} \vdash_M \text{ fwd } f \ p \ k \mapsto c : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf} \text{ then } \forall \Gamma, \theta_{inst} : \text{if } \theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst} \text{ and } \forall \theta : \Gamma \vdash \theta \text{ and } \theta \theta_{inst} (M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle \text{ then } \Gamma \vdash \text{ fwd } f \ p \ k \mapsto c : M A! \langle E \rangle.$

Proof. We prove the statement by mutual induction on these judgments. In what follows we ignore the differences in the domain of substitutions, implicitly restricting the notion of equality to the substitution domain.

Value typing

A-Var We know that $\hat{\Gamma} \vdash x : \hat{A} \dashv \emptyset$ so we can choose Γ, θ_{inst} such that $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst}$ (1) and $\forall \theta : \Gamma \vdash \theta$ and $\theta \theta_{inst} \hat{A} = A$ for some A (2).

From Lemma 11, (1) and $(x : \hat{\sigma}) \in \hat{\Gamma}$ (A-VAR), we have that $(x : \theta_{inst} \hat{\sigma}) \in \Gamma$.

From Lemma 18, $\hat{\sigma} \leq \hat{A}$ (A-VAR) and (2), we have that $\theta \theta_{inst} \hat{\sigma} \leq \theta \theta_{inst} \hat{A}$.

As θ is applied after instantiation with θ_{inst} , it has no influence on $\hat{\sigma}$. Thus $\theta\theta_{\text{inst}}\hat{\sigma} \equiv \theta_{\text{inst}}\hat{\sigma} \equiv \sigma$ (4).

Thus, we have that $(x : \sigma) \in \Gamma$ and $\sigma \leq A$ (2) (4) so that $\Gamma \vdash x : A$ (SD-VAR).

A-Unit Trivial case.

A-Pair We know that $\hat{\Gamma} \vdash (v_1, v_2) : (\theta_2\hat{A}, \hat{B}) \dashv \theta_{2..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{2..1}\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}}(\theta_2\hat{A}, \hat{B}) = (A, B)$ for some A, B (2).

From the *induction hypothesis* for v_1 we have the following:

if $\hat{\Gamma} \vdash v_1 : \hat{A} \dashv \theta_1$ (A-PAIR)
then $\theta_1\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_2$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_2\hat{A} = A$ (2)
then $\Gamma \vdash v_1 : A$.

From the *induction hypothesis* for v_2 we have the following:

if $\theta_1\hat{\Gamma} \vdash v_2 : \hat{B} \dashv \theta_2$ (A-PAIR)
then $\theta_{2..1}\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \hat{B} = B$ (2)
then $\Gamma \vdash v_2 : B$.

From (SD-PAIR) we conclude that $\Gamma \vdash (v_1, v_2) : (A, B)$.

A-Abs We know that $\hat{\Gamma} \vdash \lambda x. c : \theta_{\text{inf}} \hat{\alpha} \rightarrow \hat{C} \dashv \theta_{\text{inf}}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{\text{inf}}\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}}(\theta_{\text{inf}} \hat{\alpha} \rightarrow \hat{C}) = A \rightarrow \underline{C}$ for some $A, \underline{C}$ (2).

From the *induction hypothesis* for c we have the following:

if $\hat{\Gamma}, x : \hat{\alpha} \vdash c : \hat{C} \dashv \theta_{\text{inf}}$ (A-ABS)
then $\theta_{\text{inf}}\hat{\Gamma}, x : \hat{\alpha} \rightsquigarrow \Gamma_1; \theta_{\text{inst}}$ (Lemma 13, (1)) and $\forall \theta. \Gamma_1 \vdash \theta$ and $\theta\theta_{\text{inst}} \hat{C} = \underline{C}$ (2)
then $\Gamma_1 \vdash c : \underline{C}$.

From (Lemma 13), (1) and (2), we know that $\Gamma_1 = \Gamma, x : \theta_{\text{inst}} \hat{\alpha} = \Gamma, x : A$, where $\theta_{\text{inst}} \hat{\alpha} = \theta\theta_{\text{inst}} \theta_{\text{inf}} \hat{\alpha} = A$ because only θ_{inst} influences the instantiation of $\hat{\alpha}$.

From (SD-ABS) we conclude that $\Gamma, x : A \vdash \lambda x. c : A \rightarrow \underline{C}$.

A-Handler We know that $\hat{\Gamma} \vdash \mathbf{handler}_M \{ \mathbf{return} \ x \mapsto c_r, \mathbf{cls}, \mathbf{fwd} \ f \ p \ k \mapsto c_f \} : \hat{\alpha}_4! \langle \hat{F} \rangle \Rightarrow M \hat{\alpha}_4! \langle \theta_5 \hat{E}_3 \rangle \dashv \theta_{5..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{5..1} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} (\hat{\alpha}_4! \langle \hat{F} \rangle \Rightarrow M \hat{\alpha}_4! \langle \theta_5 \hat{E}_3 \rangle) = \alpha! \langle F \rangle \Rightarrow M \alpha! \langle E \rangle$ for some M, α, E, F (2).

From the *induction hypothesis for fwd* $f \ p \ k \mapsto c_f$ we have the following:

if $\theta_{3..1} \hat{\Gamma} \vdash_M \mathbf{fwd} \ f \ p \ k \mapsto c_f : M \hat{\alpha}_3! \langle \hat{E}_3 \rangle \dashv \theta_4$ (A-HANDLER)
then $\theta_4 \theta_{3..1} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_5$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_5 (M \hat{\alpha}_3! \langle \hat{E}_3 \rangle) = M \alpha! \langle E \rangle$ since (2) and $\hat{\alpha}_4 = \theta_5 \hat{\alpha}_3$ (A-HANDLER)
then $\Gamma \vdash \mathbf{fwd} \ f \ p \ k \mapsto c_f : M \alpha! \langle E \rangle$.

From the *induction hypothesis for cls* we have the following:

if $\theta_1 \hat{\Gamma} \vdash_M \mathbf{cls} : M \hat{\alpha}_2! \langle \hat{E}_2 \rangle \dashv \theta_2$ (A-HANDLER)
then $\theta_{2..1} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{5..3}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{5..3} (M \hat{\alpha}_2! \langle \hat{E}_2 \rangle) = M \alpha! \langle E \rangle$ (3)
then $\Gamma \vdash \mathbf{cls} : M \alpha! \langle E \rangle$.

where (3): $\theta \theta_{\text{inst}} \theta_{5..3} (M \hat{\alpha}_2! \langle \hat{E}_2 \rangle)$
 (Lemma 16, A-HANDLER)
 $= \theta \theta_{\text{inst}} \theta_5 (M \hat{\alpha}_3! \langle \hat{E}_3 \rangle)$
 (2), (A-HANDLER)
 $= M \alpha! \langle E \rangle$

From the *induction hypothesis for return* $x \mapsto c_r$ we have the following:

if $\hat{\Gamma} \vdash_M \mathbf{return} \ x \mapsto c_r : M \hat{\alpha}_1! \langle \hat{E}_1 \rangle \dashv \theta_1$ (A-HANDLER)
then $\theta_1 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{5..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{5..2} (M \hat{\alpha}_1! \langle \hat{E}_1 \rangle) = M \alpha! \langle E \rangle$ (4)
then $\Gamma \vdash \mathbf{return} \ x \mapsto c_r : M \alpha! \langle E \rangle$.

where (4): $\theta \theta_{\text{inst}} \theta_{5..2} (M \hat{\alpha}_1! \langle \hat{E}_1 \rangle)$
 (Lemma 16, A-HANDLER)
 $= \theta \theta_{\text{inst}} \theta_{5..3} (M \hat{\alpha}_2! \langle \hat{E}_2 \rangle)$
 (3)
 $= M \alpha! \langle E \rangle$

Furthermore, we have that $\langle \hat{F} \rangle = \langle \mathbf{labels} \ (cls); \theta_5 \hat{E}_3 \rangle$ and since substitutions preserve equivalence also $F = \theta \theta_{\text{inst}} \hat{F} \equiv_{\langle \rangle} \mathbf{labels} \ (cls); \theta \theta_{\text{inst}} \theta_5 \hat{E}_3 =$

$labels(cls); E.$

From (SD-H) we conclude that $\Gamma \vdash \mathbf{handler}_M \{ \text{return } x \mapsto c_r, cls, \text{fwd } f \text{ } p \text{ } k \mapsto c_f \} : A! \langle F \rangle \Rightarrow M \ A! \langle E \rangle.$

Computation typing

A-App We know that $\widehat{\Gamma} \vdash v_1 \ v_2 : \theta_3(\widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{3..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{3..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_3(\widehat{\alpha}! \langle \widehat{\mu} \rangle) = \underline{C}$ for some $\underline{C}$ (2).

From the *induction hypothesis* for v_2 we have the following:

if $\theta_1 \widehat{\Gamma} \vdash v_2 : \widehat{A}_2 \dashv \theta_2$ (A-APP)

then $\theta_2 \theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_3$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_3 \widehat{A}_2 = A$ for some A (3)

then $\Gamma \vdash v_2 : A.$

From the *induction hypothesis* for v_1 we have the following:

if $\widehat{\Gamma} \vdash v_1 : \widehat{A}_1 \dashv \theta_1$ (A-APP)

then $\theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{3..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{3..2} \widehat{A}_1 = A \rightarrow \underline{C}$ (4)

then $\Gamma \vdash v_1 : A \rightarrow \underline{C}.$

where (4): $\theta \theta_{\text{inst}} \theta_{3..2} \widehat{A}_1$

(Lemma 16, A-APP)

$= \theta \theta_{\text{inst}} \theta_3(\widehat{A}_2 \rightarrow \widehat{\alpha}! \langle \widehat{\mu} \rangle)$

$= \theta \theta_{\text{inst}} \theta_3(\widehat{A}_2) \rightarrow \theta \theta_{\text{inst}} \theta_3 \widehat{\alpha}! \langle \widehat{\mu} \rangle$

(2), (3)

$= A \rightarrow \underline{C}$

From (SD-APP) we conclude that $\Gamma \vdash v_1 \ v_2 : \underline{C}.$

A-Let We know that $\widehat{\Gamma} \vdash \mathbf{let } x = v \text{ in } c : \widehat{C} \dashv \theta_{2..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that

$\theta_{2..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \widehat{C} = \underline{C}$ for some $\underline{C}$ (2).

From the *induction hypothesis* for v we have the following:

if $\widehat{\Gamma}, \bar{\alpha}, \bar{\mu} \vdash v : \widehat{A} \dashv \theta_1$ with $\bar{\alpha}, \bar{\mu} \notin \widehat{\Gamma}$ (A-LET)

then $\theta_1 \widehat{\Gamma}, \bar{\alpha}, \bar{\mu} \rightsquigarrow \Gamma_1; \theta_{\text{inst}}$ (Lemma 13, (1)) where $\Gamma_1 = \Gamma, \theta_{\text{inst}} \theta_{2..1} \bar{\alpha}, \theta_{\text{inst}} \theta_{2..1} \overline{\text{join}} =$

$\Gamma, \bar{\alpha}, \overline{\text{join}}$, because $\bar{\alpha}, \bar{\mu} \notin \theta_{2..1}$ and $\forall \theta. \Gamma_1 \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_2 \hat{A} = A$ for some A (3)
then $\Gamma_1 \vdash v : A$.

From the *induction hypothesis* for c we have the following:

if $\theta_1 \hat{\Gamma}, x : \hat{\sigma} \vdash c : \hat{\underline{C}} \dashv \theta_2$ with $\hat{\sigma} = \forall \bar{\alpha}. \forall \overline{\text{join}}. \forall \bar{\alpha}. \forall \bar{\mu}. \hat{A}$ (A-LET)
then $\theta_2(\theta_1 \hat{\Gamma}, x : \hat{\sigma}) \rightsquigarrow \Gamma_2; \theta_{\text{inst}}$ (Lemma 13, (1)) and $\forall \theta. \Gamma_2 \vdash \theta$ and $\theta\theta_{\text{inst}} \hat{\underline{C}} = \underline{C}$ (2)
then $\Gamma \vdash c : \underline{C}$.

From (Lemma 13), (1) and (3), we know that $\Gamma_2 = \Gamma, x : \theta_{\text{inst}} \theta_2 (\forall \bar{\alpha}. \forall \overline{\text{join}}. \forall \bar{\alpha}. \forall \bar{\mu}. \hat{A}) = \Gamma, x : \forall \bar{\alpha}. \forall \overline{\text{join}}. A$, where $\text{dom}(\theta_{\text{inst}} \theta_2) \cap (uv(\hat{A}) \text{ sub } uv(\theta_1 \hat{\Gamma})) = \emptyset$.

From (SD-LET) we conclude that $\Gamma \vdash \text{let } x = v \text{ in } c : \underline{C}$.

A-Ret We know that $\hat{\Gamma} \vdash \text{return } v : \hat{A}! \langle \hat{\mu} \rangle \dashv \theta_{\text{inf}}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that
 $\theta_{\text{inf}} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} (\hat{A}! \langle \hat{\mu} \rangle) = A! \langle E \rangle$ for some A, E (2).

From the *induction hypothesis* for v we have the following:

if $\hat{\Gamma} \vdash v : \hat{A} \dashv \theta_{\text{inf}}$ (A-RET)
then $\theta_{\text{inf}} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \hat{A} = A$ (2)
then $\Gamma \vdash v : A$.

From (SD-RET) we conclude that $\Gamma \vdash \text{return } v : A! \langle E \rangle$.

A-Do We know that $\hat{\Gamma} \vdash \text{do } x \leftarrow c_1; c_2 : \theta_3(\hat{B}! \langle \hat{F} \rangle) \dashv \theta_{3..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that
 $\theta_{3..1} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_3(\hat{B}! \langle \hat{F} \rangle) = B! \langle E \rangle$ for some B, E (2).

From the *induction hypothesis* for c_1 we have the following:

if $\hat{\Gamma} \vdash c_1 : \hat{A}! \langle \hat{E} \rangle \dashv \theta_1$ (A-DO)
then $\theta_1 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{3..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_{3..2}(\hat{A}! \langle \hat{E} \rangle) = A! \langle E \rangle$ (3)
then $\Gamma \vdash c_1 : A! \langle E \rangle$.

where (3): $\theta\theta_{\text{inst}} \theta_{3..2}(\hat{A}! \langle \hat{E} \rangle)$
 $= \theta\theta_{\text{inst}} \theta_{3..2} \hat{A}! \langle \theta\theta_{\text{inst}} \theta_{3..2} \hat{E} \rangle$
 $= A! \langle \theta\theta_{\text{inst}} \theta_{3..2} \hat{E} \rangle$ for some A (4)
 (Lemma 16, A-DO)

$$\begin{aligned}
&= A! \langle \theta \theta_{\text{inst}} \theta_3 \hat{F} \rangle \\
&\text{(2)} \\
&= A! \langle E \rangle
\end{aligned}$$

From the *induction hypothesis* for c_2 we have the following:

if $\theta_1 \hat{\Gamma}, x : \hat{A} \vdash c_2 : \hat{B}! \langle \hat{F} \rangle \dashv \theta_2$ (A-DO)
then $\theta_2(\theta_1 \hat{\Gamma}, x : \hat{A}) \rightsquigarrow \Gamma_2; \theta_{\text{inst}} \theta_3$ (Lemma 13, (1)) and $\forall \theta. \Gamma_2 \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_3(\hat{B}! \langle \hat{F} \rangle) = B! \langle E \rangle$ (2)
then $\Gamma_2 \vdash c_2 : B! \langle E \rangle$.

From (Lemma 13), (1) and (4), we know that $\Gamma_2 = \Gamma, x : \theta_{\text{inst}} \theta_{3..2} \hat{A} = \Gamma, x : A$.

From (SD-DO) we conclude that $\Gamma \vdash \mathbf{do} \ x \leftarrow c_1; c_2 : B! \langle E \rangle$.

A-Op We know that $\hat{\Gamma} \vdash \mathbf{op} \ \ell^{\text{op}} \ v \ (y.c) : \theta_4(\hat{A}'! \langle \hat{E} \rangle) \dashv \theta_{4..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{4..1} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_4(\hat{A}'! \langle \hat{E} \rangle) = A! \langle E \rangle$ for some A, E (2).

From the *induction hypothesis* for v we have the following:

if $\hat{\Gamma} \vdash v : \hat{A} \dashv \theta_1$ (A-OP)
then $\theta_1 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{4..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{4..2} \hat{A} = A_{\text{op}}$ (3)
then $\Gamma \vdash v : A_{\text{op}}$.

$$\begin{aligned}
&\text{where (3): } \theta \theta_{\text{inst}} \theta_{4..2} \hat{A} \\
&\text{(Lemma 16, A-OP)} \\
&= \theta \theta_{\text{inst}} \theta_{4..2} (A_{\text{op}}) \\
&= A_{\text{op}}
\end{aligned}$$

From the *induction hypothesis* for c we have the following:

if $\theta_2 \hat{\Gamma}, y : B_{\text{op}} \vdash c : \hat{A}'! \langle \hat{E} \rangle \dashv \theta_3$ (A-OP)
then $\theta_3(\theta_{2..1} \hat{\Gamma}, y : B_{\text{op}}) \rightsquigarrow \Gamma, y : B_{\text{op}}; \theta_{\text{inst}} \theta_4$ (Lemma 13, (1)) and $\forall \theta. \Gamma, y : B_{\text{op}} \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_4(\hat{A}'! \langle \hat{E} \rangle) = A! \langle E \rangle$ (2)
then $\Gamma, y : B_{\text{op}} \vdash c : A! \langle E \rangle$.

Furthermore, from (A-OP) follows that $(\ell^{\text{op}} : A_{\text{op}} \rightarrow B_{\text{op}}) \in \Sigma$. Following (A-OP) and Lemma 16, we have that $\theta_4 \langle \hat{E} \rangle \equiv \theta_4 \langle \ell^{\text{op}}; \hat{\mu} \rangle$. Since substitutions preserve equivalence, we also have that $E = \theta \theta_{\text{inst}} \theta_4 \hat{E} \equiv_{\emptyset} \theta \theta_{\text{inst}} \theta_4 (\ell^{\text{op}}; \hat{\mu}) = \ell^{\text{op}}, E'$ for some E' so that $\ell^{\text{op}} \in E$.

From (SD-OP) we conclude that $\Gamma \vdash \mathbf{op} \ell^{\mathbf{op}} v (y.c) : A! \langle E \rangle$.

A-Sc We know that $\widehat{\Gamma} \vdash \mathbf{sc} \ell^{\mathbf{sc}} v (y.c_1) (z.c_2) : \theta_6(\widehat{A}'! \langle \widehat{F} \rangle) \dashv \theta_{6..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{6..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_6(\widehat{A}'! \langle \widehat{F} \rangle) = A! \langle E \rangle$ for some A, E (2).

From the *induction hypothesis* for v we have the following:

if $\widehat{\Gamma} \vdash v : \widehat{A} \dashv \theta_1$ (A-Sc)

then $\theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{6..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{6..2} \widehat{A} = A_{\text{sc}}$ (3)

then $\Gamma \vdash v : A_{\text{sc}}$.

where (3): $\theta \theta_{\text{inst}} \theta_{6..2} \widehat{A}$
 (Lemma 16, A-Sc)
 $= \theta \theta_{\text{inst}} \theta_{6..2} (A_{\text{sc}})$
 $= A_{\text{sc}}$

From the *induction hypothesis* for c_1 we have the following:

if $\theta_{2..1} \widehat{\Gamma}, y : B_{\text{sc}} \vdash c_1 : \widehat{B}! \langle \widehat{E} \rangle \dashv \theta_3$ (A-Sc)

then $\theta_3(\theta_{2..1} \widehat{\Gamma}, y : B_{\text{sc}}) \rightsquigarrow \Gamma, y : B_{\text{sc}}; \theta_{\text{inst}} \theta_{6..4}$ (Lemma 13, (1)) and $\forall \theta. \Gamma, y : B_{\text{sc}} \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{6..4}(\widehat{B}! \langle \widehat{E} \rangle) = B! \langle E \rangle$ (4)

then $\Gamma, y : B_{\text{sc}} \vdash c_1 : B! \langle E \rangle$.

where (4): $\theta \theta_{\text{inst}} \theta_{6..4}(\widehat{B}! \langle \widehat{E} \rangle)$
 $= \theta \theta_{\text{inst}} \theta_{6..4} \widehat{B}! \langle \theta \theta_{\text{inst}} \theta_{6..4} \widehat{E} \rangle$
 $= B! \langle \theta \theta_{\text{inst}} \theta_{6..4} \widehat{E} \rangle$ for some B (5)
 (Lemma 16, A-Sc)
 $= B! \langle \theta \theta_{\text{inst}} \theta_6 \widehat{F} \rangle$
 (2)
 $= B! \langle E \rangle$

From the *induction hypothesis* for c_2 we have the following:

if $\theta_{4..1} \widehat{\Gamma}, z : \theta_4 \widehat{B} \vdash c_2 : \widehat{A}'! \langle \widehat{F} \rangle \dashv \theta_5$ (A-Sc)

then $\theta_5(\theta_{4..1} \widehat{\Gamma}, z : \theta_4 \widehat{B}) \rightsquigarrow \Gamma_3; \theta_{\text{inst}} \theta_6$ (Lemma 13, (1)) and $\forall \theta. \Gamma_3 \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_6(\widehat{A}'! \langle \widehat{F} \rangle) = A! \langle E \rangle$ (2)

then $\Gamma_3 \vdash c_2 : A! \langle E \rangle$.

From (Lemma 13), (1) and (5), we know that $\Gamma_3 = \Gamma, z : \theta_{\text{inst}} \theta_{6..4} \widehat{B} = \Gamma, z : B$.

Furthermore, from (A-Sc) follows that $(\ell^{\text{sc}} : A_{\text{sc}} \rightarrow B_{\text{sc}}) \in \Sigma$. Following (A-Sc) and Lemma 16, we have that $\theta_4 \langle \widehat{E} \rangle \equiv \theta_4 \langle \ell^{\text{sc}}; \widehat{\mu} \rangle$. Since substitutions preserve equivalence, we also have that $E = \theta\theta_{\text{inst}} \theta_{6..4} \widehat{E} \equiv_{\langle \rangle} \theta\theta_{\text{inst}} \theta_{6..4} (\ell^{\text{sc}}; \widehat{\mu}) = \ell^{\text{sc}}, E'$ for some E' so that $\ell^{\text{sc}} \in E$.

From (SD-Sc) we conclude that $\Gamma \vdash \mathbf{sc} \ell^{\text{sc}} v (y . c_1) (z . c_2) : A! \langle E \rangle$.

A-Hand We know that $\widehat{\Gamma} \vdash v \star c : \theta_3(\widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{3..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{3..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta . \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_3(\widehat{\alpha}! \langle \widehat{\mu} \rangle) = \underline{D}$ for some $\underline{D}$ (2).

From the *induction hypothesis* for c we have the following:

if $\theta_1 \widehat{\Gamma} \vdash c : \widehat{C} \dashv \theta_2$ (A-HAND)

then $\theta_{2..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_3$ (Lemma 13, (1)) and $\forall \theta . \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_3 \widehat{C} = \underline{C}$ for some $\underline{C}$ (3)

then $\Gamma \vdash c : \underline{C}$.

From the *induction hypothesis* for v we have the following:

if $\widehat{\Gamma} \vdash v : \widehat{A} \dashv \theta_1$ (A-HAND)

then $\theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{3..2}$ (Lemma 13, (1)) and $\forall \theta . \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_{3..2} \widehat{A} = \underline{C} \Rightarrow \underline{D}$ (4)

then $\Gamma \vdash v : \underline{C} \Rightarrow \underline{D}$.

where (4): $\theta\theta_{\text{inst}} \theta_{3..2} \widehat{A}$
 (Lemma 16, A-APP)
 $= \theta\theta_{\text{inst}} \theta_3(\widehat{C} \Rightarrow \widehat{\alpha}! \langle \widehat{\mu} \rangle)$
 $= \theta\theta_{\text{inst}} \theta_3(\widehat{C}) \Rightarrow \theta\theta_{\text{inst}} \theta_3 \widehat{\alpha}! \langle \widehat{\mu} \rangle$
 (2), (3)
 $= \underline{C} \Rightarrow \underline{D}$

From (SD-HAND) we conclude that $\Gamma \vdash v \star c : \underline{D}$.

Return clause

A-Return We know that $\widehat{\Gamma} \vdash_M \text{return } x \mapsto c_r : \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{2..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{2..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta . \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) = M A! \langle E \rangle$ for some M, A, E (2).

From the *induction hypothesis* for c_r we have the following:

if $\widehat{\Gamma}, x : \widehat{\alpha} \vdash c_r : \widehat{C} \dashv \theta_1$ (A-ABS)
then $\theta_1(\widehat{\Gamma}, x : \widehat{\alpha}) \rightsquigarrow \Gamma_1; \theta_{\text{inst}} \theta_2$ (Lemma 13, (1)) and $\forall \theta. \Gamma_1 \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_2 \widehat{C} = M A! \langle E \rangle$ (3)
then $\Gamma_1 \vdash c_r : M A! \langle E \rangle$.

where (3): $\theta\theta_{\text{inst}} \theta_2 \widehat{C}$
 (Lemma 16, A-RETURN)
 $= \theta\theta_{\text{inst}} \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle)$
 (2)
 $= M A! \langle E \rangle$

From (Lemma 13), (1) and (2), we know that $\Gamma_1 = \Gamma, x : \theta_{\text{inst}} \theta_{2..1} \widehat{\alpha} = \Gamma, x : A$, where $\theta_{\text{inst}} \theta_{2..1} \widehat{\alpha} = \theta\theta_{\text{inst}} \theta_{2..1} \widehat{\alpha} = A$ because θ does not influence the instantiation of $\widehat{\alpha}$.

From (SD-RETURN) we conclude that $\Gamma \vdash \text{return } x \mapsto c_r : M A! \langle E \rangle$.

Operation clauses

A-Empty Trivial case:

We know that $\widehat{\Gamma} \vdash_M \cdot : M \widehat{\alpha}! \langle \widehat{\mu} \rangle \dashv \emptyset$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) = M A! \langle E \rangle$ for some M, A, E .

From (SD-EMPTY) we conclude that $\Gamma \vdash \cdot : M A! \langle E \rangle$.

A-OprOp We know that $\widehat{\Gamma} \vdash_M \text{op } \ell^{\text{op}} x k \mapsto c, \text{cls} : \theta_{3..2}(M \widehat{A}! \langle \widehat{E} \rangle) \dashv \theta_{3..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{3..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_{3..2}(M \widehat{A}! \langle \widehat{E} \rangle) = M A! \langle E \rangle$ for some M, A, E (2).

From the *induction hypothesis* for cls we have the following:

if $\widehat{\Gamma} \vdash_M \text{cls} : M \widehat{A}! \langle \widehat{E} \rangle \dashv \theta_1$ (A-OPROP)
then $\theta_1 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{3..2}$ (Lemma 13, (1)) and $\forall \theta. \Gamma \vdash \theta$ and $\theta\theta_{\text{inst}} \theta_{3..2} M \widehat{A}! \langle \widehat{E} \rangle = M A! \langle E \rangle$ (2)
then $\Gamma \vdash \text{cls} : M A! \langle E \rangle$.

From the *induction hypothesis* for c we have the following:

if $\theta_1 \widehat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \widehat{A}! \langle \widehat{E} \rangle \vdash c : \widehat{C} \dashv \theta_2$ (A-OPROP)

then $\theta_2(\theta_1\hat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \hat{A}!\langle\hat{E}\rangle) \rightsquigarrow \Gamma_2; \theta_{\text{inst}} \theta_3$ (Lemma 13, (1))
and $\forall \theta. \Gamma_2 \vdash \theta$ **and** $\theta\theta_{\text{inst}} \theta_3 \underline{C} = M A!\langle E \rangle$ (3)
then $\Gamma_2 \vdash c : M A!\langle E \rangle$.

where (3): $\theta\theta_{\text{inst}} \theta_3 \hat{\underline{C}}$
 (Lemma 16, A-OPROP)
 $= \theta\theta_{\text{inst}} \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle)$
 (2)
 $= M A!\langle E \rangle$

From (Lemma 13), (1) and (2), we know that $\Gamma_2 = \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow \theta_{\text{inst}} \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle) = \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M A!\langle E \rangle$.

From (SD-OPROP) we conclude that $\Gamma \vdash \mathbf{op} \ell^{\text{op}} x k \mapsto c, \text{cls} : M A!\langle E \rangle$.

A-OpSc We know that $\hat{\Gamma} \vdash_M \mathbf{sc} \ell^{\text{sc}} x p k \mapsto c, \text{cls} : \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle) \dashv \theta_{3..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{3..1}\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ **and** $\theta\theta_{\text{inst}} \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle) = M A!\langle E \rangle$ for some M, A, E (2).

From the *induction hypothesis for cls* we have the following:

if $\hat{\Gamma} \vdash_M \text{cls} : M \hat{A}!\langle\hat{E}\rangle \dashv \theta_1$ (A-OPROP)
then $\theta_1\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}} \theta_{3..2}$ (Lemma 13, (1)) **and** $\forall \theta. \Gamma \vdash \theta$ **and** $\theta\theta_{\text{inst}} \theta_{3..2} M \hat{A}!\langle\hat{E}\rangle = M A!\langle E \rangle$ (2)
then $\Gamma \vdash \text{cls} : M A!\langle E \rangle$.

From the *induction hypothesis for c* we have the following:

if $\theta_1\hat{\Gamma}, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta!\langle E \rangle, k : \beta \rightarrow M \hat{A}!\langle\hat{E}\rangle \vdash c : \underline{C} \dashv \theta_2$ (A-OPROP)
then $\theta_2(\theta_1\hat{\Gamma}, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta!\langle E \rangle, k : \beta \rightarrow M \hat{A}!\langle\hat{E}\rangle) \rightsquigarrow \Gamma_2; \theta_{\text{inst}} \theta_3$
 (Lemma 13, (1)) **and** $\forall \theta. \Gamma_2 \vdash \theta$ **and** $\theta\theta_{\text{inst}} \theta_3 \underline{C} = M A!\langle E \rangle$ (3)
then $\Gamma_2 \vdash c : M A!\langle E \rangle$.

where (3): $\theta\theta_{\text{inst}} \theta_3 \hat{\underline{C}}$
 (Lemma 16, A-OPRSC)
 $= \theta\theta_{\text{inst}} \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle)$
 (2)
 $= M A!\langle E \rangle$

From (Lemma 13), (1) and (2), we know that $\Gamma_2 = \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta!\langle\hat{E}\rangle, k : \beta \rightarrow \theta_{\text{inst}} \theta_{3..2}(M \hat{A}!\langle\hat{E}\rangle) = \Gamma, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta!\langle E \rangle, k : \beta \rightarrow M A!\langle E \rangle$ because $\beta \notin \theta_{3..2}$.

From (SD-OPRSC) we conclude that $\Gamma \vdash \mathbf{sc} \ell^{\mathbf{sc}} x p k \mapsto c, \text{cls} : M A! \langle E \rangle$.

Forwarding clause

A-Fwd We know that $\widehat{\Gamma} \vdash_M \mathbf{fwd} f p k \mapsto c_f : \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) \dashv \theta_{2..1}$ so we can choose $\Gamma, \theta_{\text{inst}}$ such that $\theta_{2..1} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$ (1) and $\forall \theta. \Gamma \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle) = M A! \langle E \rangle$ for some M, A, E (2).

From the *induction hypothesis* for c_f we have the following:

if $\widehat{\Gamma}, \alpha, \beta, p : \widehat{A}_p, k : \widehat{A}_k, f : \forall \gamma \delta. (\widehat{A}'_p, \widehat{A}'_k) \rightarrow \delta! \langle \widehat{\mu} \rangle \vdash c_f : \underline{C} \dashv \theta_1$ (A-OPROP)
then $\theta_1(\widehat{\Gamma}, \alpha, \beta, p : \widehat{A}_p, k : \widehat{A}_k, f : \forall \gamma \delta. (\widehat{A}'_p, \widehat{A}'_k) \rightarrow \delta! \langle \widehat{\mu} \rangle) \rightsquigarrow \Gamma_1; \theta_{\text{inst}} \theta_2$
 (Lemma 13, (1)) and $\forall \theta. \Gamma_1 \vdash \theta$ and $\theta \theta_{\text{inst}} \theta_2 \underline{C} = M A! \langle E \rangle$ (3)
then $\Gamma_1 \vdash c_f : M A! \langle E \rangle$.

where (3): $\theta \theta_{\text{inst}} \theta_2 \widehat{C}$
 (Lemma 16, A-FWD)
 $= \theta \theta_{\text{inst}} \theta_{2..1}(M \widehat{\alpha}! \langle \widehat{\mu} \rangle)$
 (2)
 $= M A! \langle E \rangle$

From (Lemma 13), (1) and (2), we know that $\Gamma_1 = \Gamma, \theta_{\text{inst}} \theta_{2..1} \alpha, \theta_{\text{inst}} \theta_{2..1} \beta p : \theta_{\text{inst}} \theta_{2..1} \widehat{A}_p, k : \theta_{\text{inst}} \theta_{2..1} \widehat{A}_k, f : \forall \gamma \delta. (\theta_{\text{inst}} \theta_{2..1} \widehat{A}'_p, \theta_{\text{inst}} \theta_{2..1} \widehat{A}'_k) \rightarrow \theta_{\text{inst}} \theta_{2..1} (\delta! \langle \widehat{\mu} \rangle) = \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow M A! \langle E \rangle$.

From (SD-FWD) we conclude that $\Gamma \vdash \mathbf{fwd} f p k \mapsto c_f : M A! \langle E \rangle$.

□

Completeness

Completeness states that all declarative typing judgments have a algorithmic counterpart.

Theorem 9 (Completeness). *All declarative typing judgments have an algorithmic counterpart:*

- If $\Gamma \vdash v : A$ then for all $\hat{\Gamma}$ and θ_a , if $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ then there exists $\hat{A}, \theta_{inf}, \theta_{inst}, \theta_d$ such that $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash v : \hat{A} \dashv \theta_{inf}$, and $\theta_d \theta_{inst} \hat{A} = A$.
- If $\Gamma \vdash c : \underline{C}$ then for all $\hat{\Gamma}$ and θ_a , if $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ then there exists $\hat{A}, \theta_{inf}, \theta_{inst}, \theta_d$ such that $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash c : \hat{C} \dashv \theta_{inf}$, and $\theta_d \theta_{inst} \hat{C} = \underline{C}$.
- If $\Gamma \vdash \text{return } x \mapsto c_r : M \ A! \langle E \rangle$ then for all $\hat{\Gamma}$ and θ_a , if $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ then there exists $\hat{A}, \theta_{inf}, \theta_{inst}, \theta_d$ such that $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash_M \text{return } x \mapsto c_r : M \ \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf}$, and $\theta_d \theta_{inst} M \ \hat{A}! \langle \hat{E} \rangle = M \ A! \langle E \rangle$.
- If $\Gamma \vdash \text{cls} : M \ A! \langle E \rangle$ then for all $\hat{\Gamma}$ and θ_a , if $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ then there exists $\hat{A}, \theta_{inf}, \theta_{inst}, \theta_d$ such that $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash_M \text{cls} : M \ \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf}$, and $\theta_d \theta_{inst} M \ \hat{A}! \langle \hat{E} \rangle = M \ A! \langle E \rangle$.
- If $\Gamma \vdash \text{fwd } f \ p \ k \mapsto c_f : M \ A! \langle E \rangle$ then for all $\hat{\Gamma}$ and θ_a , if $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ then there exists $\hat{A}, \theta_{inf}, \theta_{inst}, \theta_d$ such that $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \ \hat{A}! \langle \hat{E} \rangle \dashv \theta_{inf}$, and $\theta_d \theta_{inst} M \ \hat{A}! \langle \hat{E} \rangle = M \ A! \langle E \rangle$.

Proof. Note that by Lemma 13, $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$, and $\theta_a = (\theta_{inst} \theta_{inf})|_{\hat{\Gamma}}$, we can derive that $\theta_{inf} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{inst}$. In the following proof, we treat it as part of the conclusion of the theorem for simplicity. Also, we do not deal with the equivalence relations of types explicitly. All comparisons between types are considered to use equivalence relations implicitly.

Prove by mutual induction on these judgments.

Value typing

SD-Var For any $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_a$, let $\theta_{\text{inst}} = \theta_a$ and $\theta_{\text{inf}} = \emptyset$. By Lemma 12 and $(x:\sigma) \in \Gamma$ we get that there exists $(x:\widehat{\sigma}) \in \widehat{\Gamma}$ such that $\theta_a \widehat{\sigma} = \sigma$. By Lemma 7 and $\Gamma \vdash \theta_a \widehat{\sigma} \leq A$, there exists θ_d and $\widehat{A}$ such that $\theta_d \theta_a \widehat{A} = A = \theta_d \theta_{\text{inst}} \widehat{A}$ and $\widehat{\sigma} \leq \widehat{A}$. Then, $\Gamma \vdash A$ gives us $\widehat{\Gamma} \vdash \widehat{A}$. Thus, by A-VAR, we have $\widehat{\Gamma} \vdash x : \widehat{A} \dashv \theta_{\text{inf}}$. We also have $\theta_a = \theta_{\text{inst}} \theta_{\text{inf}}$ and $\theta_{\text{inf}} \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$. It is easy to check the conclusion is satisfied given $\theta_a, \theta_{\text{inf}}, \theta_{\text{inst}}, \theta_d$ defined as above.

SD-Unit Let $\theta_{\text{inst}} = \theta_a$ and $\theta_d = \theta_{\text{inf}} = \emptyset$. The conclusion follows from the assumptions.

SD-Pair By the IH on v_1 , for any $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3 \theta_2|_{\widehat{\Gamma}}(\widehat{\Gamma})$, $\widehat{\Gamma} \vdash v_1 : \widehat{A} \dashv \theta_2$, $\theta_2 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ (1), and $\theta_4 \theta_3 \widehat{A} = A$.

Then we apply the IH on v_2 to (1), there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_3 = \theta_7 \theta_6|_{\theta_2 \widehat{\Gamma}}$, $\theta_2 \widehat{\Gamma} \vdash v_2 : \widehat{B} \dashv \theta_6$, $\theta_6 \theta_2 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_7$, $\theta_8 \theta_7 \widehat{B} = B$.

By A-PAIR, we have $\widehat{\Gamma} \vdash (v_1, v_2) : (\theta_6 \widehat{A}, \widehat{B}) \dashv \theta_6 \theta_2$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_6 \theta_2$, $\theta_{\text{inst}} = \theta_7$, and $\theta_d = \theta_4 \theta_8$, the conclusion is satisfied:

- $\theta_1 = \theta_7 \theta_6 \theta_2|_{\widehat{\Gamma}}$
- $\theta_6 \theta_2 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_7$
- $\theta_4 \theta_8 \theta_7 (\theta_6 \widehat{A}, \widehat{B}) = (A, B)$ (the new unification variables in $\widehat{A}$ and $\widehat{B}$ does not overlap)

SD-Abs By IH on the function body c , for any $\widehat{\Gamma}, x:\widehat{A} \rightsquigarrow \Gamma, x:A; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3 \theta_2|_{\widehat{\Gamma}}(\widehat{\Gamma})$, $\widehat{\Gamma}, x:\widehat{A} \vdash c : \widehat{\underline{C}} \dashv \theta_2$, $\theta_2(\widehat{\Gamma}, x:\widehat{A}) \rightsquigarrow \Gamma, x:A; \theta_3$, and $\theta_4 \theta_3 \widehat{\underline{C}} = \underline{C}$.

Let the arbitrary type $\widehat{A}$ be an fresh unification variable $\widehat{\alpha}$, we have $\widehat{\Gamma}, x:\widehat{\alpha} \vdash c : \widehat{\underline{C}} \dashv \theta_2$. By A-ABS, we have $\widehat{\Gamma} \vdash \lambda x. c : \theta_2 \widehat{\alpha} \rightarrow \widehat{\underline{C}} \dashv \theta_2$.

Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_2$, $\theta_{\text{inst}} = \theta_3|_{\theta_2 \widehat{\Gamma}}$, and $\theta_d = \theta_4$, the conclusion is satisfied.

SD-H By the IH on the return clause, for any $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, taking a fresh type variable a that is not in $\widehat{\Gamma}$ and Γ . there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = (\theta_3 \theta_2)|_{\widehat{\Gamma}}$, $\widehat{\Gamma} \vdash \text{return } x \mapsto c_r : M \widehat{A}_1! \langle \widehat{E}_1 \rangle \dashv \theta_2$, $\theta_2 \widehat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, $\theta_4 \theta_3 (M \widehat{A}_1! \langle \widehat{E}_1 \rangle) = M a! \langle E \rangle$ (1). Because α is a fresh type variable, $\widehat{A}_1$ must be an unification variable $\widehat{\alpha}_1$.

Then we apply the IH on the operation clauses cls to $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_3 = (\theta_7\theta_6)|_{\theta_2\hat{\Gamma}}$ (2), $\theta_2\hat{\Gamma} \vdash cls : M \hat{A}_2! \langle \hat{E}_2 \rangle \dashv \theta_6$, $\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_7$ (3), $\theta_8\theta_7(M \hat{A}_2! \langle \hat{E}_2 \rangle) = M a! \langle E \rangle$ (4). Similarly, $\hat{A}_2$ must be an unification variable $\hat{\alpha}_2$.

By (1), (2), and (4), we have $\theta_4(\theta_7\theta_6)|_{\theta_2\hat{\Gamma}}(M \hat{\alpha}_1! \langle \hat{E}_1 \rangle) = \theta_8\theta_7(M \hat{\alpha}_2! \langle \hat{E}_2 \rangle)$. Because the new unification variables in $M \hat{\alpha}_1! \langle \hat{E}_1 \rangle$ and $M \hat{\alpha}_2! \langle \hat{E}_2 \rangle$ do not overlap, we have $\theta_8\theta_4\theta_7\theta_6(M \hat{\alpha}_1! \langle \hat{E}_1 \rangle) = \theta_8\theta_4\theta_7(M \hat{\alpha}_2! \langle \hat{E}_2 \rangle)$. By Lemma 17, there exists θ_y, θ_x such that $\theta_8\theta_4\theta_7 = \theta_y\theta_x$ and $\theta_6(M \hat{\alpha}_1! \langle \hat{E}_1 \rangle) \sim M \hat{\alpha}_2! \langle \hat{E}_2 \rangle : \theta_x$. By restricting the domain to be $\theta_6\theta_2\hat{\Gamma}$, we have $\theta_7|_{\theta_6\theta_2\hat{\Gamma}} = (\theta_y\theta_x)|_{\theta_6\theta_2\hat{\Gamma}}$ (5).

By (3) and (5), we have $\theta_x\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_9$, where $\theta_9 = \theta_y|_{\theta_x\theta_6\theta_2\hat{\Gamma}}$. By the IH on the fwd clause, there exists $\theta_{10}, \theta_{11}, \theta_{12}$ such that $\theta_9 = (\theta_{11}\theta_{10})|_{\theta_x\theta_6\theta_2\hat{\Gamma}}$ (6), $\theta_x\theta_6\theta_2\hat{\Gamma} \vdash \text{fwd } f \ p \ k \mapsto c_f : M \hat{A}_3! \langle \hat{E}_3 \rangle \dashv \theta_{10}$, $\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{11}$, $\theta_{12}\theta_{11}(M \hat{A}_3! \langle \hat{E}_3 \rangle) = M \alpha! \langle E \rangle$ (6). Similar to the above, $\hat{A}_3$ must be an unification variable $\hat{\alpha}_3$.

By (4) and (7), we have $\theta_8\theta_7(M \hat{A}_2! \langle \hat{E}_2 \rangle) = \theta_{12}\theta_{11}(M \hat{A}_3! \langle \hat{E}_3 \rangle)$. By (6) and the fact that new unification variables in $M \hat{\alpha}_2! \langle \hat{E}_2 \rangle$ and $M \hat{\alpha}_3! \langle \hat{E}_3 \rangle$ do not overlap, we have $\theta_{12}\theta_y\theta_{11}\theta_{10}\theta_x(M \hat{A}_2! \langle \hat{E}_2 \rangle) = \theta_{12}\theta_y\theta_{11}(M \hat{A}_3! \langle \hat{E}_3 \rangle)$. By Lemma 17, there exists θ'_y, θ'_x such that $\theta_{12}\theta_y\theta_{11} = \theta'_y\theta'_x$ and $\theta_{10}\theta_x(M \hat{A}_2! \langle \hat{E}_2 \rangle) \sim M \hat{A}_3! \langle \hat{E}_3 \rangle : \theta'_x$. By restricting the domain to be $\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma}$, we have $\theta_{11} = (\theta'_y\theta'_x)|_{\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma}}$. Because $\theta'_y\theta'_x(M \hat{\alpha}_3! \langle \hat{E}_3 \rangle) = \theta_{12}\theta_y\theta_{11}(M \hat{\alpha}_3! \langle \hat{E}_3 \rangle) = M \alpha! \langle E \rangle$ and α is fresh, we have that $\theta'_x\hat{\alpha}_3$ must be an unification variable $\hat{\alpha}_4$.

Because $\theta'_y\hat{\alpha}_4 = \theta_{12}\theta_y\theta_{11}\hat{\alpha}_3 = \theta_{12}\theta_{11}\hat{\alpha}_3 = \alpha$, $\theta'_y\theta'_x\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma} = \theta_{11}\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma} = \Gamma$, and $\alpha \notin \Gamma$, we have $\theta'_y\hat{\alpha}_4 \notin \theta'_y\theta'_x\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma}$, which leads to $\hat{\alpha}_4 \notin \theta'_x\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma}$ ¹.

Let $\langle \hat{F} \rangle = \langle \text{labels}(cls); \theta'_x \hat{E}_3 \rangle$, by A-HANDLER, we have $\hat{\Gamma} \vdash \mathbf{handler} \{ \dots \} : \hat{\alpha}_4! \langle \hat{F} \rangle \Rightarrow M \hat{\alpha}_4! \langle \theta'_x \hat{E}_3 \rangle \dashv \theta'_x\theta_{10}\theta_x\theta_6\theta_2$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta'_x\theta_{10}\theta_x\theta_6\theta_2$, $\theta_{\text{inst}} = \theta'_y|_{\theta'_x\theta_{10}\theta_x\theta_6\theta_2\hat{\Gamma}}$, and $\theta_d = \theta'_y$, the conclusion is satisfied.

Computation typing.

SD-Ret By the IH on the return value v , for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_a = \theta_3\theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash \mathbf{return } v : \hat{A}! \hat{\mu} \dashv \theta_2$, $\theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$,

¹here $\hat{\alpha} \notin \hat{\Gamma}$ actually means $\hat{\alpha} \notin \text{fv}(\hat{\Gamma})$

and $\theta_4\theta_3\hat{A} = A$. Take a fresh unification variable $\hat{\mu}$, by A-RET we have $\hat{\Gamma} \vdash \text{return } v : \hat{A}! \hat{\mu} \dashv \theta_2$.

Setting $\theta_{\text{inf}} = \theta_2$, $\theta_{\text{inst}} = \theta_3$, and $\theta_d = [E/\hat{\mu}]\theta_4$, it is easy to check the conclusion is satisfied.

SD-App By the IH on v_1 , for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}$ ($\hat{\Gamma}$), $\hat{\Gamma} \vdash v_1 : \hat{A}_1 \dashv \theta_2$, $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, and $\theta_4\theta_3\hat{A}_1 = A \rightarrow \underline{C}$. Then we apply the IH on v_2 to the judgement $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, which gives us that there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}}$, $\theta_2\hat{\Gamma} \vdash v_2 : \hat{A}_2 \dashv \theta_6$, $\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_7$, and $\theta_8\theta_7\hat{A}_2 = A$. Let $\underline{C} = B!\langle E \rangle$, and take fresh unification variables $\hat{\alpha}, \hat{\mu}$, we have $[B/\hat{\alpha}, E/\hat{\mu}]\theta_8\theta_7(\hat{A}_2 \rightarrow \hat{\alpha}!\langle \hat{\mu} \rangle) = A \rightarrow \underline{C} = \theta_4\theta_3\hat{A}_1 = \theta_4\theta_7\theta_6\hat{A}_1$.

Note that θ_4 only substitutes new unification variables in $\theta_3\hat{A}_1 = \theta_7\theta_6\hat{A}_1$, and $[B/\hat{\alpha}, E/\hat{\mu}]\theta_8$ only substitutes new unification variables in $\theta_7(\hat{A}_2 \rightarrow \hat{\alpha}!\langle \hat{\mu} \rangle)$, we have the equation $[B/\hat{\alpha}, E/\hat{\mu}]\theta_8\theta_4\theta_7(\theta_6\hat{A}_1) = [B/\hat{\alpha}, E/\hat{\mu}]\theta_8\theta_4\theta_7(\hat{A}_2 \rightarrow \hat{\alpha}!\langle \hat{\mu} \rangle)$. By Lemma 17, there exists θ_{10}, θ_9 such that $\theta_6\hat{A}_1 \sim (\hat{A}_2 \rightarrow \hat{\alpha}!\langle \hat{\mu} \rangle) : \theta_9$ and $[B/\hat{\alpha}, E/\hat{\mu}]\theta_8\theta_4\theta_7 = \theta_{10}\theta_9$. Restricting the domain to be $\hat{\Gamma}$, we have $\theta_7 = (\theta_{10}\theta_9)|_{\hat{\Gamma}}$. By Lemma 13, we have $\theta_9\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_{10}|_{\hat{\Gamma}}$. By A-APP, we have $\hat{\Gamma} \vdash v_1 \ v_2 : \theta_9\hat{\alpha}!\langle \hat{\mu} \rangle \dashv \theta_9\theta_6\theta_2$.

Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_9\theta_6\theta_2$, $\theta_{\text{inst}} = \theta_{10}|_{\hat{\Gamma}}$, and $\theta_d = [B/\hat{\alpha}, E/\hat{\mu}]$, it is easy to check the conclusion of the theorem is satisfied.

SD-Do By the IH on c_1 , for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash c_1 : \hat{A}!\langle \hat{E} \rangle \dashv \theta_2$, $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, and $\theta_4\theta_3(\hat{A}!\langle \hat{E} \rangle) = A!\langle E \rangle$. By $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ and the fact that θ_4 only substitutes unification variables in $\hat{A}$, we have $\theta_2\hat{\Gamma}, x : \hat{A} \rightsquigarrow \Gamma, x : A; \theta_4\theta_3$ (1).

Then we apply the IH on c_2 to (1), which gives that there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_4\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}, x : \hat{A}}$, $\theta_2\hat{\Gamma}, x : \hat{A} \vdash c_2 : \hat{B}!\hat{F} \dashv \theta_6$, $\theta_6(\theta_2\hat{\Gamma}, x : \hat{A}) \rightsquigarrow \Gamma, x : A : \theta_7$, and $\theta_8\theta_7(\hat{B}!\langle \hat{F} \rangle) = B!\langle E \rangle$. We have $\theta_8\theta_7\hat{F} = E$ and $\theta_4\theta_3\hat{E} = E = \theta_7\theta_6\hat{E}$. Note that θ_8 only substitutes new unification variables in $\theta_7\hat{F}$, we have $\theta_8\theta_7\hat{F} = \theta_8\theta_7\theta_6\hat{E}$. Thus, by Lemma 17, there exists θ_9 and θ_{10} such that $\theta_6\hat{E} \sim \hat{F} : \theta_9$, $\theta_8\theta_7 = \theta_{10}\theta_9$. Restricting the domain to be $\hat{\Gamma}$, we have $\theta_7 = (\theta_{10}\theta_9)|_{\hat{\Gamma}}$. Thus, by Lemma 13, we have $\theta_9|_{\hat{\Gamma}}\theta_6(\theta_2\hat{\Gamma}, x : \hat{A}) \rightsquigarrow \Gamma, x : A : \theta_{10}|_{\hat{\Gamma}}$. By A-DO, we also have $\hat{\Gamma} \vdash \text{do } x \leftarrow c_1; c_2 : \theta_9(\hat{B}!\langle \hat{F} \rangle) \dashv \theta_9\theta_6\theta_2$.

Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_9\theta_6\theta_2$, $\theta_{\text{inst}} = \theta_{10}|_{\hat{\Gamma}}$, and $\theta_d = \theta_8$, it is easy to check the conclusion is satisfied:

- $\theta_a = \theta_1 = \theta_3\theta_2|_{\hat{\Gamma}} = \theta_4\theta_3\theta_2|_{\hat{\Gamma}} = \theta_7\theta_6\theta_2|_{\hat{\Gamma}} = (\theta_{10}\theta_9\theta_6\theta_2)|_{\hat{\Gamma}} = (\theta_{\text{inst}} \theta_{\text{inf}})|_{\hat{\Gamma}}$
- $\hat{\Gamma} \vdash \mathbf{do} \ x \leftarrow c_1; c_2 : \theta_9(\hat{B}!\langle\hat{F}\rangle) \dashv \theta_{\text{inf}}$
- $\theta_{\text{inf}} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$
- $\theta_a\theta_{\text{inst}}(\hat{B}!\langle\hat{F}\rangle) = \theta_8\theta_{10}|_{\hat{\Gamma}}\theta_9(\hat{B}!\langle\hat{F}\rangle) = \theta_8\theta_{10}\theta_9(\hat{B}!\langle\hat{F}\rangle) = \theta_8\theta_7(\hat{B}!\langle\hat{F}\rangle) = B!\langle E \rangle$

SD-Let By the IH on v , for all $\hat{\Gamma}, \bar{\alpha}, \bar{\mu} \rightsquigarrow \Gamma, \bar{\alpha}, \bar{\mu}; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}, \hat{\Gamma}, \bar{\alpha}, \bar{\mu} \vdash v : \hat{A} \dashv \theta_2, \theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, and $\theta_4\theta_3\hat{A} = A$.

We define θ_x as the set of all substitutions in $(\theta_4\theta_3)|_{\hat{A}}$ which do not substitute any unification variable in $fv(\theta_2\hat{\Gamma})$. Let $\sigma = \forall \bar{\alpha}. \forall \bar{\mu}. A$ and $\hat{\sigma} = \forall \bar{\alpha}. \forall \bar{\mu}. \theta_x\hat{A}$. By $\theta_4\theta_3\hat{A} = A$, we have $\theta_2\hat{\Gamma}, x : \hat{\sigma} \rightsquigarrow \Gamma, x : \sigma; \theta_3$ (1).

Then we apply the IH on c to (1), which gives us that there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}}, \theta_2\hat{\Gamma}, x : \hat{\sigma} \vdash c : \hat{C} \dashv \theta_6$ (2), $\theta_6(\theta_2\hat{\Gamma}, x : \hat{\sigma}) \rightsquigarrow \Gamma, x : \sigma; \theta_7$, and $\theta_8\theta_7\hat{C} = \hat{C}$.

Let $\bar{\alpha} \bar{\mu} = uv(\hat{A}) - uv(\theta_2\hat{\Gamma})$ and $\hat{\sigma}' = \forall \bar{\alpha}. \forall \bar{\mu}. \forall \bar{\alpha}. \forall \bar{\mu}. \hat{A}$. If we replace the $\hat{\sigma}$ in (2) with $\hat{\sigma}'$, by the fact that the substitution θ_x does not substitutes anything in $\theta_2\hat{\Gamma}$, the computation c is still well-typed, and there exists θ_y such that $\theta_2\hat{\Gamma}, x : \hat{\sigma}' \vdash c : \hat{C}' \dashv \theta_6$ and $\theta_y\hat{C}' = \hat{C}$ and θ_y also does not substitutes anything in $\theta_6\theta_2\hat{\Gamma}$. Thus, $\theta_2\hat{\Gamma}, x : \hat{\sigma}' \vdash c : \hat{C} \dashv \theta_y\theta_6$. By A-LET, we have $\hat{\Gamma} \vdash \mathbf{let} \ x = v \ \mathbf{in} \ c : \hat{C} \dashv \theta_y\theta_6\theta_2$.

Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_y\theta_6\theta_2$, $\theta_{\text{inst}} = \theta_7$, and $\theta_d = \theta_8$, the conclusion is satisfied.

SD-Op By the IH on v , for all $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}, \hat{\Gamma} \vdash v : \hat{A}' \dashv \theta_2, \theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, and $\theta_4\theta_3\hat{A}' = A_{\text{op}}$. By Lemma 17 and $\theta_4\theta_3\hat{A}' = A_{\text{op}}$, there exists θ_x, θ_y such that $\theta_4\theta_3 = \theta_y\theta_x$ and $\hat{A}' \sim A_{\text{op}} : \theta_x$. By $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ and the fact that θ_4 only substitutes new unification variables in $\hat{A}'$, we have $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_4\theta_3$. By restricting the domain to be $\theta_2\hat{\Gamma}$ and the fact that θ_4 only substitutes new unification variables in $\hat{A}'$, we have $(\theta_y\theta_x)|_{\theta_2\hat{\Gamma}} = (\theta_4\theta_3)|_{\theta_2\hat{\Gamma}} = \theta_3|_{\theta_2\hat{\Gamma}} = \theta_3$. By $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ and Lemma 13, we have $\theta_x\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_z$, where $\theta_z = \theta_y|_{\theta_2\hat{\Gamma}}$.

Then, we apply the IH on c to the judgement $\theta_x\theta_2\hat{\Gamma}, y : B_{\text{op}} \rightsquigarrow \Gamma, y : B_{\text{op}}; \theta_z$, there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_z = \theta_7\theta_6, \theta_x\theta_2\hat{\Gamma}, y : B_{\text{op}} \vdash c : \hat{A}!\langle\hat{E}\rangle \dashv \theta_6$,

$\theta_6(\theta_x\theta_2\widehat{\Gamma}, y : B_{\text{op}}) \rightsquigarrow \Gamma, y : B_{\text{op}}; \theta_7$, and $\theta_8\theta_7(\widehat{A}!\langle\widehat{E}\rangle) = A!\langle E \rangle$. By Lemma 17 and $\theta_8\theta_7\widehat{E} = E = \theta_8\theta_7E$, there exists θ'_x, θ'_y such that $\theta_8\theta_7 = \theta'_y\theta'_x$ and $\widehat{E} \sim E : \theta'_x$ (1). Note that since $\ell^{\text{op}} \in E$ and the row type $\langle\ell^{\text{op}}; \widehat{\mu}\rangle$ is the most general type that explicitly contains label ℓ^{op} , the unification of $\widehat{E}$ and $\langle\ell^{\text{op}}; \widehat{\mu}\rangle$ must succeed. Suppose $\widehat{E} \sim \langle\ell^{\text{op}}; \widehat{\mu}\rangle : \theta_9$ (2) and $E \equiv_{\langle} \ell^{\text{op}}; E'$, by A-OP we have $\widehat{\Gamma} \vdash \text{op } \ell^{\text{op}} v(y.c) : \theta_9(\widehat{A}!\langle\widehat{E}\rangle) \dashv \theta_9\theta_6\theta_x\theta_2$. Now we do a case analysis to prove that there exists θ_{10} such that $\theta_7 = (\theta_{10}\theta_9)|_{\theta_6\theta_x\theta_2\widehat{\Gamma}}$:

$\widehat{\mu} \notin \text{dom}(\theta_9)$: We have $[E' / \widehat{\mu}]\theta_9\widehat{E} = E$. Thus, $[E' / \widehat{\mu}]\theta_9 = (\theta_8\theta_7)|_{\widehat{E}}$. By Lemma 15, there exists θ_{10} such that $\theta_8\theta_7 = \theta_{10}[E' / \widehat{\mu}]\theta_9$. By restricting the domains of the substitutions on both side of the equation to $\theta_6\theta_x\theta_2\widehat{\Gamma}$ and simplifying the equation, we have $\theta_7 = (\theta_{10}\theta_9)|_{\theta_6\theta_x\theta_2\widehat{\Gamma}}$.

$\widehat{\mu} \in \text{dom}(\theta_9)$: Let θ'_9 be the substitution that generated from removing the substitution of $\widehat{\mu}$ from θ_9 . Suppose $\langle E \rangle = \langle\ell^{\text{op}}; E'\rangle$, by (1), we have $\theta'_x[E' / \widehat{\mu}]\widehat{E} = \theta'_x[E' / \widehat{\mu}]\langle\ell^{\text{op}}; \widehat{\mu}\rangle$. By Lemma 17 and (2), there exists θ_{11} such that $\theta'_x[E' / \widehat{\mu}] = \theta_{11}\theta_9$. Removing the substitution of $\widehat{\mu}$ from both sides of the equation, we have $\theta'_x = \theta_{11}\theta'_9$, where θ'_9 is the substitution generated by removing the substitution of $\widehat{\mu}$ from θ_9 . Thus, by $\theta_8\theta_7 = \theta'_y\theta'_x$, we have $\theta_8\theta_7 = \theta'_y\theta_{11}\theta'_9$. By restricting the domain to be $\theta_6\theta_x\theta_2\widehat{\Gamma}$, we have $\theta_7 = (\theta'_y\theta_{11}\theta'_9)|_{\theta_6\theta_x\theta_2\widehat{\Gamma}} = (\theta'_y\theta_{11}\theta_9)|_{\theta_6\theta_x\theta_2\widehat{\Gamma}}$.

Finally, setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_9\theta_6\theta_x\theta_2$, $\theta_{\text{inst}} = \theta_{10}|_{\theta_9\theta_6\theta_x\theta_2\widehat{\Gamma}}$ and $\theta_d = \theta_8$, it is easy to check the conclusion is satisfied.

SD-Sc By the IH on v , for any $\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$, such that $\theta_1 = \theta_3\theta_2|_{\widehat{\Gamma}}$, $\widehat{\Gamma} \vdash v : \widehat{A} \dashv \theta_2$, $\theta_2\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_3$, and $\theta_4\theta_3\widehat{A} = A_{\text{sc}}$. By Lemma 17, there exists θ_y, θ_x such that $\theta_4\theta_3 = \theta_y\theta_x$ and $\widehat{A} \sim (A_{\text{sc}}) : \theta_x$. By restricting the domain to be $\widehat{\Gamma}$, we have $\theta_3 = (\theta_4\theta_3)|_{\theta_2\widehat{\Gamma}} = (\theta_y\theta_x)|_{\theta_2\widehat{\Gamma}}$. By Lemma 13, we have $\theta_x|_{\widehat{\Gamma}}\theta_2\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_5$ (1), where $\theta_5 = \theta_y|_{\theta_x\widehat{\Gamma}}$.

Then we apply the IH on c_1 to (1), which gives us that there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_5 = \theta_7\theta_6$, $\theta_x\theta_2\widehat{\Gamma}, y : B_{\text{sc}} \vdash c_1 : \widehat{B}!\langle\widehat{E}\rangle \dashv \theta_6$, $\theta_6(\theta_x\theta_2\widehat{\Gamma}, y : B_{\text{sc}}) \rightsquigarrow \Gamma, y : B_{\text{sc}}; \theta_7$, and $\theta_8\theta_7\widehat{B}!\langle\widehat{E}\rangle = B!\langle E \rangle$. By Lemma 17, there exists θ'_y, θ'_x such that $\theta_8\theta_7 = \theta'_y\theta'_x$ and $\widehat{E} \sim E : \theta'_x$. Because $\ell^{\text{sc}} \in E$ and $\langle\ell^{\text{sc}}; \widehat{\mu}\rangle$ is the most general row type containing label ℓ^{sc} , there exists θ_9 such that $\widehat{E} \sim \langle\ell^{\text{sc}}; \widehat{\mu}\rangle : \theta_9$. By a similar case analysis to the SD-SC case, we get that there exists θ_{10} such that $\theta_7 = \theta_{10}\theta_9|_{\theta_6\theta_x\theta_2\widehat{\Gamma}}$. By Lemma 13 and $\theta_6\theta_x\theta_2\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_7$, we have that $\theta_9\theta_6\theta_x\theta_2\widehat{\Gamma} \rightsquigarrow \Gamma; \theta_{10}$, which implies $\theta_9\theta_6\theta_x\theta_2\widehat{\Gamma}, z : \theta_9\widehat{B} \rightsquigarrow \Gamma, z : B; \theta_{11}$ (2) where $\theta_{11} = \theta_8\theta_{10}$.

Then we apply the IH on c_2 to (2), which gives us that there exists $\theta_{12}, \theta_{13}, \theta_{14}$ such that $\theta_{11} = \theta_{13}\theta_{12}$, $\theta_9\theta_6\theta_x\theta_2\hat{\Gamma}, z : \theta_9\hat{B} \vdash c_2 : \hat{A}!\langle\hat{F}\rangle \dashv \theta_{12}$, $\theta_{12}(\theta_9\theta_6\theta_x\theta_2\hat{\Gamma}, z : \theta_9\hat{B}) \rightsquigarrow \Gamma, z : B; \theta_{13}$, and $\theta_{14}\theta_{13}(\hat{A}!\langle\hat{F}\rangle) = A!\langle E \rangle$. By Lemma 17, there exists θ_z, θ_w such that $\theta_{14}\theta_{13} = \theta_z\theta_w$ and $\hat{F} \sim E : \theta_w$. By restricting the domain to be $\hat{\Gamma}' = \theta_{12}\theta_9\theta_6\theta_x\theta_2\hat{\Gamma}$, we have $\theta_{13} = (\theta_{14}\theta_{13})|_{\hat{\Gamma}'} = (\theta_z\theta_w)|_{\hat{\Gamma}'}$.

By A-SC, we have $\hat{\Gamma} \vdash \mathbf{sc} \ell^{\text{sc}} v (y. c_1) (z. c_2) : \theta_w(\hat{A}!\langle\hat{F}\rangle) \dashv \theta_w\theta_{12}\theta_9\theta_6\theta_x\theta_2$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_w\theta_{12}\theta_9\theta_6\theta_x\theta_2$, $\theta_{\text{inst}} = \theta_z|_{\theta_w\hat{\Gamma}'}$, and $\theta_d = \theta_{14}$, the conclusion is satisfied.

SD-Hand By the IH on v , there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash v : \hat{A} \dashv \theta_2$, $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ (1), and $\theta_4\theta_3\hat{A} = \underline{C} \Rightarrow \underline{D}$. By $\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}}$, we have $\theta_4\theta_7(\theta_6\hat{A}) = \underline{C} \Rightarrow \underline{D}$ (1).

Then we apply the IH on c to (1), which gives us that there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}}$, $\theta_2\hat{\Gamma} \vdash c : \hat{C} \dashv \theta_6$, $\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_7$ (3), and $\theta_8\theta_7\hat{C} = \underline{C}$. Taking two fresh unification variables $\hat{\alpha}, \hat{\mu}$ and supposing $\underline{D} = A!\langle E \rangle$, we have $[A / \hat{\alpha}, E / \hat{\mu}]\theta_8\theta_7(\hat{C} \Rightarrow \alpha!\langle\mu\rangle) = \underline{C} \Rightarrow \underline{D}$ (2).

Notice that θ_4 only substitutes new unification variables in $\hat{A}$, and $[A / \hat{\alpha}, E / \hat{\mu}]\theta_8$ only substitutes new unification variables in $(\hat{C} \Rightarrow \alpha!\langle\mu\rangle)$, we can combine (1) and (2) to get the equation $[A / \hat{\alpha}, E / \hat{\mu}]\theta_8\theta_4\theta_7(\theta_6\hat{A}) = [A / \hat{\alpha}, E / \hat{\mu}]\theta_8\theta_4\theta_7(\hat{C} \Rightarrow \alpha!\langle\mu\rangle)$. By Lemma 17, there exists θ_x, θ_y such that $[A / \hat{\alpha}, E / \hat{\mu}]\theta_8\theta_4\theta_7 = \theta_y\theta_x$ and $\theta_6\hat{A} \sim \hat{C} \Rightarrow \hat{\alpha}!\langle\hat{\mu}\rangle : \theta_x$. By restricting the domain to be $\theta_6\theta_2\hat{\Gamma}$, we have $\theta_7 = (\theta_y\theta_x)|_{\theta_6\theta_2\hat{\Gamma}}$.

By A-HAND, we have $\hat{\Gamma} \vdash v \star c : \theta_x(\hat{\alpha}!\langle\hat{\mu}\rangle) \dashv \theta_x\theta_6\theta_2$. By Lemma 13 and (3), we have $\theta_x\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_y|_{\theta_x\theta_6\theta_2\hat{\Gamma}}$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_x\theta_6\theta_2$, $\theta_{\text{inst}} = \theta_y|_{\theta_x\theta_6\theta_2\hat{\Gamma}}$, and $\theta_d = \theta_4$, it is easy to check the conclusion is satisfied.

Return clause

SD-Return Taking a fresh unification variable $\hat{\alpha}$, for any $\hat{\Gamma}, x : \hat{\alpha} \rightsquigarrow \Gamma, x : A; \theta_1$, there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma}, x : \hat{\alpha} \vdash c_r : \hat{C} \dashv \theta_2$, $\theta_2(\hat{\Gamma}, x : \hat{\alpha}) \rightsquigarrow \Gamma, x : A; \theta_3$ (1), and $\theta_4\theta_3\hat{C} = M A!\langle E \rangle$. By (1), we have $\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_3|_{\theta_2\hat{\Gamma}}$ (2).

Because $\theta_1 = \theta_3\theta_2|_{\hat{\Gamma}}$ and $\theta_1\hat{\alpha} = A$, we have $M A! E = [E / \hat{\mu}]\theta_3\theta_2(M \hat{\alpha}!\langle\hat{\mu}\rangle) = \theta_4\theta_3\hat{C}$. Note that θ_4 only substitutes new unification variables in $\hat{C}$, we have $\theta_4[E / \hat{\mu}]\theta_3\theta_2(M \hat{\alpha}!\langle\hat{\mu}\rangle) = \theta_4[E / \hat{\mu}]\theta_3\hat{C}$. By Lemma 17, there exists θ_x, θ_y such

that $\theta_4[E / \hat{\mu}] \theta_3 = \theta_y \theta_x$ and $\hat{C} \sim \hat{\alpha}! \langle \hat{\mu} \rangle : \theta_x$. By restricting the domain to be $\theta_2 \hat{\Gamma}$, we have $\theta_3|_{\theta_2 \hat{\Gamma}} = (\theta_y \theta_x)|_{\theta_2 \hat{\Gamma}}$.

By A-RETURN, we have $\hat{\Gamma} \vdash \text{return } x \mapsto c_r : \theta_x \theta_2(M \hat{\alpha}! \langle \hat{\mu} \rangle) \dashv \theta_x \theta_2$. By (2) and Lemma 13, we have $\theta_x \theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_x \theta_2$, $\theta_{\text{inst}} = \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$, and $\theta_d = [E / \hat{\mu}]$, it is easy to check the conclusion is satisfied.

Operation clauses

SD-Empty Our goal is for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$ and fresh unification variables $\hat{\alpha}, \hat{\mu}$, there exists $\theta_{\text{inf}}, \theta_{\text{inst}}, \theta_d$ such that $\theta_a = \theta_{\text{inst}} \theta_{\text{inf}}$, $\hat{\Gamma} \vdash \cdot : M \hat{\alpha}! \langle \hat{\mu} \rangle \dashv \theta_{\text{inf}}$, $\theta_{\text{inf}} \hat{\Gamma} \rightsquigarrow \Gamma; \theta_{\text{inst}}$, $\theta_d \theta_{\text{inst}}(M \hat{\alpha}! \langle \hat{\mu} \rangle) = M A! \langle E \rangle$. It is easy to check that setting $\theta_{\text{inf}} = \emptyset$, $\theta_{\text{inst}} = \theta_a$, and $\theta_d = [A / \hat{\alpha}, E / \hat{\mu}]$ satisfies our goal.

SD-OprOp By the IH on *cls*, for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4, \hat{A}, \hat{E}$ such that $\theta_1 = \theta_3 \theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash \text{cls} : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_2$, $\theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ (1), and $\theta_4 \theta_3(M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle$ (2). Note that here we want θ_4 to be minimal, i.e. $\theta_4 = \theta_4|_{\theta_3(M \hat{A}! \langle \hat{E} \rangle)}$.

By (1) and (2), we have $\theta_2 \hat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \hat{A}! \langle \hat{E} \rangle \rightsquigarrow \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M A! \langle E \rangle; \theta_4 \theta_3$ (3). Let $\hat{\Gamma}' = \theta_2 \hat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \hat{A}! \langle \hat{E} \rangle$ and $\Gamma' = \Gamma, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M A! \langle E \rangle$. Applying the IH on *c* to (3), there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_4 \theta_3 = \theta_7 \theta_6|_{\theta_2 \hat{\Gamma}, x : A_{\text{op}}, k : B_{\text{op}} \rightarrow M \hat{A}! \langle \hat{E} \rangle}$, $\hat{\Gamma}' \vdash c : \hat{C} \dashv \theta_6$, $\theta_6 \hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$, and $\theta_8 \theta_7 \hat{C} = M A! \langle E \rangle$ (4).

By (2), (4), $\theta_4 \theta_3 = \theta_7 \theta_6$, and the fact that θ_8 only substitutes new unification variables in $\hat{C}$, we have $\theta_8 \theta_7 \theta_6(M \hat{A}! \langle \hat{E} \rangle) = \theta_8 \theta_7 \hat{C}$. By Lemma 17, there exists θ_y, θ_x such that $\theta_8 \theta_7 = \theta_y \theta_x$ and $\theta_6(M \hat{A}! \langle \hat{E} \rangle) \sim \hat{C} : \theta_x$. By restricting the domain to be $\theta_2 \hat{\Gamma}$, we have $\theta_7|_{\theta_2 \hat{\Gamma}} = (\theta_y \theta_x)|_{\theta_2 \hat{\Gamma}}$ (5).

By A-OPROP, we have $\hat{\Gamma} \vdash_M \text{op } \ell^{\text{op}} x k \mapsto c, \text{cls} : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_x \theta_6 \theta_2$. By $\theta_6 \hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$ and (5), we have $\theta_x \theta_6 \theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_x \theta_6 \theta_2$, $\theta_{\text{inst}} = \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$, and $\theta_d = \theta_4$, the conclusion is satisfied.

SD-OprSc Similar to the case of SD-OPROP.

By the IH on *cls*, for any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_1$, there exists $\theta_2, \theta_3, \theta_4, \hat{A}, \hat{E}$ such that $\theta_1 = \theta_3 \theta_2|_{\hat{\Gamma}}$, $\hat{\Gamma} \vdash \text{cls} : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_2$, $\theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_3$ (1), and $\theta_4 \theta_3(M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle$ (2). Note that here we want θ_4 to be minimal, i.e. $\theta_4 = \theta_4|_{\theta_3(M \hat{A}! \langle \hat{E} \rangle)}$.

Take a fresh type variable β . By (1) and (2), we have $\theta_2\hat{\Gamma}, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle \hat{E} \rangle, k : \beta \rightarrow M \hat{A}! \langle \hat{E} \rangle \rightsquigarrow \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle E \rangle, k : \beta \rightarrow M A! \langle E \rangle; \theta_4\theta_3$ (3). Let $\hat{\Gamma}' = \theta_2\hat{\Gamma}, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle \hat{E} \rangle, k : \beta \rightarrow M \hat{A}! \langle \hat{E} \rangle$ and $\Gamma' = \Gamma, \beta, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle E \rangle, k : \beta \rightarrow M A! \langle E \rangle$. Applying the IH on c to (3), there exists $\theta_6, \theta_7, \theta_8$ such that $\theta_4\theta_3 = \theta_7\theta_6|_{\theta_2\hat{\Gamma}, x : A_{\text{sc}}, p : B_{\text{sc}} \rightarrow M \beta! \langle \hat{E} \rangle, k : \beta \rightarrow M \hat{A}! \langle \hat{E} \rangle}$, $\hat{\Gamma}' \vdash c : \hat{C} \dashv \theta_6$, $\theta_6\hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$, and $\theta_8\theta_7\hat{C} = M A! \langle E \rangle$ (4).

By (2), (4), $\theta_4\theta_3 = \theta_7\theta_6$, and the fact that θ_8 only substitutes new unification variables in $\hat{C}$, we have $\theta_8\theta_7\theta_6(M \hat{A}! \langle \hat{E} \rangle) = \theta_8\theta_7\hat{C}$. By Lemma 17, there exists θ_y, θ_x such that $\theta_8\theta_7 = \theta_y\theta_x$ and $\theta_6(M \hat{A}! \langle \hat{E} \rangle) \sim \hat{C} : \theta_x$. By restricting the domain to be $\theta_2\hat{\Gamma}$, we have $\theta_7|_{\theta_2\hat{\Gamma}} = (\theta_y\theta_x)|_{\theta_2\hat{\Gamma}}$ (5).

Since β is fresh, we have $\beta \notin \text{rng}(\theta_2)$. By $\theta_6\hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$, we have $\beta \notin \text{rng}(\theta_6|_{\hat{\Gamma}'})$. Because $\theta_7\theta_6(M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle$, we have $\beta \notin \theta_6(M \hat{A}! \langle \hat{E} \rangle)$. Thus, $\beta \notin \theta_x$ and $\beta \notin \theta_6|_{\hat{C}}$. Thus, by Lemma 14, we have $\beta \notin \text{rng}(\theta_6)$. Finally, we have $\beta \notin \text{rng}(\theta_x\theta_6\theta_2)$.

By A-OPRSC, we have $\hat{\Gamma} \vdash_M \text{sc } \ell^{\text{sc}} x p k \mapsto c, \text{cls} : M \hat{A}! \langle \hat{E} \rangle \dashv \theta_x\theta_6\theta_2$. By $\theta_6\hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$ and (5), we have $\theta_x\theta_6\theta_2\hat{\Gamma} \rightsquigarrow \Gamma; \theta_y|_{\theta_x\theta_2\hat{\Gamma}}$. Setting $\theta_a = \theta_1$, $\theta_{\text{inf}} = \theta_x\theta_6\theta_2$, $\theta_{\text{inst}} = \theta_y|_{\theta_x\theta_2\hat{\Gamma}}$, and $\theta_d = \theta_4$, the conclusion is satisfied.

Forwarding clause

SD-Fwd Take fresh unification variables $\hat{\alpha}, \hat{\mu}$, fresh type variables α, β , and fresh type variable names γ, δ . For any $\hat{\Gamma} \rightsquigarrow \Gamma; \theta_a$, we have $\hat{\Gamma}, \alpha, \beta, p : \hat{A}_p, k : \hat{A}_k, f : \forall \gamma \delta. (\hat{A}'_p, \hat{A}'_k) \rightarrow \delta! \langle \hat{\mu} \rangle \rightsquigarrow \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow \delta! \langle E \rangle; [A / \hat{\alpha}, E / \hat{\mu}] \theta_a$ (1). All $A_p, A_k, A'_p, A'_k, \hat{A}_p, \hat{A}_k, \hat{A}'_p, \hat{A}'_k$ are defined the same as what are defined in rules SD-FWD and A-FWD. Let $\hat{\Gamma}' = \hat{\Gamma}, \alpha, \beta, p : \hat{A}_p, k : \hat{A}_k, f : \forall \gamma \delta. (\hat{A}'_p, \hat{A}'_k) \rightarrow \delta! \langle \hat{\mu} \rangle$ and $\Gamma' = \Gamma, \alpha, \beta, p : A_p, k : A_k, f : \forall \gamma \delta. (A'_p, A'_k) \rightarrow \delta! \langle E \rangle$.

Applying the IH on c_f to (1), there exists $\theta_2, \theta_3, \theta_4$ such that $\theta_3\theta_2|_{\hat{\Gamma}, p : \hat{A}_p, k : \hat{A}_k, f : \forall \gamma \delta. (\hat{A}'_p, \hat{A}'_k) \rightarrow \delta! \langle \hat{\mu} \rangle} = [A / \hat{\alpha}, E / \hat{\mu}] \theta_a$, $\hat{\Gamma}' \vdash c_f : \hat{C} \dashv \theta_2$, $\theta_2\hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_3$ (2), and $\theta_4\theta_3\hat{C} = M A! \langle E \rangle$.

Because $\hat{\alpha}$ and $\hat{\mu}$ are fresh, we have $\theta_4[A / \hat{\alpha}, E / \hat{\mu}] \theta_a(m \hat{\alpha}! \langle \hat{\mu} \rangle) = M A! \langle E \rangle$, which leads to $\theta_4\theta_3\theta_2(m \hat{\alpha}! \langle \hat{\mu} \rangle) = M A! \langle E \rangle = \theta_4\theta_3\hat{C}$. By Lemma 17, there exists θ_x, θ_y such that $\theta_4\theta_3 = \theta_y\theta_x$ and $\hat{C} \sim \theta_2(M \hat{\alpha}! \langle \hat{\mu} \rangle) : \theta_x$. By restricting the domain to be $\theta_2\hat{\Gamma}$, we have $\theta_3|_{\theta_2\hat{\Gamma}} = (\theta_y, \theta_x)|_{\theta_2\hat{\Gamma}}$.

It is obvious that $\gamma, \delta \notin \text{rng}(\theta_x \theta_2)$ because they are bounded by universal quantifiers. By $\theta_6 \hat{\Gamma}' \rightsquigarrow \Gamma'; \theta_7$, we have $\alpha, \beta \notin \text{rng}(\theta_6|_{\hat{\Gamma}'})$. Because $\theta_7 \theta_6(M \hat{A}! \langle \hat{E} \rangle) = M A! \langle E \rangle$, we have $\alpha, \beta \notin \theta_6(M \hat{A}! \langle \hat{E} \rangle)$. Thus, $\alpha, \beta \notin \theta_x$ and $\alpha, \beta \notin \theta_6|_{\hat{C}}$. Thus, by Lemma 14, we have $\alpha, \beta \notin \text{rng}(\theta_6)$. Finally, we have $\alpha, \beta, \gamma, \delta \notin \text{rng}(\theta_x \theta_6 \theta_2)$.

By A-FWD, we have $\hat{\Gamma} \vdash_M \text{fwd } f \ p \ k \mapsto c_f : \theta_x \theta_2(M \hat{A}! \langle \hat{\mu} \rangle) \dashv \theta_x \theta_2$. By (1) and Lemma 13, we have $\theta_x \theta_2 \hat{\Gamma} \rightsquigarrow \Gamma; \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$. Setting $\theta_{\text{inf}} = \theta_x \theta_2$, $\theta_{\text{inst}} = \theta_y|_{\theta_x \theta_2 \hat{\Gamma}}$, and $\theta_d = [A / \hat{A}, E / \hat{\mu}]$, the conclusion is satisfied.

□

Discussion and Limitations

Our type inference algorithm works as is for most effects. However, the type operators M might have additional type variables, which go beyond the basic type inference algorithm we have presented.

For example, the type of a handler of the reader effect with local state is

$$h_{\text{Read}} : \forall \alpha \ \mu. \alpha ! \langle \text{ask}; \text{local}; \mu \rangle \Rightarrow (\text{Int} \rightarrow^\mu (\alpha, \text{Int}))! \langle \mu \rangle$$

Thus, for type inference, h_{Read} has the type annotation $\lambda \alpha. \text{Int} \rightarrow^\mu (\alpha, \text{Int})$ which contains type variable μ that is bounded by $\forall \mu$ in the type of h_{Read} . Our basic algorithm does not allow free type variables in the annotation.

We see several ways to generalize our algorithm to accommodate these examples. The first approach is to allow free type variables to occur in the type annotation and add the condition that the free variables in the annotation are not in the range of the inferred substitution of the handler typing judgment. This condition ensures that the free type variables in the annotation will not escape the lexical scope of the handler type. The new handler typing rule is shown in Figure D.23.

Another approach is to extend the type annotation of handlers from $\lambda \alpha. A$ to $\lambda \alpha. \lambda \mu. A$ to bind the extra type variable μ . This way, we also need to adjust the typing rules of the handler and handler clauses to pass extra parameters to the type operator M . This approach is less general than the first.

$$\begin{array}{c}
\hat{\Gamma} \vdash_M \text{return } x \mapsto c_r : M \hat{A}_1! \langle \hat{E}_1 \rangle \dashv \theta_1 \\
\hat{A}_1 = \hat{\alpha}_1 \quad \theta_1 \hat{\Gamma} \vdash_M \text{cls} : M \hat{A}_2! \langle \hat{E}_2 \rangle \dashv \theta_2 \\
\hat{A}_2 = \hat{\alpha}_2 \quad \theta_2(\hat{\alpha}_1! \langle \hat{E}_1 \rangle) \sim \hat{\alpha}_2! \langle \hat{E}_2 \rangle : \theta_3 \\
\theta_{3..1} \hat{\Gamma} \vdash_M \text{fwd } f \ p \ k \mapsto c_f : M \hat{A}_3! \langle \hat{E}_3 \rangle \dashv \theta_4 \quad \hat{A}_3 = \hat{\alpha}_3 \\
\theta_{4..3}(\hat{\alpha}_2! \langle \hat{E}_2 \rangle) \sim \hat{\alpha}_3! \langle \hat{E}_3 \rangle : \theta_5 \quad \hat{\alpha}_4 = \theta_5 \hat{\alpha}_3 \quad \hat{\alpha}_4 \notin \text{fv}(\theta_{5..1} \hat{\Gamma}) \\
\langle \hat{F} \rangle = \langle \text{labels}(\text{cls}); \theta_5 \hat{E}_3 \rangle \quad \text{fv}(M) \cap \text{fv}(\theta_{5..1} \hat{\Gamma}) = \emptyset \\
\hline
\hat{\Gamma} \vdash \text{handler}_M \{ \text{return } x \mapsto c_r, \text{cls}, \text{fwd } f \ p \ k \mapsto c_f \} : \\
\hat{\alpha}_4! \langle \hat{F} \rangle \Rightarrow M \hat{\alpha}_4! \langle \theta_5 \hat{E}_3 \rangle \dashv \theta_{5..1}
\end{array} \text{A-H},$$

Figure D.23: The new type inference rule for handlers.

Appendix E

Derivation of the Second-Order Derivative

This Appendix expands on Section 11.4’s claim that we can derive from the specification

$$\begin{aligned} \text{derive}_{2nd} &:: (Eq\ v, Semiring\ d) \\ &\Rightarrow (v \rightarrow d) \rightarrow Expr\ v \rightarrow Dense\ v\ (Dense\ v\ d) \\ \text{derive}_{2nd}\ var\ e &= \lambda x\ y \rightarrow eval\ var\ (\tan^N\ (fwdAD_{dense}\ Var \\ &\quad (\tan^N\ (fwdAD_{dense}\ Var\ e)\ x))\ y) \end{aligned}$$

that consists of three successive appeals to *eval*, an equivalent version that is an instance of *abstractD* (which consists of a single *eval*):

$$\begin{aligned} \text{derive}'_{2nd}\ var\ e &= \lambda x\ y \rightarrow \tan^N\ (\tan^N\ (\text{abstractD}\ gen\ e)\ x)\ y \\ &\quad \textbf{where}\ gen\ z = N\ (var\ z)\ (\delta z) \end{aligned}$$

We start the derivation from a point-free formulation of *derive*_{2nd}:

$$\begin{aligned} \lambda x\ y \rightarrow &\quad eval\ var \circ (\$y) \circ \tan^N \circ fwdAD_{dense}\ Var \\ &\quad \circ (\$x) \circ \tan^N \circ fwdAD_{dense}\ Var \\ = \{ &\text{-naturality of function application -} \} \\ \lambda x\ y \rightarrow &\quad (\$y) \circ \text{fmap}\ (eval\ var) \circ \tan^N \circ fwdAD_{dense}\ Var \\ &\quad \circ (\$x) \circ \tan^N \circ fwdAD_{dense}\ Var \end{aligned}$$

$$\begin{aligned}
&= \{-\eta\text{-reduction -}\} \\
&\quad \lambda x \rightarrow \quad \text{fmap } (eval \text{ var}) \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&\quad \quad \circ (\$x) \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&= \{-\text{naturality of function application -}\} \\
&\quad \lambda x \rightarrow \quad (\$x) \circ \text{fmap } (\text{fmap } (eval \text{ var}) \circ tan^N \circ fwdAD_{dense} \text{ Var}) \\
&\quad \quad \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&= \{-\eta\text{-reduction -}\} \\
&\quad \text{fmap } (\text{fmap } (eval \text{ var}) \circ tan^N \circ fwdAD_{dense} \text{ Var}) \\
&\quad \quad \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&= \{-\text{naturality of } tan^N; \text{ see Equation (E.1) below -}\} \\
&\quad \text{fmap } (tan^N \circ \text{bimap } (eval \text{ var}) (\text{fmap } (eval \text{ var}))) \circ fwdAD_{dense} \text{ Var} \\
&\quad \quad \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&= \{-eval \text{ fusion -}\} \\
&\quad \text{fmap } (tan^N \circ eval \text{ gen}') \circ tan^N \circ fwdAD_{dense} \text{ Var} \\
&\quad \quad \textbf{where } gen' \text{ z} = N \text{ (var z) (delta z)} \\
&= \{-\text{fmap fission + naturality of } tan^N \text{ -}\} \\
&\quad \text{fmap } tan^N \circ tan^N \circ \text{bimap } (eval \text{ gen}') (\text{fmap } (eval \text{ gen}')) \\
&\quad \quad \circ fwdAD_{dense} \text{ Var} \\
&\quad \quad \textbf{where } gen' \text{ z} = N \text{ (var z) (delta z)} \\
&= \{-eval \text{ fusion -}\} \\
&\quad \text{fmap } tan^N \circ tan^N \circ eval \text{ gen}'' \\
&\quad \quad \textbf{where } gen'' \text{ z} = N \text{ (N (var z) (delta z)) (delta z)} \\
&= \{-abstractD \text{ definition -}\} \\
&\quad \text{fmap } tan^N \circ tan^N \circ abstractD \\
&\quad \quad \textbf{where } gen \text{ z} = N \text{ (var z) (delta z)}
\end{aligned}$$

The steps involving the naturality of tan^N concern the *Bifunctor* instance of Nagata numbers, which allows to vary both components.

instance *Bifunctor* $\cdot \ltimes \cdot$ **where**
 $\text{bimap } f \text{ g } (N \text{ d } e) = N \text{ (f d) (g d)}$

The general naturality property of $tan^N :: \forall d \ e. d \ltimes e \rightarrow e$ for this bifunctor is:

$$g \cdot tan^N = tan^N \cdot \text{bimap } f \text{ g}$$

However, for the specialized type $tan^N :: \forall d. d \ltimes (Dense \text{ v } d) \rightarrow Dense \text{ v } d$ where the two components are not independent, we also have this specialized naturality property:

$$\text{fmap } f \cdot tan^N = tan^N \cdot \text{bimap } f \text{ (fmap } f) \tag{E.1}$$

Appendix F

Proof of Lemma 1

Proof. We prove this rule by means of the chain rule for multi-variate functions:

$$\frac{\partial f(e_1, \dots, e_n)}{\partial x} = \sum_{i=1}^n \left(\frac{\partial f(x_1, \dots, x_n)}{\partial x_i} \Big|_{x_1=e_1, \dots, x_n=e_n} \times \frac{\partial e_i}{\partial x} \right)$$

Define f as $f(y, x_1, \dots, x_n) = e_2$ where $x_1, \dots, x_n$ comprise all the free variables of e_1 and e_2 . Then we have that $f(e_1, x_1, \dots, x_n) = \mathbf{let } y = e_1 \mathbf{ in } e_2$.

Using this in combination with the chain rule yields:

$$\begin{aligned} \frac{\partial (\mathbf{let } y = e_1 \mathbf{ in } e_2)}{\partial x_i} &= \frac{\partial f(e_1, x_1, \dots, x_n)}{\partial x_i} \\ &= \frac{\partial f(y, x_1, \dots, x_n)}{\partial y} \Big|_{y=e_1, x_1=x_1, \dots, x_n=x_n} \times \frac{\partial e_1}{\partial x_i} \\ &\quad + \sum_{j=1}^n \left(\frac{\partial f(y, x_1, \dots, x_n)}{\partial x_j} \Big|_{y=e_1, x_1=x_1, \dots, x_n=x_n} \times \frac{\partial x_j}{\partial x_i} \right) \\ &= \frac{\partial e_2}{\partial y} \Big|_{y=e_1} \times \frac{\partial e_1}{\partial x_i} + \frac{\partial e_2}{\partial x_i} \Big|_{y=e_1} \end{aligned}$$

□

Appendix G

Functions of Different Haskell Libraries

Maps

The functions below are from Haskell's *Data.Map* library¹, copied with their type signature and explanation.

- *empty* :: *Map k a*
 $\mathcal{O}(1)$. The empty map.
- *(!)* :: *Ord k* $\Rightarrow$ *Map k a* $\rightarrow$ *k* $\rightarrow$ *a*
 $\mathcal{O}(\log n)$. Find the value at a key. Calls *error* when the element can not be found.
- *findWithDefault* :: *Ord k* $\Rightarrow$ *a* $\rightarrow$ *k* $\rightarrow$ *Map k a* $\rightarrow$ *a*
 $\mathcal{O}(\log n)$. The expression *findWithDefault def k map* returns the value at key *k* or returns default value *def* when the key is not in the *map*.
- *fromList* :: *Ord k* $\Rightarrow$ [(*k*, *a*)] $\rightarrow$ *Map k a*
 $\mathcal{O}(n \times \log n)$. Build a map from a list of key/value pairs. If the list contains more than one value for the same key, the last value for the key is retained.

¹<https://hackage.haskell.org/package/containers-0.4.0.0/docs/Data-Map.html>

- *unionWith* :: $Ord\ k \Rightarrow (a \rightarrow a \rightarrow a) \rightarrow Map\ k\ a \rightarrow Map\ k\ a \rightarrow Map\ k\ a$
 $\mathcal{O}(n + m)$. Union with a combining function. The implementation uses the efficient hedge-union algorithm.
- *insertWith* :: $Ord\ k \Rightarrow (a \rightarrow a \rightarrow a) \rightarrow k \rightarrow a \rightarrow Map\ k\ a \rightarrow Map\ k\ a$
 $\mathcal{O}(\log n)$. Insert with a function, combining new value and old value.
insertWith *f* *key* *value* *mp* will insert the pair (*key*, *value*) into *mp* if *key* does not exist in the map. If the key does exist, the function will insert the pair (*key*, *f new_value old_value*).

Arrays

The functions below are from Haskell’s *Data.Array.MArray* library², copied with their type signature and explanation.

- *newArray* :: $Ix\ i \Rightarrow (i, i) \rightarrow e \rightarrow m\ (a\ i\ e)$
Builds a new array, with every element initialised to the supplied value.
- *readArray* :: $(MArray\ a\ e\ m, Ix\ i) \Rightarrow a\ i\ e \rightarrow i \rightarrow m\ e$
Read an element from a mutable array
- *writeArray* :: $(MArray\ a\ e\ m, Ix\ i) \Rightarrow a\ i\ e \rightarrow i \rightarrow e \rightarrow m\ ()$
Write an element in a mutable array
- *getAssocs* :: $(MArray\ a\ e\ m, Ix\ i) \Rightarrow a\ i\ e \rightarrow m\ [(i, e)]$
Return a list of all the associations of a mutable array, in index order.

Below, we document the *ST* monad from Haskell’s *Control.Monad.ST* library³.

- **data** *ST* *s* *a*
The *ST* monad allows for destructive updates, but is escapable. A computation of type *ST s a* returns a value of type *a*, and execute in “thread” *s*. It serves to keep the internal states of different invocations of *runST* separate from each other.
- *runST* :: $(\forall\ s.\ ST\ s\ a) \rightarrow a$
Return the value computed by a state thread. The $\forall$ ensures that the internal state used by the *ST* computation is inaccessible to the rest of the program.

²<https://hackage.haskell.org/package/array-0.5.1.1/candidate/docs/Data-Array-MArray.html>

³<https://hackage.haskell.org/package/base-4.18.0.0/docs/Control-Monad-ST.html>

Below, we document the *STArray* arrays from Haskell's *Data.Array.ST* library⁴.

- **data** *STArray s i e*
Mutable, boxed, non-strict arrays in the *ST* monad. *s* is the state variable argument for the *ST* type, *i* is the index type of the array (should be an instance of *Ix*), and *e* is the element type of the array.

Lists

The functions below are from Haskell's *Data.List* library⁵, copied with their type signature and explanation.

- *concatMap* :: *Foldable t* $\Rightarrow$ (*a* $\rightarrow$ [*b*]) $\rightarrow$ *t a* $\rightarrow$ [*b*]
Map a function over all the elements of a container and concatenate the resulting lists.
- *transpose* :: [[*a*]] $\rightarrow$ [[*a*]]
The transpose function transposes the rows and columns of its argument.
- *replicate* :: *Int* $\rightarrow$ *a* $\rightarrow$ [*a*]
replicate n x is a list of length *n* with *x* the value of every element.
- *foldr* :: *Foldable t* $\Rightarrow$ (*a* $\rightarrow$ *b* $\rightarrow$ *b*) $\rightarrow$ *b* $\rightarrow$ *t a* $\rightarrow$ *b*
Right-associative fold of a structure, lazy in the accumulator. In the case of lists, *foldr*, when applied to a binary operator, a starting value (typically the right-identity of the operator), and a list, reduces the list using the binary operator, from right to left. Note that since the head of the resulting expression is produced by an application of the operator to the first element of the list, given an operator lazy in its right argument, *foldr* can produce a terminating expression from an unbounded list.
- *zip* :: [*a*] $\rightarrow$ [*b*] $\rightarrow$ [(*a*, *b*)]
zip takes two lists and returns a list of corresponding pairs.
- *zipWith* :: (*a* $\rightarrow$ *b* $\rightarrow$ *c*) $\rightarrow$ [*a*] $\rightarrow$ [*b*] $\rightarrow$ [*c*]
zipWith generalizes *zip* by zipping with the function given as the first argument, instead of a tupling function.
- *subsequences* :: [*a*] $\rightarrow$ [[*a*]]
The *subsequences* function returns the list of all subsequences of the argument.

⁴<https://hackage.haskell.org/package/array-0.5.5.0/docs/Data-Array-ST.html>

⁵<https://hackage.haskell.org/package/base-4.18.0.0/docs/Data-List.html>

Sets

The functions below are from Haskell's *Data.Set* library⁶, copied with their type signature and explanation.

- *empty* :: *Set a*
 $\mathcal{O}(1)$. The empty set.
- *fromList* :: *Ord a* $\Rightarrow$ $[a] \rightarrow \text{Set } a$
 $\mathcal{O}(n \log n)$. Create a set from a list of elements.
- *toList* :: *Set a* $\rightarrow [a]$
 $\mathcal{O}(n)$. Convert the set to a list of elements. Subject to list fusion.
- *member* :: *Ord a* $\Rightarrow a \rightarrow \text{Set } a \rightarrow \text{Bool}$
 $\mathcal{O}(\log n)$. Is the element in the set?
- *filter* :: $(a \rightarrow \text{Bool}) \rightarrow \text{Set } a \rightarrow \text{Set } a$
 $\mathcal{O}(n)$. Filter all elements that satisfy the predicate.
- *union* :: *Ord a* $\Rightarrow \text{Set } a \rightarrow \text{Set } a \rightarrow \text{Set } a$
 $\mathcal{O}(m \log \frac{n+1}{m+1})$, $m \leq n$. The union of two sets, preferring the first set when equal elements are encountered.
- *difference* :: *Ord a* $\Rightarrow \text{Set } a \rightarrow \text{Set } a \rightarrow \text{Set } a$
 $\mathcal{O}(m \log \frac{n+1}{m+1})$, $m \leq n$. Difference of two sets. Return elements of the first set not existing in the second set.
- *intersection* :: *Ord a* $\Rightarrow \text{Set } a \rightarrow \text{Set } a \rightarrow \text{Set } a$
 $\mathcal{O}(m \log \frac{n+1}{m+1})$, $m \leq n$. The intersection of two sets. Elements of the result come from the first set

Other

Below, we document the *foldMap* function from Haskell's *Data.Foldable* library⁷.

- *foldMap* :: *Monoid m* $\Rightarrow (a \rightarrow m) \rightarrow t \ a \rightarrow m$
 Map each element of the structure into a monoid, and combine the results with $(\diamond)$. This fold is right-associative and lazy in the accumulator.

⁶<https://hackage.haskell.org/package/containers-0.6.7/docs/Data-Set.html>

⁷<https://hackage.haskell.org/package/base-4.18.0.0/docs/Data-Foldable.html>

Below, we document the *ReaderT* monad transformer taken from Haskell's *Control.Monad.Trans.Reader* library⁸.

- **newtype** *ReaderT* *r m a* = *ReaderT* { *runReaderT* :: *r* → *m a* }
The reader monad transformer, which adds a read-only environment to the given monad *m*. The **return** function ignores the environment, while (**>>=**) passes the inherited environment to both subcomputations.

⁸<https://hackage.haskell.org/package/transformers-0.6.1.0/docs/Control-Monad-Trans-Reader.html>

Appendix H

Derivation of fwdad/4 from Symbolic Differentiation

Define `ad_spec/5` as the naive implementation of forward-mode automatic differentiation that first symbolically derives expression `E` with respect to `X` and subsequently evaluates the expression and the derivative with respect to environment `Env`.

```
ad_spec(E,X,Env,N,DN) :-  
    symb(E,X,F,DF), eval(F,Env,N), eval(DF,Env,DN).
```

We can derive `fwdad/5` by unfold/fold transformations; we start with unfolding `symb/4`.

- Case `lit/1`

```
ad_spec(lit(M),X,Env,N,DN) :-  
    lit(M),X,lit(N),lit(0),  
    eval(lit(M),Env,N), eval(lit(0),Env,DN).
```

(unfold of `eval`)

```
ad_spec(lit(M),X,Env,M,0).
```

- Case `var/1`

```
ad_spec(var(Y),X,Env,N,DN) :-  
    ( X == Y -> DF = 1 ; DF = 0 ),  
    eval(var(Y),Env,N), eval(lit(DF),Env,DN).
```

```
(commutativity of ,/2 and distributivity w.r.t. (->))
```

```
ad_spec(var(Y),X,Env,N,DN) :-
  eval(var(Y),Env,N),
  ( X == Y -> eval(lit(1),Env,DN);
    eval(lit(0),Env,DN)).
```

```
(unfold eval)
```

```
ad_spec(var(Y),X,Env,N,DN) :-
  arg(Y,Env,N),
  ( X == Y -> DN = 1; DN = 0).
```

- Case add/2

```
ad_spec(add(E1,E2),X,Env,N,DN) :-
  symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
  eval(add(F1,F2),Env,N), eval(add(DF1,DF2),Env,DN).
```

```
(unfold eval)
```

```
ad_spec(add(E1,E2),X,Env,N,DN) :-
  symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
  eval(F1,Env,N1), eval(F2,Env,N2),
  N is N1 + N2,
  eval(DF1,Env,DN1), eval(DF2,Env,DN2),
  DN is DN1 + DN2.
```

```
(commutativity of ,/2)
```

```
ad_spec(add(E1,E2),X,Env,N,DN) :-
  symb(E1,X,F1,DF1),
  eval(F1,Env,N1), eval(DF1,Env,DN1),
  symb(E2,X,F2,DF2),
  eval(DF2,Env,DN2), eval(F2,Env,N2),
  N is N1 + N2, DN is DN1 + DN2.
```

```
(fold of ad_spec)
```

```
ad_spec(add(E1,E2),X,Env,N,DN) :-
  ad_spec(E1,X,N1,DN1),
  symb(E2,X,N2,DN2),
  N is N1 + N2, DN is DN1 + DN2.
```

- Case mul/2


```

ad_spec(mul(E1,E2),X,Env,N,DN) :-
    symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
    eval(mul(F1,F2),Env,N),
    eval(add(mul(F1,DF2),mul(F2,DF1)),Env,DN).

```

(unfold eval)

```

ad_spec(mul(E1,E2),X,Env,N,DN) :-
    symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
    eval(F1,Env,N1), eval(F2,Env,N2),
    N is N1 * N2,
    eval(F1,Env,M1), eval(DF2,Env,DN2),
    eval(F2,Env,M2), eval(DF1,Env,DN1),
    DN is M1 * DN2 + M2 * DN1.

```

(functional dependency of eval and idempotence)

```

ad_spec(mul(E1,E2),X,Env,N,DN) :-
    symb(E1,X,F1,DF1), symb(E2,X,F2,DF2),
    eval(F1,Env,N1), eval(F2,Env,N2),
    N is N1 * N2,
    eval(DF2,Env,DN2), eval(DF1,Env,DN1),
    DN is N1 * DN2 + N2 * DN1.

```

(commutativity of ,/2)

```

ad_spec(mul(E1,E2),X,Env,N,DN) :-
    symb(E1,X,F1,DF1),
    eval(F1,Env,N1), eval(DF1,Env,DN1),
    symb(E2,X,F2,DF2),
    eval(F2,Env,N2), eval(DF2,Env,DN2),
    N is N1 * N2, DN is N1 * DN2 + N2 * DN1.

```

(fold of ad_spec)

```

ad_spec(mul(E1,E2),X,Env,N,DN) :-
    ad_spec(E1,X,N1,DN1), ad_spec(E2,X,N2,DN2),
    N is N1 * N2, DN is N1 * DN2 + N2 * DN1.

```


FACULTY OF ENGINEERING SCIENCE
DEPARTMENT OF COMPUTER SCIENCE



DTAI
Celestijnenlaan 200A box 2402
B-3001 Leuven
birthe.vandenberg@kuleuven.be